

Lecture 1: Introduction

6.006 pre-requisite:

- Data structures such as heaps, trees, graphs
- Algorithms for sorting, shortest paths, graph search, dynamic programming

Course Overview

This course covers several modules:

1. Divide and Conquer - FFT, Randomized algorithms
2. Optimization - greedy and dynamic programming
3. Network Flow
4. Intractibility (and dealing with it)
5. Linear programming
6. Sublinear algorithms, approximation algorithms
7. Advanced topics

Theme of today's lecture

Very similar problems can have very different complexity. Recall:

- **P**: class of problems solvable in polynomial time. $O(n^k)$ for some constant k .

Shortest paths in a graph can be found in $O(V^2)$ for example.

- **NP**: class of problems verifiable in polynomial time.

Hamiltonian cycle in a directed graph $G(V, E)$ is a simple cycle that contains each vertex in V .

Determining whether a graph has a hamiltonian cycle is NP-complete but verifying that a cycle is hamiltonian is easy.

- **NP-complete**: problem is in NP and is as hard as any problem in NP.

If any NPC problem can be solved in polynomial time, then every problem in NP has a polynomial time solution.

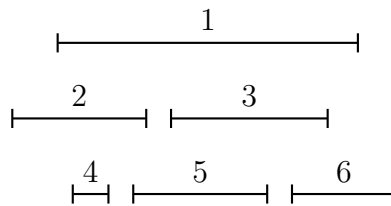
Interval Scheduling

Requests $1, 2, \dots, n$, single resource

$s(i)$ start time, $f(i)$ finish time, $s(i) < f(i)$ (start time must be less than finish time for a request)

Two requests i and j are compatible if they don't overlap, i.e., $f(i) \leq s(j)$ or $f(j) \leq s(i)$.

In the figure below, requests 2 and 3 are compatible, and requests 4, 5 and 6 are compatible as well, but requests 2 and 4 are not compatible.



Goal: Select a compatible subset of requests of maximum size.

Claim: We can solve this using a greedy algorithm.

A greedy algorithm is a myopic algorithm that processes the input one piece at a time with no apparent look ahead.

Greedy Interval Scheduling

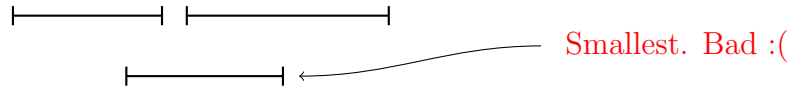
1. Use a simple rule to select a request i .
2. Reject all requests incompatible with i .
3. Repeat until all requests are processed.

Possible rules?

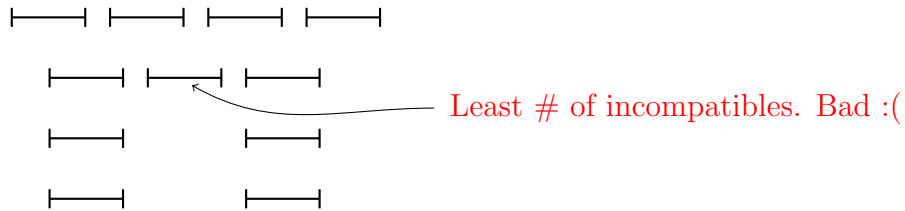
1. Select request that starts earliest, i.e., minimum $s(i)$.



2. Select request that is smallest, i.e., minimum $f(i) - s(i)$.



3. For each request, find number of incompatibles, and select request with minimum such number.



4. Select request with earliest finish time, i.e., minimum $f(i)$.

Claim 1. Greedy algorithm outputs a list of intervals

$$\langle s(i_1), f(i_1) \rangle, \langle s(i_2), f(i_2) \rangle, \dots, \langle s(i_k), f(i_k) \rangle$$

such that

$$s(i_1) < f(i_1) \leq s(i_2) < f(i_2) \leq \dots \leq s(i_k) < f(i_k)$$

Proof. Simple proof by contradiction – if $f(i_j) > s(i_{j+1})$, interval j and $j+1$ intersect, which is a contradiction of Step 2 of the algorithm! $\square$

Claim 2. Given list of intervals L , greedy algorithm with earliest finish time produces k^* intervals, where k^* is optimal.

Proof. Induction on k^* .

Base case: $k^* = 1$ – this case is easy, any interval works.

Inductive step: Suppose claim holds for k^* and we are given a list of intervals whose optimal schedule has $k^* + 1$ intervals, namely

$$S^*[1, 2, \dots, k^* + 1] = \langle s(j_1), f(j_1) \rangle, \dots, \langle s(j_{k^*+1}), f(j_{k^*+1}) \rangle$$

Say for some generic k , the greedy algorithm gives a list of intervals

$$S[1, 2, \dots, k] = \langle s(i_1), f(i_1) \rangle, \dots, \langle s(i_k), f(i_k) \rangle$$

By construction, we know that $f(i_1) \leq f(j_1)$, since the greedy algorithm picks the **earliest finish time**.

Now we can create a schedule

$$S^{**} = \langle s(i_1), f(i_1) \rangle, \langle s(j_2), f(j_2) \rangle, \dots, \langle s(j_{k^*+1}), f(j_{k^*+1}) \rangle$$

since the interval $\langle s(i_1), f(i_1) \rangle$ does not overlap with the interval $\langle s(j_2), f(j_2) \rangle$ and all intervals that come after that. Note that since the length of S^{**} is $k^* + 1$, this schedule is also optimal.

Now we proceed to define L' as the set of intervals with $s(i) \geq f(i_1)$.

Since S^{**} is optimal for L , $S^{**}[2, 3, \dots, k^* + 1]$ is optimal for L' , which implies that the optimal schedule for L' has k^* size.

We now see by our initial inductive hypothesis that running the greedy algorithm on L' should produce a schedule of size k^* . Hence, by our construction, running the greedy algorithm on L' gives us $S[2, \dots, k]$.

This means $k - 1 = k^*$ or $k = k^* + 1$, which implies that $S[1, \dots, k]$ is indeed optimal, and we are done.

□

Weighted Interval Scheduling

Each request i has weight $w(i)$. Schedule subset of requests that are non-overlapping with maximum weight.

A key observation here is that the greedy algorithm no longer works.

Dynamic Programming

We can define our sub-problems as

$$R^x = \{j \in R \mid s(j) \geq x\}$$

Here, R is the set of all requests.

If we set $x = f(i)$, then R^x is the set of requests later than request i .

Total number of sub-problems = n (one for each request)

Only need to solve each subproblem once and memoize.

We try each request i as a possible first. If we pick a request as the first, then the remaining requests are $R^{f(i)}$.

Note that even though there may be requests compatible with i that are not in $R^{f(i)}$, we are picking i as the first request, i.e., we are going in order.

$$\text{opt}(R) = \max_{1 \leq i \leq n} (w(i) + \text{opt}(R^{f(i)}))$$

Total running time is $O(n^2)$ since we need $O(n)$ time to solve each sub-problem.

Turns out that we can actually reduce the overall complexity to $O(n \log n)$. We leave this as an exercise.

Non-identical machines

As before, we have n requests $\{1, 2, \dots, n\}$. Each request i is associated with a start time $s(i)$ and finish time $f(i)$, m different machine types as well $\tau = \{T_1, \dots, T_m\}$. Each request i is associated with a set $Q(i) \subseteq \tau$ that represents the set of machines that request i can be serviced on.

Each request has a weight of 1. We want to maximize the number of jobs that can be scheduled on the m machines.

This problem is in NP, since we can clearly check that a given subset of jobs with machine assignments is legal.

Can $k \leq n$ requests be scheduled? This problem is NP-complete.

Maximum number of requests that should be scheduled? This problem is NP-hard.

Dealing with intractability

1. Approximation algorithms: Guarantee within some factor of optimal in polynomial time.
2. Pruning heuristics to reduce (possible exponential) runtime on “real-world” examples.
3. Greedy or other sub-optimal heuristics that work well in practice but provide no guarantees.

MIT OpenCourseWare
<http://ocw.mit.edu>

6.046J / 18.410J Design and Analysis of Algorithms
Spring 2015

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Lecture 2: Divide and Conquer

- Paradigm
- Convex Hull
- Median finding

Paradigm

Given a problem of size n divide it into subproblems of size $\frac{n}{b}$, $a \geq 1$, $b > 1$. Solve each subproblem recursively. Combine solutions of subproblems to get overall solution.

$$T(n) = aT\left(\frac{n}{b}\right) + [\text{work for merge}]$$

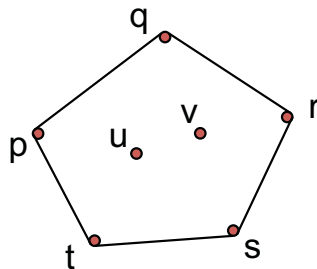
Convex Hull

Given n points in plane

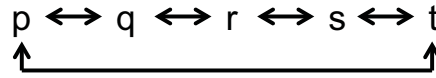
$$S = \{(x_i, y_i) | i = 1, 2, \dots, n\}$$

assume no two have same x coordinate, no two have same y coordinate, and no three in a line for convenience.

Convex Hull ($\text{CH}(S)$): smallest polygon containing all points in S .



$\text{CH}(S)$ represented by the sequence of points on the boundary in order clockwise as doubly linked list.



Brute force for Convex Hull

Test each line segment to see if it makes up an edge of the convex hull

- If the rest of the points are on one side of the segment, the segment is on the convex hull.
- else the segment is not.

$O(n^2)$ edges, $O(n)$ tests $\Rightarrow O(n^3)$ complexity

Can we do better?

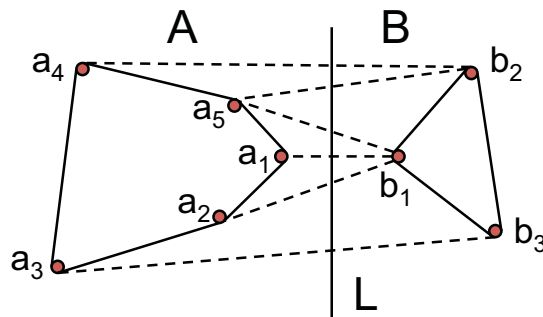
Divide and Conquer Convex Hull

Sort points by x coord (once and for all, $O(n \log n)$)

For input set S of points:

- Divide into left half A and right half B by x coords
- Compute $CH(A)$ and $CH(B)$
- Combine CH's of two halves (merge step)

How to Merge?



- Find upper tangent (a_i, b_j) . In example, (a_4, b_2) is U.T.
- Find lower tangent (a_k, b_m) . In example, (a_3, b_3) is L.T.

- Cut and paste in time $\Theta(n)$.

First link a_i to b_j , go down b list till you see b_m and link b_m to a_k , continue along the a list until you return to a_i . In the example, this gives (a_4, b_2, b_3, a_3) .

Finding Tangents

Assume a_i maximizes x within $CH(A)$ ($a_1, a_2, \dots, a_p$). b_1 minimizes x within $CH(B)$ ($b_1, b_2, \dots, b_q$)

L is the vertical line separating A and B . Define $y(i, j)$ as y-coordinate of intersection between L and segment (a_i, b_j) .

Claim: (a_i, b_j) is uppertangent iff it maximizes $y(i, j)$.

If $y(i, j)$ is not maximum, there will be points on both sides of (a_i, b_j) and it cannot be a tangent.

Algorithm: Obvious $O(n^2)$ algorithm looks at all a_i, b_j pairs. $T(n) = 2T(n/2) + \Theta(n^2) = \Theta(n^2)$.

```

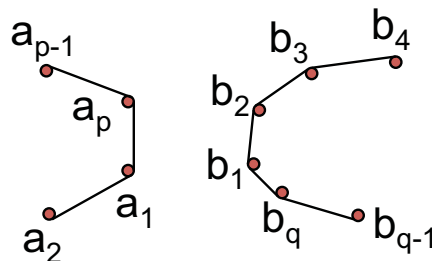
1  i = 1
2  j = 1
3  while (y(i, j + 1) > y(i, j) or y(i - 1, j) > y(i, j))
4      if (y(i, j + 1) > y(i, j)) ▷ move right finger clockwise
5          j = j + 1 ( mod q)
6      else
7          i = i - 1 ( mod p) ▷ move left finger anti-clockwise
8      return (a_i, b_j) as upper tangent

```

Similarly for lower tangent.

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n) = \Theta(n \log n)$$

Intuition for why Merge works



a_1, b_1 are right most and left most points. We move anti clockwise from a_1 , clockwise from b_1 . $a_1, a_2, \dots, a_q$ is a convex hull, as is $b_1, b_2, \dots, b_q$. If a_i, b_j is such that moving from either a_i or b_j decreases $y(i, j)$ there are no points above the (a_i, b_j) line.

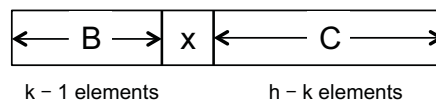
The formal proof is quite involved and won't be covered.

Median Finding

Given set of n numbers, define $rank(x)$ as number of numbers in the set that are $\leq x$. Find element of rank $\lfloor \frac{n+1}{2} \rfloor$ (lower median) and $\lceil \frac{n+1}{2} \rceil$ (upper median).

Clearly, sorting works in time $\Theta(n \log n)$.

Can we do better?



SELECT(S, i)

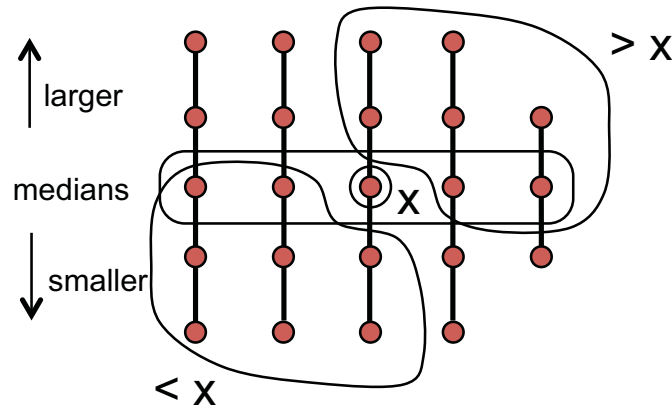
```

1  Pick  $x \in S$   $\triangleright$  cleverly
2  Compute  $k = rank(x)$ 
3   $B = \{y \in S | y < x\}$ 
4   $C = \{y \in S | y > x\}$ 
5  if  $k = i$ 
6      return  $x$ 
7  else if  $k > i$ 
8      return Select( $B, i$ )
9  else if  $k < i$ 
10     return Select( $C, i - k$ )
```

Picking x Cleverly

Need to pick x so $rank(x)$ is not extreme.

- Arrange S into columns of size 5 ($\lceil \frac{n}{5} \rceil$ cols)
- Sort each column (bigger elements on top) (linear time)
- Find “median of medians” as x



How many elements are guaranteed to be $> x$?

Half of the $\lceil \frac{n}{5} \rceil$ groups contribute at least 3 elements $> x$ except for 1 group with less than 5 elements and 1 group that contains x .

At least $3(\lceil \frac{n}{10} \rceil - 2)$ elements are $> x$, and at least $3(\lceil \frac{n}{10} \rceil - 2)$ elements are $< x$

Recurrence:

$$T(n) = \begin{cases} O(1), & \text{for } n \leq 140 \\ T(\lceil \frac{n}{5} \rceil) + T(\frac{7n}{10} + 6), \Theta(n), & \text{for } n > 140 \end{cases} \quad (1)$$

Solving the Recurrence

Master theorem does not apply. Intuition $\frac{n}{5} + \frac{7n}{10} < n$.

Prove $T(n) \leq cn$ by induction, for some large enough c .

True for $n \leq 140$ by choosing large c

$$T(n) \leq c\lceil \frac{n}{5} \rceil + c(\frac{7n}{10} + 6) + an \quad (2)$$

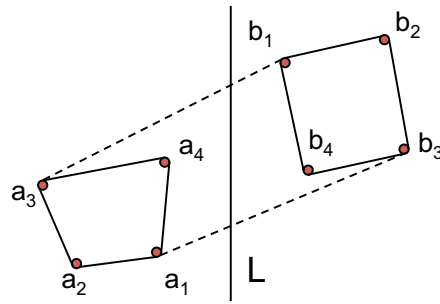
$$\leq \frac{cn}{5} + c + \frac{7nc}{10} + 6c + an \quad (3)$$

$$= cn + (-\frac{cn}{10} + 7c + an) \quad (4)$$

If $c \geq \frac{70c}{n} + 10a$, we are done. This is true for $n \geq 140$ and $c \geq 20a$.

Appendix 1

Example



a_3, b_1 is upper tangent. $a_4 > a_3, b_2 > b_1$ in terms of Y coordinates.

a_1, b_3 is lower tangent, $a_2 < a_1, b_4 < b_3$ in terms of Y coordinates.

a_i, b_j is an upper tangent. Does not mean that a_i or b_j is the highest point. Similarly, for lower tangent.

MIT OpenCourseWare
<http://ocw.mit.edu>

6.046J / 18.410J Design and Analysis of Algorithms
Spring 2015

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Lecture 3: Divide and Conquer: Fast Fourier Transform

- Polynomial Operations vs. Representations
- Divide and Conquer Algorithm
- Collapsing Samples / Roots of Unity
- FFT, IFFT, and Polynomial Multiplication

Polynomial operations and representation

A polynomial $A(x)$ can be written in the following forms:

$$\begin{aligned} A(x) &= a_0 + a_1x + a_2x^2 + \cdots + a_{n-1}x^{n-1} \\ &= \sum_{k=0}^{n-1} a_k x^k \\ &= \langle a_0, a_1, a_2, \dots, a_{n-1} \rangle \quad (\text{coefficient vector}) \end{aligned}$$

The degree of A is $n - 1$.

Operations on polynomials

There are three primary operations for polynomials.

1. **Evaluation:** Given a polynomial $A(x)$ and a number x_0 , compute $A(x_0)$. This can be done in $O(n)$ time using $O(n)$ arithmetic operations via Horner's rule.
 - **Horner's Rule:** $A(x) = a_0 + x(a_1 + x(a_2 + \cdots x(a_{n-1}) \cdots))$. At each step, a sum is evaluated, then multiplied by x , before beginning the next step. Thus $O(n)$ multiplications and $O(n)$ additions are required.
2. **Addition:** Given two polynomials $A(x)$ and $B(x)$, compute $C(x) = A(x) + B(x)$ ($\forall x$). This takes $O(n)$ time using basic arithmetic, because $c_k = a_k + b_k$.

3. **Multiplication:** Given two polynomials $A(x)$ and $B(x)$, compute $C(x) = A(x) \cdot B(x)$ ($\forall x$). Then $c_k = \sum_{j=0}^k a_j b_{k-j}$ for $0 \leq k \leq 2(n-1)$, because the degree of the resulting polynomial is twice that of A or B . This multiplication is then equivalent to a convolution of the vectors A and $\text{reverse}(B)$. The *convolution* is the inner product of all relative shifts, an operation also useful for smoothing etc. in digital signal processing.

- Naive polynomial multiplication takes $O(n^2)$.
- $O(n^{\lg 3})$ or even $O(n^{1+\varepsilon})$ ($\forall \varepsilon > 0$) is possible via Strassen-like divide-and-conquer tricks.
- Today, we will compute the product in $O(n \lg n)$ time via Fast Fourier Transform!

Representations of polynomials

First, consider the different representations of polynomials, and the time necessary to complete operations based on the representation.

There are 3 main representations to consider.

1. Coefficient vector with a monomial basis
2. Roots and a scale term
 - $A(x) = (x - r_0) \cdot (x - r_1) \cdot \dots \cdot (x - r_{n-1}) \cdot c$
 - However, it is impossible to find exact roots with only basic arithmetic operations and k th root operations. Furthermore, addition is extremely hard with this representation, or even impossible. Multiplication simply requires roots to be concatenated, and evaluation can be completed in $O(n)$.
3. Samples: $(x_0, y_0), (x_1, y_1), \dots, (x_{n-1}, y_{n-1})$ with $A(x_i) = y_i$ ($\forall i$) and each x_i is distinct. These samples uniquely determine a degree $n - 1$ polynomial A , according to the Lagrange and Fundamental Theorem of Algebra. Addition and multiplication can be computed by adding and multiplying the y_i terms, assuming that the x_i 's match. However, evaluation requires interpolation.

The runtimes for the representations and the operations is described in the table below, with algorithms for the operations versus the representations.

Algorithms vs.	Representations		
	Coefficients	Roots	Samples
Evaluation	$O(n)$	$O(n)$	$O(n^2)$
Addition	$O(n)$	∞	$O(n)$
Multiplication	$O(n^2)$	$O(n)$	$O(n)$

We combine the best of each representation by converting between coefficients and samples in $O(n \lg n)$ time.

How? Consider the polynomial in matrix form.

$$V \cdot A = \begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{n-1} \\ 1 & x_1 & x_1^2 & \cdots & x_1^{n-1} \\ 1 & x_2 & x_2^2 & \cdots & x_2^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \cdots & x_{n-1}^{n-1} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_{n-1} \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ \vdots \\ y_{n-1} \end{bmatrix}$$

where V is the Vandermonde matrix with entries $v_{jk} = x_j^k$.

Then we can convert between coefficients and samples using the matrix vector product $V \cdot A$, which is equivalent to evaluation. This takes $O(n^2)$.

Similarly, we can samples to coefficients by solving $V \setminus Y$ (in **MATLAB**[®] notation). This takes $O(n^3)$ via Gaussian elimination, or $O(n^2)$ to compute $A = V^{-1} \cdot Y$, if V^{-1} is precomputed.

To do better than $\Theta(n^2)$ when converting between coefficients and samples, and vice versa, we need to choose special values for $x_0, x_1, \dots, x_{n-1}$. Thus far, we have only made the assumption that the x_i values are distinct.

Divide and Conquer Algorithm

We can formulate polynomial multiplication as a divide and conquer algorithm with the following steps for a polynomial $A(x) \forall x \in X$.

1. Divide the polynomial A into its even and odd coefficients:

$$A_{\text{even}}(x) = \sum_{k=0}^{\lceil \frac{n}{2} - 1 \rceil} a_{2k} x^k = \langle a_0, a_2, a_4, \dots \rangle$$

$$A_{\text{odd}}(x) = \sum_{k=0}^{\lfloor \frac{n}{2} - 1 \rfloor} a_{2k+1} x^k = \langle a_1, a_3, a_5, \dots \rangle$$

2. Recursively conquer $A_{\text{even}}(y)$ for $y \in X^2$ and $A_{\text{odd}}(y)$ for $y \in X^2$, where $X^2 = \{x^2 \mid x \in X\}$.
3. Combine the terms. $A(x) = A_{\text{even}}(x^2) + x \cdot A_{\text{odd}}(x^2)$ for $x \in X$.

However, the recurrences for this algorithm is

$$\begin{aligned} T(n, |X|) &= 2 \cdot T\left(\frac{n}{2}, |X|\right) + O(n + |X|) \\ &= O(n^2) \end{aligned}$$

which is no better than before.

We can do better if X is *collapsing*: either $|X| = 1$ (base case), or $|X^2| = \frac{|X|}{2}$ and X^2 is (recursively) collapsing. Then the recurrence is of the form

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + O(n) = O(n \lg n).$$

Roots of Unity

Collapsing sets can be constructed via square roots. Each of the following collapsing sets is computing by taking all square roots of the previous set.

1. $\{1\}$
2. $\{1, -1\}$
3. $\{1, -1, i, -i\}$
4. $\{1, -1, \pm \frac{\sqrt{2}}{2}(1 + i), \pm \frac{\sqrt{2}}{2}(-1 + i)\}$, which lie on a unit circle

We can repeat this process and make our set larger and larger by finding more and more points on this circle. These points are called the n th roots of unity. Formally, the n th roots of unity are n x 's such that $x^n = 1$. These points are uniformly spaced around the unit circle in the complex plane (including 1). These points are of the form $(\cos \theta, \sin \theta) = \cos \theta + i \sin \theta = e^{i\theta}$ by Euler's Formula, for $\theta = 0, \frac{1}{n}\tau, \frac{2}{n}\tau, \dots, \frac{n-1}{n}\tau$ (where $\tau = 2\pi$).

The n th roots of unity where $n = 2^\ell$ form a collapsing set, because $(e^{i\theta})^2 = e^{i(2\theta)} = e^{i(2\theta \bmod \tau)}$. Therefore the even n th roots of unity are equivalent to the $\frac{n}{2}$ nd roots of unity.

FFT, IFFT, and Polynomial Multiplication

We can take advantage of the n th roots of unity to improve the runtime of our polynomial multiplication algorithm. The basis for the algorithm is called the Discrete Fourier Transform (DFT).

The DFT allows the transformation between coefficients and samples, computing $A \rightarrow A^* = V \cdot A$ for $x_k = e^{i\tau k/n}$ where $n = 2^\ell$, where A is the set of coefficients and A^* is the resulting samples. The individual terms $a_j^* = \sum_{k=0}^{n-1} e^{i\tau jk/n} \cdot a_j$.

Fast Fourier Transform (FFT)

The FFT algorithm is an $O(n \lg n)$ divide and conquer algorithm for DFT, used by Gauss circa 1805, and popularized by Cooley and Turkey in 1965. Gauss used the algorithm to determine periodic asteroid orbits, while Cooley and Turkey used it to detect Soviet nuclear tests from offshore readings.

A practical implementation of FFT is FFTW, which was described by Frigo and Johnson at MIT. The algorithm is often implemented directly in hardware, for fixed n .

Inverse Discrete Fourier Transform

The Inverse Discrete Fourier Transform is an algorithm to return the coefficients of a polynomial from the multiplied samples. The transformation is of the form $A^* \rightarrow V^{-1} \cdot A^* = A$.

In order to compute this, we need to find V^{-1} , which in fact has a very nice structure.

Claim 1. $V^{-1} = \frac{1}{n} \bar{V}$, where $\bar{V}$ is the complex conjugate of V .¹

¹Recall the complex conjugate of $p + qi$ is $p - qi$.

Proof. We claim that $P = V \cdot \bar{V} = nI$:

$$\begin{aligned}
 p_{jk} &= (\text{row } j \text{ of } V) \cdot (\text{col. } k \text{ of } \bar{V}) \\
 &= \sum_{m=0}^{n-1} e^{ij\tau m/n} \overline{e^{ik\tau m/n}} \\
 &= \sum_{m=0}^{n-1} e^{ij\tau m/n} e^{-ik\tau m/n} \\
 &= \sum_{m=0}^{n-1} e^{i(j-k)\tau m/n}
 \end{aligned}$$

Now if $j = k$, $p_{jk} = \sum_{m=0}^{n-1} 1 = n$. Otherwise it forms a geometric series.

$$\begin{aligned}
 p_{jk} &= \sum_{m=0}^{n-1} (e^{i(j-k)\tau/n})^m \\
 &= \frac{(e^{i\tau(j-k)/n})^n - 1}{e^{i\tau(j-k)/n} - 1} \\
 &= 0
 \end{aligned}$$

because $e^{i\tau} = 1$. Thus $V^{-1} = \frac{1}{n}\bar{V}$, because $V \cdot \bar{V} = nI$. □

This claim says that the Inverse Discrete Fourier Transform is equivalent to the Discrete Fourier Transform, but changing x_k from $e^{ik\tau/n}$ to its complex conjugate $e^{-ik\tau/n}$, and dividing the resulting vector by n . The algorithm for IFFT is analogous to that for FFT, and the result is an $O(n \lg n)$ algorithm for IDFT.

Fast Polynomial Multiplication

In order to compute the product of two polynomials A and B , we can perform the following steps.

1. Compute $A^* = FFT(A)$ and $B^* = FFT(B)$, which converts both A and B from coefficient vectors to a sample representation.
2. Compute $C^* = A^* \cdot B^*$ in sample representation in linear time by calculating $C_k^* = A_k^* \cdot B_k^*$ ($\forall k$).
3. Compute $C = IFFT(C^*)$, which is a vector representation of our final solution.

Applications

Fourier (frequency) space many applications. The polynomial $A^* = FFT(A)$ is complex, and the amplitude $|a_k^*|$ represents the amplitude of the frequency- k signal, while $\arg(a_k^*)$ (the angle of the 2D vector) represents the phase shift of that signal. For example, this perspective is particularly useful for audio processing, as used by Adobe Audition, Audacity, etc.:

- High-pass filters zero out high frequencies
- Low-pass filters zero out low frequencies
- Pitch shifts shift the frequency vector
- Used in MP3 compression, etc.

MIT OpenCourseWare
<http://ocw.mit.edu>

6.046J / 18.410J Design and Analysis of Algorithms
Spring 2015

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Lecture 4: Divide and Conquer: van Emde Boas Trees

- Series of Improved Data Structures
- Insert, Successor
- Delete
- Space

This lecture is based on personal communication with Michael Bender, 2001.

Goal

We want to maintain n elements in the range $\{0, 1, 2, \dots, u - 1\}$ and perform Insert, Delete and Successor operations in $\mathcal{O}(\log \log u)$ time.

- If $n = n^{\mathcal{O}(1)}$ or $n^{(\log n)^{\mathcal{O}(1)}}$, then we have $\mathcal{O}(\log \log n)$ time operations
 - Exponentially faster than Balanced Binary Search Trees
 - Cooler queries than hashing
- Application: Network Routing Tables
 - u = Range of IP Addresses $\rightarrow$ port to send ($u = 2^{32}$ in IPv4)

Where might the $\mathcal{O}(\log \log u)$ bound arise ?

- Binary search over $\mathcal{O}(\log u)$ elements
- Recurrences
 - $T(\log u) = T\left(\frac{\log u}{2}\right) + \mathcal{O}(1)$
 - $T(u) = T(\sqrt{u}) + \mathcal{O}(1)$

Improvements

We will develop the van Emde Boas data structure by a series of improvements on a very simple data structure.

Bit Vector

We maintain a vector V of size u such that $V[x] = 1$ if and only if x is in the set. Now, inserts and deletes can be performed by just flipping the corresponding bit in the vector. However, successor/predecessor requires us to traverse through the vector to find the next 1-bit.

- Insert/Delete: $\mathcal{O}(1)$
- Successor/Predecessor: $\mathcal{O}(u)$

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	1	0	0	0	0	0	0	0	1	1	0	0	0	0	1

Figure 1: Bit vector for $u = 16$. The current set is $\{1, 9, 10, 15\}$.

Split Universe into Clusters

We can improve performance by splitting up the range $\{0, 1, 2, \dots, u-1\}$ into $\sqrt{u}$ clusters of size $\sqrt{u}$. If $x = i\sqrt{u} + j$, then $V[x] = V.Cluster[i][j]$.

$$low(x) = x \bmod \sqrt{u} = j$$

$$high(x) = \left\lfloor \frac{x}{\sqrt{u}} \right\rfloor = i$$

$$index(i, j) = i\sqrt{u} + j$$

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	1	0	0	0	0	0	0	0	1	1	0	0	0	0	1
$V.Cluster[0]$				$V.Cluster[1]$				$V.Cluster[2]$				$V.Cluster[3]$			

Figure 2: Bit vector ($u = 16$) split into $\sqrt{16} = 4$ clusters of size 4.

- Insert:
 - Set $V.cluster[high(x)][low(x)] = 1$ $\mathcal{O}(1)$

- Mark cluster $high(x)$ as non-empty $\mathcal{O}(1)$
- Successor:
 - Look within cluster $high(x)$ $\mathcal{O}(\sqrt{u})$
 - Else, find next non-empty cluster i $\mathcal{O}(\sqrt{u})$
 - Find minimum entry j in that cluster $\mathcal{O}(\sqrt{u})$
 - Return $index(i, j)$ Total = $\mathcal{O}(\sqrt{u})$

Recurse

The three operations in Successor are also Successor calls to vectors of size $\sqrt{u}$. We can use recursion to speed things up.

- $V.cluster[i]$ is a size- $\sqrt{u}$ van Emde Boas structure ($\forall 0 \leq i < \sqrt{u}$)
- $V.summary$ is a size- $\sqrt{u}$ van Emde Boas structure
- $V.summary[i]$ indicates whether $V.cluster[i]$ is nonempty

INSERT(V, x)

- 1 $Insert(V.cluster[high(x)], low[x])$
- 2 $Insert(V.summary, high[x])$

So, we get the recurrence:

$$\begin{aligned}
 T(u) &= 2T(\sqrt{u}) + \mathcal{O}(1) \\
 T'(\log u) &= 2T'\left(\frac{\log u}{2}\right) + \mathcal{O}(1) \\
 \implies T(u) &= T'(\log u) = \mathcal{O}(\log u)
 \end{aligned}$$

SUCCESSOR(V, x)

- 1 $i = high(x)$
- 2 $j = Successor(V.cluster[i], j)$
- 3 **if** $j == \infty$
- 4 $i = Successor(V.summary, i)$
- 5 $j = Successor(V.cluster[i], -\infty)$
- 6 **return** $index(i, j)$

$$\begin{aligned}
T(u) &= 3T(\sqrt{u}) + \mathcal{O}(1) \\
T'(\log u) &= 3T'\left(\frac{\log u}{2}\right) + \mathcal{O}(1) \\
\implies T(u) &= T'(\log u) = \mathcal{O}((\log u)^{\log 3}) \approx \mathcal{O}((\log u)^{1.585})
\end{aligned}$$

To obtain the $\mathcal{O}(\log \log u)$ running time, we need to reduce the number of recursions to one.

Maintain Min and Max

We store the minimum and maximum entry in each structure. This gives an $\mathcal{O}(1)$ time overhead for each *Insert* operation.

SUCCESSOR(V, x)

```

1   $i = \text{high}(x)$ 
2  if  $\text{low}(x) < V.\text{cluster}[i].\text{max}$ 
3       $j = \text{Successor}(V.\text{cluster}[i], \text{low}(x))$ 
4  else  $i = \text{Successor}(V.\text{summary}, \text{high}(x))$ 
5       $j = V.\text{cluster}[i].\text{min}$ 
6  return  $\text{index}(i, j)$ 

```

$$\begin{aligned}
T(u) &= T(\sqrt{u}) + \mathcal{O}(1) \\
\implies T(u) &= \mathcal{O}(\log \log u)
\end{aligned}$$

Don't store Min recursively

The *Successor* call now needs to check for the min separately.

if $x < V.\text{min}$: **return** $V.\text{min}$ (1)

INSERT(V, x)

```

1  if  $V.min == None$ 
2       $V.min = V.max = x$       ▷  $\mathcal{O}(1)$  time
3      return
4  if  $x < V.min$ 
5       $swap(x \leftrightarrow V.min)$ 
6  if  $x > V.max$ 
7       $V.max = x$ 
8  if  $V.cluster[high(x)] == None$ 
9       $Insert(V.summary, high(x))$       ▷ First Call
10  $Insert(V.cluster[high(x)], low(x))$     ▷ Second Call
```

If the **first call** is executed, the **second call** only takes $\mathcal{O}(1)$ time. So

$$T(u) = T(\sqrt{u}) + \mathcal{O}(1)$$

$$\implies T(u) = \mathcal{O}(\log \log u)$$

DELETE(V, x)

```

1  if  $x == V.min$       ▷ Find new min
2       $i = V.summary.min$ 
3      if  $i = None$ 
4           $V.min = V.max = None$       ▷  $\mathcal{O}(1)$  time
5          return
6       $V.min = index(i, V.cluster[i].min)$       ▷ Unstore new min
7       $Delete(V.cluster[high(x)], low(x))$       ▷ First Call
8  if  $V.cluster[high(x)].min == None$ 
9       $Delete(V.summary, high(x))$       ▷ Second Call
10 ▷ Now we update  $V.max$ 
11 if  $x == V.max$ 
12 if  $V.summary.max = None$ 
13 else
14      $i = V.summary.max$ 
15      $V.max = index(i, V.cluster[i].max)$ 
```

If the **second call** is executed, the **first call** only takes $\mathcal{O}(1)$ time. So

$$T(u) = T(\sqrt{u}) + \mathcal{O}(1)$$

$$\implies T(u) = \mathcal{O}(\log \log u)$$

Lower Bound [Patrascu & Thorup 2007]

Even for static queries (no Insert/Delete)

- $\Omega(\log \log u)$ time per query for $u = n^{(\log n)^{\mathcal{O}(1)}}$
- $\mathcal{O}(n \cdot \text{poly}(\log n))$ space

Space Improvements

We can improve from $\Theta(u)$ to $\mathcal{O}(n \log \log u)$.

- Only create nonempty clusters
 - If $V.min$ becomes *None*, deallocate V
- Store $V.cluster$ as a hashtable of nonempty clusters
- Each insert may create a new structure $\Theta(\log \log u)$ times (each empty insert)
 - Can actually happen [Vladimir Čunát]
- Charge pointer to structure (and associated hash table entry) to the structure

This gives us $\mathcal{O}(n \log \log u)$ space (but randomized).

Indirection

We can further reduce to $\mathcal{O}(n)$ space.

- Store vEB structure with $n = \mathcal{O}(\log \log u)$ using BST or even an array
 - $\implies \mathcal{O}(\log \log n)$ time once in base case
- We use $\mathcal{O}(n / \log \log u)$ such structures (disjoint)
 - $\implies \mathcal{O}(\frac{n}{\log \log u} \cdot \log \log u) = \mathcal{O}(n)$ space for small
- Larger structures “store” pointers to them
 - $\implies \mathcal{O}(\frac{n}{\log \log u} \cdot \log \log u) = \mathcal{O}(n)$ space for large
- Details: Split/Merge small structures

MIT OpenCourseWare
<http://ocw.mit.edu>

6.046J / 18.410J Design and Analysis of Algorithms
Spring 2015

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Lecture 5: Amortization

Amortized analysis is a powerful technique for data structure analysis, involving the total runtime of a sequence of operations, which is often what we really care about. This lecture covers:

- Different techniques of amortized analysis
 - aggregate method
 - accounting method
 - charging method
 - potential method
- Examples of amortized analysis
 - table doubling
 - binary counter
 - 2-3 tree and 2-5 tree

Table doubling

(Recall from 6.006) We want to store n elements in a table of size $m = \Theta(n)$. One idea is to double m whenever n becomes larger than m (due to insertions). The cost to double a table of size m is clearly $\Theta(m) = \Theta(n)$, which is also the worst case cost of an insertion.

But what is the total cost of n insertions? It is at most

$$2^0 + 2^1 + 2^2 + \dots + 2^{\lceil \lg n \rceil} = \Theta(n).$$

In this case, we say each insertion has $\Theta(n)/n = \Theta(1)$ *amortized cost*.

Aggregate Method

The method we used in the above analysis is the aggregate method: just add up the cost of all the operations and then divide by the number of operations.

$$\text{amortized cost per operation} = \frac{\text{total cost of } k \text{ operations}}{k}$$

Aggregate method is the simplest method. Because it's simple, it may not be able to analyze more complicated algorithms.

Amortized Bound Definition

Amortized cost can be, but does not have to be, average cost. We can assign any amortized cost to each operation, as long as they “preserve the total cost”, i.e., for any sequence of operations,

$$\sum \text{amortized cost} \geq \sum \text{actual cost}$$

where the sum is taken over all operations.

For example, we can say a 2-3 tree achieves $O(1)$ amortized cost per create, $O(\lg n^*)$ amortized cost per insert, and 0 amortized cost per delete, where n^* is the maximum size of the 2-3 tree during the entire sequence of operations. The reason we can claim this is that for any sequence of operations, suppose there are c creations, i insertions and $d \leq i$ deletions (cannot delete from an empty tree), the total amortized cost is asymptotically the same as the total actual cost:

$$O(c + i \lg n^* + 0d) = O(c + i \lg n^* + d \lg n^*)$$

Later, we will tighten the amortized cost per insert to $O(\lg n)$ where n is the *current* size.

Accounting Method

This method allows an operation to store credit into a bank for future use, if its assigned amortized cost $>$ its actual cost; it also allows an operation to pay for its extra actual cost using existing credit, if its assigned amortized cost $<$ its actual cost.

Table doubling

For example, in table doubling:

- if an insertion does not trigger table doubling, store a coin representing $c = O(1)$ work for future use.
- if an insertion does trigger table doubling, there must be $n/2$ elements that are inserted after the previous table doubling, whose coins have not been consumed. Use up these $n/2$ coins to pay for the $O(n)$ table doubling. See figure below.
- amortized cost for table doubling: $O(n) - c \cdot n/2 = 0$ for large enough c .
- amortized cost per insertion: $1 + c = O(1)$.

 an element
  a unused coin



table doubling due to the next insert



2-3 trees

Now let's try the accounting method on 2-3 trees. Our goal is to show that insert has $O(\lg n)$ amortized cost and delete has 0 amortized cost. Let's try a natural approach: save a $O(\lg n)$ coin for inserting an element, and use this coin when we delete this element later. However, we will run into a problem: by the time we delete the element, the size of the tree may have got bigger $n' > n$, and the coin we saved is not enough to pay for the $\lg n'$ actual cost of that delete operation! This problem can be solved using the charging method in the next section.

Charging Method

The charging method allows operations to charge cost retroactively to *past* operations.

$$\begin{aligned}
 \text{amortized cost of an operation} &= \text{actual cost of this operation} \\
 &\quad - \text{total cost charged to past operations} \\
 &\quad + \text{total cost charged by future operations}
 \end{aligned}$$

Table doubling and halving

For example, in table doubling, when the table doubles from m to $2m$, we can charge $\Theta(m)$ cost to the $m/2$ insert operations since the last doubling. Each insert is charged by $\Theta(1)$, and will not be charged again. So the amortized cost per insert is $\Theta(1)$.

Now let's extend the above example with table halving. The motivation is to save space when with deletes. If the table is down to $1/4$ full, $n = m/4$, we shrink the table size from m to $m/2$ at $\Theta(m)$ cost. This way, the table is half full again after any resize (doubling or shrinking). Now each table doubling still has $\geq m/2$ insert operations to charge to, and each table halving has $\geq m/4$ delete operations to charge to. So the amortized cost per insert or delete is still $\Theta(1)$.

Free deletion in 2-3 trees

For another example, let's consider insertion and deletion in 2-3 trees. Again, our goal is to show that insert has $O(\lg n)$ amortized cost, where n is the size of the tree when that insert happens, and delete has 0 amortized cost.

Insert does not need to charge anything.

Delete will charge an insert operation. But we will not charge the insert of the element to be deleted, because we will run into the same problem as the accounting method. Instead, each delete operation will charge the insert operation that brought the tree to its current size n . Each insert is still charged at most once, because for the tree size to reach n again, another insert must happen.

Potential Method

This method defines a potential function Φ that maps a data structure (DS) configuration to a value. This function Φ is equivalent to the total unused credits stored up by all past operations (the bank account balance). Now

$$\text{amortized cost of an operation} = \text{actual cost of this operation} + \Delta\Phi$$

and

$$\sum \text{amortized cost} = \sum \text{actual cost} + \Phi(\text{final DS}) - \Phi(\text{initial DS}).$$

In order for the amortized bound to hold, Φ should never go below $\Phi(\text{initial DS})$ at any point. If $\Phi(\text{initial DS}) = 0$, which is usually the case, then Φ should never go negative (intuitively, we cannot "owe the bank").

Relation to accounting method

In accounting method, we specify $\Delta\Phi$, while in potential method, we specify Φ . One determines the other, so the two methods are equivalent. But sometimes one is more intuitive than the other.

Binary counter

Our first example of potential method is incrementing a binary counter. E.g.,

```

0011010111
increment   ↓
0011011000
  
```


Cost of increment is $\Theta(1 + \#1)$, where $\#1$ represents the number of trailing 1 bits. So the intuition is that 1 bits are bad.

Define $\Phi = c \cdot \#1$. Then for large enough c ,

$$\begin{aligned}\text{amortized cost} &= \text{actual cost} + \Delta\Phi \\ &= \Theta(1 + \#1) + c(-\#1 + 1) \\ &= \Theta(1)\end{aligned}$$

$\Phi(\text{initial DS}) = 0$ if the counter starts at $000 \dots 0$. This is necessary for the above amortized analysis. Otherwise, Φ may become smaller than $\Phi(\text{initial DS})$.

Insert in 2-3 trees

Insert can cause $O(\lg n)$ splits in the worst case, but we can show it causes only $O(1)$ amortized splits. First consider what causes a split: insertion into a 3-node (a node with 3 children). In that case, the 3-node needs to split into two 2-nodes.

So 3-nodes are bad. We define $\Phi =$ the number of 3-nodes. Then $\Delta\Phi \leq 1 -$ the number of splits. Amortized number of splits = actual number of splits + $\Delta\Phi = 1$. $\Phi(\text{initial DS}) = 0$ if the tree is empty initially.

The above analysis holds for any (a, b) -tree, if we define Φ to be the number of b -nodes.

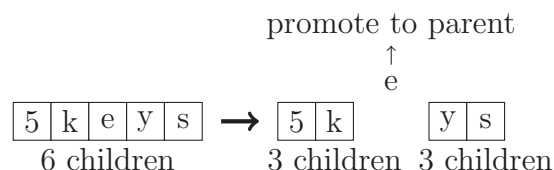
If we consider both insertion and deletion in 2-3 trees, can we claim both $O(1)$ splits for insert, and $O(1)$ merges for delete? The answer is no, because a split creates two 2-nodes, which are bad for merge. In the worse case, they may be merged by the next delete, and then need split again on the next insert, and so on.

What do we solve this problem? We need to prevent split and merge from creating ‘bad’ nodes.

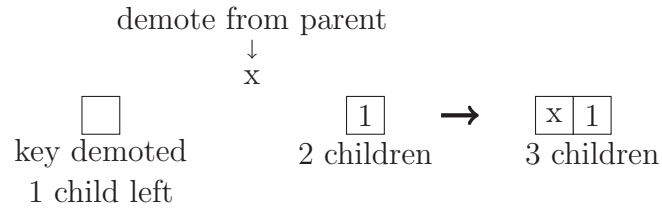
Insert and delete in 2-5 trees

We can claim $O(1)$ splits for insert, and $O(1)$ merges for delete in 2-5 trees.

In 2-5 trees, insertion into a 5-node (a node with 5 children) causes it to split into two 3-nodes.



Deletion from a 2-node causes it to merge with another 2-node to form a 3-node.



5-nodes and 2-nodes are bad. We define $\Phi = \# \text{ of 5-nodes} + \# \text{ of 2-nodes}$. Amortized splits and merges = 1. $\Phi(\text{initial DS}) = 0$ if the tree is empty initially.

The above analysis holds for any (a, b) -tree where $b > 2a$, because splits and merges do not produce bad nodes. We define Φ to be the number of b -nodes plus the number of a nodes.

Note: The potential examples could also be done with the accounting method by placing coins on 1s (binary counter) or 2/5-nodes $((2, 5)$ -trees).

MIT OpenCourseWare
<http://ocw.mit.edu>

6.046J / 18.410J Design and Analysis of Algorithms
Spring 2015

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Lecture 6: Randomized Algorithms

- Check matrix multiplication
- Quicksort

Randomized or Probabilistic Algorithms

What is a randomized algorithm?

- Algorithm that generates a random number $r \in \{1, \dots, R\}$ and makes decisions based on r 's value.
- On the same input on different executions, a randomized algorithm may
 - Run a different number of steps
 - Produce a different output

Randomized algorithms can be broadly classified into two types- Monte Carlo and Las Vegas.

<u>Monte Carlo</u>	<u>Las Vegas</u>
runs in polynomial time always	runs in expected polynomial time
output is correct with high probability	output always correct

Matrix Product

$$C = A \times B$$

Simple algorithm: $O(n^3)$ multiplications.

Strassen: multiply two 2×2 matrices in 7 multiplications: $O(n^{\log_2 7}) = O(n^{2.81})$

Coppersmith-Winograd: $O(n^{2.376})$

Matrix Product Checker

Given $n \times n$ matrices A, B, C , the goal is to check if $A \times B = C$ or not.

Question. Can we do better than carrying out the full multiplication?

We will see an $O(n^2)$ algorithm that:

- if $A \times B = C$, then $Pr[\text{output}=\text{YES}] = 1$.
- if $A \times B \neq C$, then $Pr[\text{output}=\text{YES}] \leq \frac{1}{2}$.

We will assume entries in matrices $\in \{0, 1\}$ and also that the arithmetic is mod 2.

Frievald's Algorithm

Choose a random binary vector $r[1...n]$ such that $Pr[r_i = 1] = 1/2$ independently for $r = 1, \dots, n$. The algorithm will output 'YES' if $A(Br) = Cr$ and 'NO' otherwise.

Observation

The algorithm will take $O(n^2)$ time, since there are 3 matrix multiplications Br , $A(Br)$ and Cr of a $n \times n$ matrix by a $n \times 1$ matrix.

Analysis of Correctness if $AB \neq C$

Claim. If $AB \neq C$, then $Pr[ABr \neq Cr] \geq 1/2$.

Let $D = AB - C$. Our hypothesis is thus that $D \neq 0$. Clearly, there exists r such that $Dr \neq 0$. Our goal is to show that there are **many** r such that $Dr \neq 0$. Specifically, $Pr[Dr \neq 0] \geq 1/2$ for randomly chosen r .

$D = AB - C \neq 0 \implies \exists i, j$ s.t. $d_{ij} \neq 0$. Fix vector v which is 0 in all coordinates except for $v_j = 1$. $(Dv)_i = d_{ij} \neq 0$ implying $Dv \neq 0$. Take any r that can be chosen by our algorithm. We are looking at the case where $Dr = 0$. Let

$$r' = r + v$$

Since v is 0 everywhere except v_j , r' is the same as r except $r'_j = (r_j + v_j) \bmod 2$. Thus, $Dr' = D(r + v) = 0 + Dv \neq 0$. We see that there is a 1 to 1 correspondence between r and r' , as if $r' = r + V = r'' + V$ then $r = r''$. This implies that

$$\text{number of } r' \text{ for which } Dr' \neq 0 \geq \text{number of } r \text{ for which } Dr = 0$$

From this we conclude that $Pr[Dr \neq 0] \geq 1/2$

Quicksort

Divide and conquer algorithm but work mostly in the divide step rather than combine. Sorts “in place” like insertion sort and unlike mergesort (which requires $O(n)$ auxiliary space).

Different variants:

- Basic: good in average case
- Median-based pivoting: uses median finding
- Random: good for all inputs in expectation (Las Vegas algorithm)

Steps of quicksort:

- Divide: pick a pivot element x in A , partition the array into sub-arrays L , consisting of all elements $< x$, G consisting of all elements $> x$ and E consisting of all elements $= x$.
- Conquer: recursively sort subarrays L and G
- Combine: trivial

Basic Quicksort

Pivot around $x = A[1]$ or $A[n]$ (first or last element)

- Remove, in turn, each element y from A
- Insert y into L , E or G depending on the comparison with pivot x
- Each insertion and removal takes $O(1)$ time
- Partition step takes $O(n)$ time
- To do this in place: see CLRS p. 171

Basic Quicksort Analysis

If input is sorted or reverse sorted, we are partitioning around the min or max element each time. This means one of L or G has $n - 1$ elements, and the other 0. This gives:

$$\begin{aligned} T(n) &= T(0) + T(n - 1) + \Theta(n) \\ &= \Theta(1) + T(n - 1) + \Theta(n) \\ &= \Theta(n^2) \end{aligned}$$

However, this algorithm does well on random inputs in practice.

Pivot Selection Using Median Finding

Can **guarantee** balanced L and G using rank/median selection algorithm that runs in $\Theta(n)$ time. The first $\Theta(n)$ below is for the pivot selection and the second for the partition step.

$$\begin{aligned} T(n) &= 2T\left(\frac{n}{2}\right) + \Theta(n) + \Theta(n) \\ T(n) &= \Theta(n \log n) \end{aligned}$$

This algorithm is slow in practice and loses to mergesort.

Randomized Quicksort

x is chosen at random from array A (at each recursion, a random choice is made). Expected time is $O(n \log n)$ for all input arrays A . See CLRS p.181-184 for the analysis of this algorithm; we will analyze a variant of this.

“Paranoid” Quicksort

Repeat

choose pivot to be random element of A

perform Partition

Until

resulting partition is such that

$$|L| \leq \frac{3}{4}|A| \text{ and } |G| \leq \frac{3}{4}|A|$$

Recurse on L and G

“Paranoid” Quicksort Analysis

Let's define a "good pivot" and a "bad pivot" -

Good pivot: sizes of L and $G \leq \frac{3}{4}n$ each

Bad pivot: one of L and G is $\leq \frac{3}{4}n$ each

bad pivots

good pivots

bad pivots

$\frac{n}{4}$	$\frac{n}{2}$	$\frac{n}{4}$
---------------	---------------	---------------

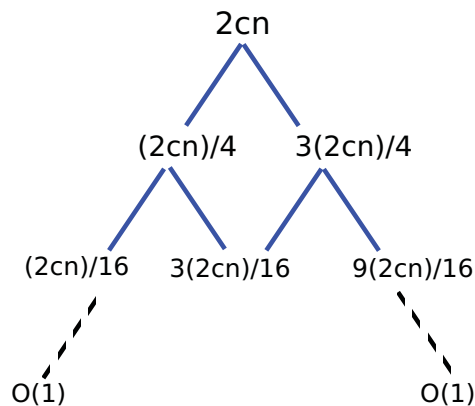
We see that a pivot is good with probability $> 1/2$.

Let $T(n)$ be an upper bound on the expected running time on any array of n size.

$T(n)$ comprises:

- time needed to sort left subarray
- time needed to sort right subarray
- the number of iterations to get a good call. Denote as $c \cdot n$ the cost of the partition step

Expectations



$$T(n) \leq \max_{n/4 \leq i \leq 3n/4} (T(i) + T(n-i)) + E(\#iterations) \cdot cn$$

Now, since probability of good pivot $> \frac{1}{2}$,

$$E(\#iterations) \leq 2$$

$$T(n) \leq T\left(\frac{n}{4}\right) + T\left(\frac{3n}{4}\right) + 2cn$$

We see in the figure that the height of the tree can be at most $\log_{\frac{4}{3}}(2cn)$ no matter what branch we follow to the bottom. At each level, we do a total of $2cn$ work. Thus, expected runtime is $T(n) = \Theta(n \log n)$

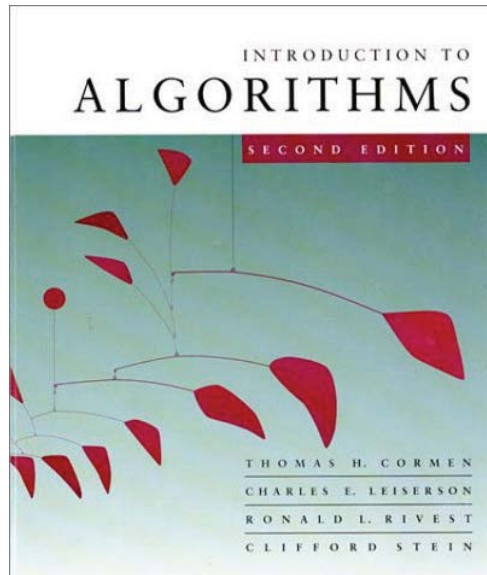
MIT OpenCourseWare
<http://ocw.mit.edu>

6.046J / 18.410J Design and Analysis of Algorithms
Spring 2015

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Design and Analysis of Algorithms

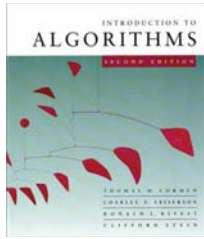
6.046J/18.401J



LECTURE 7

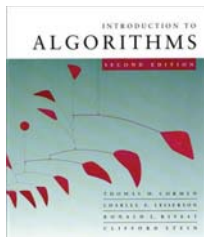
Skip Lists

- Data structure
- Randomized insertion
- With high probability (w.h.p.) bound



Skip lists

- Simple randomized dynamic search structure
 - Invented by William Pugh in 1989
 - Easy to implement
- Maintains a dynamic set of n elements in $O(\lg n)$ time per operation in expectation and *with high probability*
 - Strong guarantee on tail of distribution of $T(n)$
 - $O(\lg n)$ “almost always”

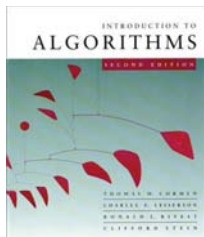


One linked list

Start from simplest data structure:
(sorted) linked list

- Searches take $\Theta(n)$ time in worst case
- How can we speed up searches?

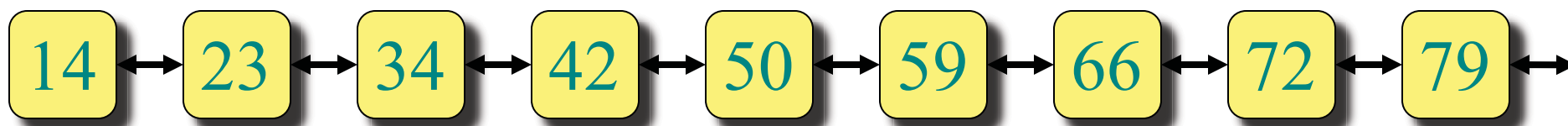


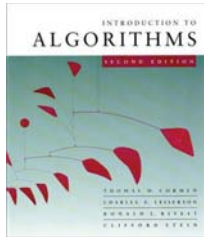


Two linked lists

Suppose we had *two* sorted linked lists
(on subsets of the elements)

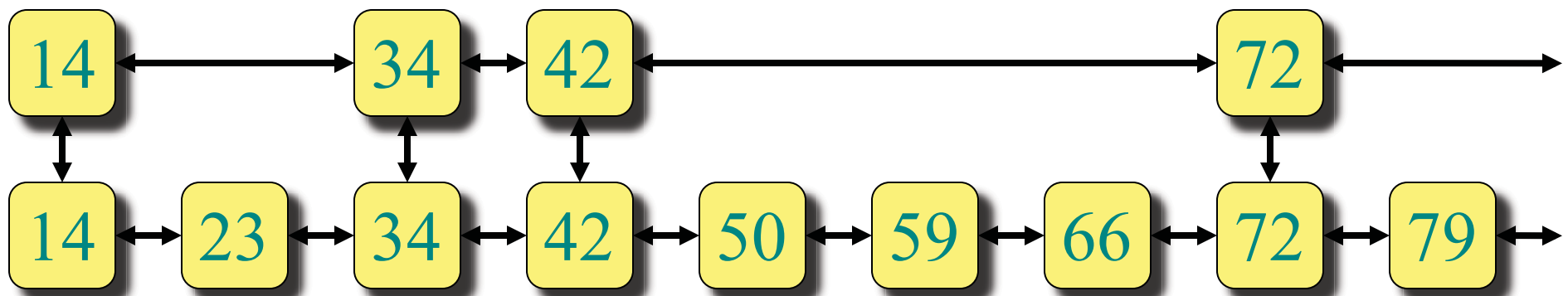
- Each element can appear in one or both lists
- How can we speed up searches?

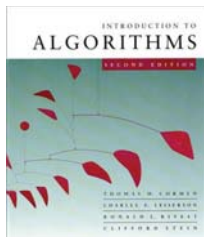




Two linked lists as a subway

- IDEA:** Express and local subway lines
(à la New York City 7th Avenue Line)
- Express line connects a few of the stations
 - Local line connects all stations
 - Links between lines at common stations

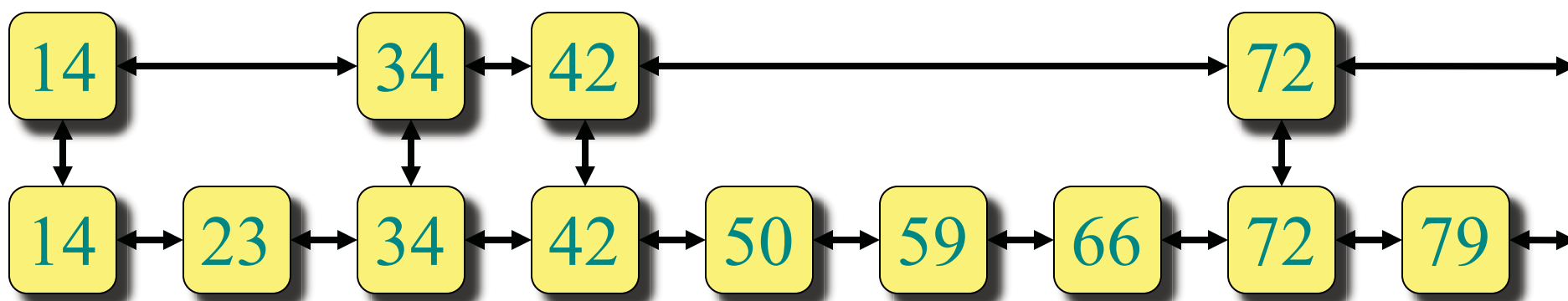


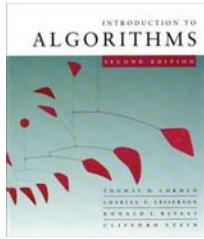


Searching in two linked lists

SEARCH(x):

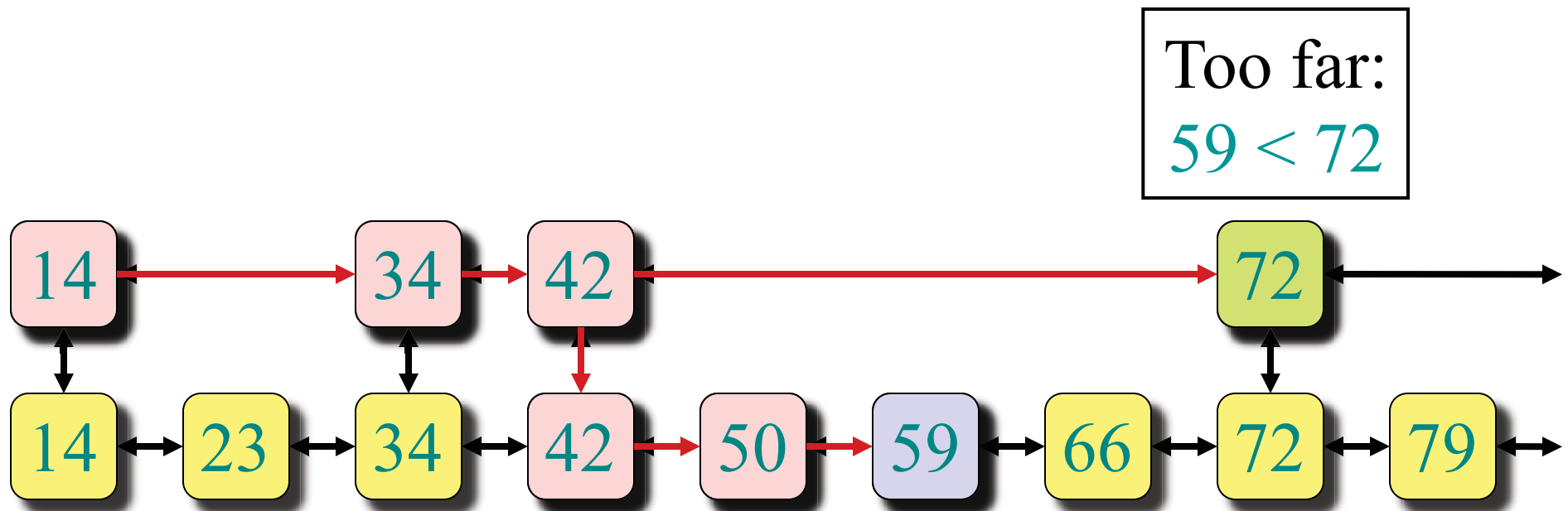
- Walk right in top linked list (L_1) until going right would go too far
- Walk down to bottom linked list (L_2)
- Walk right in L_2 until element found (or not)

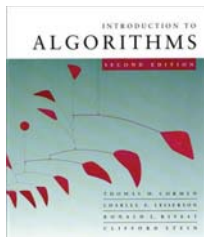




Searching in two linked lists

EXAMPLE: SEARCH(59)

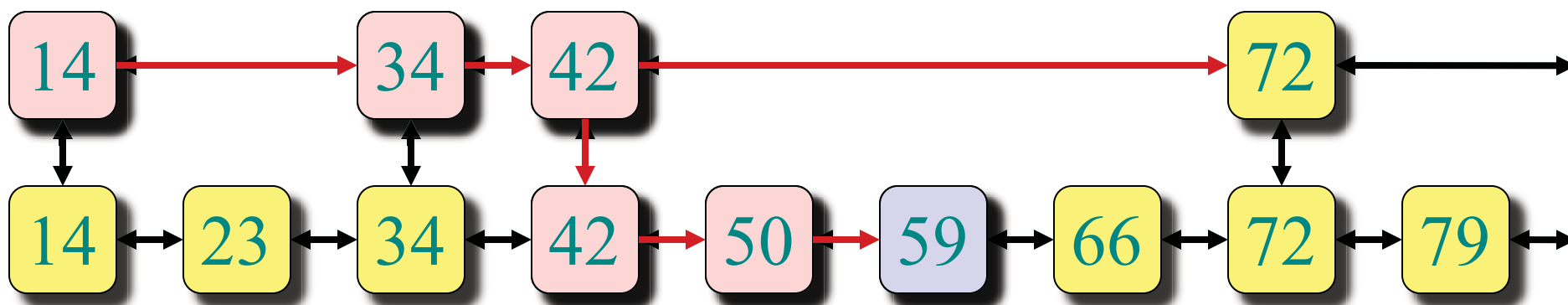


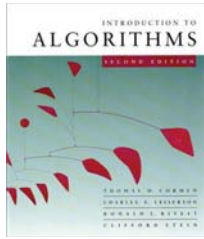


Design of two linked lists

QUESTION: Which nodes should be in L_1 ?

- In a subway, the “popular stations”
- Here we care about *worst-case performance*
- **Best approach:** Evenly space the nodes in L_1
- But *how many nodes* should be in L_1 ?

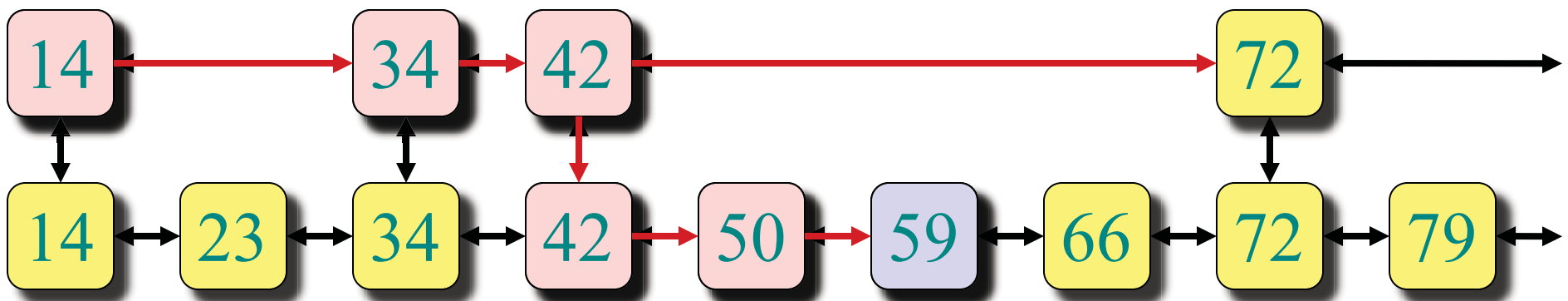


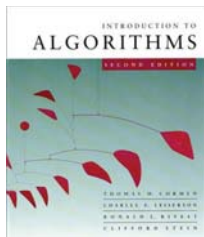


Analysis of two linked lists

ANALYSIS:

- Search cost is roughly $|L_1| + \frac{|L_2|}{|L_1|}$
- Minimized (up to constant factors) when terms are equal
- $|L_1|^2 = |L_2| = n \Rightarrow |L_1| = \sqrt{n}$



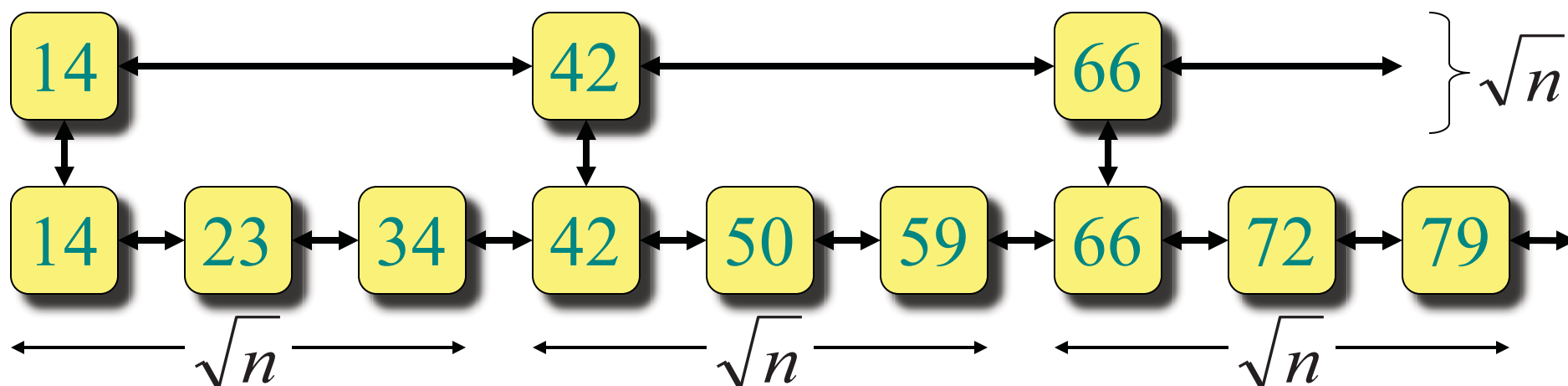


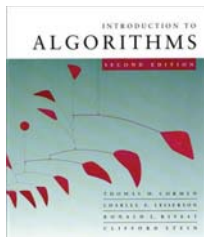
Analysis of two linked lists

ANALYSIS:

- $|L_1| = \sqrt{n}$, $|L_2| = n$
- Search cost is roughly

$$|L_1| + \frac{|L_2|}{|L_1|} = \sqrt{n} + \frac{n}{\sqrt{n}} = 2\sqrt{n}$$

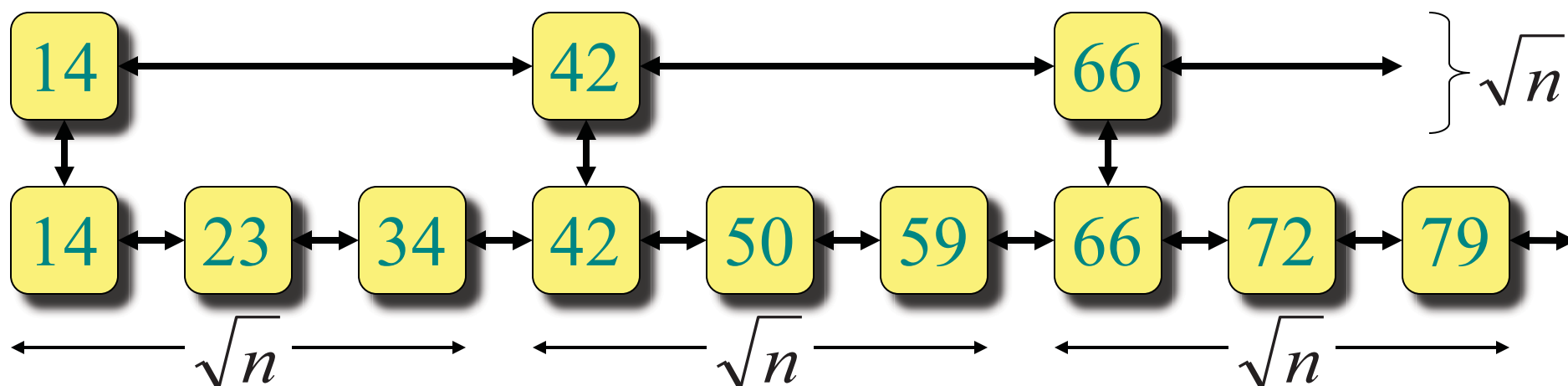


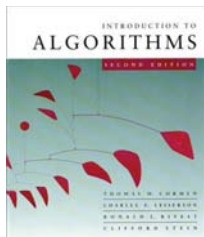


More linked lists

What if we had more sorted linked lists?

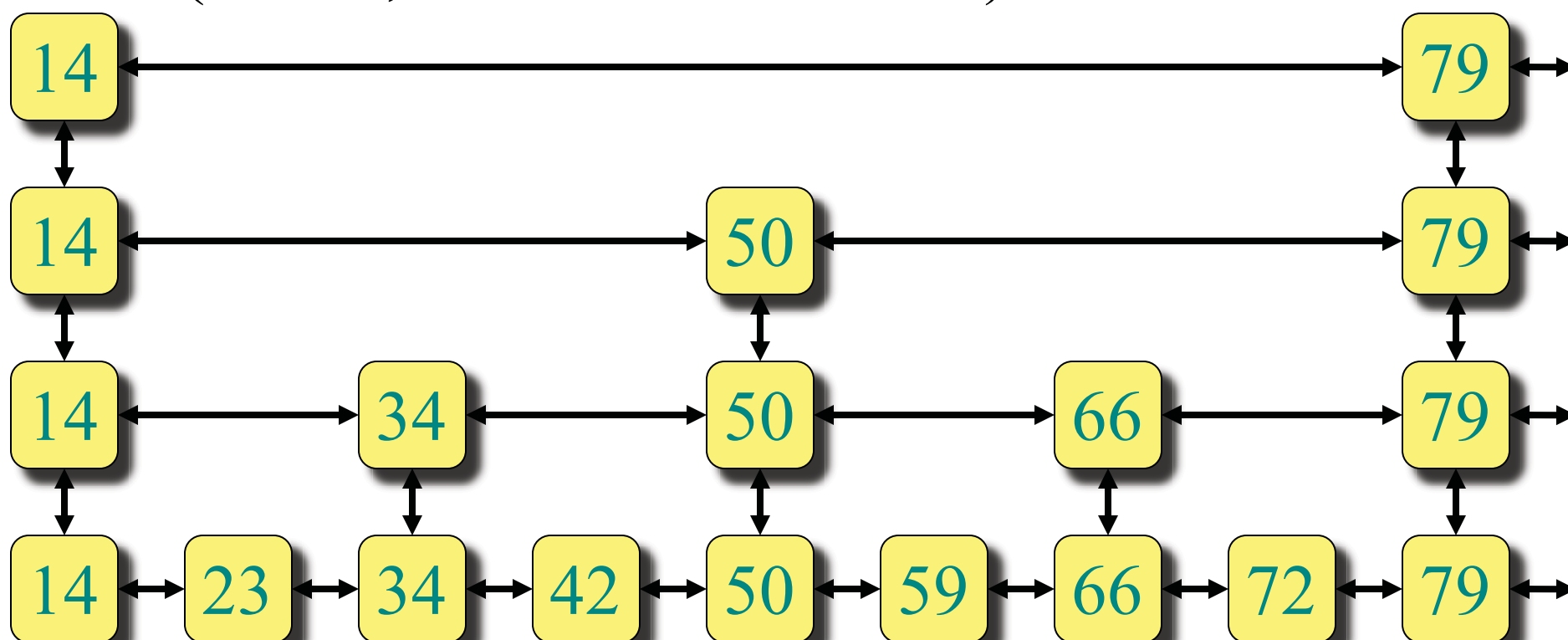
- 2 sorted lists $\Rightarrow 2 \cdot \sqrt{n}$
- 3 sorted lists $\Rightarrow 3 \cdot \sqrt[3]{n}$
- k sorted lists $\Rightarrow k \cdot \sqrt[k]{n}$
- $\lg n$ sorted lists $\Rightarrow \lg n \cdot \sqrt[\lg n]{n} = 2 \lg n$

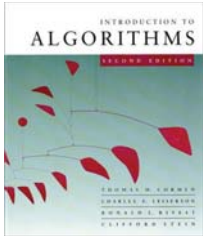




$\lg n$ linked lists

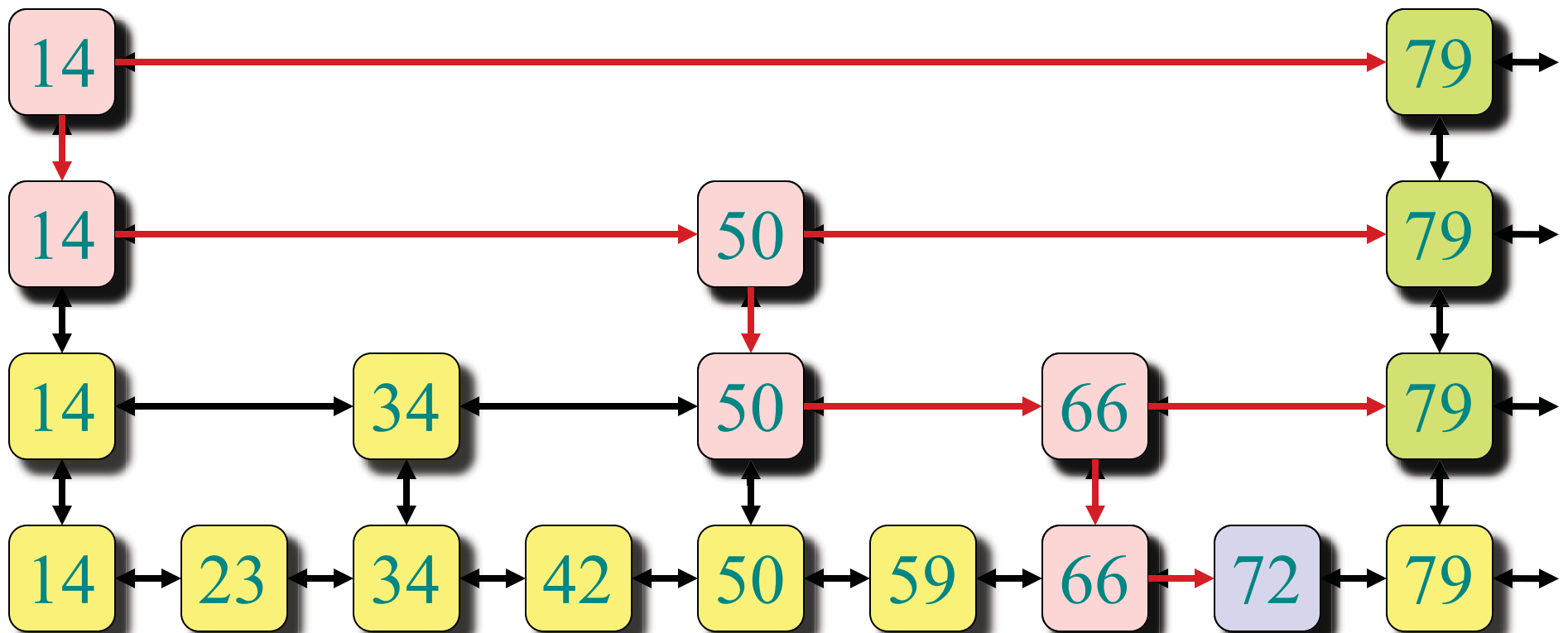
$\lg n$ sorted linked lists are like a binary tree
(in fact, level-linked B^+ -tree)

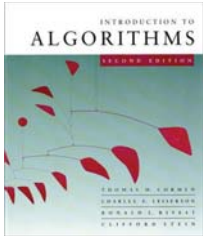




Searching in $\lg n$ linked lists

EXAMPLE: SEARCH(72)

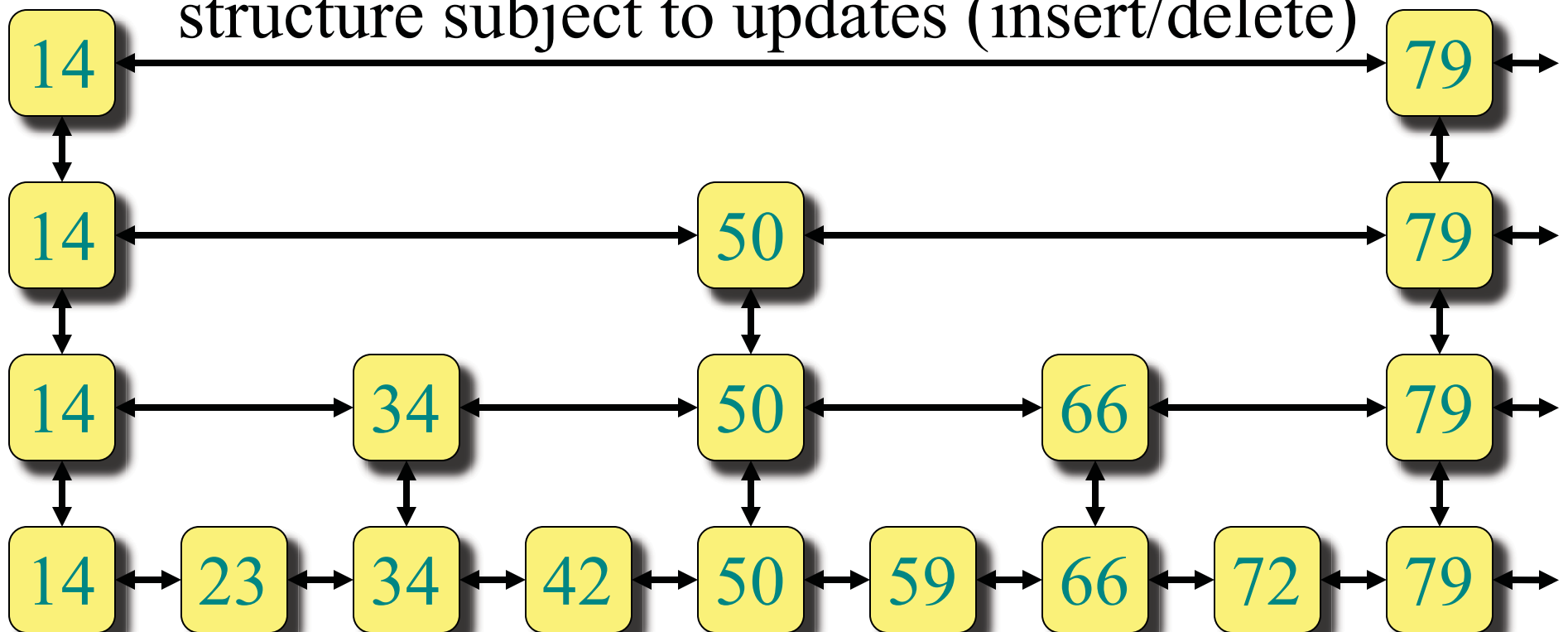


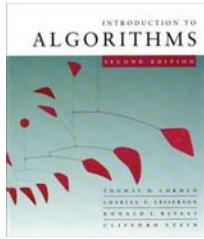


Skip lists

Ideal skip list is this $\lg n$ linked list structure

Skip list data structure maintains roughly this structure subject to updates (insert/delete)





INSERT(x)

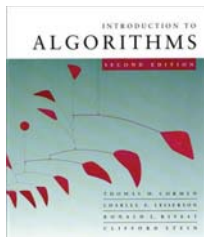
To insert an element x into a skip list:

- SEARCH(x) to see where x fits in bottom list
- Always insert into bottom list

INVARIANT: Bottom list contains all elements

- Insert into some of the lists above...

QUESTION: To which other lists should we add x ?



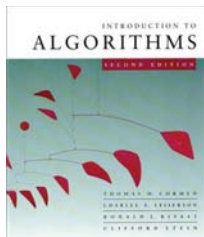
INSERT(x)

QUESTION: To which other lists should we add x ?

IDEA: Flip a (fair) coin; if HEADS,
promote x to next level up and flip again

- Probability of promotion to next level = $p = 1/2$
- On average:
 - 1/2 of the elements promoted 0 levels
 - 1/4 of the elements promoted 1 level
 - 1/8 of the elements promoted 2 levels
 - etc.

Approx.
balance
d?

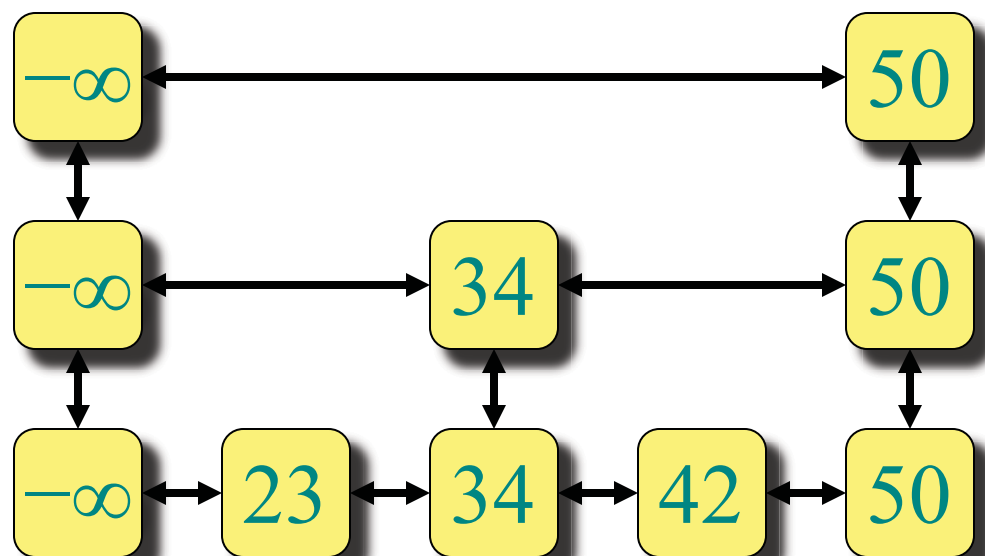


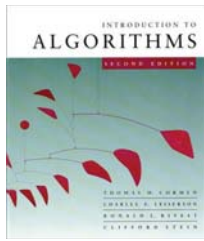
Example of skip list

EXERCISE: Try building a skip list from scratch by repeated insertion using a real coin

Small change:

- Add special $-\infty$ value to *every* list $\Rightarrow$ can search with the same algorithm

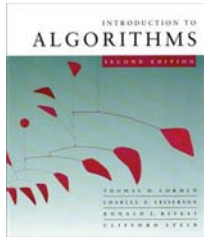




Skip lists

A *skip list* is the result of insertions (and deletions) from an initially empty structure (containing just $-\infty$)

- INSERT(x) uses random coin flips to decide promotion level
- DELETE(x) removes x from all lists containing it



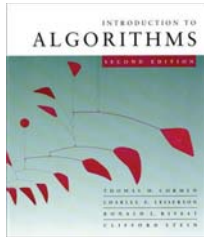
Skip lists

A *skip list* is the result of insertions (and deletions) from an initially empty structure (containing just $-\infty$)

- INSERT(x) uses random coin flips to decide promotion level
- DELETE(x) removes x from all lists containing it

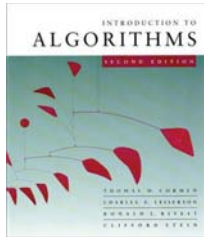
How good are skip lists? (speed/balance)

- **INTUITIVELY:** Pretty good on average
- **Expected Time for Search:** $O(\lg n)$



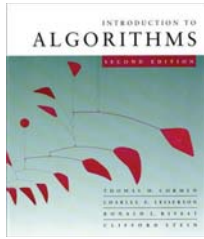
Expected Time for SEARCH

- Search for target begins with head element in top list
- Proceed horizontally until current element greater than or equal to target
- If the current element is equal to the target, it has been found. If the current element is greater than the target, go back to the previous element and drop down vertically to the next lower list and repeat the procedure.
- The expected number of steps in each linked list is seen to be $1/p$, by tracing the search path **backwards** from the target until reaching an element that appears in the next higher list.
- The total *expected* cost of a search is $O(\log_{1/p} n) \cdot (1/p)$ which is $O(\lg n)$ when p is a constant



With-high-probability theorem

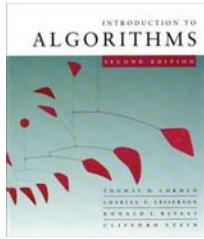
THEOREM: *With high probability, every search in an n -element skip list costs $O(\lg n)$*



With-high-probability theorem

THEOREM: *With high probability*, every search in a skip list costs $O(\lg n)$

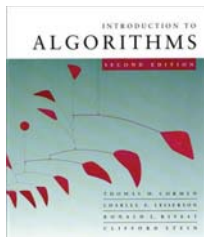
- **INFORMALLY:** Event E occurs *with high probability* (w.h.p.) if, for any $\alpha \geq 1$, there is an appropriate choice of constants for which E occurs with probability at least $1 - O(1/n^\alpha)$
 - In fact, constant in $O(\lg n)$ depends on α
- **FORMALLY:** Parameterized event E_α occurs *with high probability* if, for any $\alpha \geq 1$, there is an appropriate choice of constants for which E_α occurs with probability at least $1 - c_\alpha/n^\alpha$



With-high-probability theorem

THEOREM: With high probability, every search in a skip list costs $O(\lg n)$

- **INFORMALLY:** Event E occurs *with high probability* (*w.h.p.*) if, for any $\alpha \geq 1$, there is an appropriate choice of constants for which E occurs with probability at least $1 - O(1/n^\alpha)$
- **IDEA:** Can make *error probability* $O(1/n^\alpha)$ very small by setting α large, e.g., 100
- Almost certainly, bound remains true for entire execution of polynomial-time algorithm



Boole's inequality / union bound

Recall:

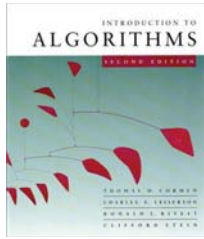
BOOLE'S INEQUALITY / UNION BOUND:

For any random events $E_1, E_2, \dots, E_k$,

$$\Pr\{E_1 \cup E_2 \cup \dots \cup E_k\} \\ \leq \Pr\{E_1\} + \Pr\{E_2\} + \dots + \Pr\{E_k\}$$

Application to with-high-probability events:

If $k = n^{O(1)}$, and each E_i occurs with high probability, then so does $E_1 \cap E_2 \cap \dots \cap E_k$



Analysis Warmup

LEMMA: n -element skip list has $O(\lg n)$ expected number of levels

PROOF:

- Probability that x has been promoted once is p
- Probability that x has been promoted k times is p^k
- Expected number of promotions is
- $\sum_{i=0}^{\infty} i \cdot p^i = O(\log \text{Error probability for having at most } c \lg n \text{ levels})$

$$= \Pr\{\text{more than } c \lg n \text{ levels}\}$$

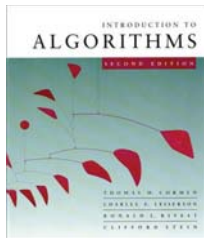
$$\leq n \cdot \Pr\{\text{element } x \text{ promoted at least } c \lg n \text{ times}\}$$

(by Boole's Inequality)

$$= n \cdot (1/2^{c \lg n})$$

$$= n \cdot (1/n^c)$$

$$= 1/n^{c-1}$$

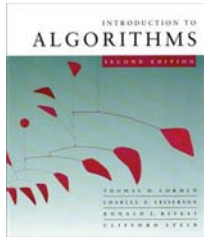


Analysis Warmup

LEMMA: With high probability,
 n -element skip list has $O(\lg n)$ levels

PROOF:

- Error probability for having at most $c \lg n$ levels
 $\leq 1/n^{c-1}$
- This probability is *polynomially small*,
i.e., at most n^α for $\alpha = c - 1$.
- We can make α arbitrarily large by choosing the
constant c in the $O(\lg n)$ bound accordingly. $\square$

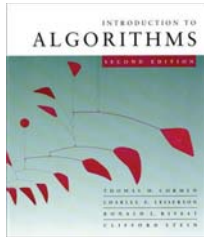


Proof of theorem

THEOREM: Every search in an n -element skip list costs $O(\lg n)$ expected time

COOL IDEA: Analyze search backwards—leaf to root

- Search starts [ends] at leaf (node in bottom level)
- At each node visited:
 - If node wasn't promoted higher (got TAILS here), then we go [came from] left
 - If node was promoted higher (got HEADS here), then we go [came from] up
- Search stops [starts] at the root (or $-\infty$)



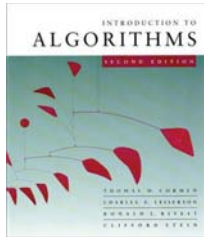
Proof of theorem

THEOREM: With high probability, every search
in an n -element skip list costs $O(\lg n)$

COOL IDEA: Analyze search backwards—leaf to root

PROOF:

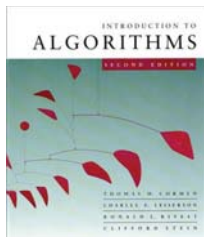
- Search makes “up” and “left” moves until it reaches the root (or $-\infty$)
- Expected number of “up” moves $<$ num. of levels $\leq O(\lg n)$ (*Lemma*)
- $\Rightarrow$ w.h.p., number of moves is at most the number of times we need to flip a coin to get $c \lg n$ HEADS



Chernoff Bounds

THEOREM (CHERNOFF): Let Y be a random variable representing the total number of heads (tails) in a series of m independent coin flips, where each flip has a probability p of coming up heads (tails). Then, for all $r > 0$,

$$Pr[Y \geq E[Y] + r] \leq e^{-2r^2/m}$$



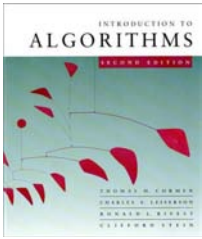
Lemma

LEMMA: For any c there is a constant d such that w.h.p. the number of heads in flipping $d \lg n$ fair coins is at least $c \lg n$.

PROOF: Let Y be the number of tails when flipping a fair coin $d \lg n$ times. $p = 1/2$.

$m = d \lg n$, so $E[Y] = 1/2 m = 1/2 d \lg n$

We want to bound the probability of $\leq c \lg n$ heads = probability of $\geq d \lg n - c \lg n$ tails.



Lemma Proof (contd.)

$$\Pr[Y \geq (d - c) \lg n] =$$

$$\Pr[Y \geq E[Y] + (\tfrac{1}{2} d - c) \lg n]$$

$$\text{Choose } d = 3c \Rightarrow r = 3c \lg n$$

By Chernoff, probability of $\leq c \lg n$ heads is

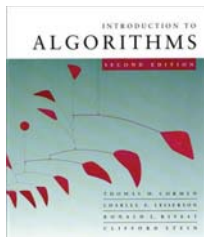
$$\leq e^{-2r^2/m} = e^{-2(3c \lg n)^2 / 8c \lg n} = e^{-9/4 c \lg n}$$

$$\leq e^{-c \lg n}$$

$$\leq 2^{-c \lg n} \quad (e > 2)$$

$$= 1/n^c$$





Proof of theorem (finally!)

THEOREM: With high probability, every search in an n -element skip list costs $O(\lg n)$

event A : number of levels $\leq c \lg n$ w.h.p.

event B : number of moves until $c \lg n$ “up” moves $\leq d \lg n$ w.h.p.

A and B are not independent!

Want to show $A \& B$ occurs w.h.p. to prove theorem

$$\begin{aligned} Pr(\overline{A \& B}) = Pr(\overline{A} + \overline{B}) &\leq Pr(\overline{A}) + Pr(\overline{B}) \text{ (union bound)} \\ &\leq 1/n^{c-1} + 1/n^c \\ &= O(1/n^{c-1}) \end{aligned}$$



MIT OpenCourseWare
<http://ocw.mit.edu>

6.046J / 18.410J Design and Analysis of Algorithms
Spring 2015

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Lecture 8: Hashing

Course Overview

This course covers several modules:

1. Review: dictionaries, chaining, simple uniform
2. Universal hashing
3. Perfect hashing

Review

Dictionary Problem

A dictionary is an Abstract Data Type (ADT) that maintains a set of items. Each item has a key. The dictionary supports the following operations:

- **insert(item)**: add item to set
- **delete(item)**: remove item from set
- **search(key)**: return item with key if it exists

We assume that items have distinct keys (or that inserting new ones clobbers old ones).

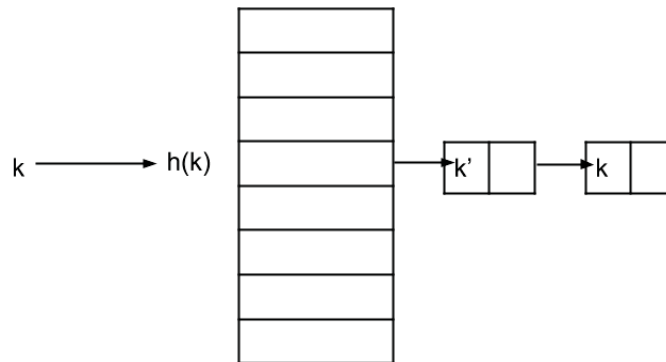
This problem is easier than predecessor/successor problems solved in previous lecture (by van Emde Boas trees, or by AVL/2-3 trees/skip lists).

Hashing from 6.006

Goal: $O(1)$ time per operation and $O(n)$ space complexity.

Definitions:

- u = number of keys over all possible items
- n = number of keys/items currently in the table
- m = number of slots in the table



Solution: hashing with chaining

Assuming simple uniform hashing,

$$\Pr_{k_1 \neq k_2} \{h(k_1) = h(k_2)\} = \frac{1}{m}$$

we achieve $\Theta(1 + \alpha)$ time per operation, where $\alpha = \frac{n}{m}$ is called load factor. The downside of the algorithm is that it requires assuming input keys are random, and it only works in average case, like basic quicksort. Today we are going to remove the unreasonable simple uniform hashing assumption.

Etymology

The English ‘hash’ (1650s) means “cut into small pieces”, which comes from the French ‘hacher’ which means “chop up”, which comes from the Old French ‘hache’ which means “axe” (cf. English ‘hatchet’). Alternatively, perhaps they come from Vulcan ‘la’ash’, which means “axe”. (R.I.P. Leonard Nimoy.)

Universal Hashing

The idea of universal hashing is listed as following:

- choose a random hash function h from $\mathcal{H}$
- require $\mathcal{H}$ to be a *universal hashing family* such that

$$\Pr_{h \in \mathcal{H}} \{h(k) = h(k')\} \leq \frac{1}{m} \text{ for all } k \neq k'$$

- now we just assume h is random, and make no assumption about input keys. (like Randomized Quicksort)

Theorem: For n arbitrary distinct keys and random $h \in \mathcal{H}$, where $\mathcal{H}$ is a universal hashing family,

$$E[\text{number of keys colliding in a slot}] \leq 1 + \alpha \quad \text{where} \quad \alpha = \frac{n}{m}$$

Proof: Consider keys $k_1, k_2, \dots, k_n$. Let $I_{i,j} = \begin{cases} 1 & \text{if } h(k_i) = h(k_j) \\ 0 & \text{otherwise} \end{cases}$. Then we have

$$\begin{aligned} E[I_{i,j}] &= \Pr\{I_{i,j} = 1\} \\ &= \Pr\{h(k_i) = h(k_j)\} \\ &\leq \frac{1}{m} \quad \text{for any } j \neq i \end{aligned} \tag{1}$$

$$\begin{aligned} E[\# \text{ keys hashing to the same slot as } k_i] &= E\left[\sum_{j=1}^n I_{i,j}\right] \\ &= \sum_{j=1}^n E[I_{i,j}] \quad (\text{linearity of expectation}) \\ &= \sum_{j \neq i} E[I_{i,j}] + E[I_{i,i}] \\ &\leq \frac{n}{m} + 1 \end{aligned} \tag{2}$$

□

From the above theorem, we know that Insert, Delete, and Search all take $O(1 + \alpha)$ expected time. Here we give some examples of universal hash functions.

All hash functions: $\mathcal{H} = \{\text{all hash functions } h : \{0, 1, \dots, u-1\} \rightarrow \{0, 1, \dots, m-1\}\}$. Apparently, $\mathcal{H}$ is universal, but it is useless. On one hand, storing a single hashing function h takes $\log(m^u) = u \log(m)$ bits $\gg n$ bits. On the other hand, we would need to precompute u values, which takes $\Omega(u)$ time.

Dot-product hash family:

Assumptions

- m is a prime

- $u = m^r$ where r is an integer

In real cases, we can always round up m and u to satisfy the above assumptions. Now let's view keys in base m : $k = \langle k_0, k_1, \dots, k_{r-1} \rangle$. For key $a = \langle a_0, a_1, a_2, \dots, a_{r-1} \rangle$, define

$$\begin{aligned} h_a(k) &= a \cdot k \bmod m \text{ (dot product)} \\ &= \sum_{i=0}^{r-1} a_i k_i \bmod m \end{aligned} \tag{3}$$

Then our hash family is $\mathcal{H} = \{h_a \mid a \in \{0, 1, \dots, u-1\}\}$

Storing $h_a \in \mathcal{H}$ requires just storing one key, which is a . In the **word RAM model**, manipulating $O(1)$ machine words takes $O(1)$ time and “objects of interest” (here, keys) fit into a machine word. Thus computing $h_a(k)$ takes $O(1)$ time.

Theorem: Dot-product hash family $\mathcal{H}$ is universal.

Proof: Take any two keys $k \neq k'$. They must differ in some digits. Say $k_d \neq k'_d$. Define *not* $d = \{0, 1, \dots, r-1\} \setminus \{d\}$. Now we have

$$\begin{aligned} \Pr_a \{h_a(k) = h_a(k')\} &= \Pr_a \left\{ \sum_{i=0}^{r-1} a_i k_i = \sum_{i=0}^{r-1} a_i k'_i \pmod{m} \right\} \\ &= \Pr_a \left\{ \sum_{i \neq d} a_i k_i + a_d k_d = \sum_{i \neq d} a_i k'_i + a_d k'_d \pmod{m} \right\} \\ &= \Pr_a \left\{ \sum_{i \neq d} a_i (k_i - k'_i) + a_d (k_d - k'_d) = 0 \pmod{m} \right\} \\ &= \Pr_a \left\{ a_d = -(k_d - k'_d)^{-1} \sum_{i \neq d} a_i (k_i - k'_i) \pmod{m} \right\} \end{aligned} \tag{4}$$

(m is prime $\Rightarrow \mathbb{Z}_m$ has multiplicative inverses)

$$\begin{aligned} &= E_{a_{not\ d} \ a_d} [\Pr_{a_d} \{a_d = f(k, k', a_{not\ d})\}] \\ &= \sum_x \Pr_{a_{not\ d}} \{a_{not\ d} = x\} \Pr_{a_d} \{a_d = f(k, k', x)\} \\ &= E_{a_{not\ d}} \left[\frac{1}{m} \right] \\ &= \frac{1}{m} \end{aligned}$$

□

Another universal hash family from CLRS: Choose prime $p \geq u$ (once). Define $h_{ab}(k) = [(ak + b) \bmod p] \bmod m$. Let $\mathcal{H} = \{h_{ab} \mid a, b \in \{0, 1, \dots, u - 1\}\}$.

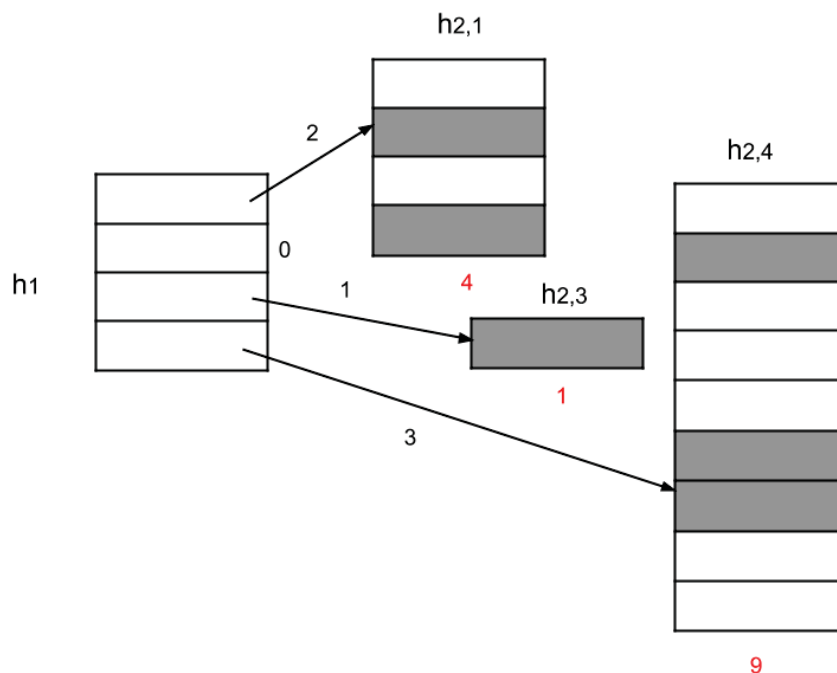
Perfect Hashing

Static dictionary problem: Given n keys to store in table, only need to support $\text{search}(k)$. No insertion or deletion will happen.

Perfect hashing: [Fredman, Komlós, Szemerédi 1984]

- polynomial build time with high probability (w.h.p.)
- $O(1)$ time for search in worst case
- $O(n)$ space in worst case

Idea: 2-level hashing



The algorithm contains the following two major steps:

Step 1: Pick $h_1 : \{0, 1, \dots, u - 1\} \rightarrow \{0, 1, \dots, m - 1\}$ from a universal hash family for $m = \Theta(n)$ (e.g., nearby prime). Hash all items with chaining using h_1 .

Step 2: For each slot $j \in \{0, 1, \dots, m-1\}$, let l_j be the number of items in slot j . $l_j = |\{i \mid h(k_i) = j\}|$. Pick $h_{2,j} : \{0, 1, \dots, u-1\} \rightarrow \{0, 1, \dots, m_j\}$ from a universal hash family for $l_j^2 \leq m_j \leq O(l_j^2)$ (e.g., nearby prime). Replace chain in slot j with hashing-with-chaining using $h_{2,j}$.

The space complexity is $O(n + \sum_{j=0}^{m-1} l_j^2)$. In order to reduce it to $O(n)$, we need to add two more steps:

Step 1.5: If $\sum_{j=0}^{m-1} l_j^2 > cn$ where c is a chosen constant, then redo Step 1.

Step 2.5: While $h_{2,j}(k_i) = h_{2,j}(k_{i'})$ for any $i \neq i', j$, repick $h_{2,j}$ and rehash those l_j .

The above two steps guarantee that there are no collisions at second level, and the space complexity is $O(n)$. As a result, search time is $O(1)$. Now let's look at the build time of the algorithm. Both Step 1 and Step 2 are $O(n)$. How about Step 1.5 and Step 2.5?

For Step 2.5,

$$\begin{aligned} \Pr_{h_{2,j}} \{h_{2,j}(k_i) = h_{2,j}(k_{i'}) \text{ for some } i \neq i'\} &\leq \sum_{i \neq i'} \Pr_{h_{2,j}} \{h_{2,j}(k_i) = h_{2,j}(k_{i'})\} \text{ (union bound)} \\ &\leq \binom{l_j}{2} \cdot \frac{1}{l_j^2} \\ &< \frac{1}{2} \end{aligned} \tag{5}$$

As a result, each trial is like a coin flip. If the outcome is “tail”, we move to the next step. By Lecture 7, we have $E[\text{\#trials}] \leq 2$ and $\text{\#trials} = O(\log n)$ w.h.p. By a Chernoff bound, $l_j = O(\log n)$ w.h.p., so each trial takes $O(\log n)$ time. Because we have to do this for each j , the total time complexity is $O(\log n) \cdot O(\log n) \cdot O(n) = O(n \log^2 n)$ w.h.p.

For Step 1.5, we define $I_{i,i'} = \begin{cases} 1 & \text{if } h(k_i) = h(k_{i'}) \\ 0 & \text{otherwise} \end{cases}$. Then we have

$$\begin{aligned}
 E \left[\sum_{j=0}^{m-1} l_j^2 \right] &= E \left[\sum_{i=1}^n \sum_{i'=1}^n I_{i,i'} \right] \\
 &= \sum_{i=1}^n \sum_{i'=1}^n E[I_{i,i'}] \quad (\text{linearity of expectation}) \\
 &\leq n + 2 \binom{n}{2} \cdot \frac{1}{m} \\
 &= O(n) \text{ because } m = \Theta(n)
 \end{aligned} \tag{6}$$

By Markov inequality, we have

$$\Pr_{h_1} \left\{ \sum_{j=0}^{m-1} l_j^2 \leq cn \right\} \leq \frac{\sum_{j=0}^{m-1} l_j^2}{cn} \leq \frac{1}{2}$$

for a sufficiently large constant c . By Lecture 7, we have $E[\text{\#trials}] \leq 2$ and $\text{\#trials} = O(\log n)$ w.h.p. As a result, Step 1 and Step 1.5 combined takes $O(n \log n)$ time w.h.p.

MIT OpenCourseWare
<http://ocw.mit.edu>

6.046J / 18.410J Design and Analysis of Algorithms
Spring 2015

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Lecture 9: Augmentation

This lecture covers augmentation of data structures, including

- easy tree augmentation
- order-statistics trees
- finger search trees, and
- range trees

The main idea is to modify “off-the-shelf” common data structures to store (and update) additional information.

Easy Tree Augmentation

The goal here is to store $x.f$ at each node x , which is a function of the node, namely $f(\text{subtree rooted at } x)$. Suppose $x.f$ can be computed (updated) in $O(1)$ time from x , *children* and *children.f*. Then, modifying a set S of nodes costs $O(\# \text{ of ancestors of } S)$ to update $x.f$, because we need to walk up the tree to the root. Two examples of $O(\lg n)$ updates are

- AVL trees: after rotating two nodes, first update the new bottom node and then update the new top node
- 2-3 trees: after splitting a node, update the two new nodes.
- In both cases, then update up the tree.

Order-Statistics Trees (from 6.006)

The goal of order-statistics trees is to design an Abstract Data Type (ADT) interface that supports the following operations

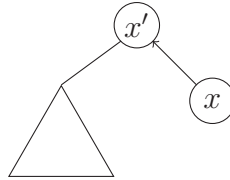
- $\text{insert}(x)$, $\text{delete}(x)$, $\text{successor}(x)$,
- $\text{rank}(x)$: find x 's index in the sorted order, i.e., $\#$ of elements $< x$,
- $\text{select}(i)$: find the element with rank i .

We can implement the above ADT using easy tree augmentation on AVL trees (or 2-3 trees) to store subtree size: $f(\text{subtree}) = \#$ of nodes in it. Then we also have $x.\text{size} = 1 + \sum c.\text{size}$ for c in $x.\text{children}$.

As a comparison, we cannot store the rank for each node. In that case, $\text{insert}(-\infty)$ will change the ranks for *all* nodes.

$\text{rank}(x)$ can be computed as follows:

- initialize $\text{rank} = x.\text{left.size} + 1$ ¹
- walk up from x to root, whenever taking a left move ($x \rightarrow x'$), $\text{rank} += x'.\text{left.size} + 1$



$\text{select}(i)$ can be implemented as follows:

- $x = \text{root}$
- $\text{rank} = x.\text{left.size} + 1$ ²
- if $i = \text{rank}$: return x
- if $i < \text{rank}$: $x = x.\text{left}$
- if $i > \text{rank}$: $x = x.\text{right}$, $i -= \text{rank}$

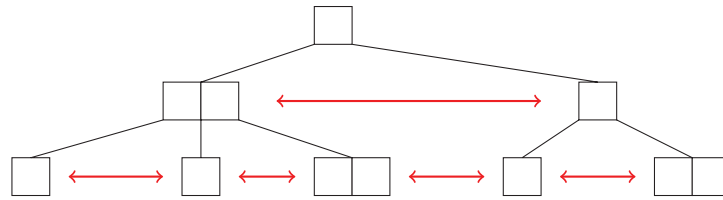
Finger Search Trees

The goal of finger search trees [Brown and Tarjan, 1980] is that, if we already have node y , we want to search x from y in $O(\lg |\text{rank}(y) - \text{rank}(x)|)$ time. Intuitively, we would like the search of x to be fast if we already have a node y that is close to x .

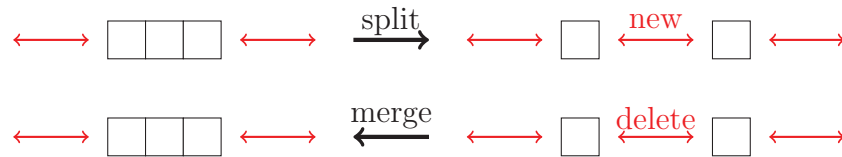
One idea is to use **level-linked** 2-3 trees, where each node has pointers to its next and previous node on the same level.

¹omit the +1 term if indices start at 0.

²same as above.



The level links can be maintained during split and merge.



We store all keys in the leaves. Non-leaf nodes do not store keys; instead, they store the min and max key of the subtree (via easy tree augmentation).

Then the original top-down search(x) (without being given y) can be implemented as follows:

- start from the root, look at min & max of each child c_i
- if $c_i.\text{min} \leq x \leq c_i.\text{max}$, go down to c_i
- if $c_i.\text{max} \leq x \leq c_{i+1}.\text{min}$, return $c_i.\text{max}$ (as predecessor) or $c_{i+1}.\text{min}$ (as successor)

search(x) from y can be implemented as follows. Initialize v to the leaf node containing y (given), and then in a loop do

- if $v.\text{min} \leq x \leq v.\text{max}$ (this means x is in the subtree rooted at v), do top-down search for x from v and return
- elif $x < v.\text{min}$: $v = v.\text{prev}$ (the previous node in this level)
- elif $x > v.\text{max}$: $v = v.\text{next}$ (the next node in this level)
- $v = v.\text{parent}$

Analysis. We start at the leaf level, and go up by 1 level in each iteration. At step i , level link at height i skips roughly c^i keys (ranks), where $c \in [2, 3]$. Therefore, if $|\text{rank}(y) - \text{rank}(x)| = k$, we will reach the subtree containing x in $O(\lg k)$ steps, and the top-down search that follows is also $O(\lg k)$.

Orthogonal Range Searching and Range Trees

Suppose we have n points in a d -dimension space. We would like a data structure that supports **range query** on these points: find all the points in a give **axis-aligned box**. An axis-aligned box is simply an interval in 1D, a rectangle in 2D, and a cube in 3D.

To be more precise, each point x_i (for i from 1 to n) is a d -dimension vector $x_i = (x_{i1}, x_{i2}, \dots, x_{id})$. Range-query(a, b) takes two points $a = (a_1, a_2, \dots, a_d)$ and $b = (b_1, b_2, \dots, b_d)$ as input, and should return a set of indices $\{i \mid \forall j, a_j \leq x_{ij} \leq b_j\}$.

1D case

We start with the simple case of 1D points, i.e., all x_i 's and a and b are scalars.

Then, we can simply use a sorted array. To do range-query(a, b), we simply perform two binary searches for a and b , respectively, and then return all the points in between (say there are k of them). The complexity is $O(\lg n + k)$.

Sorted arrays are inefficient for insertion and deletion. For a dynamic data structure that supports range queries, we can use finger search tree from the previous section. Finger search trees support efficient insertion and deletion. To do range-query(a, b), we first search for a , and then keep doing finger search to the right by 1 until we exceed b . Each finger search by 1 takes $O(1)$, so the total complexity is also $O(\lg n + k)$.

However, neither of the above approaches generalizes to high dimensions. That's why we now introduce range trees.

1D range trees

A 1D range tree is a complete binary search tree (for dynamic, use an AVL tree). Range-query(a, b) can be implemented as follows:

- search(a)
- search(b)
- find the least common ancestor (LCA) of a and b , v_{split}
- return the nodes and subtrees “in between”. There are $O(\lg n)$ nodes and $O(\lg n)$ subtrees “in between”.

Analysis. $O(\lg n)$ to implicitly represent the answer. $O(\lg n + k)$ to output all k answers. $O(\lg n)$ to report k via subtree size augmentation.

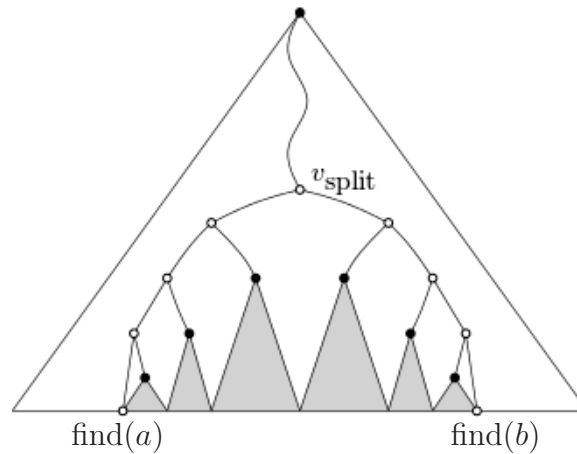


Figure 1: 1D range tree. Range-query(a, b) returns all hollow nodes and shaded subtrees. Image from Wikipedia http://en.wikipedia.org/wiki/Range_tree

2D range trees

A 2D range tree consists of a primary 1D range tree and many secondary 1D range trees. The primary range stores all points, keyed on the first coordinate. Every node v in the primary range tree stores all points in v 's subtree in a secondary range tree, keyed on the second coordinate.

Range-query(a, b) can be implemented as follows:

- use the primary range tree to find all points with the correct range on the first coordinate. Only implicitly represent the answer, so this takes $O(\lg n)$.
- for the $O(\lg n)$ nodes, manually check whether their second coordinate lie in the correct range.
- for the $O(\lg n)$ subtrees, use their secondary range tree to find all points with the correct range on the second coordinate.

Analysis. $O(\lg^2 n)$ to implicitly represent the answer, because we will find $O(\lg^2 n)$ nodes and subtrees in secondary range trees. $O(\lg^2 n + k)$ to output all k answers. $O(\lg^2 n)$ to report k via subtree size augmentation.

Space complexity is $O(n \lg n)$. The primary subtree is $O(n)$. Each point is duplicated up to $O(\lg n)$ times in secondary subtrees, one per ancestor.

d -D range trees

Just recurse: primary 1D range tree $\rightarrow$ secondary 1D range trees $\rightarrow$ tertiary 1D range trees $\rightarrow \dots$

Range-query complexity: $O(\lg^d n + k)$.

Space complexity: $O(n \lg^{d-1} n)$.

See 6.851 for Chazelle's improved results: $O(\lg^{d-1} n + k)$ range-query complexity and $O\left(n \left(\frac{\lg n}{\lg \lg n}\right)^{d-1}\right)$ space complexity.

MIT OpenCourseWare
<http://ocw.mit.edu>

6.046J / 18.410J Design and Analysis of Algorithms
Spring 2015

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Lecture 10: Dynamic Programming

- Longest palindromic sequence
- Optimal binary search tree
- Alternating coin game

DP notions

1. Characterize the structure of an optimal solution
2. Recursively define the value of an optimal solution based on optimal solutions of subproblems
3. Compute the value of an optimal solution in bottom-up fashion (recursion & memoization)
4. Construct an optimal solution from the computed information

Longest Palindromic Sequence

Definition: A palindrome is a string that is unchanged when reversed.

Examples: radar, civic, t, bb, redder

Given: A string $X[1 \cdots n]$, $n \geq 1$

To find: Longest palindrome that is a subsequence

Example: Given “c h a r a c t e r”

output “c a r a c”

Answer will be ≥ 1 in length

Strategy

$L(i, j)$: length of longest palindromic subsequence of $X[i \cdots j]$ for $i \leq j$.

```
1  def L(i, j) :
2    if i == j: return 1
3    if X[i] == X[j]:
4      if i + 1 == j: return 2
5      else : return 2 + L(i + 1, j - 1)
6    else :
7      return max(L(i + 1, j), L(i, j - 1))
```

Exercise: compute the actual solution

Analysis

As written, program can run in exponential time: suppose all symbols $X[i]$ are distinct.

$$\begin{aligned} T(n) &= \text{running time on input of length } n \\ T(n) &= \begin{cases} 1 & n = 1 \\ 2T(n-1) & n > 1 \end{cases} \\ &= 2^{n-1} \end{aligned}$$

Subproblems

But there are only $\binom{n}{2} = \theta(n^2)$ distinct subproblems: each is an (i, j) pair with $i < j$. By solving each subproblem only once, running time reduces to

$$\theta(n^2) \cdot \theta(1) = \theta(n^2)$$

where $\theta(n^2)$ is the number of subproblems and $\theta(1)$ is the time to solve each subproblem, given that smaller ones are solved.

Memoize $L(i, j)$, hash inputs to get output value, and lookup hash table to see if the subproblem is already solved, else recurse.

Memoizing Vs. Iterating

1. Memoizing uses a dictionary for $L(i, j)$ where value of L is looked up by using i, j as a key. Could just use a 2-D array here where null entries signify that the problem has not yet been solved.
2. Can solve subproblems in order of increasing $j - i$ so smaller ones are solved first.

Optimal Binary Search Trees: CLRS 15.5

Given: keys $K_1, K_2, \dots, K_n, K_1 < K_2 < \dots < K_n$, WLOG $K_i = i$

weights $W_1, W_2, \dots, W_n$

Find: BST T that minimizes:

$$\sum_{i=1}^n W_i \cdot (\text{depth}_T(K_i) + 1)$$

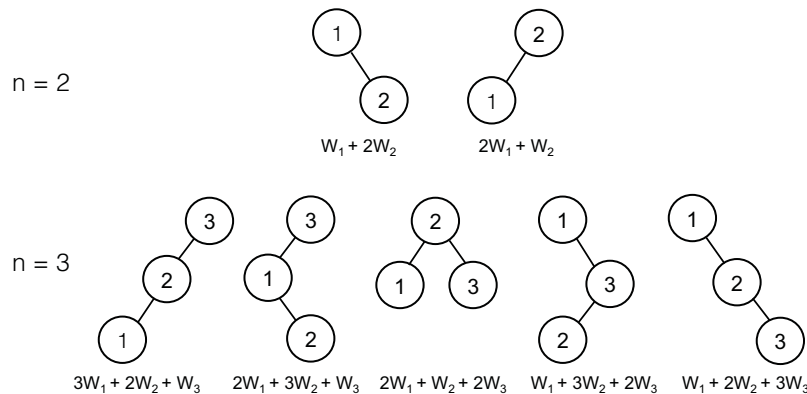
Example: $W_i = p_i = \text{probability of searching for } K_i$

Then, we are minimizing expected search cost.

(say we are representing an English $\rightarrow$ French dictionary and common words should have greater weight)

Enumeration

Exponentially many trees



Strategy

$$W(i, j) = W_i + W_{i+1} + \dots + W_j$$

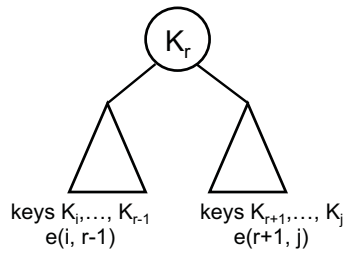
$e(i, j)$ = cost of optimal BST on $K_i, K_{i+1}, \dots, K_j$

Want $e(1, n)$

Greedy solution?

Pick K_r in some greedy fashion, e.g., W_r is maximum.

greedy doesn't work, see example at the end of the notes.



DP Strategy: Guess all roots

$$e(i, j) = \begin{cases} W_i & \text{if } i = j \\ \min_{i \leq r \leq j} (e(i, r-1) + e(r+1, j) + W(i, j)) & \text{else} \end{cases}$$

$+W(i, j)$ accounts for W_r of root K_r as well as the increase in depth by 1 of all the other keys in the subtrees of K_r (DP tries all ways of making local choices and takes advantage of overlapping subproblems)

Complexity: $\theta(n^2) \cdot \theta(n) = \theta(n^3)$

where $\theta(n^2)$ is the number of subproblems and $\theta(n)$ is the time per subproblem.

Alternating Coin Game

Row of n coins of values $V_1, \dots, V_n$, n is even. In each turn, a player selects either the first or last coin from the row, removes it permanently, and receives the value of the coin.

Question

Can the first player always win?

Try: 4 42 39 17 25 6

Strategy

$V_1, V_2, \dots, V_{n-1}, V_n$

1. Compare $V_1 + V_3 + \dots + V_{n-1}$ against $V_2 + V_4 + \dots + V_n$ and pick whichever is greater.
2. During the game only pick from the chosen subset (you will always be able to!)

How to maximize the amount of money won assuming you move first?

Optimal Strategy

$V(i, j)$: max value we can definitely win if it is our turn and only coins $V_i, \dots, V_j$ remain.

$V(i, i)$: just pick i .

$V(i, i + 1)$: pick the maximum of the two.

$V(i, i + 2), V(i, i + 3), \dots$

$$V(i, j) = \max\{\langle \text{range becomes } (i + 1, j) \rangle + V_i, \langle \text{range becomes } (i, j - 1) \rangle + V_j\}$$

Solution

$V(i + 1, j)$ subproblem with opponent picking

we are guaranteed $\min\{V(i + 1, j - 1), V(i + 2, j)\}$

Where $V(i + 1, j - 1)$ corresponds to opponent picking V_j and $V(i + 2, j)$ corresponds to opponent picking V_{i+1}

We have

$$V(i, j) = \max\left\{\min\left\{\begin{array}{c} V(i + 1, j - 1), \\ V(i + 2, j) \end{array}\right\} + V_i, \min\left\{\begin{array}{c} V(i, j - 2), \\ V(i + 1, j - 1) \end{array}\right\} + V_j\right\}$$

Complexity?

$$\Theta(n^2) \cdot \Theta(1) = \Theta(n^2)$$

Example of Greedy Failing for Optimal BST problem

Thanks to Nick Davis!

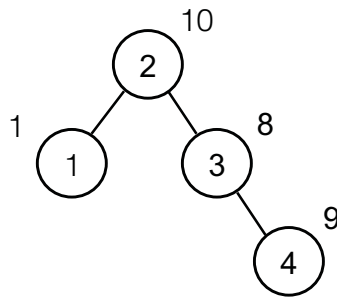


Figure 1: $\text{cost} = 1 \times 2 + 10 \times 1 + 8 \times 2 + 9 \times 3 = 55$

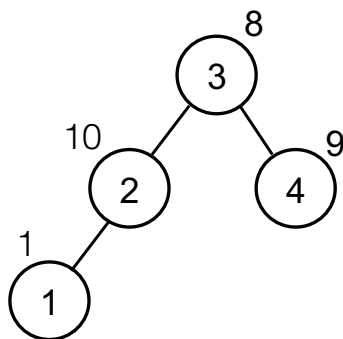


Figure 2: $\text{cost} = 1 \times 3 + 10 \times 2 + 8 \times 1 + 9 \times 2 = 49$

MIT OpenCourseWare
<http://ocw.mit.edu>

6.046J / 18.410J Design and Analysis of Algorithms
Spring 2015

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Lecture 11: All-Pairs Shortest Paths

Introduction

Different types of algorithms can be used to solve the all-pairs shortest paths problem:

- Dynamic programming
- Matrix multiplication
- Floyd-Warshall algorithm
- Johnson's algorithm
- Difference constraints

Single-source shortest paths

- given directed graph $G = (V, E)$, vertex $s \in V$ and edge weights $w : E \rightarrow \mathbb{R}$
- find $\delta(s, v)$, equal to the shortest-path weight $s \rightarrow v$, $\forall v \in V$ (or $-\infty$ if negative weight cycle along the way, or ∞ if no path)

Situation	Algorithm	Time
unweighted ($w = 1$)	BFS	$O(V + E)$
non-negative edge weights	Dijkstra	$O(E + V \lg V)$
general	Bellman-Ford	$O(VE)$
acyclic graph (DAG)	Topological sort + one pass of B-F	$O(V + E)$

All of the above results are the best known. We achieve a $O(E + V \lg V)$ bound on Dijkstra's algorithm using Fibonacci heaps.

All-pairs shortest paths

- given edge-weighted graph, $G = (V, E, w)$
- find $\delta(u, v)$ for all $u, v \in V$

A simple way of solving All-Pairs Shortest Paths (APSP) problems is by running a single-source shortest path algorithm from each of the V vertices in the graph.

Situation	Algorithm	Time	$E = \Theta(V^2)$
unweighted ($w = 1$)	$ V \times$ BFS	$O(VE)$	$O(V^3)$
non-negative edge weights	$ V \times$ Dijkstra	$O(VE + V^2 \lg V)$	$O(V^3)$
general	$ V \times$ Bellman-Ford	$O(V^2E)$	$O(V^4)$
general	Johnson's	$O(VE + V^2 \lg V)$	$O(V^3)$

These results (apart from the third) are also best known — don't know how to beat $|V| \times$ Dijkstra

Algorithms to solve APSP

Note that for all the algorithms described below, we assume that $w(u, v) = \infty$ if $(u, v) \notin E$.

Dynamic Programming, attempt 1

1. **Sub-problems:** $d_{uv}^{(m)}$ = weight of shortest path $u \rightarrow v$ using $\leq m$ edges
2. **Guessing:** What's the last edge (x, v) ?
3. **Recurrence:**

$$d_{uv}^{(m)} = \min(d_{ux}^{(m-1)} + w(x, v) \text{ for } x \in V)$$

$$d_{uv}^{(0)} = \begin{cases} 0 & \text{if } u = v \\ \infty & \text{otherwise} \end{cases}$$

4. **Topological ordering:** for $m = 0, 1, 2, \dots, n - 1$: for u and v in V :
5. **Original problem:**

If graph contains no negative-weight cycles (by Bellman-Ford analysis), then shortest path is simple $\Rightarrow \delta(u, v) = d_{uv}^{(n-1)} = d_{uv}^{(n)} = \dots$

Time complexity

In this Dynamic Program, we have $O(V^3)$ total sub-problems.

Each sub-problem takes $O(V)$ time to solve, since we need to consider V possible choices. This gives a total runtime complexity of $O(V^4)$.

Note that this is no better than $|V| \times$ Bellman-Ford

Bottom-up via relaxation steps

```

1  for  $m = 1$  to  $n$  by 1
2      for  $u$  in  $V$ 
3          for  $v$  in  $V$ 
4              for  $x$  in  $V$ 
5                  if  $d_{uv} > d_{ux} + d_{xv}$ 
6                       $d_{uv} = d_{ux} + d_{xv}$ 

```

In the above pseudocode, we omit superscripts because more relaxation can never hurt.

Note that we can change our relaxation step to $d_{uv}^{(m)} = \min(d_{ux}^{[m/2]} + d_{xv}^{[m/2]})$ for $x \in V$. This change would produce an overall running time of $O(n^3 \lg n)$ time. (student suggestion)

Matrix multiplication

Recall the task of standard matrix multiplication,

Given $n \times n$ matrices A and B , compute $C = A \cdot B$, such that $c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj}$.

- $O(n^3)$ using standard algorithm
- $O(n^{2.807})$ using Strassen's algorithm
- $O(n^{2.376})$ using Coppersmith-Winograd algorithm
- $O(n^{2.3728})$ using Vassilevska Williams algorithm

Connection to shortest paths

- Define $\oplus = \min$ and $\odot = +$
- Then, $C = A \odot B$ produces $c_{ij} = \min_k (a_{ik} + b_{kj})$
- Define $D^{(m)} = (d_{ij}^{(m)})$, $W = (w(i, j))$, $V = \{1, 2, \dots, n\}$

With the above definitions, we see that $D^{(m)}$ can be expressed as $D^{(m-1)} \odot W$. In other words, $D^{(m)}$ can be expressed as the circle-multiplication of W with itself m times.

Matrix multiplication algorithm

- $n - 2$ multiplications $\Rightarrow O(n^4)$ time (stil no better)
- Repeated squaring: $((W^2)^2)^{2^{\dots}} = W^{2^{\lg n}} = W^{n-1} = (\delta(i, j))$ if no negative-weight cycles. Time complexity of this algorithm is now $O(n^3 \lg n)$.

We can't use Strassen, etc. since our new multiplication and addition operations don't support negation.

Floyd-Warshall: Dynamic Programming, attempt 2

1. **Sub-problems:** $c_{uv}^{(k)}$ = weight of shortest path $u \rightarrow v$ whose intermediate vertices $\in \{1, 2, \dots, k\}$
2. **Guessing:** Does shortest path use vertex k ?
3. **Recurrence:**

$$c_{uv}^{(k)} = \min(c_{uv}^{(k-1)}, c_{uk}^{(k-1)} + c_{kv}^{(k-1)})$$

$$c_{uv}^{(0)} = w(u, v)$$

4. **Topological ordering:** for k : for u and v in V :
5. **Original problem:** $\delta(u, v) = c_{uv}^{(n)}$. Negative weight cycle $\Leftrightarrow$ negative $c_{uu}^{(n)}$

Time complexity

This Dynamic Program contains $O(V^3)$ problems as well. However, in this case, it takes only $O(1)$ time to solve each sub-problem, which means that the total runtime of this algorithm is $O(V^3)$.

Bottom up via relaxation

```

1  C = (w(u, v))
2  for k = 1 to n by 1
3      for u in V
4          for v in V
5              if cuv > cuk + ckv
6                  cuv = cuk + ckv
```

As before, we choose to ignore subscripts.

Johnson's algorithm

1. Find function $h : V \rightarrow \mathbb{R}$ such that $w_h(u, v) = w(u, v) + h(u) - h(v) \geq 0$ for all $u, v \in V$ or determine that a negative-weight cycle exists.
2. Run Dijkstra's algorithm on (V, E, w_h) from every source vertex $s \in V \Rightarrow$ get $\delta_h(u, v)$ for all $u, v \in V$
3. Given $\delta_h(u, v)$, it is easy to compute $\delta(u, v)$

Claim. $\delta(u, v) = \delta_h(u, v) - h(u) + h(v)$

Proof. Look at any $u \rightarrow v$ path p in the graph G

- Say p is $v_0 \rightarrow v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k$, where $v_0 = u$ and $v_k = v$.

$$\begin{aligned}
 w_h(p) &= \sum_{i=1}^k w_h(v_{i-1}, v_i) \\
 &= \sum_{i=1}^k [w(v_{i-1}, v_i) + h(v_{i-1}) - h(v_i)] \\
 &= \sum_{i=1}^k w(v_{i-1}, v_i) + h(v_0) - h(v_k) \\
 &= w(p) + h(u) - h(v)
 \end{aligned}$$

- Hence all $u \rightarrow v$ paths change in weight by the same offset $h(u) - h(v)$, which implies that the shortest path is preserved (but offset).

□

How to find h ?

We know that

$$w_h(u, v) = w(u, v) + h(u) - h(v) \geq 0$$

This is equivalent to,

$$h(v) - h(u) \leq w(u, v)$$

for all $(u, v) \in V$. This is called a **system of difference constraints**.

Theorem. *If (V, E, w) has a negative-weight cycle, then there exists no solution to the above system of difference constraints.*

Proof. Say $v_0 \rightarrow v_1 \rightarrow \cdots \rightarrow v_k \rightarrow v_0$ is a negative weight cycle.

Let us assume to the contrary that the system of difference constraints has a solution; let's call it h .

This gives us the following system of equations,

$$\begin{aligned} h(v_1) - h(v_0) &\leq w(v_0, v_1) \\ h(v_2) - h(v_1) &\leq w(v_1, v_2) \\ &\vdots \\ h(v_k) - h(v_{k-1}) &\leq w(v_{k-1}, v_k) \\ h(v_0) - h(v_k) &\leq w(v_k, v_0) \end{aligned}$$

Summing all these equations gives us

$$0 \leq w(\text{cycle}) < 0$$

which is obviously not possible.

From this, we can conclude that no solution to the above system of difference constraints exists if the graph (V, E, w) has a negative weight cycle.

□

Theorem. *If (V, E, w) has no negative-weight cycle, then we can find a solution to the difference constraints.*

Proof. Add a new vertex s to G , and add edges (s, v) of weight 0 for all $v \in V$.

- Clearly, these new edges do not introduce any new negative weight cycles to the graph
- Adding these new edges ensures that there now exists at least one path from s to v . This implies that $\delta(s, v)$ is finite for all $v \in V$
- We now claim that $h(v) = \delta(s, v)$. This is obvious from the triangle inequality:
 $\delta(s, u) + w(u, v) \geq \delta(s, v) \Leftrightarrow \delta(s, v) - \delta(s, u) \leq w(u, v) \Leftrightarrow h(v) - h(u) \leq w(u, v)$

□

Time complexity

1. The first step involves running Bellman-Ford from s , which takes $O(VE)$ time. We also pay a pre-processing cost to reweight all the edges ($O(E)$)

2. We then run Dijkstra's algorithm from each of the V vertices in the graph; the total time complexity of this step is $O(VE + V^2 \lg V)$
3. We then need to reweight the shortest paths for each pair; this takes $O(V^2)$ time.

The total running time of this algorithm is $O(VE + V^2 \lg V)$.

Applications

Bellman-Ford consult any system of difference constraints (or report that it is unsolvable) in $O(VE)$ time where V = variables and E = constraints.

An exercise is to prove the Bellman-Ford minimizes $\max_i x_i - \min_i x_i$.

This has applications to

- Real-time programming
- Multimedia scheduling
- Temporal reasoning

For example, you can bound the duration of an event via difference constraint $LB \leq t_{end} - t_{start} \leq UB$, or bound a gap between events via $0 \leq t_{start2} - t_{end1} \leq \varepsilon$, or synchronize events via $|t_{start1} - t_{start2}| \leq \varepsilon$ or 0.

MIT OpenCourseWare
<http://ocw.mit.edu>

6.046J / 18.410J Design and Analysis of Algorithms
Spring 2015

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Lecture 12: Greedy Algorithms and Minimum Spanning Tree

Introduction

- Optimal Substructure
- Greedy Choice Property
- Prim's algorithm
- Kruskal's algorithm

Definitions

Recall that a *greedy algorithm* repeatedly makes a locally best choice or decision, but ignores the effects of the future.

A *tree* is a connected, acyclic graph.

A *spanning tree* of a graph G is a subset of the edges of G that form a tree and include all vertices of G .

Finally, the *Minimum Spanning Tree* problem: Given an undirected graph $G = (V, E)$ and edge weights $W : E \rightarrow \mathbb{R}$, find a spanning tree T of minimum weight $\sum_{e \in T} w(e)$.

A naive algorithm

The obvious MST algorithm is to compute the weight of every tree, and return the tree of minimum weight. Unfortunately, this can take exponential time in the worst case. Consider the following example:

If we take the top two edges of the graph, the minimum spanning tree can consist of any combination of the left and right edges that connect the middle vertices to the left and right vertices. Thus in the worst case, there can be an exponential number of spanning trees.

Instead, we consider greedy algorithms and dynamic programming algorithms to solve MST. We will see that greedy algorithms can solve MST in nearly linear time.

Properties of Greedy Algorithms

Problems that can be solved by greedy algorithms have two main properties:

- Optimal Substructure: the optimal solution to a problem incorporates the optimal solution to subproblem(s)
- Greedy choice property: locally optimal choices lead to a globally optimal solution

We can see how these properties can be applied to the MST problem

Optimal substructure for MST

Consider an edge $e = \{u, v\}$, which is an edge of some MST. Then we can contract e by merging the vertices u and v to create a new vertex. Then any edge adjacent to vertex u or v is adjacent to the newly created vertex, and the process could result in a multiedge if u and v have a mutual neighbor. We resolve the multiedge problem by creating a single edge with the minimum weight between the two edges.

This leads us to the following lemma:

Lemma 1. *If T' is a minimum spanning tree of G/e , then $T' \cup \{e\}$ is an MST of G .*

Proof. Let T^* be an MST of G containing the edge e . Then T^*/e is a spanning tree of $G' = G/\{e\}$. By definition, T' is an MST of G' . Thus the total weight of T' is less than or equal to that of T^*/e , or $w(T') \leq w(T^*/e)$. Then

$$w(T) = w(T') + w(e) \leq w(T^*/e) + w(e) = w(T^*)$$

□

The statement can be used as the basis for a dynamic programming algorithm, in which we guess an edge that belongs to the MST, retract the edge, and recurse. At the end, we decontract the edge and add e to the MST.

The lemma guarantees that this algorithm is correct. However, this algorithm requires exponential time, because there are an exponential number of edges that we can guess to form our MST.

We make the algorithm polynomial time by removing the guessing process.

Greedy Choice Property

The MST problem can be solved by a greedy algorithm because the the locally optimal solution is also the globally optimal solution. This fact is described by the Greedy-Choice Property for MSTs, and its proof of correctness is given via a “cut and paste” argument common for greedy proofs.

Lemma 2 (Greedy-Choice Property for MST). *For any cut $(S, V \setminus S)$ in a graph $G = (V, E, w)$, any least-weight crossing edge $e = \{u, v\}$ with $u \in S$ and $v \notin S$ is in some MST of G .*

Proof. First, consider an MST T of G . Then T has a path from u to v . Because $u \in S$ and $v \notin S$, the path has some edge $e' = \{u', v'\}$ which also crosses the cut. Then $T' = T \setminus \{e'\} \cup \{e\}$ is a spanning tree of G and $w(T') = w(T) - w(e') + w(e)$, but e is a least-weight edge crossing $(S, V \setminus S)$. Thus $w(e) \leq w(e')$, so $w(T') \leq w(T)$. Therefore T' is an MST too. $\square$

Prim's Algorithm

Now, we can apply the insights from the optimal structure and greedy choice property to build a polynomial-time, greedy algorithm to solve the minimum spanning tree problem.

Prim's Algorithm Psuedocode

```

1  Maintain priority queue  $Q$  on  $V \setminus S$ , where  $v.key = \min\{w(u, v) \mid u \in S\}$ 
2   $Q = V$ 
3  Choose arbitrary start vertex  $s \in V$ ,  $s.key = \emptyset$ 
4  for  $v$  in  $V \setminus \{s\}$ 
5       $v.key = \infty$ 
6  while  $Q$  is not empty
7       $u = \text{Extract-Min}(Q)$ , add  $u$  to  $S$ 
8      for  $v \in \text{Adj}[u]$ 
9          if  $v \in Q$  and  $v \notin S$  and  $w(u, v) < v.key$ :
10              $v.key = w(u, v)$  (via a Decrease-Key operation)
11              $v.parent = u$ 
12 return  $\{\{v, v.parent\} \mid v \in V \setminus \{s\}\}$ 

```

In the above pseudocode, we choose an arbitrary start vertex, and attempt to sequentially reduce the distance to all vertices. After attempting to find the lowest

weight edge to connect all vertices, we return our MST

Correctness

We prove the correctness of Prim's Algorithm with the following invariants.

1. $v \notin S \implies v.key = \min\{w(u, v) \mid u \in S\}$
2. Tree T_S within $S \subseteq \text{MST of } G$.

The first invariant follows from Step 8 of the algorithm above. A proof of the second invariant follows:

Proof. Assume by induction that $T_S \subseteq \text{MST } T^*$. Then $S \rightarrow S' \cup \{e\}$, where e is a least-weight edge crossing the cut $(S, V \setminus S)$. Then we can greedily cut and paste e , which implies that we can modify T^* to include e without removing T_S , since the edges of T_S do not cross the cut. Therefore $T_S \cup \{e\} = T'_S \subseteq T^*$. $\square$

Thus Prim's Algorithm always adds edges that have the lowest weight and gradually builds a tree that is always a subset of some MST, and returns a correct answer.

Runtime

Prim's algorithm runs in

$$O(V) \cdot T_{\text{Extract-Min}} + O(E) \cdot T_{\text{Decrease-Key}}$$

The $O(E)$ term results from the fact that Step 8 is repeated a number of times equal to the sum of the number of adjacent vertices in the graph, which is equal to $2|E|$, by the handshaking lemma.

Then the effective runtime of the algorithm varies with the data structures used to implement the algorithm. The table below describes the runtime with the different implementations of the priority queue.

Priority Queue	$T_{\text{Extract-Min}}$	$T_{\text{Decrease-Key}}$	Total
Array	$O(V)$	$O(1)$	$O(V^2)$
Binary heap	$O(\lg V)$	$O(\lg V)$	$O(E \lg V)$
Fibonacci heap [CLRS ch. 19]	$O(\lg V)$ (amortized)	$O(1)$ (amortized)	$O(E + V \lg V)$

Kruskal's Algorithm

Kruskal's Algorithm is another algorithm to solve the MST problem. It constructs an MST by taking the globally lowest-weight edge and contracting it.

Kruskal's Algorithm Pseudocode

```
1  Maintain connected components that have been added to the
   MST so far  $T$ , in a Union-Find structure
2  Initialize  $T = \emptyset$ 
3  for  $v$  in  $V$ 
4      Make-Set( $v$ )
5  Sort  $E$  by weight
6  For  $e = (u, v) \in E$  (in increasing-weight order):
7      if Find-Set( $u$ )  $\neq$  Find-Set( $v$ ):
8          Add  $e$  to  $T$ 
9          Union( $u, v$ )
```

Correctness

We use the following invariant to prove the correctness of Kruskal's Algorithm.

Claim 3. *The tree-so-far $T \subseteq \text{MST } T^*$.*

Proof. We give an induction proof. We begin by assuming that the tree-so-far $T \subseteq T^*$, via the inductive hypothesis. When we add an edge e between some components C_1 and C_2 , we use the greedy-choice property on the cut $(C_1, V \setminus C_2)$. Thus we have added the edge without removing T , and our new tree-so-far remains a subset of the MST T^* . $\square$

Runtime

Kruskal's algorithm has an overall runtime of

$$T_{\text{sort}}(E) + O(V) \cdot T_{\text{Make-Set}} + O(E)(T_{\text{Find}} + T_{\text{Union}}) = O(E \lg E + E\alpha(V))$$

We note that $T_{\text{Make-Set}}$ is $O(1)$ and $T_{\text{find}} + T_{\text{Union}}$ is amortized $O(\alpha(V))$ for Union-Find data structures.

Furthermore, if all weights are integer weights, or all weights are in the range $[0, E^{O(1)}]$, then the runtime of the sorting step is $O(E)$, using Counting Sort or a similar algorithm, and the runtime of Kruskal's Algorithm will be better than that of Prim's Algorithm.

Other MST Algorithms

Currently, the fastest MST algorithm is a randomized algorithm with an expected runtime of $O(V + E)$. The algorithm was proposed by Karger, Klein, and Tarjan in 1993.

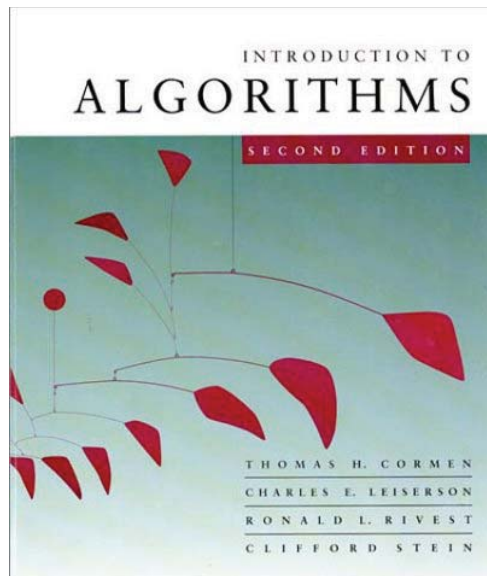
MIT OpenCourseWare
<http://ocw.mit.edu>

6.046J / 18.410J Design and Analysis of Algorithms
Spring 2015

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Design and Analysis of Algorithms

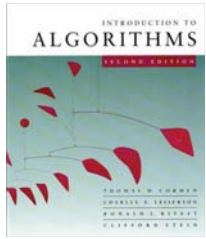
6.046J/18.401J



LECTURE 13

Network Flow

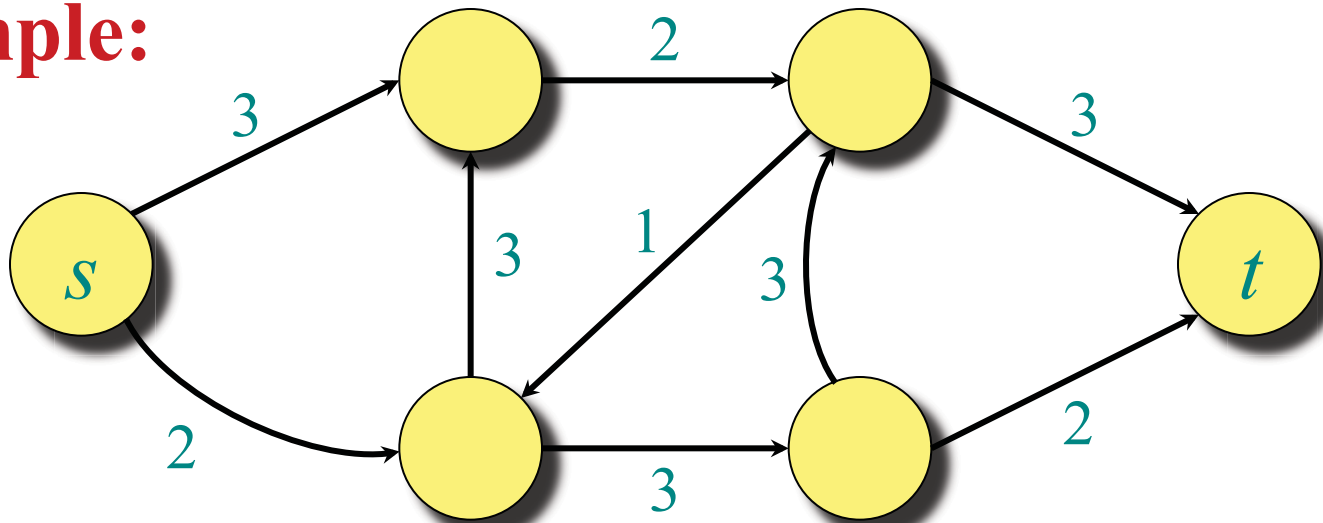
- Flow networks
- Maximum-flow problem
- Cuts
- Residual networks
- Augmenting paths
- Max-flow min-cut theorem
- Ford Fulkerson algorithm

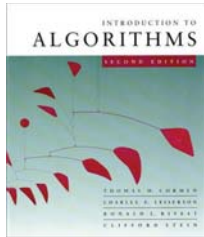


Flow networks

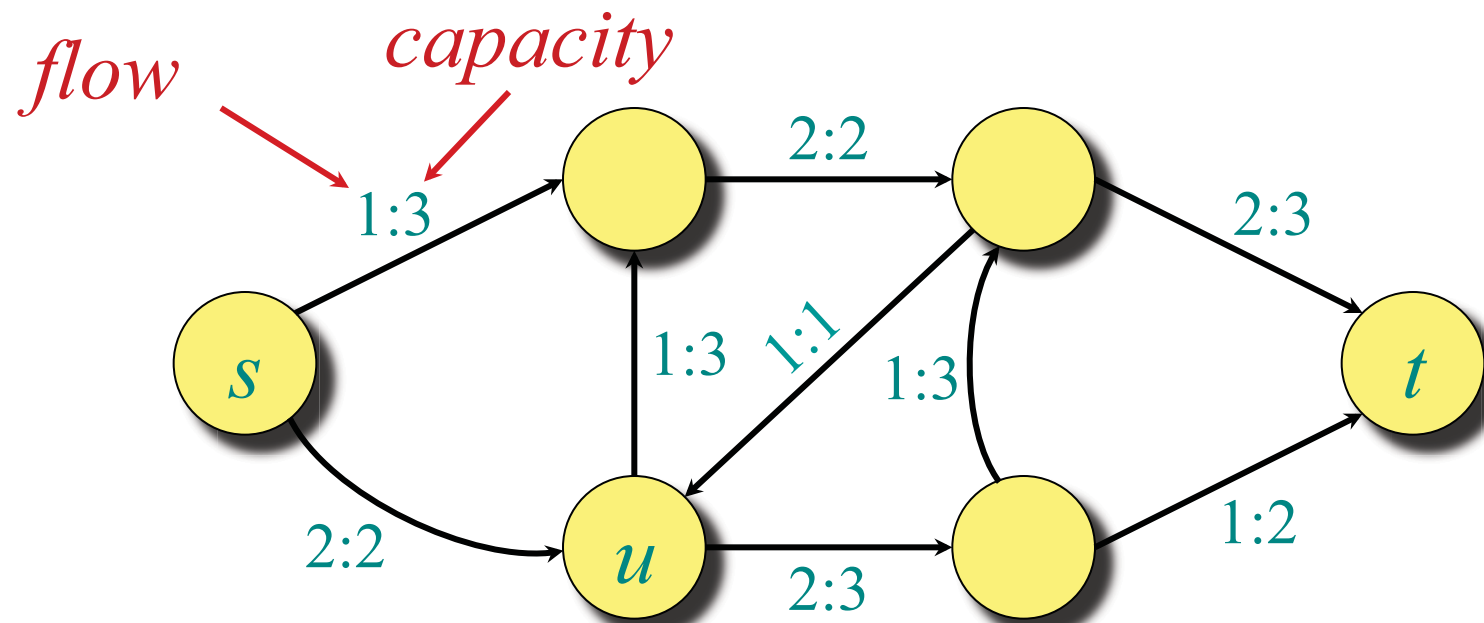
Definition. A *flow network* is a directed graph $G = (V, E)$ with two distinguished vertices: a *source* s and a *sink* t . Each edge $(u, v) \in E$ has a nonnegative *capacity* $c(u, v)$. If $(u, v) \notin E$, then $c(u, v) = 0$.

Example:





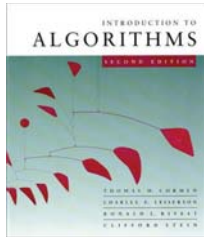
A flow on a network



Flow conservation (like Kirchhoff's current law):

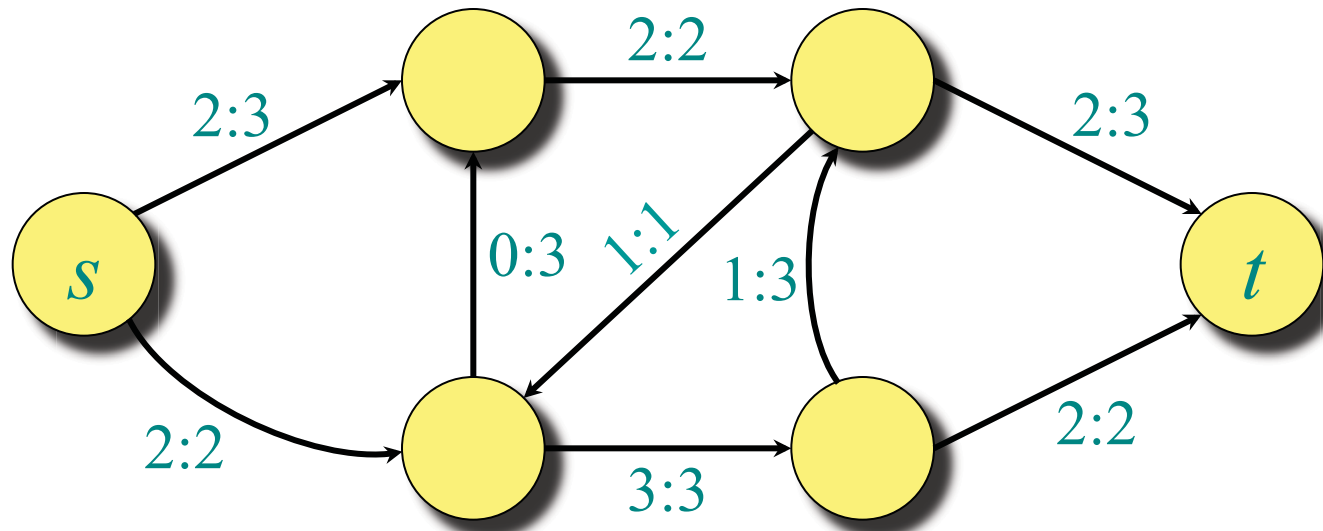
- Flow into u is $2 + 1 = 3$.
- Flow out of u is $1 + 2 = 3$.

INTUITION: View flow as a *rate*, not a *quantity*.

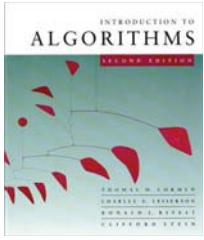


The maximum-flow problem

Maximum-flow problem: Given a flow network G , find a flow of maximum value on G .



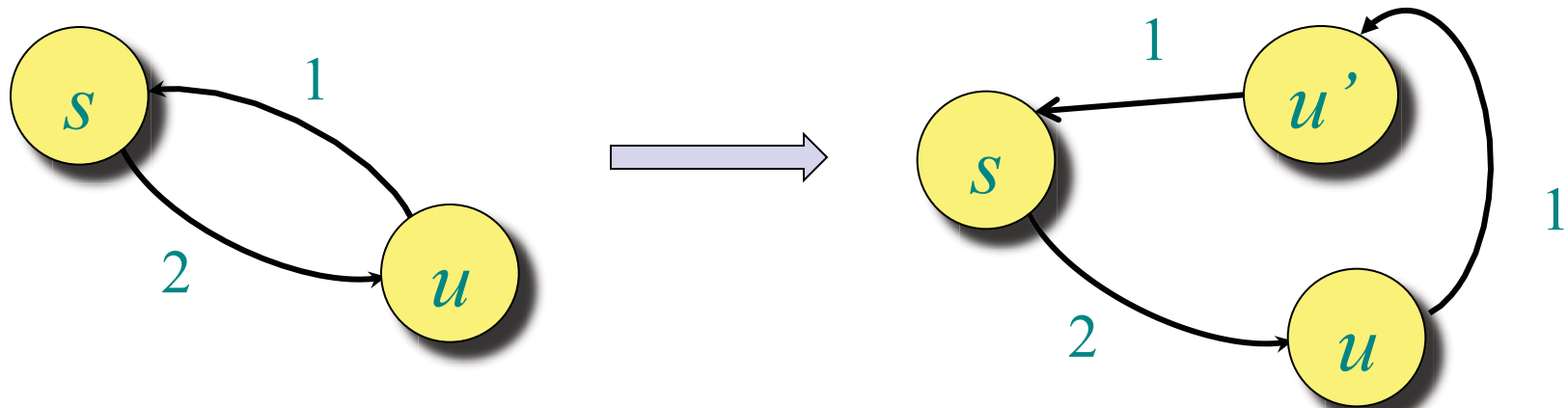
The value of the maximum flow is 4.

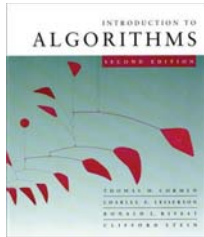


Flow network Assumptions

Assumption. If edge $(u, v) \in E$ exists, then $(v, u) \notin E$.

Assumption. No self-loop edges (u, u) exist





Net Flow

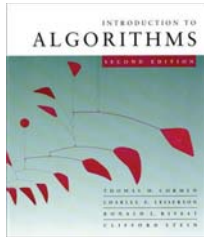
Definition. A *(net) flow* on G is a function $f : V \times V \rightarrow \mathbb{R}$ satisfying the following:

- **Capacity constraint:** For all $u, v \in V$,
$$f(u, v) \leq c(u, v).$$
- **Flow conservation:** For all $u \in V - \{s, t\}$,

$$\sum_{v \in V} f(u, v) = 0.$$

- **Skew symmetry:** For all $u, v \in V$,
$$f(u, v) = -f(v, u).$$

Note: CLRS defines positive flows and net flows; these are equivalent for our flow networks obeying our assumptions.



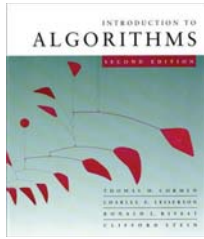
Notation

Definition. The *value* of a flow f , denoted by $|f|$, is given by

$$\begin{aligned}|f| &= \sum_{v \in V} f(s, v) \\ &= f(s, V).\end{aligned}$$

Implicit summation notation: A set used in an arithmetic formula represents a sum over the elements of the set.

- **Example** — flow conservation:
 $f(u, V) = 0$ for all $u \in V - \{s, t\}$.



Simple properties of flow

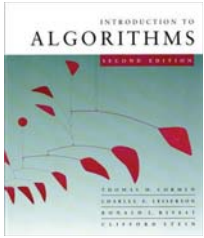
Lemma.

- $f(X, X) = 0$,
- $f(X, Y) = -f(Y, X)$,
- $f(X \cup Y, Z) = f(X, Z) + f(Y, Z)$ if $X \cap Y = \emptyset$. □

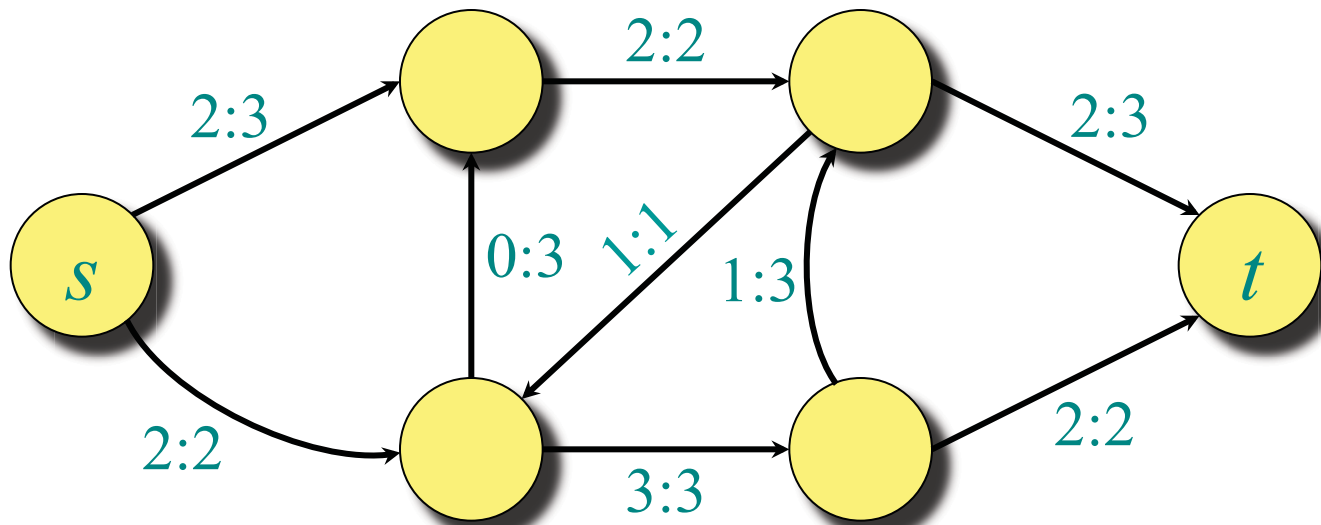
Theorem. $|f| = f(V, t)$.

Proof.

$$\begin{aligned} |f| &= f(s, V) \\ &= f(V, V) - f(V-s, V) && \text{Omit braces.} \\ &= f(V, V-s) \\ &= f(V, t) + f(V, V-s-t) \\ &= f(V, t). \quad \text{□} \end{aligned}$$

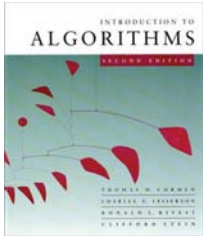


Flow into the sink



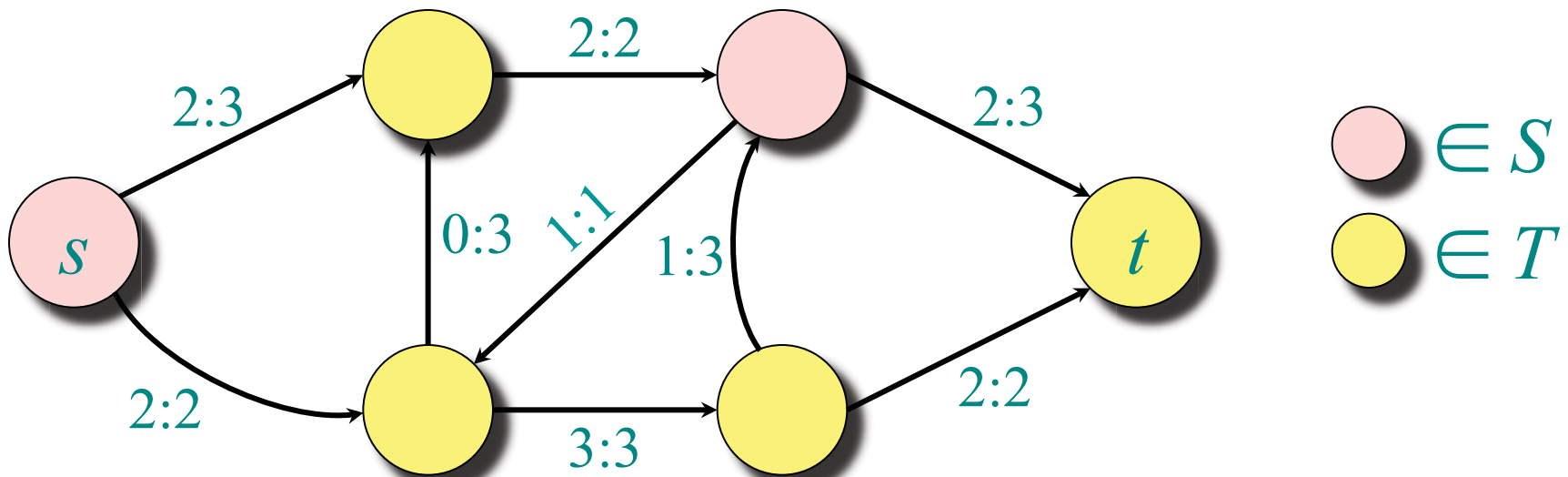
$$|f| = f(s, V) = 4$$

$$f(V, t) = 4$$

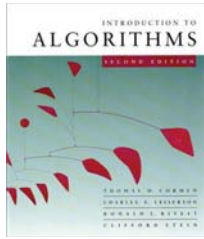


Cuts

Definition. A *cut* (S, T) of a flow network $G = (V, E)$ is a partition of V such that $s \in S$ and $t \in T$. If f is a flow on G , then the *flow across the cut* is $f(S, T)$.



$$f(S, T) = (2 + 2) + (-2 + 1 - 1 + 2) = 4$$

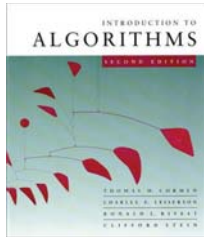


Another characterization of flow value

Lemma. For any flow f and any cut (S, T) , we have $|f| = f(S, T)$.

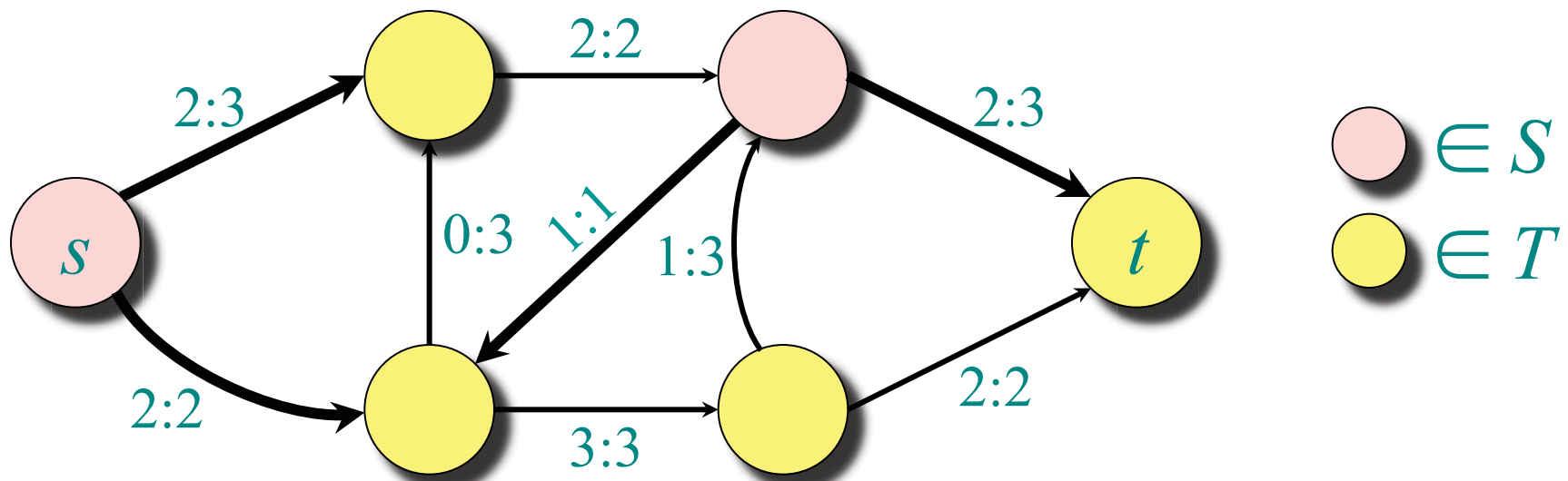
Proof.

$$\begin{aligned} f(S, T) &= f(S, V) - f(S, S) \\ &= f(S, V) \\ &= f(s, V) + f(S-s, V) \\ &= f(s, V) \\ &= |f|. \quad \square \end{aligned}$$

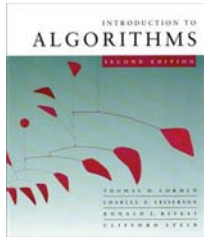


Capacity of a cut

Definition. The *capacity of a cut* (S, T) is $c(S, T)$.



$$c(S, T) = (3 + 2) + (1 + 3) = 9$$



Upper bound on the maximum flow value

Theorem. The value of any flow is bounded above by the capacity of any cut.

Proof.

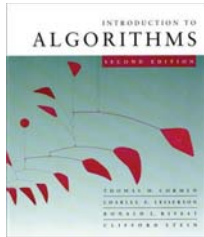
$$|f| = f(S, T)$$

$$= \sum_{u \in S} \sum_{v \in T} f(u, v)$$

$$\leq \sum_{u \in S} \sum_{v \in T} c(u, v)$$

$$= c(S, T) .$$





Residual network

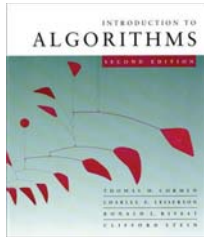
Definition. Let f be a flow on $G = (V, E)$. The *residual network* $G_f(V, E_f)$ is the graph with strictly positive *residual capacities*

$$c_f(u, v) = c(u, v) - f(u, v) > 0.$$

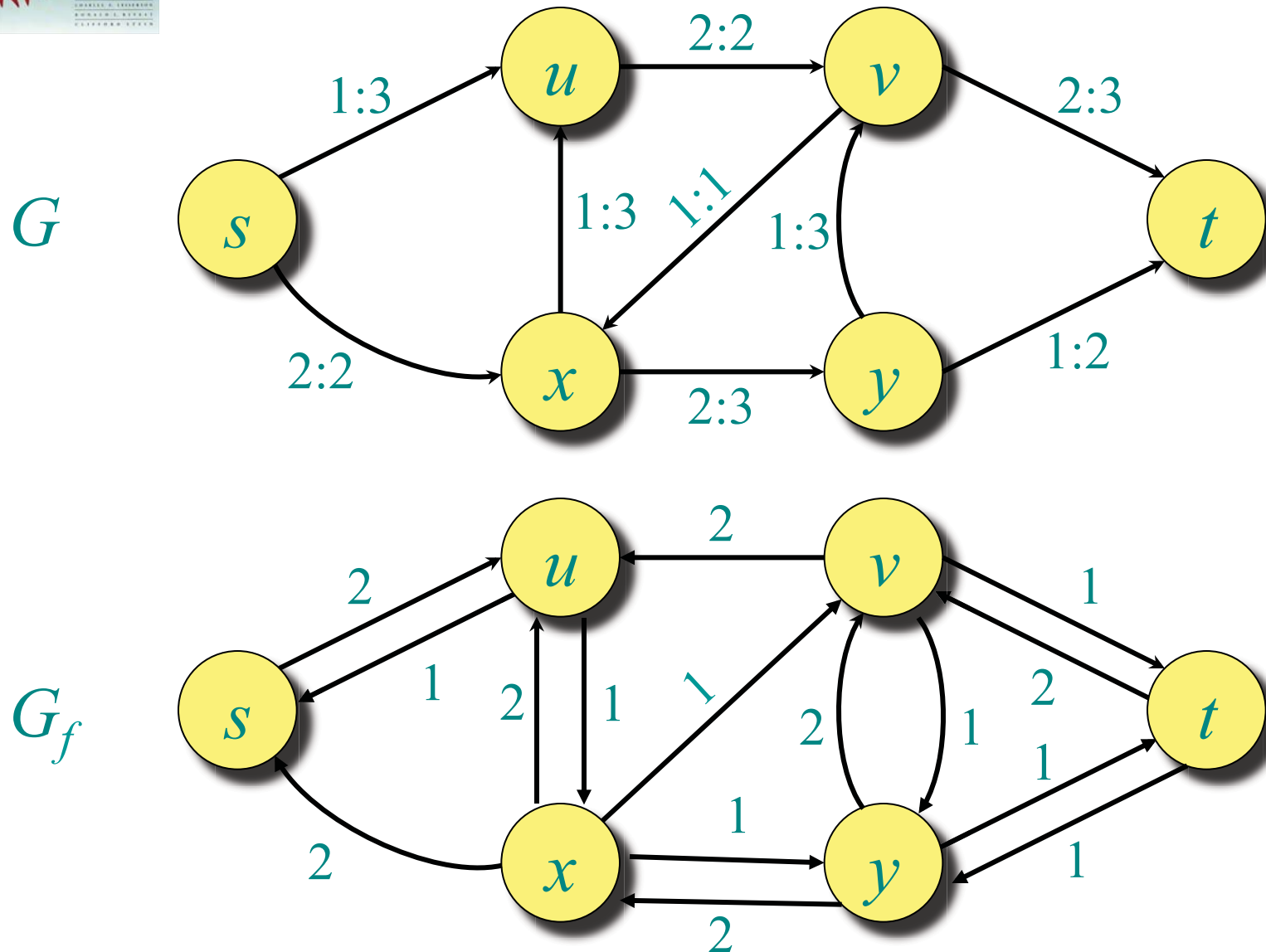
Edges in E_f admit more flow.

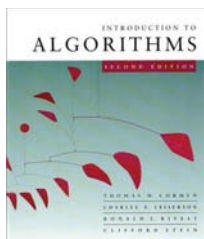
If $(v, u) \notin E$, $c(v, u) = 0$, but $f(v, u) = -f(u, v)$.

$$|E_f| \leq 2 |E|.$$



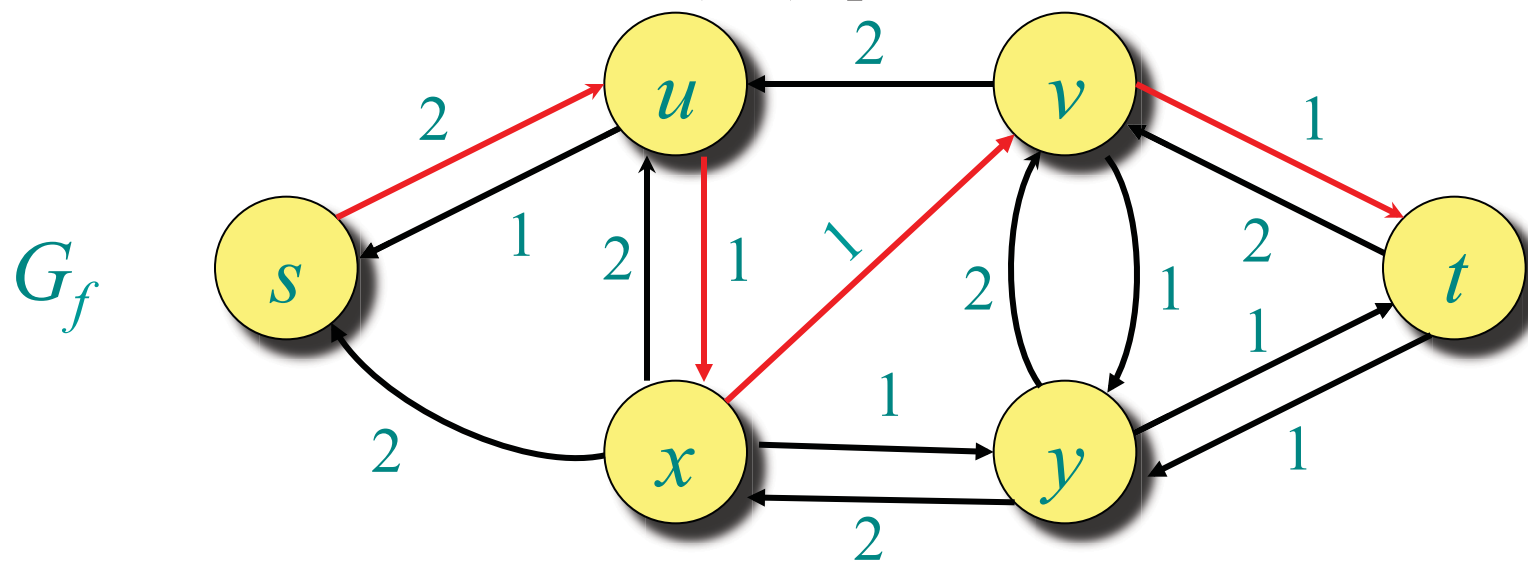
Flow and Residual Network



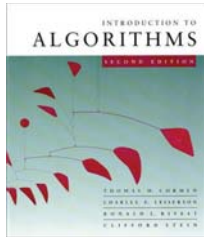


Augmenting paths

Definition. Any path from s to t in G_f is an *augmenting path* in G with respect to f . The flow value can be increased along an augmenting path p by $c_f(p) = \min_{(u,v) \in p} \{c_f(u,v)\}$.

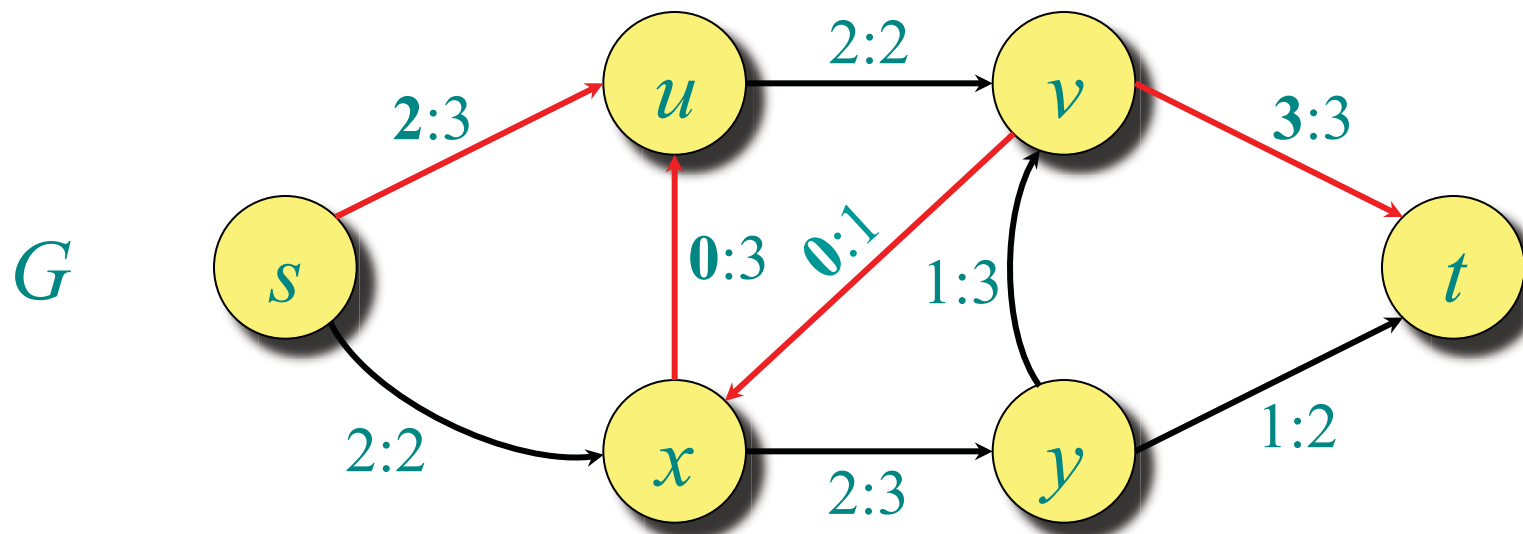


$$p = \{s, u, x, v, t\}, \quad c_f(p) = 1$$



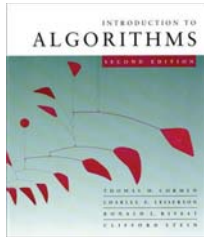
Augmented Flow Network

$$p = \{s, u, x, v, t\}, \quad c_f(p) = 1$$



The value of the maximum flow is 4.

Note: Some flows on edges *decreased*.

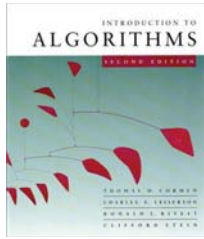


Max-flow, min-cut theorem

Theorem. The following are equivalent:

1. $|f| = c(S, T)$ for some cut (S, T) .
2. f is a maximum flow.
3. f admits no augmenting paths.

Proof. Next time!



Ford-Fulkerson max-flow algorithm

Algorithm:

$f[u, v] \leftarrow 0$ for all $u, v \in V$

while an augmenting path p in G wrt f exists

do augment f by $c_f(p)$

MIT OpenCourseWare
<http://ocw.mit.edu>

6.046J / 18.410J Design and Analysis of Algorithms
Spring 2015

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Lecture 15: Linear Programming

Linear programming (LP) is a method to achieve the optimum outcome under some requirements represented by linear relationships. More precisely, LP can solve the problem of maximizing or minimizing a linear objective function subject to some linear constraints.

In general, the standard form of LP consists of

- Variables: $\mathbf{x} = (x_1, x_2, \dots, x_d)^\top$
- Objective function: $\mathbf{c} \cdot \mathbf{x}$
- Inequalities (constraints): $A\mathbf{x} \leq \mathbf{b}$, where A is a $n \times d$ matrix

and we maximize the objective function subject to the constraints and $\mathbf{x} \geq 0$.

LP has many different applications, such as flow, shortest paths, and even politics. In this lecture, we will be covering different examples of LP, and present an algorithm for solving them. We will also learn how to convert any LP to the standard form in this lecture.

1 Examples of Linear Programming: Politics

In this example, we will be studying how to campaign to win an election. In general, there are n demographics, each with p_i people, and m issues that the voters are interested in. Given the information on how many votes can be obtained per dollar spent advertising in support of an issue, how can we guarantee victory by ensuring a majority vote in all demographics?¹

In particular, consider the example with 3 demographics and 4 issues shown in Table 1. What is the minimum amount of money we can spend to guarantee majority in all demographics?

Let x_1, x_2, x_3, x_4 denote the dollars spent per issue. We can now formulate this problem as an LP problem:

$$\begin{aligned}
 &\text{Minimize } x_1 + x_2 + x_3 + x_4 \\
 &\text{Subject to } -2x_1 + 8x_2 + 0x_3 + 10x_4 \geq 50,000 \text{ (Urban Majority)} & (1) \\
 &\quad 5x_1 + 2x_2 + 0x_3 + 0x_4 \geq 100,000 \text{ (Suburban Majority)} & (2) \\
 &\quad 3x_1 - 5x_2 + 10x_3 - 2x_4 \geq 25,000 \text{ (Rural Majority)} & (3) \\
 &\quad x_1, x_2, x_3, x_4 \geq 0 \text{ (Can't unadvertise)} & (4)
 \end{aligned}$$

¹We will assume that the votes obtained by advertising different issues are disjoint.

Policy	Demographic		
	Urban	Suburban	Rural
Building roads	-2	5	3
Gun Control	8	2	-5
Farm Subsidies	0	0	10
Gasoline Tax	10	0	2
Population	100,000	200,000	50,000

Table 1: Votes per dollar spent on advertising, and population.

1.1 Certificate of Optimality

Though we have not presented an algorithm for solving this problem, given a solution, we can verify that the solution is optimal with a proper certificate. For example,

$$\begin{aligned}
 x_1 &= \frac{2050000}{111} \\
 x_2 &= \frac{425000}{111} \\
 x_3 &= 0 \\
 x_4 &= \frac{625000}{111} \\
 x_1 + x_2 + x_3 + x_4 &= \frac{3100000}{111}
 \end{aligned}$$

is a solution to this problem. Now consider the following equation (certificate):

$$\begin{aligned}
 \frac{25}{222} \cdot (1) + \frac{46}{222} \cdot (2) + \frac{14}{222} \cdot (3) &= x_1 + x_2 + \frac{140}{222}x_3 + x_4 \\
 &\geq \frac{25}{222} \cdot 50000 + \frac{46}{222} \cdot 100000 + \frac{14}{222} \cdot 25000 \\
 &= \frac{3100000}{111} \\
 \Rightarrow x_1 + x_2 + \frac{140}{222}x_3 + x_4 &\geq \frac{3100000}{111}
 \end{aligned}$$

We also know that $x_1 + x_2 + x_3 + x_4 \geq x_1 + x_2 + \frac{140}{222}x_3 + x_4$. Therefore, the given solution is an optimal solution to the problem.

2 Linear Programming Duality

The short certificate provided in the last section is not a coincidence, but a consequence of *duality* of LP problems. For every primal LP problem in the form of

$$\begin{array}{ll}\text{Maximize} & \mathbf{c} \cdot \mathbf{x} \\ \text{Subject to} & A\mathbf{x} \leq \mathbf{b}, \mathbf{x} \geq 0,\end{array}$$

there exists an equivalent dual LP problem

$$\begin{array}{ll}\text{Minimize} & \mathbf{b} \cdot \mathbf{y} \\ \text{Subject to} & A^\top \mathbf{y} \geq \mathbf{c}, \mathbf{y} \geq 0.\end{array}$$

This property of LP can be used to show many important theorems. For instance, the max-flow min-cut theorem can be proven by formulating the max-flow problem as the primal LP problem.

3 Converting to Standard Form

The natural LP formulation of a problem may not result in the standard LP form. In these cases, we can convert the problem to standard LP form without affecting the answers by using the following rules.

- **Minimize an objective function:** Negate the coefficients and maximize.
- **Variable x_j does not have a non-negativity constraint:** Replace x_j with $x'_j - x''_j$, and $x'_j, x''_j \geq 0$.
- **Equality constraints:** Split into two different constraints; $x = b \Rightarrow x \leq b, x \geq b$.
- **Greater than or equal to constraints:** Negate the coefficients, and translate to less than or equal to constraint.

4 Formulating LP Problems

In this section, we will give brief descriptions of how to formulate some problems seen previously in this class as LP problems. Once we have a LP formulation, we can convert the problem into the standard form as described in Section 3.

4.1 Maximum Flow

We can model the max flow problem as maximization of sum of flows, under some constraints which will model different properties of the flow. Given $G(V, E)$, the capacity $c(e)$ for each $e \in E$, the source s , and the sink t ,

$$\begin{aligned} &\text{Maximize } \sum_{v \in V} f(s, v) \\ &\text{Subject to } f(u, v) = -f(v, u) \quad \forall u, v \in V \quad \text{skew symmetry} \\ &\quad \sum_{v \in V} f(u, v) = 0 \quad \forall u \in V - \{s, t\} \quad \text{conservation} \\ &\quad f(u, v) \leq c(u, v) \quad \forall u, v \in V \quad \text{capacity.} \end{aligned}$$

4.2 Shortest Paths

We can model the shortest paths problem as minimization of the sum of all distances from a node. Note that this sum is minimized only when all distances are minimized. Given $G(V, E)$, the weight $w(e)$ for each $e \in E$, and the source s ,

$$\begin{aligned} &\text{Minimize } \sum_{v \in V} d(v) \\ &\text{Subject to } d(v) - d(u) \leq w(u, v) \quad \forall u, v \in V \quad \text{triangular inequality} \\ &\quad \sum_{v \in V} d(s) = 0. \end{aligned}$$

Note the maximization above, so all distances don't end up being zero. There is no solution to this LP if and only if there exists a negative weight cycle reachable from s .

5 Algorithms for LP

There are many algorithms for solving LP problems:

- **Simplex algorithm:** In the feasible region, $\mathbf{x}$ moves from vertex to vertex in direction of $\mathbf{c}$. The algorithm is simple, but runs in exponential time in the worst case.
- **Ellipsoid algorithm:** It starts with an ellipsoid that includes the optimal solution, and keeps shrinking the ellipsoid until the optimal solution is found. This was the first poly-time algorithm, and was a theoretical breakthrough. However, the algorithm is impractical in practice.

- **Interior Point Method:** $\mathbf{x}$ moves inside the polytope following $\mathbf{c}$. This algorithm runs in poly-time, and is practical.

In this lecture, we will study only the simplex algorithm.

5.1 Simplex Algorithm

As mentioned before, the simplex algorithm works well in practice, but runs in exponential time in the worst case. At a high level, the algorithm works as Gaussian elimination on the inequalities or constraints. The simplex algorithm works as follows:

- Represent LP in “slack” form.
- Convert one slack form into an equivalent slack form, while likely increasing the value of the objective function, and ensuring that the value does not decrease.
- Repeat until the optimal solution becomes apparent.

5.1.1 Simplex Example

Consider the following example:

$$\begin{aligned} &\text{Minimize } 3x_1 + x_2 + x_3 \\ &\text{Subject to } x_1 + x_2 + 3x_3 \leq 30 \\ &\quad 2x_1 + 2x_2 + 5x_3 \leq 24 \\ &\quad 4x_1 + x_2 + 2x_3 \leq 36 \\ &\quad x_1, x_2, x_3 \geq 0 \end{aligned}$$

Change the given LP problem to slack form, consisting of the original variables called *nonbasic* variables, and new variables representing slack called *basic* variables.

$$\begin{aligned} z &= 3x_1 + x_2 + 2x_3 \\ x_4 &= 30 - x_1 - x_2 - 3x_3 \\ x_5 &= 24 - 2x_1 - 2x_2 - 5x_3 \\ x_6 &= 36 - 4x_1 - x_2 - 2x_3 \end{aligned}$$

We start with a *basic solution*: we set all nonbasic variables on the right hand side to some feasible value, and compute the values of the basic variables. For instance, we can set $x_1 = x_2 = x_3 = 0$. Note that the all 0 solution satisfies all constraints in this problem, but may not do so in the general case.

We now perform the pivoting step:

- Select a nonbasic variable x_e whose coefficient in the objective function is positive.
- Increase the value of x_e as much as possible without violating any constraints.
- Set x_e to be basic, while some other basic variable becomes nonbasic.

In this example, we can increase the value of x_1 . The third constraint will limit the value of x_1 to 9. We then get

$$x_1 = 9 - \frac{x_2}{4} - \frac{x_3}{2} - \frac{x_6}{4}.$$

Now rewrite the other constraints with x_6 on the right hand side.

$$\begin{aligned} z &= 27 + \frac{x_2}{4} + \frac{x_3}{2} - \frac{3x_6}{4} \\ x_1 &= 9 - \frac{x_2}{4} - \frac{x_3}{2} - \frac{x_6}{4} \\ x_4 &= 21 - \frac{3x_2}{4} - \frac{5x_3}{2} + \frac{x_6}{4} \\ x_5 &= 6 - \frac{3x_2}{2} - 4x_3 + \frac{x_6}{2} \end{aligned}$$

We note the equivalence of the solutions. That is, the original basic solution $(0, 0, 0, 30, 24, 36)$ satisfies the rewritten constraints, and has the objective value of 0. The second basic solution $(9, 0, 0, 21, 6, 0)$ has the objective value of 27.

At this point, pivoting on x_6 will actually cause the objective value to decrease (though the computation is not shown here). Thus let us pick x_3 as the next pivot to get

$$\begin{aligned} z &= \frac{111}{4} + \frac{x_2}{16} - \frac{x_5}{8} - \frac{11x_6}{16} \\ x_1 &= \frac{33}{4} - \frac{x_2}{16} + \frac{x_5}{8} - \frac{5x_6}{16} \\ x_2 &= \frac{3}{2} - \frac{3x_2}{8} - \frac{x_5}{4} + \frac{x_6}{8} \\ x_4 &= \frac{69}{4} + \frac{3x_2}{16} + \frac{5x_5}{8} - \frac{x_6}{16} \end{aligned}$$

which results in basic solution $(\frac{33}{4}, 0, \frac{3}{2}, \frac{69}{4}, 0, 0)$ with objective value of $\frac{111}{4}$.

Finally, pivoting on x_2 yields

$$\begin{aligned}z &= 28 - \frac{x_3}{6} - \frac{x_5}{6} - \frac{2x_6}{3} \\x_1 &= 8 + \frac{x_3}{6} + \frac{x_5}{6} - \frac{x_6}{3} \\x_2 &= 4 - \frac{8x_3}{3} - \frac{2x_5}{3} + \frac{x_6}{3} \\x_4 &= 18 - \frac{x_3}{2} + \frac{x_5}{2}\end{aligned}$$

Though we will not prove the correctness of this algorithm in this lecture, when all coefficients of all nonbasic variables are negative, the simplex algorithm has found the optimal solution. In general, simplex algorithm is guaranteed to converge in $\binom{n+m}{n}$ iterations where n is the number of variables, and $n+m$ is the number of constraints. This for general n and m can be exponential.

6 More Topics of LP

There are several important questions regarding LP that were not discussed in this lecture:

- How do we determine if LP is feasible?
- What if LP is feasible, but the initial basic solution is infeasible?
- How do we determine if the LP is unbounded?
- How do we choose the pivot?

These questions are answered in the textbook and other LP literature.

MIT OpenCourseWare
<http://ocw.mit.edu>

6.046J / 18.410J Design and Analysis of Algorithms
Spring 2015

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Lecture 16: NP-Completeness

Introduction

- NP-hardness and NP-completeness
- 3SAT
 - ↳ Super Mario Brothers
 - ↳ 3 Dimensional Matching
 - ↳ Subset Sum (*weak*)
 - ↳ Partition (*weak*)
 - ↳ Rectangle Packing (*weak*)
 - ↳ 4-Partition (*strong*)
 - ↳ Rectangle Packing (*strong*)
 - ↳ Jigsaw Puzzles

NP-Hard and NP-Complete problems

Today, we discuss NP-Completeness.

Recall from 6.006:

- **P** = the set of problems that are solvable in polynomial time. If the problem has size n , the problem should be solved in $n^{O(1)}$.
- **NP** = the set of decision problems solvable in nondeterministic polynomial time. The output of these problems is a YES or NO answer. Nondeterministic refers to the fact that a solution can be guessed out of polynomially many options in $O(1)$ time. If any guess is a YES instance, then the nondeterministic algorithm will make that guess.

In this model of nondeterminism, we can assume that all guessing is done first. This is equivalent to finding a polynomial-time verifier of polynomial-size certificates for YES answers.

Note that there is an asymmetry between YES and NO inputs

- A problem X is **NP-complete** if $X \in \text{NP}$ and X is NP-hard.

- A problem X is NP-hard if every problem $Y \in \text{NP}$ reduces to X .
 - If $P \neq \text{NP}$, then $X \notin P$.
- A **reduction** from problem A to problem B is a polynomial-time algorithm that converts inputs to problem A into equivalent inputs to problem B . Equivalent means that both problem A and problem B must output the same YES or NO answer for the input and converted input.
 - If $B \in P$, then $A \in \text{NP}$
 - If $B \in \text{NP}$, then $A \in \text{NP}$
 - If A is NP-hard, then B is NP-hard.

We can show that problems are NP-complete via the following steps.

1. **Show $X \in \text{NP}$.** Show that $X \in \text{NP}$ by finding a nondeterministic algorithm, or giving a valid verifier for a certificate.
2. **Show X is NP-hard.** Reduce from a known NP-complete problem Y to X . This is sufficient because all problems $Z \in \text{NP}$ can be reduced to Y , and the reduction demonstrates inputs to Y can be modified to become inputs to X , which implies X is NP-hard. We must demonstrate the following properties for a complete reduction.
 - (a) Give an polynomial-time conversion from Y inputs to X inputs.
 - (b) If Y 's answer is YES, then X 's answer is YES.
 - (c) If X 's answer is YES, then Y 's answer is YES.

Finally, a **gadget** transforms features in an input problem to a feature in an output problem.

3SAT

The 3SAT was discovered to be NP-complete by Cook in 1971.

Definition 1. 3SAT: Given a boolean formula of the form:

$$(x_1 \vee x_3 \vee \bar{x}_6) \wedge (\bar{x}_2 \vee x_3 \vee \bar{x}_7) \wedge \dots$$

is there an assignment of variables to True and False, such that the entire formula evaluates to True?

We note that a literal is of the form $\{x_i, \bar{x}_i\}$, and both forms of the literal correspond to the variable x_i . A clause is made up of the OR of 3 literals, and a formula is the AND of clauses.

3SAT $\in$ NP because we can create a verifier for a certificate. For a given instance of 3SAT, a certificate corresponds to a list of assignments for each variable, and a verifier can compute whether the instances is satisfied, or can be evaluated to true. Thus the verifier is polynomial time, and the certificate has polynomial length.

It is important to note that this verifier only guarantees that a 3SAT instance is verifiable. To ensure that a 3SAT instance is not verifiable, the algorithm would have to check every variable assignment, which cannot be done in polynomial time.

3SAT is also NP-hard. We give some intuition for this result. Consider any problem in NP. Because it belongs in NP, a nondeterministic polynomial time algorithm exists to solve this problem, or a verifier to check a solution. The verifier is an algorithm that can be implemented as a circuit. Now, the circuit consists of AND, OR and NOT gates, which can be represented as a formula. This formula can be converted to 3SAT form, where each clause has 3 literals, which is equivalent to the original formula. Thus all problems in NP can be converted to 3SAT, and the inputs to the original problem are equivalent to the converted inputs to 3SAT, thus 3SAT is NP-complete.

Super Mario Brothers [Aloupis, Demaine, Guo, Viglietta 2014]

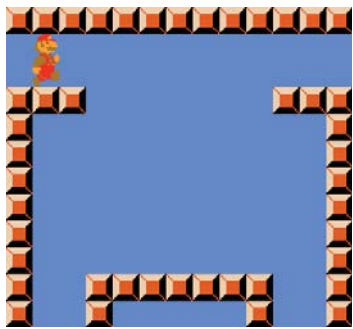
We show that Super Mario Brothers is NP-hard by giving a reduction from 3SAT. This version of Super Mario Brothers is generalized to an arbitrary screen size of $n \times n$, so we remove limits on the number of items on screen. We have the following problem definition

Definition 2. Super Mario Brothers: *Given a level of Super Mario Brothers, can we advance to the next level?*

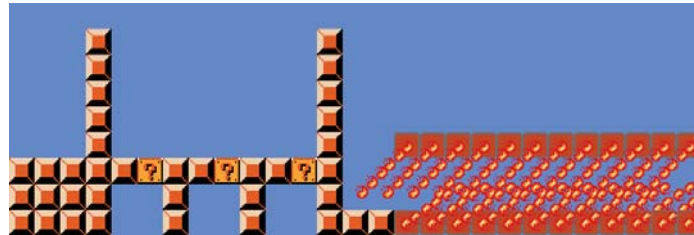
Because we reduce from 3SAT, we are given a 3SAT instance, and we must generate a level of Super Mario Brothers that corresponds to that 3SAT instance.

We construct the level by constructing gadgets for each of the variables in the 3SAT formula, as pictured in Figure 1. Mario jumps down from the ledge, and cannot jump back up. He can fall to the left or right, corresponding to assigning the variable to True or False. The remainder of the level is set up such that this choice cannot be reversed.

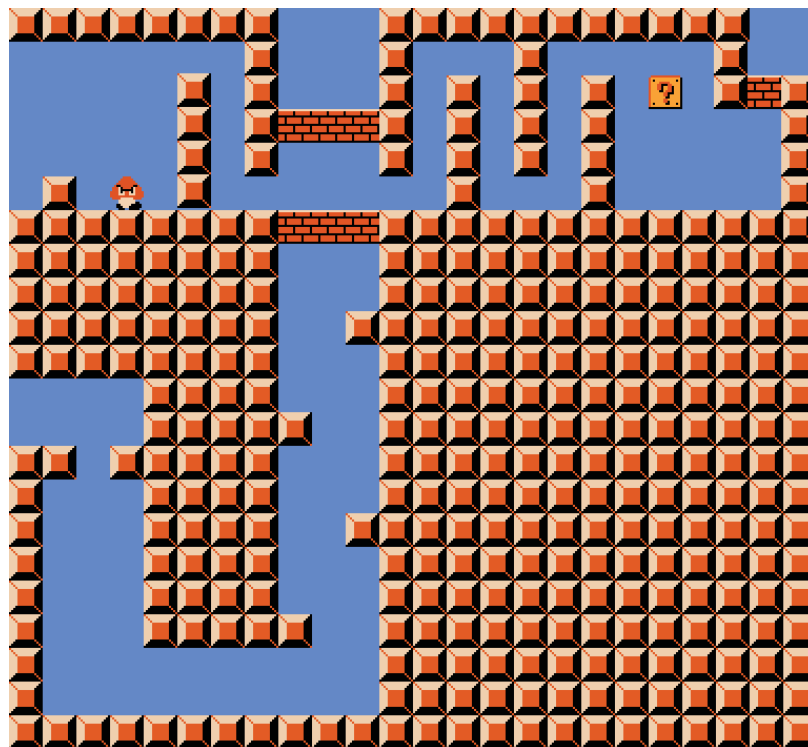
We also create the following gadget for clauses. After choosing the assignment for a given variable, Mario visits all of the clause gadgets with the same literal value,



(a) Variable gadget



(b) Clause gadget



(c) Crossover gadget

Figure 1: Gadgets for Super Mario Brothers.

© Nintendo. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <http://ocw.mit.edu/fairuse>.

then moves to the next variable gadget. By visiting a clause, Mario can release a star.

Finally, after visiting all of the variable gadgets, Mario must re-traverse the clause gadgets. If the clause gadget was previously visited, a star is available, and he can pass through the fire. Otherwise, he will not be able to traverse the clause gadget and die.

Thus, winning the level is equivalent to passing through all the clause gadgets on the second pass through. Mario can only pass all the clause gadgets if they have all been satisfied by a variable assignment. The actions throughout the variable gadgets correspond to the solution to the 3SAT formula, so if Mario can pass the level, we have a solution to the 3SAT problem.

The final gadget needed is the crossover gadget. It ensures that Mario does not switch between variable and clause gadgets when it is not allowed. The total size of all these gadgets within the polynomial size required by the reduction.

Thus, Mario can win the level if and only if the original 3SAT formula could be satisfied. Therefore have a reduction from 3SAT, and Super Mario Brothers is NP-hard.

3 Dimensional Matching (3DM)

Definition 3. 3DM: Given disjoint sets X , Y , and Z , each of n elements and triples $T \subseteq X \times Y \times Z$ is there a subset $S \subseteq T$ such that each element $\in X \cup Y \cup Z$ is in exactly one $s \in S$?

3DM is NP. Given a certificate, which lists a candidate list of triples, a verifier can check that each triple belongs to T and every element of $X \cup Y \cup Z$ is in one triple.

3DM is also NP-complete, via a reduction from 3SAT. We build gadgets for the variables and clauses.

The variable gadget for variable x_i is displayed in the picture. Only red or blue triangles can be chosen, where the red triangles correspond to the true literal, while the blue triangles correspond to the false literal. Otherwise, there will be overlap, or some inner elements will not be covered. There is an $2n_{x_i}$ “wheel” for each variable gadget, where n_{x_i} corresponds to the number of occurrences of x_i in the formula.

The clause gadget for the clause $x_i \wedge \bar{x}_j \wedge x_k$ is displayed in the picture. The dot that is unshared in each variable’s triangle within the clause gadget is also the single dot within the variable gadget.

Then, if we set x_i to true, we take all of the red false triangles in the variable gadget, leaving a blue true triangle available to cover the clause gadget. However, this still potentially leaves $\bar{x}_j$ and x_k uncovered, so we need a garbage collection

gadget, pictured below. There are $\sum_x n_x$ clauses of these gadgets, because there are n_x unnecessary elements of the variable gadget that won't be covered. However, of the remaining elements, one of these per clause will be covered, so the remaining need to be covered by the garbage collection clause.

Thus, if a solution exists to the 3SAT formula, we can find the solution to the 3DM problem by choosing the points in 3DM corresponding to the variable values. If we have a 3DM solution, we can map this back to the satisfying assignment for 3SAT. Thus, our reduction is complete and 3DM is NP-hard.

Subset Sum

Definition 4. Subset Sum Given n integers $A = \{a_1, a_2, \dots, a_n\}$ and a target sum t , is there a subset $S \subseteq A$ such that

$$\sum_{a_i \in S} a_i = t$$

The Subset Sum problem is NP-complete. It is in NP, because a verifier can simply check that the given subset is a subset of A and that its sum is equivalent to the target in polynomial time.

It is NP-hard via a reduction from 3DM. View the numbers in base $b = 1 + \max_i n_{x_i}$. Then the triple (x_i, x_j, x_k) can be written in base b in the form $000100100001000 = b^i + b^j + b^k$, where the first 1 corresponds to i , the second to j , and the third to k . The target sum $t = 11111111111111 = \sum_i b^i$.

This prevents any 1s from colliding, which ensures that each element is used exactly once, because multiple 1s correspond to reusing an element, and the base is sufficiently large so that no smaller numbers can be summed to create the next power of b . This completes the reduction.

In fact, the Subset Sum problem is only **weakly NP-hard**. The number of digits in t is $O(n)$. This means that the values of the numbers used in the reduction are exponential in input, making this problem weakly NP-hard. Problems which are **strongly NP-hard** must only use number values that are polynomial in the size of the input.

This also implies that the problem can be solved by a pseudopolynomial algorithm.

Partition

Definition 5. *Partition:* Given $A = \{a_1, a_2, \dots, a_n\}$, is there a subset $S \subseteq A$ such that

$$\sum S = \sum A \setminus S = \frac{1}{2} \sum A?$$

Partition is also weakly NP-complete. It is a special case of the Subset Sum problem, where we set $t = \frac{1}{2} \sum A$. In fact, we can reduce Partition to Subset Sum, though this is not the direction we want for the reduction.

We can reduce from Subset Sum to Partition as follows. Let $\sigma = \sum A$. Add elements $a_{n+1} = \sigma + t$ and $a_{n+2} = 2\sigma - t$ to A . Then a_{n+1} and a_{n+2} must be on different sides of the partition. In order to balance the two sides, $\sigma + t$ must be added to a_{n+1} and t must be added to a_{n+2} , so each subset has sum 2σ . Thus if we can solve Partition, we also have the subset of elements that sum to t , the target for the Subset Sum problem, completing our reduction.

Rectangle Packing

Definition 6. *Rectangle Packing:* Given a set of rectangles R_i and a target rectangle T , can we pack the rectangles in T such that there is no overlap? Note that sum of the area of the rectangles R_i is equivalent to the area of the target rectangle or $\sum_i R_i = T$.

Rectangle packing is weakly NP-hard via a reduction from Partition. For every element a_i in Partition, we create a rectangle R_i with height 1 and width $3a_i$. The target rectangle has height 2 and width $3t = \frac{3}{2} \sum A$. Because each rectangle has width at least 3, all rectangles must be packed horizontally. Thus to solve the Rectangle packing problem, we must separate the blocks into two groups with total width $3t$, which will correspond to two subsets with total sum t in the Partition problem.

Jigsaw Puzzles [Demaine & Demaine 2007]

Definition 7. Given square tiles with no patterns, can these tiles be arranged to fit a target rectangular shape? Note that the tiles can have a side tab, pocket, or boundary, but tabs and pockets must have matching shapes.

The most obvious reduction is from Partition. For every number, create a set of square rectangles with a unique tab and pocket, where the number of tiles is equivalent to the value of a_i , and the two end pieces of the rectangle have boundaries.

However, this reduction cannot be completed because the inputs to Partition can be exponentially large.

Instead, the reduction comes from 4-Partition.

Definition 8. 4-Partition: Given n integers $A = \{a_1, a_2, \dots, a_n\} \in (\frac{t}{5}, \frac{t}{3})$, is there a partition into $\frac{n}{4}$ subsets of 4 elements, each with the same sum $t = \sum A/\frac{n}{4}$?

4-Partition is a strongly NP-complete problem. We reduce from 4-Partition to Jigsaw Puzzles to show that Jigsaw Puzzles are NP-hard.

For each a_i , we create the following set of pieces. This ensures that we cannot mix pieces from different a_i .

These pieces are set into the following target board. There are $\frac{n}{4}$ rows, each of which will hold pieces corresponding to 4 a_i terms in the 4-Partition problem. The width of the target board is t , because each row must hold t pieces, so that the corresponding a_i form a group of sum t in the 4-Partition problem. Thus, the reduction is complete.

MIT OpenCourseWare
<http://ocw.mit.edu>

6.046J / 18.410J Design and Analysis of Algorithms
Spring 2015

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Lecture 17: Approximation Algorithms

- Definitions
- Vertex Cover
- Set Cover
- Partition

Approximation Algorithms and Schemes

Let C_{opt} be the cost of the optimal algorithm for a problem of size n . An approximation algorithm for this problem has an approximation ratio $\varrho(n)$ if, for any input, the algorithm produces a solution of cost C such that:

$$\max\left(\frac{C}{C_{opt}}, \frac{C_{opt}}{C}\right) \leq \varrho(n)$$

Such an algorithm is called a $\varrho(n)$ -approximation algorithm.

An approximation scheme that takes as input $\epsilon > 0$ and produces a solution such that $C = (1 + \epsilon)C_{opt}$ for any fixed ϵ , is a $(1 + \epsilon)$ -approximation algorithm.

A Polynomial Time Approximation Scheme (PTAS) is an approximation algorithm that runs in time polynomial in the size of the input, n . A Fully Polynomial Time Approximation Scheme (FPTAS) is an approximation algorithm that runs in time polynomial in both n and ϵ . For example, a $O(n^{2/\epsilon})$ approximation algorithm is a PTAS but not a FPTAS. A $O(n/\epsilon^2)$ approximation algorithm is a FPTAS.

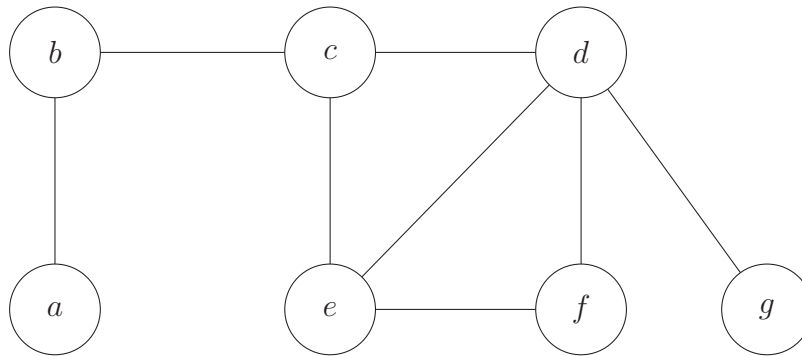
Vertex Cover

Given an undirected graph $G(V, E)$, find a subset $V' \subseteq V$ such that, for every edge $(u, v) \in E$, either $u \in V'$ or $v \in V'$ (or both). Furthermore, find a V' such that $|V'|$ is minimum. This is an NP-Complete problem.

Approximation Algorithm For Vertex Cover

Here we define algorithm *Approx_Verex_Cover*, an approximation algorithm for Vertex Cover. Start with an empty set V' . While there are still edges in E , pick an edge (u, v) arbitrarily. Add both u and v into V' . Remove all edges incident on u or v . Repeat until there are no more edges left in E . *Approx_Verex_Cover* runs in polynomial time.

Take for example the following graph G :



Approx_Verex_Cover could pick edges (b, c) , (e, f) and (d, g) , such that $V' = \{b, c, e, f, d, g\}$ and $|V'| = 6$. Hence, the cost is $C = |V'| = 6$. The optimal solution for this example is $\{b, d, e\}$, hence $C_{opt} = 3$.

Claim: *Approx_Verex_Cover* is a 2-approximation algorithm.

Proof: Let $U \subseteq E$ be the set of all the edges that are picked by *Approx_Verex_Cover*. The optimal vertex cover must include at least one endpoint of each edge in U (and other edges). Furthermore, no two edges in U share an endpoint. Therefore, $|U|$ is a lower bound for C_{opt} . i.e. $C_{opt} \geq |U|$. The number of vertices in V' returned by *Approx_Verex_Cover* is $2 \cdot |U|$. Therefore, $C = |V'| = 2 \cdot |U| \leq 2C_{opt}$. Hence $C \leq 2 \cdot C_{opt}$. $\square$

Set Cover

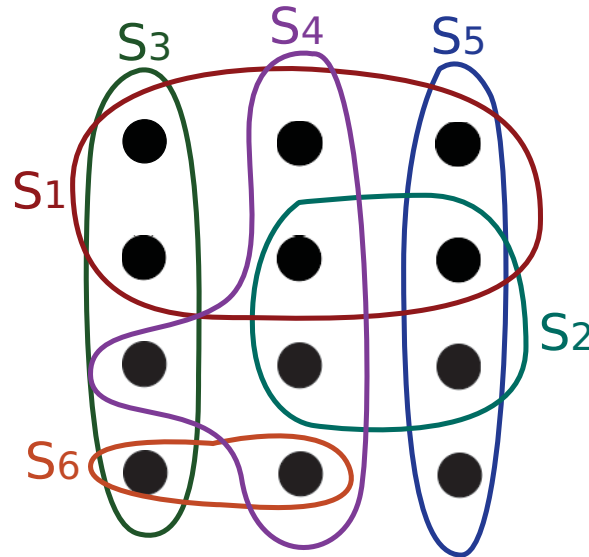
Given a set X and a family of (possibly overlapping) subsets $S_1, S_2, \dots, S_m \subseteq X$ such that $\cup_{i=1}^m S_i = X$, find a set $P \subseteq \{1, 2, 3, \dots, m\}$ such that $\cup_{i \in P} S_i = X$. Furthermore find a P such that $|P|$ is minimum.

Set Cover is an NP-Complete problem.

Approximation Algorithm for Set Cover

Here we define algorithm *Approx_Set_Cover*, an approximation algorithm for Set Cover. Start by initializing the set P to the empty set. While there are still elements in X , pick the largest set S_i and add i to P . Then remove all elements in S_i from X and all other subsets S_j . Repeat until there are no more elements in X . *Approx_Set_Cover* runs in polynomial time.

In the following example, each dot is an element in X and each S_i are subsets of X .



Approx_Set_Cover selects sets S_1, S_4, S_5, S_3 in that order. Therefore it returns $P = \{1, 4, 5, 3\}$ and its cost $C = |P| = 4$. The optimal solution is $P_{opt} = \{S_3, S_4, S_5\}$ and $C_{opt} = |P_{opt}| = 3$.

Claim: *Approx_Set_Cover* is a $(\ln(n) + 1)$ -approximation algorithm (where $n = |X|$).

Proof: Let the optimal cover be P_{opt} such that $C_{opt} = |P_{opt}| = t$. Let X_k be the set of elements remaining in iteration k of *Approx_Set_Cover*. Hence, $X_0 = X$. Then:

- for all k , X_k can be covered by t sets (from the optimal solution)
- one of them covers at least $\frac{|X_k|}{t}$ elements
- *Approx_Set_Cover* picks a set of (current) size $\geq \frac{|X_k|}{t}$

- for all k , $|X_{k+1}| \leq (1 - \frac{1}{t})|X_k|$ (More careful analysis (see CLRS, Ch. 35) relates $\varrho(n)$ to harmonic numbers. t should shrink.)
- for all k , $|X_{k+1}| \leq (1 - \frac{1}{t})^k \cdot n \leq e^{-k/t} \cdot n$ ($n = |X_0|$)

Algorithm terminates when $|X_k| < 1$, i.e., $|X_k| = 0$ and will have cost $C = k$.

$$e^{-k/t} \cdot n < 1$$

$$e^{k/t} > n$$

Hence algorithm terminates when $\frac{k}{t} > \ln(n)$. Therefore $\frac{k}{t} = \frac{C}{C_{opt}} \leq \ln(n) + 1$. Hence *Approx_Set_Cover* is a $(\ln(n) + 1)$ -approximation algorithm for Set Cover. $\square$

Notice that the approximation ratio gets worse for larger problems as it changes with n .

Partition

The input is a set $S = \{1, 2, \dots, n\}$ of n items with weights $s_1, s_2, \dots, s_n$. Assume, without loss of generality, that the items are ordered such that $s_1 \geq s_2 \geq \dots \geq s_n$. Partition S into sets A and B to minimize $\max(w(A), w(B))$, where $w(A) = \sum_{i \in A} S_i$ and $w(B) = \sum_{j \in B} S_j$.

Define $2L = \sum_{i=1}^n s_i = w(S)$. Then optimal solution will have cost $C_{opt} \geq L$ by definition.

Partition is an NP-Complete problem. Want to find a PTAS $(1 + \epsilon)$ -approximation. (Note that 2-approximation in this case is trivial). Also, an FPTAS also exists for this problem.

Approximation Algorithm for Partition

Here we define *Approx_Partition*. Define $m = \lceil \frac{1}{\epsilon} \rceil - 1$. ($\epsilon \approx \frac{1}{m+1}$) The algorithm proceeds in two phases.

First Phase: Find an optimal partition A', B' of $s_1, \dots, s_m$. This takes $O(2^m)$ time.

Second Phase: Initialize sets A and B to A' and B' respectively. Hence they already contain a partition of elements $s_1, \dots, s_m$. Then, for each i , where i goes

from $m + 1$ to n , if $w(A) \leq w(B)$, add i to A , otherwise add i to B .

Claim: *Approx_Partition* is a PTAS for Partition.

Proof: Without loss of generality, assume $w(A) \geq w(B)$. Then the approximation ratio is $\frac{C}{C_{opt}} = \frac{w(A)}{L}$. Let k be the last item added to A . There are two cases, either k was added in the first phase, or in the second phase.

Case 1: k is added to A in the first phase. This means that $A = A'$. We have an optimal partition since we can't do better than $w(A')$ when we have $n \geq m$ items, and we know that $w(A')$ is optimal for the m items.

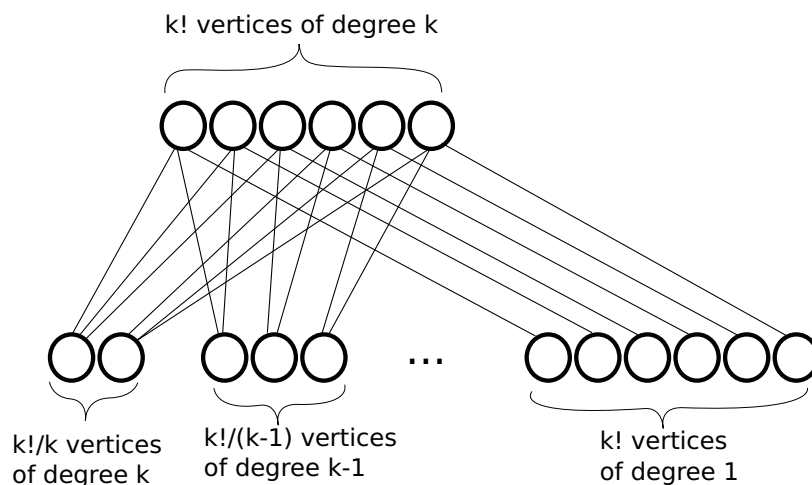
Case 2: k is added to A in the second phase. Here we know $w(A) - s_k \leq w(B)$ since this is why k was added to A and not to B . (Note that $w(B)$ may have increased after this last addition to A). Now, because $w(A) + w(B) = 2L$, $w(A) - s_k \leq w(B) = 2L - w(A)$. Therefore $w(A) \leq L + \frac{s_k}{2}$. Since $s_1 \geq s_2 \geq \dots \geq s_n$, we can say that $s_1, s_2, \dots, s_m \geq s_k$. Now since $k > m$, $2L \geq (m + 1)s_k$.

Now, $\frac{w(A)}{L} \leq \frac{L + \frac{s_k}{2}}{L} = 1 + \frac{s_k}{2L} \leq 1 + \frac{s_k}{(m+1)s_k} = 1 + \frac{1}{m+1} = 1 + \epsilon$. Hence *Approx_Partition* is a $(1 + \epsilon)$ -approximation for Partition. $\square$

Natural Vertex Cover Approximation

Here we describe *Approx_Verex_Cover_Natural*, a different approximation algorithm for Vertex Cover. Start with an empty set V' . While there are still edges left in E , pick the vertex $v \in V$ that has maximum degree and add it to V' . Then remove v and all incident edges from E . Repeat until no more edges left in E . In the end, return V' .

The following example shows a bad-case example for *Approx_Verex_Cover_Natural*. In the example, the optimal cover will pick the $k!$ vertices at the top.



Approx_Verex_Cover_Natural could possibly pick all the bottom vertices from left to right in order. Hence the cost could be $k! \cdot (\frac{1}{k} + \frac{1}{k-1} + \cdots + 1) \approx k! \log k$. Which is a factor of $\log k$ worse than optimal.

Claim: *Approx_Verex_Cover_Natural* is a $(\log n)$ -approximation.

Proof: Let G_k be the graph after iteration k of the algorithm. And let n be the number of edges in the graph, i.e. $|G| = n = |E|$. With each iteration, the algorithm selects a vertex and deletes it along with all incident edges. Let $m = C_{opt}$ be the number of vertices in the optimal vertex cover for G . Then let's look at the first m iterations of the algorithm: $G_0 \rightarrow G_1 \rightarrow G_2 \rightarrow \cdots \rightarrow G_m$.

Let d_i be the degree of the maximum degree vertex of G_{i-1} . Then the algorithm deletes all edges incident on that vertex to get G_i . Therefore:

$$|G_m| = |G_0| - \sum_{i=1}^m d_i$$

Also:

$$\sum_{i=1}^m d_i \geq \sum_{i=1}^m \frac{|G_{i-1}|}{m}$$

This is true because given $|G_{i-1}|$ edges that can be covered by m vertices, we know that there is a vertex with degree at least $\frac{|G_{i-1}|}{m}$. Then:

$$\sum_{i=1}^m \frac{|G_{i-1}|}{m} \geq \sum_{i=1}^m \frac{|G_m|}{m} = |G_m|$$

This is true since $|G_i| \leq |G_{i-1}|$ for all i . Then, it follows:

$$|G_0| - |G_m| \geq |G_m|$$

Because $|G_m| \leq \sum_{i=1}^m d_i$. Hence after m iterations, the algorithm will have deleted half or more edges from G_0 . And generally, since every m iterations it will halve the number of edges in the graph, in $m \cdot \log |G_0|$ iterations, it will have deleted all the edges. And since with each iteration it adds 1 vertex to the cover, it will end up with a vertex cover of size $m \cdot \log |G_0| = m \cdot \log n$. Since we assumed that m was the size of the optimal vertex cover, $\frac{C}{C_{opt}} = \frac{m \log n}{m} = \log n$. Hence *Approx_Verex_Cover_Natural* is a $(\log n)$ -approximation. $\square$

Note that since $n \approx k! \log k$ in the example of Figure , the worst-case example is $\log k \approx \log \log n$ worse, but we have only shown an $O(\log n)$ approximation.

MIT OpenCourseWare
<http://ocw.mit.edu>

6.046J / 18.410J Design and Analysis of Algorithms
Spring 2015

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Lecture 18: Fixed-Parameter Algorithms

- Vertex Cover
- Fixed-Parameter Tractability
- Kernelization
- Connection to Approximation

Fixed Parameter Algorithms

Fixed Parameter Algorithms are an alternative way to deal with NP-hard problems instead of approximation algorithms. There are three general desired features of an algorithm:

1. Solve (NP-)hard problems
2. Run in polynomial time (fast)
3. Get exact solutions

In general, unless $P = NP$, an algorithm can have two of these three features, but not all three. An algorithm that has Features 2 and 3 is an algorithm in P (poly-time exact). An approximation algorithm has Features 1 and 2. It solves hard problems, and it runs fast, but it does not give exact solutions. Fixed-parameter algorithms will have Features 1 and 3. They will solve hard problems and give exact solutions, but they will not run very fast.

Idea: The idea is to aim for an exact algorithm but isolate exponential terms to a specific *parameter*. When the value of this parameter is small, the algorithm gets fast instances. Hopefully, this parameter will be small in practice.

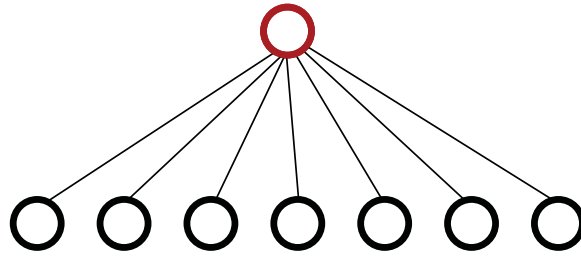
Parameter: A parameter is a nonnegative integer $k(x)$ where x is the problem input. Typically, the parameter is a natural property of the problem (some k in input). It may not necessarily be efficiently computable (e.g., OPT).

Parameterized Problem: A parameterized problem is simply the problem plus the parameter or the problem as seen with respect to the parameter. There are potentially many interesting parameterizations for any given problem.

Goal: The goal of fixed-parameter algorithms is to have an algorithm that is polynomial in the problem size n but possibly exponential in the parameter k , and still get an exact solution.

k -Vertex Cover

Given a graph $G = (V, E)$ and a nonnegative integer k , is there a set $S \subseteq V$ of vertices of size at most k , $|S| \leq k$, that covers all edges. This is a decision problem for Vertex Cover and is also NP-hard. We will use k as the parameter to develop a fixed-parameter algorithm for k -Vertex Cover. Note that we can have $k \ll |V|$ as the figure below shows:



Brute-force solution (bad)

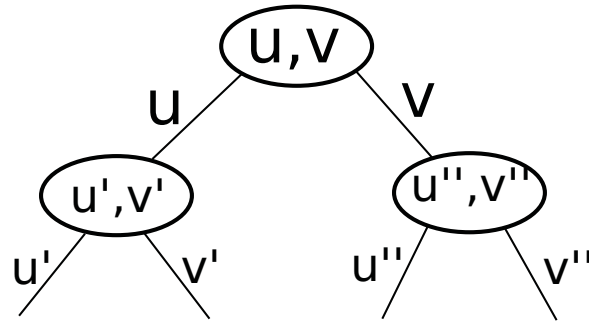
Try all $\binom{v}{k} + \binom{v}{k-1} + \dots + \binom{v}{0}$ sets of $\leq k$ vertices. Can skip all terms smaller than $\binom{v}{k}$ because bigger sets have more coverage. Testing coverage takes $O(m)$ time where m is the number of edges. Therefore, the total runtime is $O(V^k |E|)$. It is polynomial for fixed k but not the same polynomial for all k 's. It is inefficient in most cases. Hence we define $n^{f(k)}$ to be **bad**, where $n = |V| + |E|$ is the input size.

Bounded search-tree algorithm (good)

This is a general technique used to improve brute force searches. It works as follows:

- pick arbitrary edge $e = (u, v)$
- we know that either $u \in S$ or $v \in S$ (or both) but don't know which
- guess which one: try both possibilities
 1. add u to S , delete u and incident edges from G , and recurse with $k' = k - 1$.
 2. do the same but with v instead of u
 3. return the OR of the two outcomes

This is like guessing in dynamic programming but memoization doesn't help here. The recursion tree looks like the following:



At a leaf ($k = 0$), return YES if $|E| = 0$ (all edges covered). It takes $O(V)$ time to delete u or v . Therefore this has a total runtime of $(2^k|V|)$.

- $O(V)$ for fixed k
- degree of polynomial is independent of k
- also polynomial for $k = O(\lg |V|)$
- practical for e.g. $k \leq 32$
- Hence we define $f(k) \cdot n^{O(1)}$ to be **good**

Fixed Parameter Tractability

A parameterized problem is fixed-parameter tractable (FPT) if there is an algorithm with running time $\leq f(k) \cdot n^{O(1)}$, such that $f : \mathbb{N} \rightarrow \mathbb{N}$ (non negative) and k is the parameter, and the $O(1)$ degree of the polynomial is independent of k and n .

Question: Why $f(k) \cdot n^{O(1)}$ and not $f(k) + n^{O(1)}$?

Theorem: $\exists f(k) \cdot n^c$ algorithm $\iff \exists f'(k) + n^{c'}$

Proof:

$(\Leftarrow)$

Trivial (assuming $f'(k)$ and $n^{c'}$ are ≥ 1)

$(\Rightarrow)$

if $n \leq f(k)$, then $f(k) \cdot n^c \leq f(k)^{c+1}$

if $f(k) \leq n$ then $f(k) \cdot n^c \leq n^{c+1}$

Therefore $f(k) \cdot n^c \leq \max(f(k)^{c+1}, n^{c+1}) \leq f(k)^{c+1} + n^{c+1} = f'(k) + n^{c'} \quad \square$

Alternatively, since $xy \leq x^2 + y^2$, can just make $f'(k) = (f(k))^2$ and $c' = 2c$.

Example: $O(2^k \cdot n) \leq O(4^k + n^2)$

Kernelization

Kernelization is a simplifying self-reduction. It is a polynomial time algorithm that converts an input (x, k) into a *small* and *equivalent* input (x', k') . Here, *small* means $|x'| \leq f(k)$ and *equivalent* means the answer to x is the same as the answer to x' .

Theorem: a problem is FPT $\iff \exists$ a kernelization

Proof:

($\Leftarrow$)

Kernelize $\Rightarrow n' \leq f(k)$

Run any finite $g(n')$ algorithm

Totals to $n^{O(1)} + g(f(k))$ time

($\Rightarrow$)

let A be an $f(k) \cdot n^c$ algorithm, then assuming k is known:

if $n \leq f(k)$, it's already kernelized.

if $f(k) \leq n$, then

1. run $A \rightarrow f(k) \cdot n^c \leq n^{c+1}$ time
2. output $O(1)$ -sized YES/NO instance as appropriate (to kernelize)

if k is unknown: run A for n^{c+1} time and if it is still not done, we know it is already kernelized.

So we know (exponential) kernel exists. Recent work aims to find polynomial (even linear) kernels when possible.

Polynomial kernel for k-Vertex Cover

To create a kernel for k -Vertex Cover, the algorithm follows the following steps:

- Make graph simple by removing all self loops and multi-edges
- Any vertex of degree $> k$ must be in the cover (else would need to add $> k$ vertices to cover incident edges)

- Remove such vertices (and incident edges) one at a time, decreasing k accordingly
- Remaining graph has maximum degree $\leq k$
- Each remaining vertex covers $\leq k$ edges
- If the number of remaining edges is $> k$, answer NO and output canonical NO instance.
- Else, $|E'| \leq k^2$
- Remove all isolated vertices (degree 0 vertices)
- Now $|V'| \leq 2k^2$
- The input has been reduced to instance (V', E') of size $O(k^2)$

The runtime of the kernelization algorithm is naively $O(VE)$. ($O(V + E)$ with more work.) After this, we can apply either a brute-force algorithm on the kernel, which yields an overall runtime $O(V + E + (2k^2)^k k^2) = O(V + E + 2^k k^{2k+2})$. Or we can apply a bounded search-tree solution, which yields a runtime of $O(V + E + 2^k k^2)$.

The best algorithm to date: $O(kV + 1.274^k)$ by [Chen, Kanj, Xia - TCS 2010].

Connection to Approximation Algorithms

Take an optimization problem, integral OPT and consider its associated decision problem: “OPT $\leq k$?” and parameterize by k .

Theorem: optimization problem has EPTAS

(EPTAS: efficient PTAS, $f(\frac{1}{\epsilon}) \cdot n^{O(1)}$ e.g. *Approxpartition*[L17])

$\Rightarrow$ decision problem is FPT

Proof: (like FPTAS, pseudopolynomial algorithm)

- Say maximization problem (and $\leq k$ decision)
- run EPTAS with $\epsilon = \frac{1}{2k}$ in $f(2k) \cdot n^{O(1)}$ time.
- relative error $\leq \frac{1}{2k} < \frac{1}{k}$
- $\Rightarrow$ absolute error < 1 if OPT $\leq k$

- So if we find a solution with value $\leq k$, then $\text{OPT} \leq (1 + \frac{1}{2k}) \cdot k \leq k + \frac{1}{2}$
Integral $\Rightarrow \text{OPT} \leq k \Rightarrow \text{YES}$
- else $\text{OPT} > k$

□

Also: $=, \leq, \geq$ decision problems are equivalent with respect to FPT.

(Can use this relation to prove that EPTASs don't exist in some cases)

MIT OpenCourseWare
<http://ocw.mit.edu>

6.046J / 18.410J Design and Analysis of Algorithms
Spring 2015

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

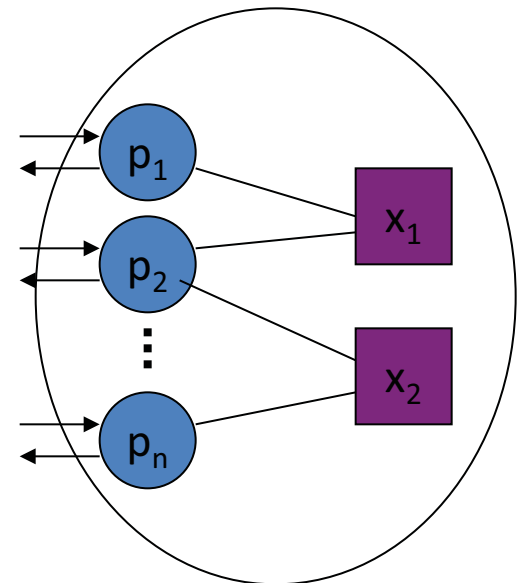
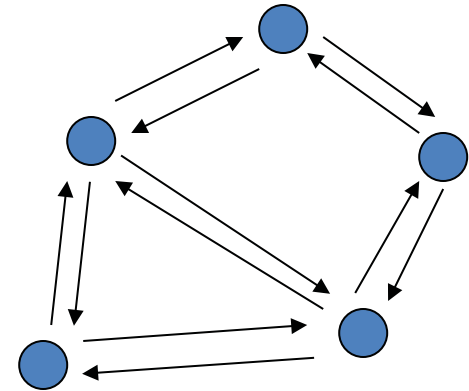
Distributed Algorithms

6.046J, Spring, 2015

Nancy Lynch

What are Distributed Algorithms?

- Algorithms that run on networked processors, or on multiprocessors that share memory.
- They solve many kinds of problems:
 - Communication
 - Data management
 - Resource management
 - Synchronization
 - Reaching consensus
 - ...
- They work in difficult settings:
 - Concurrent activity at many processing locations
 - Uncertainty of timing, order of events, inputs
 - Failure and recovery of processors, of channels.
- So they can be complicated:
 - Hard to design, prove correct, and analyze.

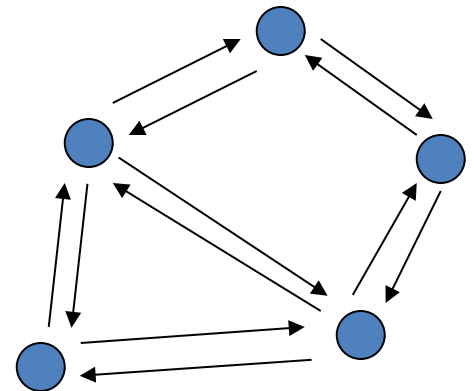


Distributed Algorithms

- Studied since 1967, starting with Dijkstra and Lamport.
 - Edsger Dijkstra: 1972 Turing award winner
 - Leslie Lamport: 2013 Turing award winner
- Some sources:
 - Lynch, Distributed Algorithms
 - Attiya and Welch, Distributed Computing: Fundamentals, Simulations, and Advanced Topics
 - Morgan Claypool series of monographs on Distributed Computing Theory
 - Conferences:
 - Principles of Distributed Computing (PODC)
 - Distributed Computing (DISC)

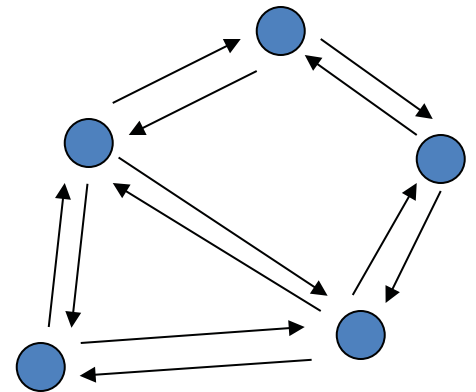
This Week

- A quick introduction:
 - Two common distributed computing models,
 - A few fundamental algorithms, and
 - Techniques for analyzing algorithms.
- Synchronous distributed algorithms:
 - Leader Election
 - Maximal Independent Set
 - Breadth-First Spanning Trees
 - Shortest Paths Trees
- Asynchronous distributed algorithms:
 - Breadth-First Spanning Trees
 - Shortest Paths Trees



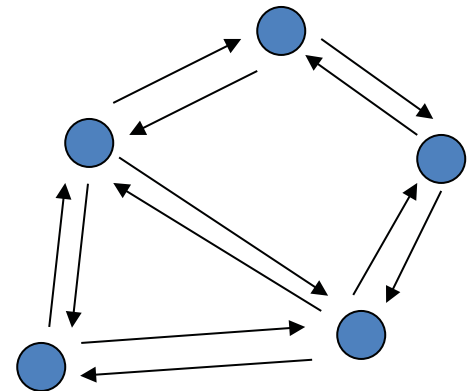
Modeling, Proofs, Analysis

- Important for distributed algorithms, because they are complicated and error-prone.
- Infinite-state interacting state machines.
- Proofs use invariants, proved by induction.
- Abstraction relations.
- New complexity measures:
 - Time complexity: Rounds, or real time.
 - Communication complexity: Messages, or bits.

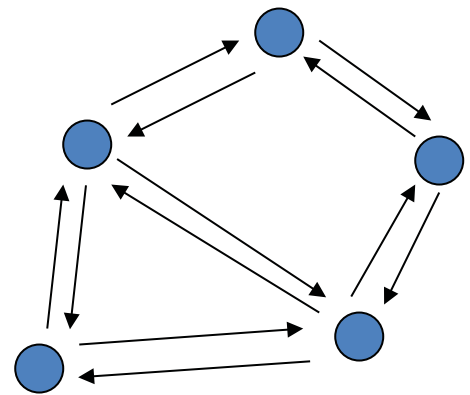


Distributed Networks

- Based on an undirected graph $G = (V, E)$.
 - $n = |V|$
 - $\Gamma(u)$, set of neighbors of vertex u .
 - $\deg(u) = |\Gamma(u)|$, number of neighbors of vertex u .
- Associate a **process** with each graph vertex.
 - An infinite-state automaton.
 - Sometimes refer to processes, or vertices, as **nodes**.
- Associate two directed **communication channels** with each edge.
- **Note:** We won't consider failures.

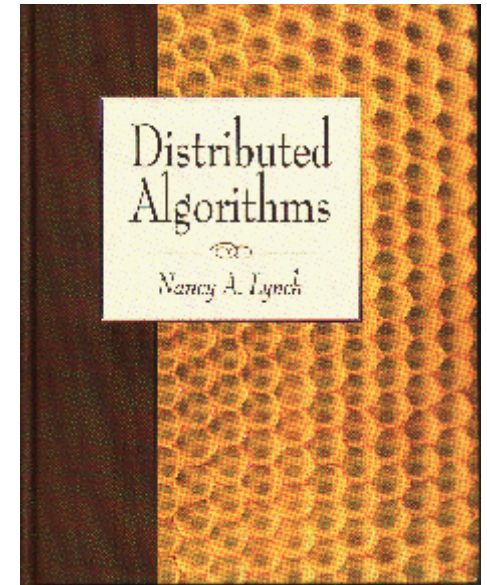


Synchronous Distributed Algorithms



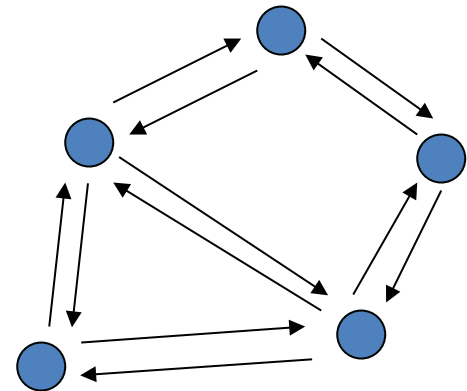
Synchronous Network Model

- Lynch, Distributed Algorithms, Chapter 2.
- Processes at nodes of an undirected graph, communicate using messages.
- Each process has **output ports, input ports** that connect to communication channels.
 - Process at vertex u doesn't know who its ports' channels connect to.
 - Knows the ports only by local names, such as $1, 2, \dots, k$, where $k = \deg(u)$.
- Processes need not be distinguishable.
 - E.g., they needn't have **Unique Identifiers (UIDs)**.
 - They just know how many ports they have, and their local names.

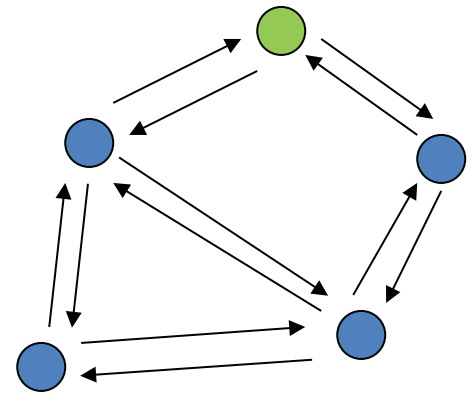


Execution

- An algorithm executes in **synchronous rounds**.
 - In each round, each process determines, based on its state, the messages to send on all its ports.
 - At most one message per port per round.
 - Each message gets put into its channel, and delivered to the process at the other end.
 - Then each process computes a new state based on its old state and the arriving messages.
-
- **Note on costs:**
 - Generally ignore costs of local computation (time and space).
 - Focus on time (number of rounds), and communication (number of messages or total number of bits).

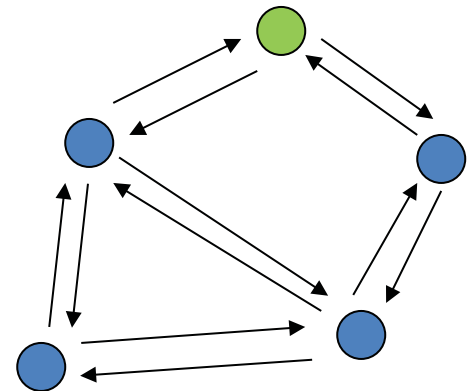


Leader Election



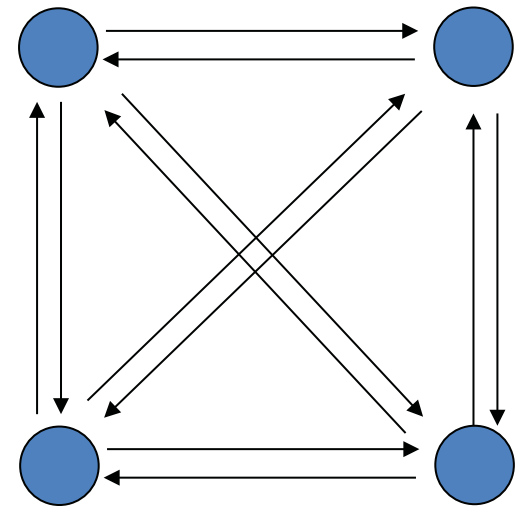
Leader Election

- $G = (V, E)$ is an arbitrary connected (undirected) graph.
- Goal is for exactly one process to output a special **leader** signal.
- **Motivation:** Leader can take charge of:
 - Communication
 - Coordinating data processing
 - Allocating resources
 - Scheduling tasks
 - Reaching consensus
 - ...

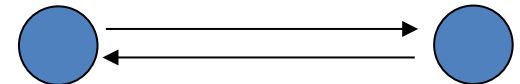


Simple case: Clique Network

- All vertices are directly connected to all others.
- **Theorem 1:** Let $G = (V, E)$ be an n -vertex clique. Then there is no algorithm consisting of deterministic, indistinguishable processes that is guaranteed to elect a leader in G .



- **Proof:**
 - For example, consider $n = 2$.
 - Two identical, deterministic processes.
 - Show by induction that the processes remain in the same state forever...

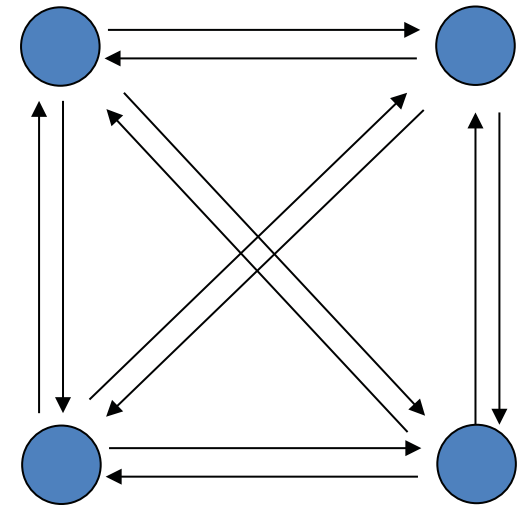


Proof

- By contradiction. Assume an algorithm A that solves the problem.
- Both processes begin in the same start state.
- Prove by induction on the number r of completed rounds that both processes are in identical states after r rounds.
 - Generate the same message.
 - Receive the same message.
 - Make the same state change.
- Since the algorithm solves the leader election problem, eventually one of them gets elected.
- Then they both get elected, contradiction.

Clique Network

- **Theorem 1:** Let $G = (V, E)$ be an n -vertex clique. Then there is no algorithm consisting of deterministic, indistinguishable processes that is guaranteed to elect a leader in G .



- **Proof:**
 - Now consider $n > 2$.
 - Processes have $n - 1$ output ports and $n - 1$ input ports, each set numbered $1, 2, \dots, n - 1$.
 - Assume ports are connected “consistently”: output port k at a process is connected to input port k at the other end.
 - Show by induction that all the processes remain in the same state forever...

Proof

- Assume an algorithm A that solves the leader election problem.
 - All processes start in the same start state.
 - Prove by induction on the number r of completed rounds that all processes are in identical states after r rounds.
 - For each k , they generate the same message on port k .
 - For each k , they receive the same message on port k .
 - Make the same state changes.
 - Since the algorithm solves the leader election problem, eventually some process gets elected.
 - Then everyone gets elected, contradiction.
-
- A basic problem for distributed algorithms: **Breaking symmetry**
 - Deterministic, indistinguishable processes can't do it.
 - So we need something more...

So we need something more...

- **Unique Identifiers (UIDs)**
 - Assume processes have unique identifiers (UIDs), which they “know”, e.g., each process starts with its own UID in a special state variable.
 - UIDs are elements of some totally-ordered set, e.g., the natural numbers.
 - Different UIDs can appear anywhere in the graph, but each can appear only once.
- **Randomness**

Algorithm Using UIDs

- **Theorem 2:** Let $G = (V, E)$ be an n -vertex clique. Then there is an algorithm consisting of deterministic processes with UIDs that is guaranteed to elect a leader in G .
- The algorithm takes only 1 round and uses only n^2 point-to-point messages.
- **Algorithm:**
 - Everyone sends its UID on all its output ports, and collects UIDs received on all its input ports.
 - The process with the maximum UID elects itself the leader.

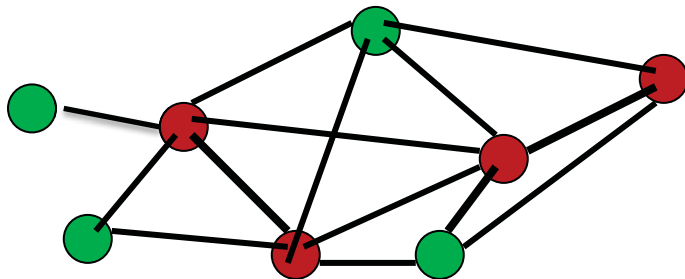
Algorithm Using Randomness

- **Idea:** Processes choose IDs randomly, from a sufficiently large set so that it is likely that all are different. Then use them like UUIDs.
- **Q:** What is a “sufficiently large” set?
- **Lemma 3:** Let ϵ be a real, $0 < \epsilon < 1$. Suppose that n processes choose ids uniformly at random, independently, from $\{1, \dots, r\}$, where $r = \lceil n^2 / 2\epsilon \rceil$. Then with probability at least $1 - \epsilon$, all the chosen numbers are different.
- **Proof:** The probability of any two particular processes choosing the same number is $\frac{1}{r}$. Taking a union bound for all $\frac{n^2}{2}$ pairs, the probability is still $\leq \epsilon$.

Algorithm Using Randomness

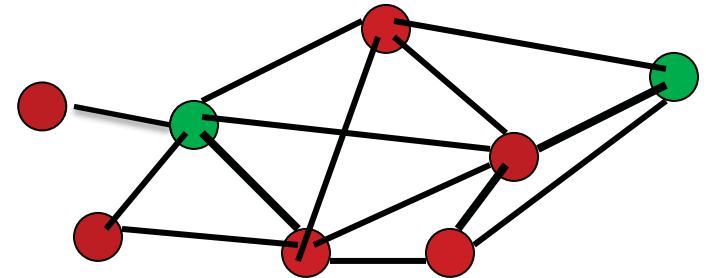
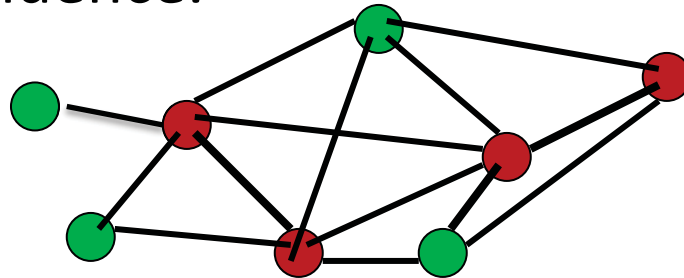
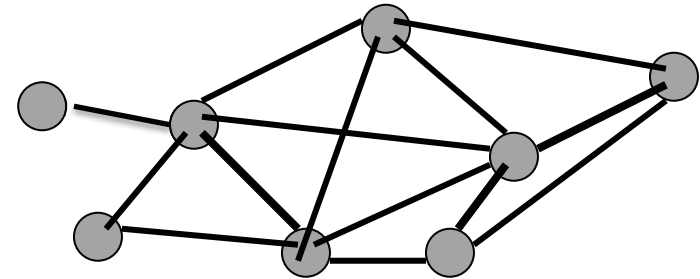
- **Theorem 4:** Let $G = (V, E)$ be an n -vertex clique. Then there is an algorithm consisting of randomized, indistinguishable processes that eventually elects a leader in G , with probability 1.
- The algorithm takes expected time $\leq \frac{1}{1-\epsilon}$.
- Also, with probability $\geq 1 - \epsilon$, the algorithm finishes in only one round.
- **Algorithm:**
 - Processes choose random ids from a sufficiently large space.
 - Exchange ids; if the maximum is unique, the maximum wins.
 - Otherwise repeat, as many times as necessary.

Maximal Independent Set



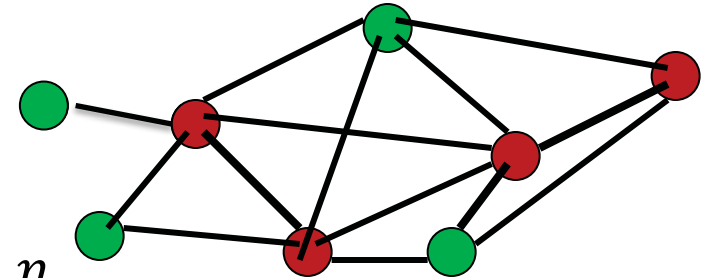
Maximal Independent Set (MIS)

- General undirected graph network:
- **Problem:** Select a subset S of the nodes, so that they form a Maximal Independent Set.
- **Independent:** No two neighbors are both in the set.
- **Maximal:** We can't add any more nodes without violating independence.



- Every node is either in S or has a neighbor in S .

Distributed MIS

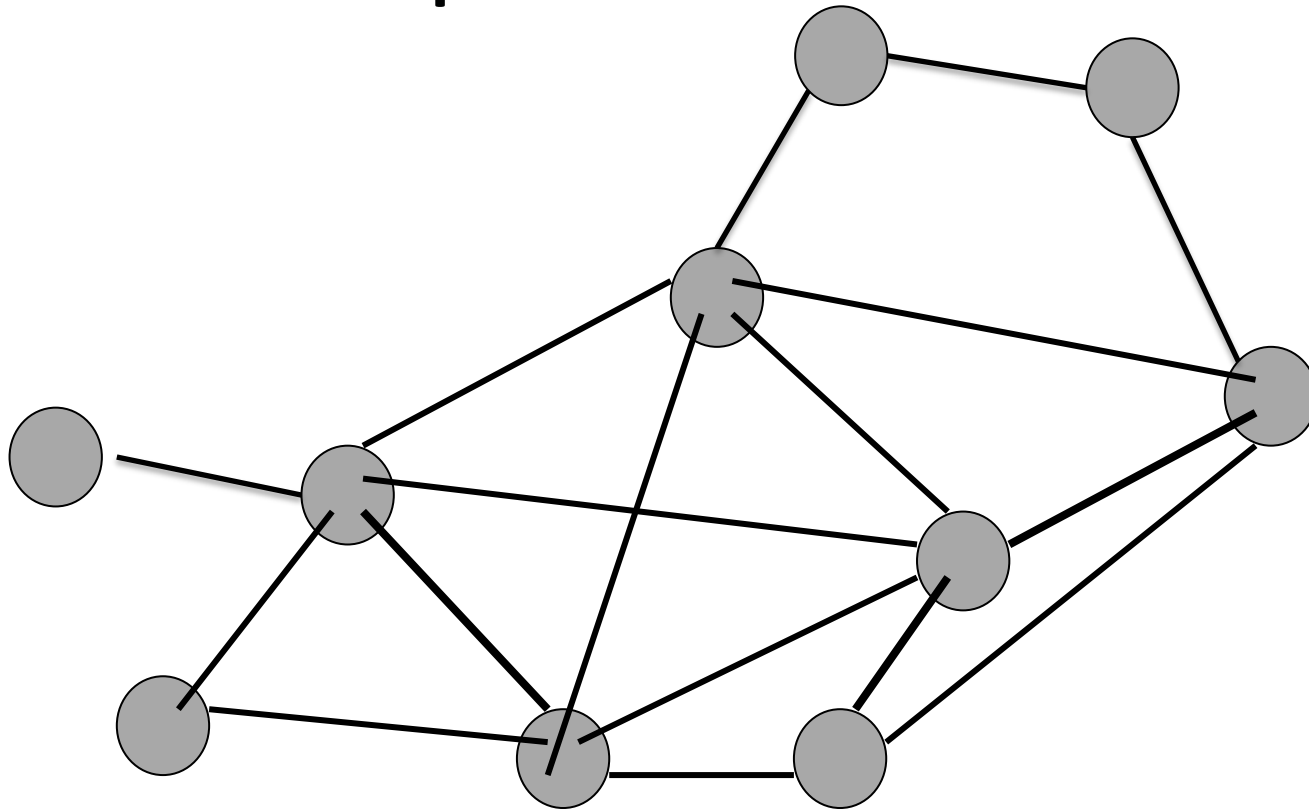


- **Assume:**
 - No UUIDs
 - Processes know a good upper bound on n .
- **Require:**
 - Compute an MIS S of the entire network graph.
 - Each process in S should output **in**, others output **out**.
- Unsolvable by deterministic algorithms, in some graphs.
- So consider randomized algorithms.
- **Applications of distributed MIS:**
 - **Communication networks:** Selected processes can take charge of communication, convey information to their neighbor processes.
 - **Developmental biology:** Distinguish cells in fruit fly's nervous system to become "Sensory Organ Precursor" cells [Afek, Alon, et al., Science].

Luby's MIS Algorithm

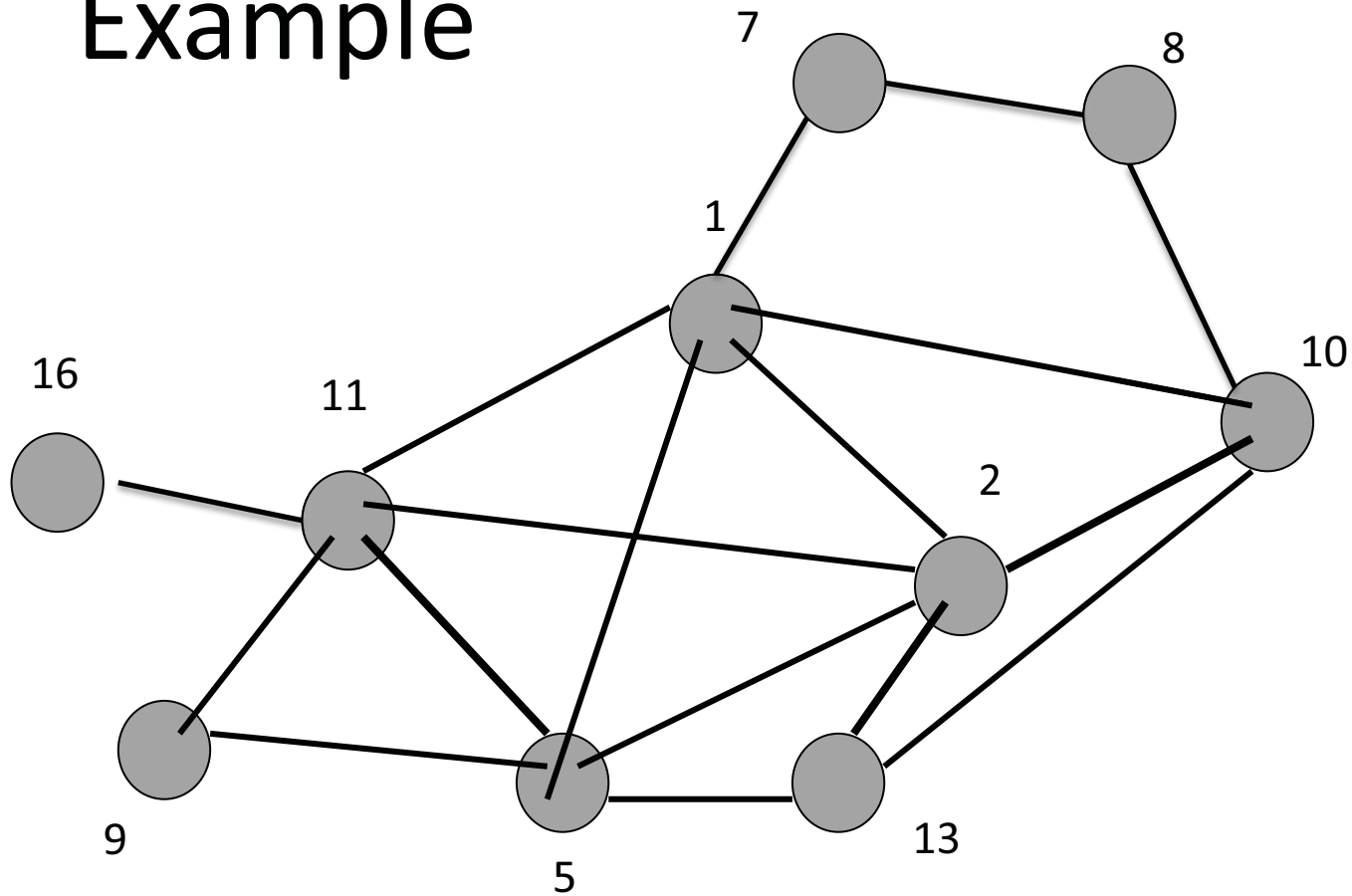
- Executes in 2-round **phases**.
- Initially all nodes are **active**.
- At each phase, some active nodes decide to be **in**, others decide to be **out**, algorithm continues to the next phase with a smaller graph.
- Repeat until all nodes have decided.
- **Behavior of active node u at phase ph :**
- **Round 1:**
 - Choose a random value r in $\{1, 2, \dots, n^5\}$, send it to all neighbors.
 - Receive values from all active neighbors.
 - If r is strictly greater than all received values, then join the MIS, output **in**.
- **Round 2:**
 - If you joined the MIS, announce it in messages to all (active) neighbors.
 - If you receive such an announcement, decide not to join the MIS, output **out**.
 - If you decided one way or the other at this phase, become **inactive**.

Example



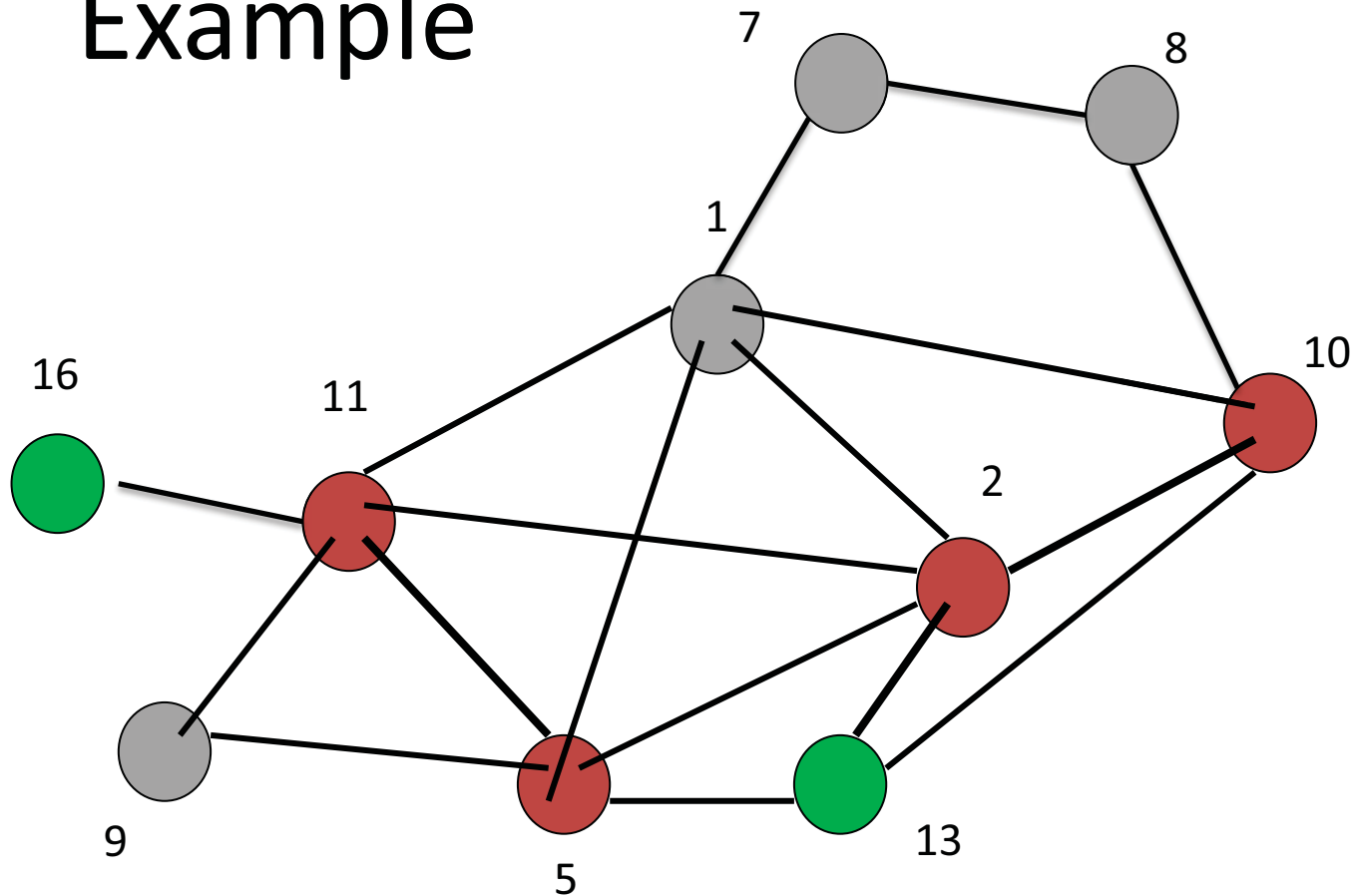
- All nodes start out identical.

Example

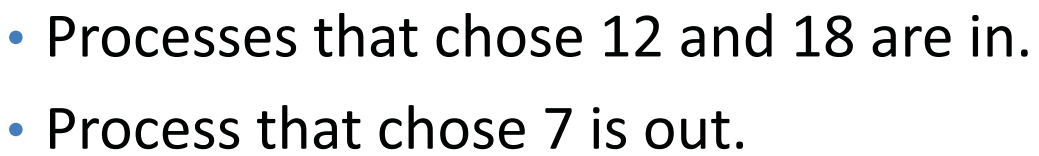


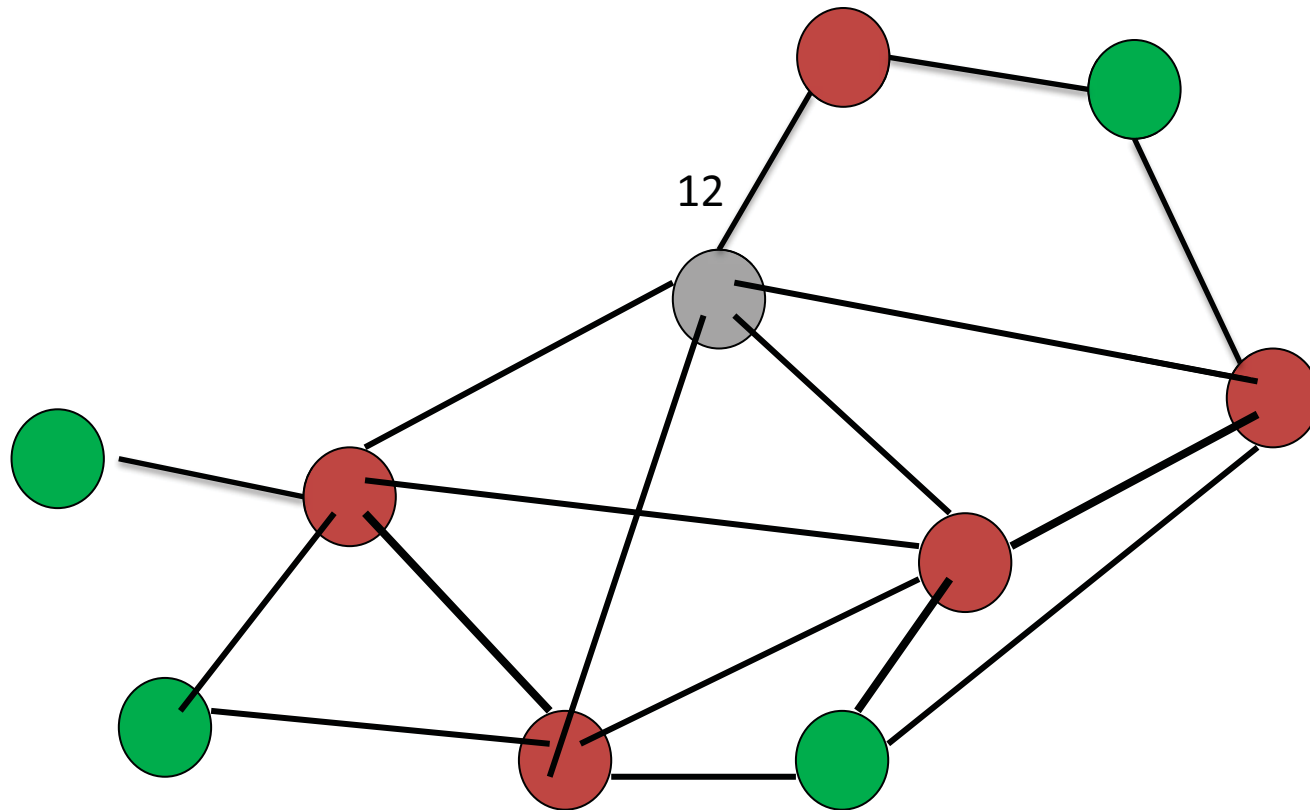
- Everyone chooses an ID.

Example

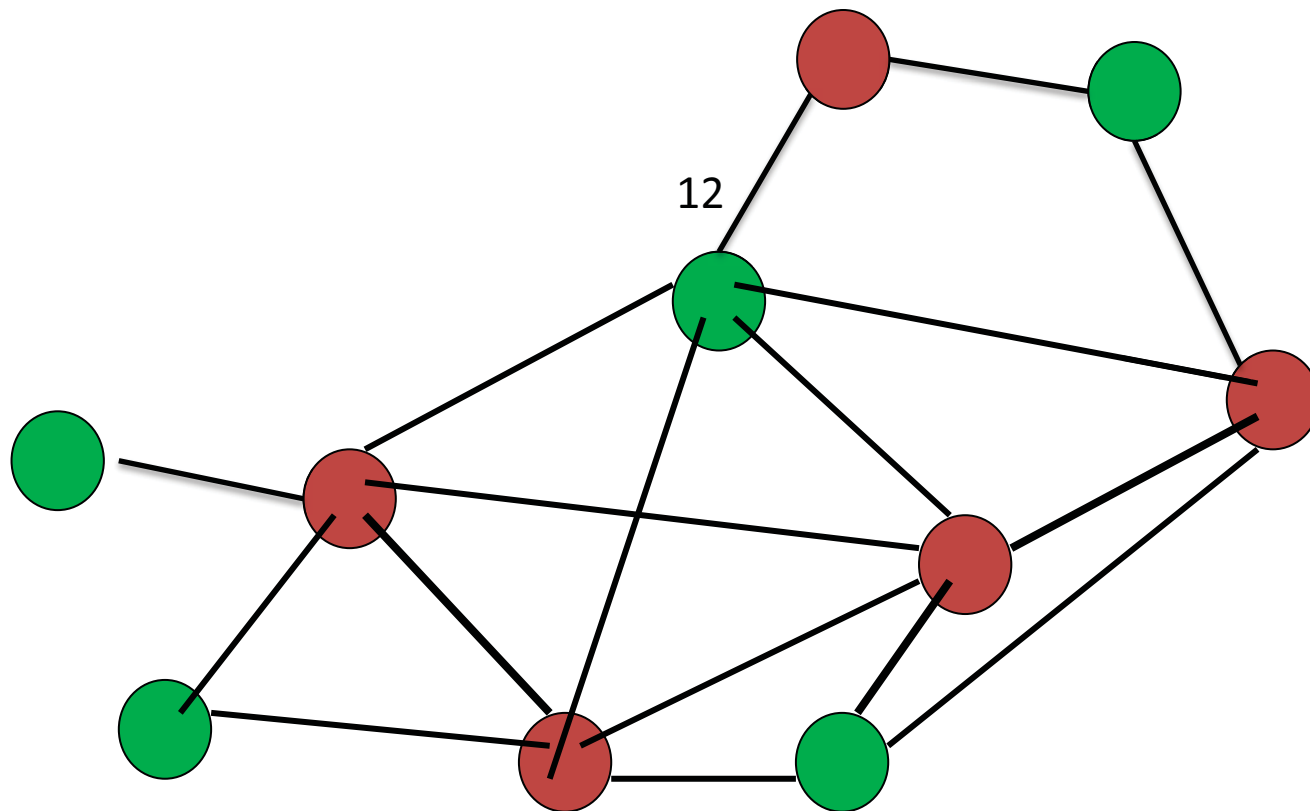


- Processes that chose 16 and 13 are in.
- Processes that chose 11, 5, 2, and 10 are out.





- Undecided (gray) process chooses a new ID.



- It's in.

Independence

- **Theorem 5:** If Luby's algorithm ever terminates, then the final set S satisfies the independence property.
- **Proof:**
 - Each node joins S only if it has the unique maximum value in its neighborhood, at some phase.
 - When it does, all its neighbors become inactive.

Maximality

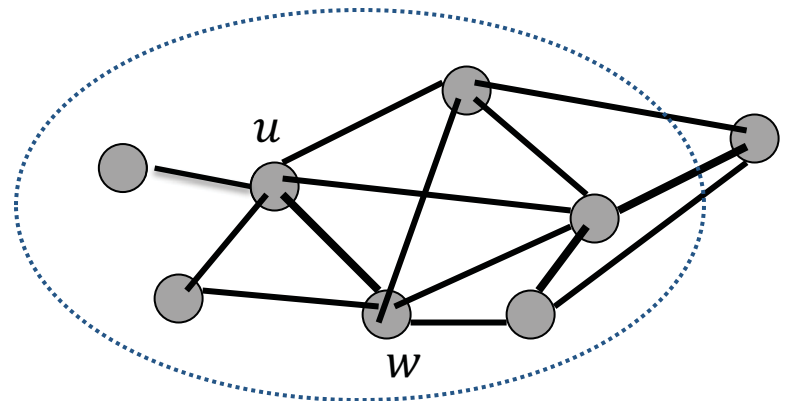
- **Theorem 6:** If Luby's algorithm ever terminates, then the final set S satisfies the maximality property.
- **Proof:**
 - A node becomes inactive only if it joins S or a neighbor joins S .
 - We continue until all nodes are inactive.

Termination

- With probability 1, Luby's MIS algorithm eventually terminates.
- **Theorem 7:** With probability at least $1 - \frac{1}{n}$, all nodes decide within $4 \log n$ phases.
- Proof uses a lemma similar to before:
- **Lemma 8:** With probability at least $1 - \frac{1}{n^2}$, in each phase $1, \dots, 4 \log n$, all nodes choose different random values.
- So we can essentially pretend that, in each phase, all the random numbers chosen are different.
- **Key idea:** Show the graph gets sufficiently “smaller” in each phase.
- **Lemma 9:** For each phase ph , the expected number of **edges** that are live (connect two active nodes) at the end of the phase is at most half the number that were live at the beginning of the phase.

Termination

- **Lemma 9:** For each phase ph , the expected number of edges that are live (connect two active nodes) after the phase is at most half the number that were live before the phase.
- **Proof:**
 - If node u has some neighbor w whose chosen value is greater than all of w 's neighbors and all of u 's other neighbors, then u must decide **out** in phase ph .
 - Probability that w chooses such a value is at least $\frac{1}{\deg(u) + \deg(w)}$.
 - Then the probability node u is “killed” by some neighbor in this way is at least $\sum_{w \in \Gamma(u)} \frac{1}{\deg(u) + \deg(w)}$.
 - Now consider an edge $\{u, v\}$.



Termination, cont'd

- **Proof:**

- Probability u killed $\geq \sum_{w \in \Gamma(u)} \frac{1}{\deg(u) + \deg(w)}$.
- Probability that edge $\{u, v\}$ “dies”
 $\geq \frac{1}{2}$ (probability u killed + probability v killed).
- So the expected number of edges that die
 $\geq \frac{1}{2} \sum_{\{u,v\}} (\text{probability } u \text{ killed} + \text{probability } v \text{ killed})$.
- The sum includes the “kill probability” for each node u exactly $\deg(u)$ times.
- So rewrite the sum as:
 $\frac{1}{2} \sum_u \deg(u) (\text{probability } u \text{ killed})$.
- Plug in the probability lower bound:
 $\geq \frac{1}{2} \sum_u \deg(u) \sum_{w \in \Gamma(u)} \frac{1}{\deg(u) + \deg(w)}$.
 $= \frac{1}{2} \sum_u \sum_{w \in \Gamma(u)} \frac{\deg(u)}{\deg(u) + \deg(w)}$.

Termination, cont'd

- **Proof:**

- Expected number of edges that die $\geq \frac{1}{2} \sum_u \sum_{w \in \Gamma(u)} \frac{\deg(u)}{\deg(u) + \deg(w)}$.

- Write this expression equivalently as a sum over **directed edges** (u, v) :

$$\frac{1}{2} \sum_{(u,v)} \frac{\deg(u)}{\deg(u) + \deg(v)}.$$

- Here each undirected edge is counted twice, once for each direction, so this is the same as the following sum over **undirected edges** $\{u, v\}$.

$$\frac{1}{2} \sum_{\{u,v\}} \frac{\deg(u) + \deg(v)}{\deg(u) + \deg(v)}.$$

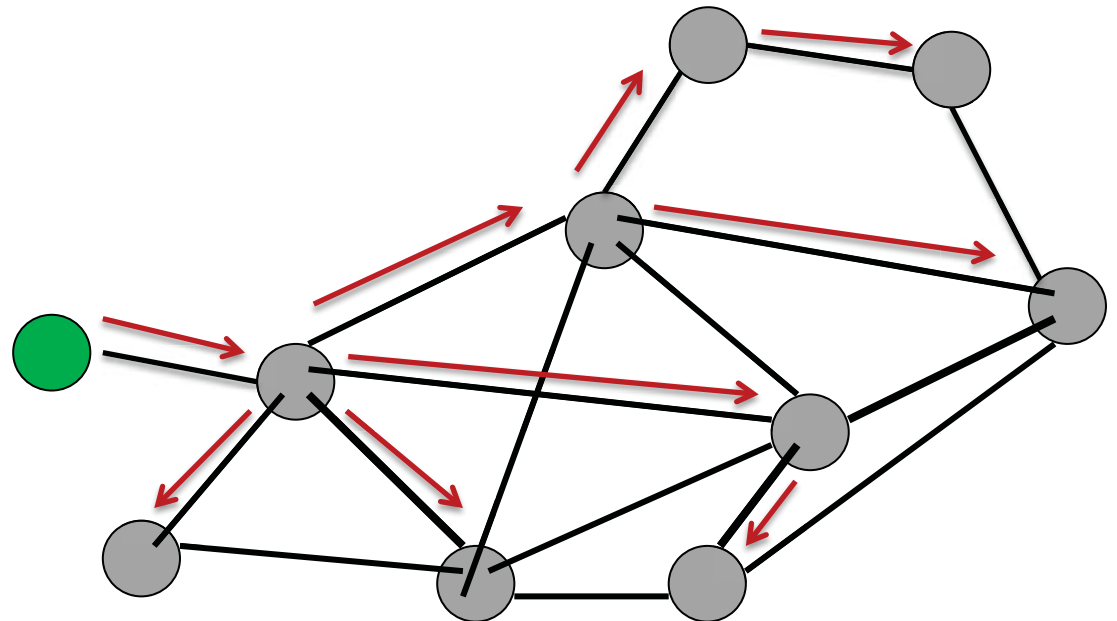
- This is half the total number of undirected edges, as needed!

- **Lemma 9:** For each phase ph , the expected number of edges that are live (connect two active nodes) at the end of the phase is at most half the number that were live at the beginning of the phase.

Termination, cont'd

- **Lemma 9:** For each phase ph , the expected number of edges that are live (connect two active nodes) at the end of the phase is at most half the number that were live before the phase.
- **Theorem 7:** With probability at least $1 - \frac{1}{n}$, all nodes decide within $4 \log n$ phases.
- **Proof sketch:**
 - Lemma 9 implies that the expected number of edges still live after $4 \log n$ phases is at most $\frac{n^2}{2} \div 2^{4 \log n} = \frac{1}{2n^2}$.
 - Then the probability that *any* edges remain live is $\leq \frac{1}{2n^2}$ (by Markov).
 - The probability that the algorithm doesn't terminate within $4 \log n$ phases $\leq \frac{1}{2n^2} + \frac{1}{n^2} < \frac{1}{n}$.

Breadth-First Spanning Trees



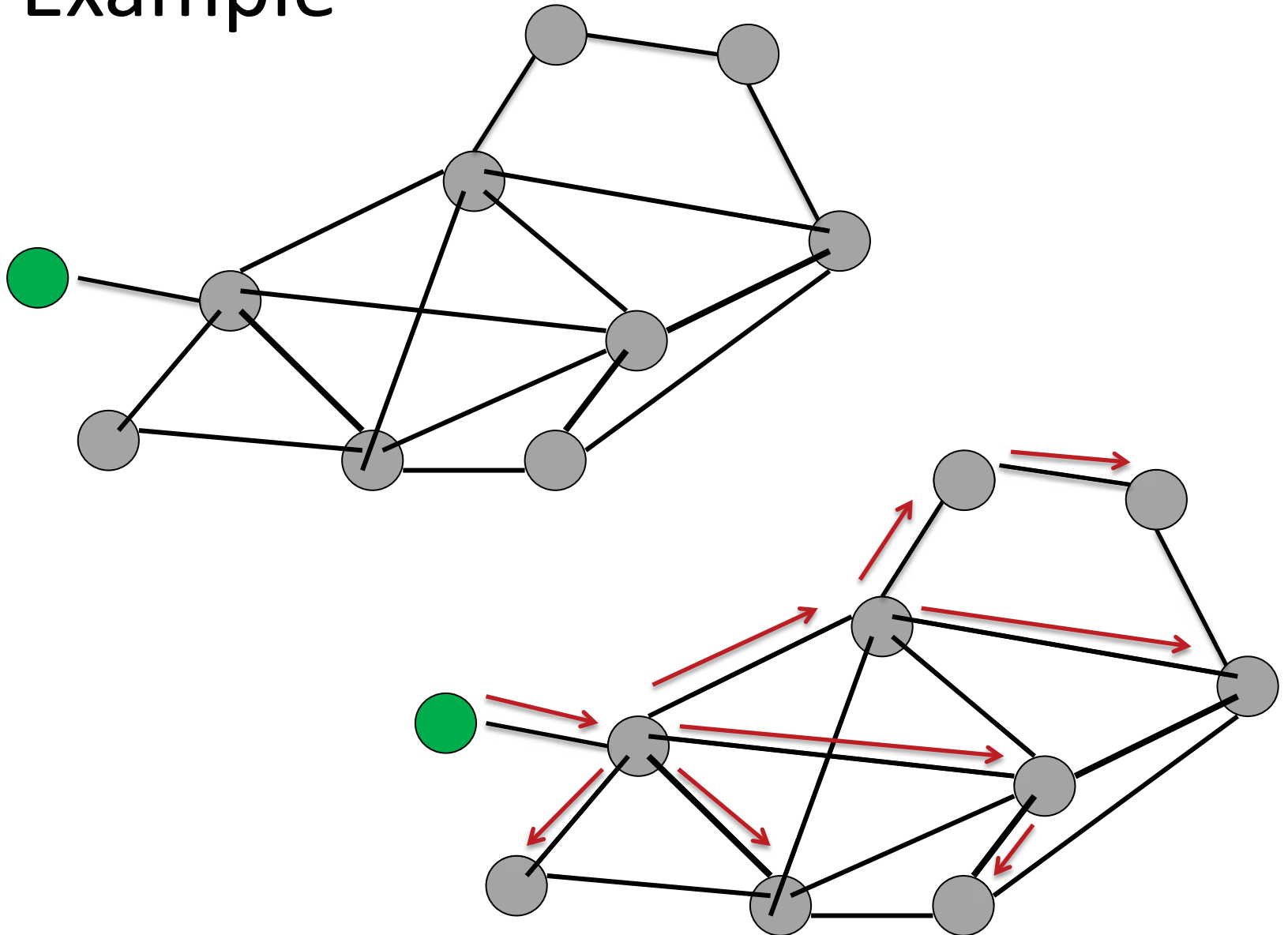
Breadth-First Spanning Trees

- New problem, new setting.
- Assume graph $G = (V, E)$ is connected.
- V includes a distinguished vertex v_0 , which will be the origin (root) of the BFS tree.
- Generally, processes have no knowledge about the graph.
- Processes have UIDs.
 - Each process knows its own UID.
 - i_0 is the UID of the root v_0 .
 - Process with UID i_0 knows it is located at the root.
- We may assume (WLOG) that processes know the UIDs of their neighbors, and know which input and output ports are connected to each neighbor.
- Algorithms will be deterministic (or nondeterministic), but not randomized.

Breadth-First Spanning Trees

- Processes must produce a Breadth-First Spanning Tree rooted at vertex v_0 .
- Branches are directed paths from v_0 .
 - **Spanning:** Branches reach all vertices.
 - **Breadth-first:** Vertex at distance d from v_0 appears at depth exactly d in the tree.
- **Output:** Each process $i \neq i_0$ should output *parent(j)*, meaning that j 's vertex is the parent of i 's vertex in the BFS tree.

Example



Simple BFS Algorithm

- Processes **mark** themselves as they get incorporated into the tree.
- Initially, only i_0 is marked.
- **Algorithm for process i :**
 - **Round 1:**
 - If $i = i_0$ then process i sends a **search** message to its neighbors.
 - If process i receives a message, then it:
 - Marks itself.
 - Selects i_0 as its parent, outputs **parent(i_0)**.
 - Plans to send at the next round.
 - **Round $r > 1$:**
 - If process i planned to send, then it sends a **search** message to its neighbors.
 - If process i is not marked and receives a message, then it:
 - Marks itself.
 - Selects one sending neighbor, j , as its parent, outputs **parent(j)**.
 - Plans to send at the next round.

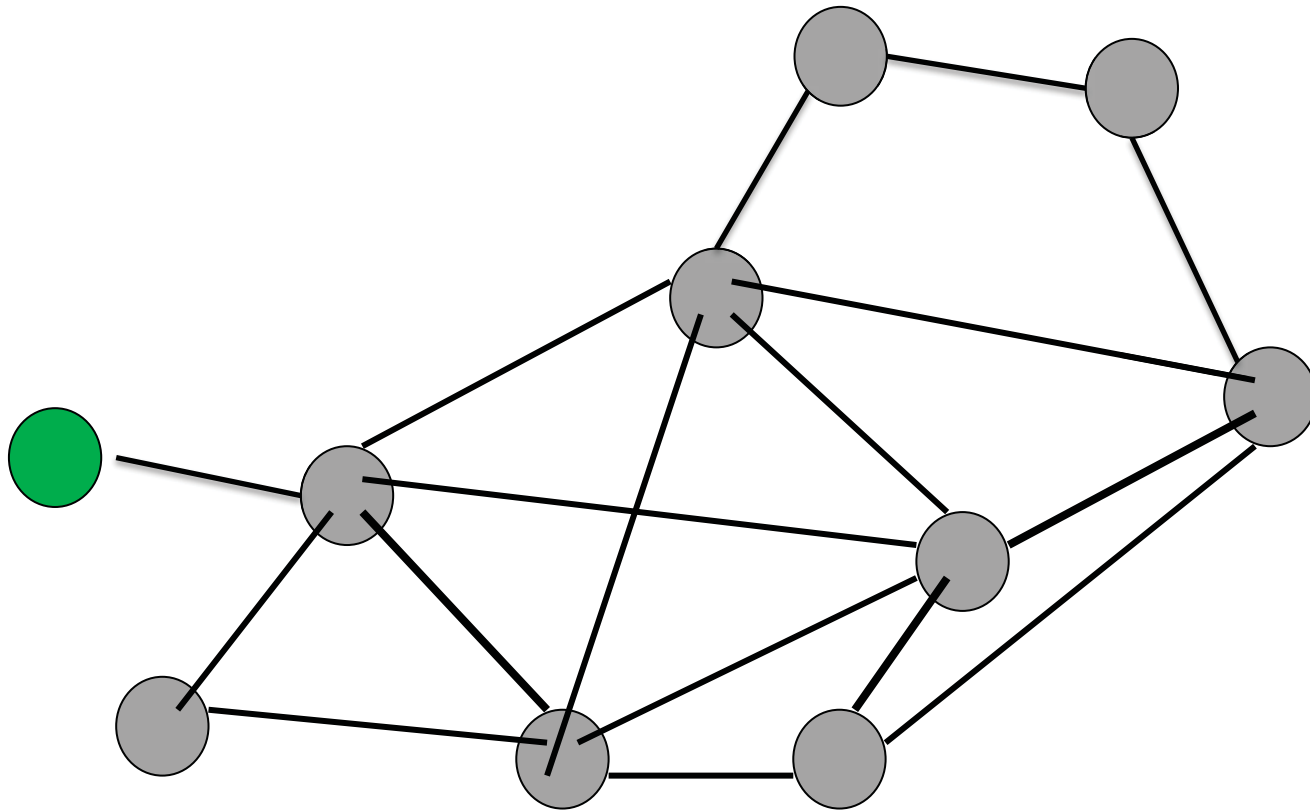
Nondeterminism

- The algorithm is **slightly nondeterministic**, in that a process can choose arbitrarily among several possible parents.
- We could make this deterministic, by using a default, like “always choose the sender with the smallest UID”.
- But it’s also OK to leave it nondeterministic.
- For distributed algorithms, nondeterminism is regarded differently from the way it is for sequential algorithms.
- **A distributed algorithm should work correctly for all ways of resolving the nondeterministic choices.**

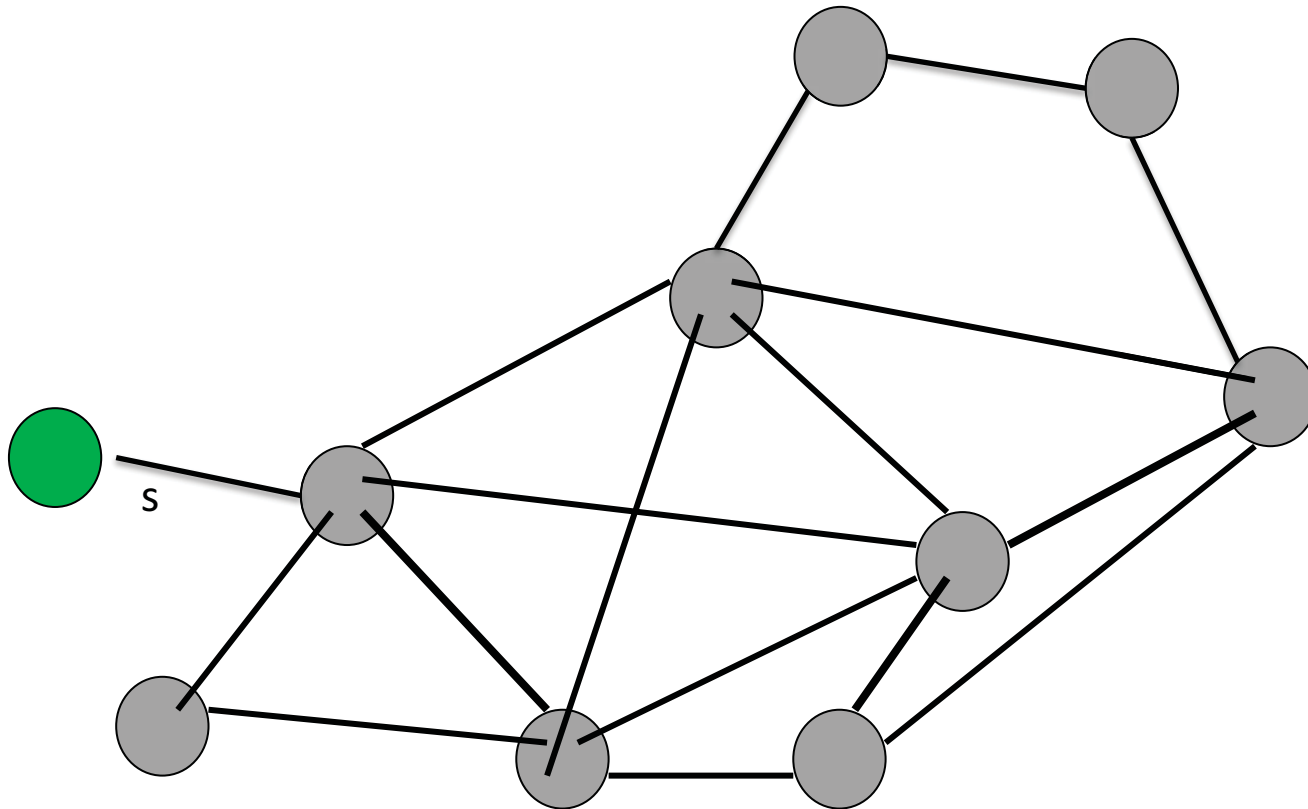
Simple BFS Algorithm

- **Q:** Why does this algorithm yield a BFS tree?
- Because all branches are created synchronously, growing one hop at each round.
- **Q:** Why does it eventually span all the nodes?
- Because the graph is connected, and any marked node sends messages to its neighbors.

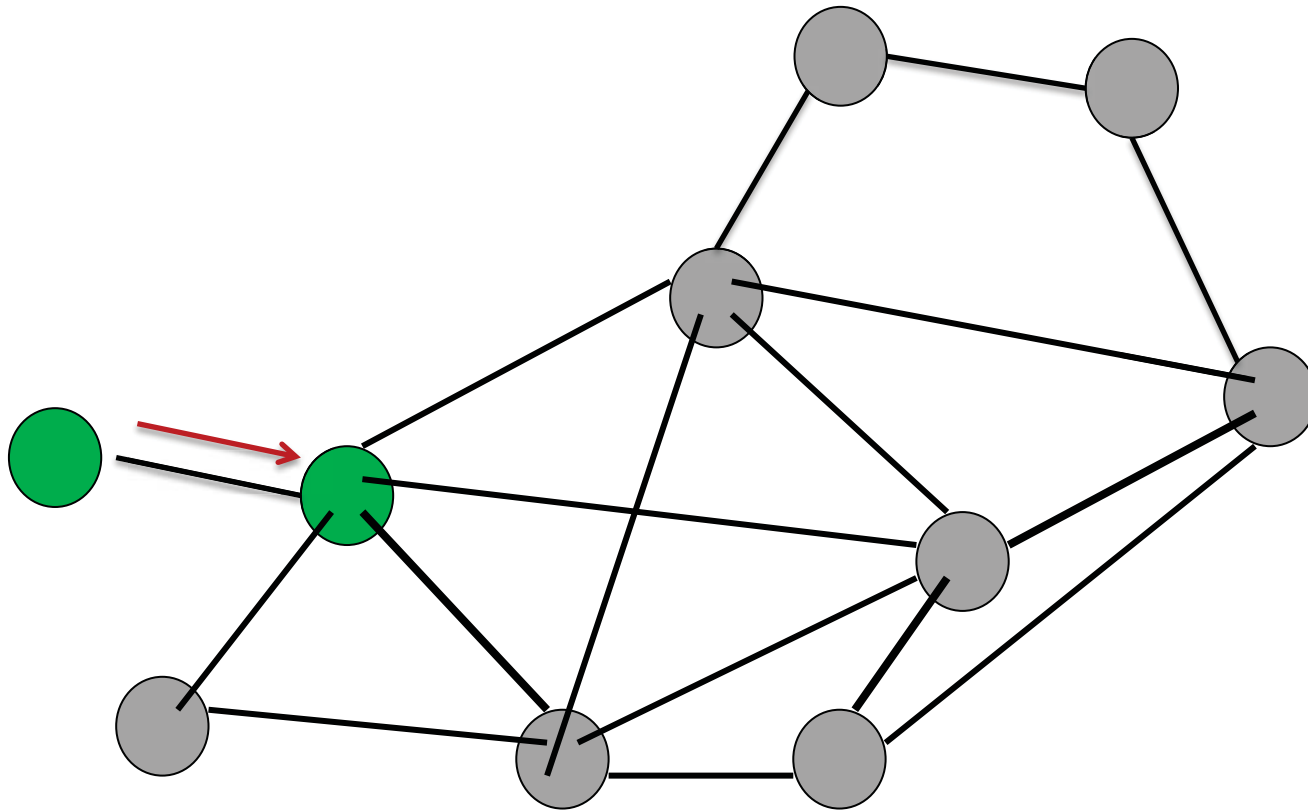
Example: Simple BFS



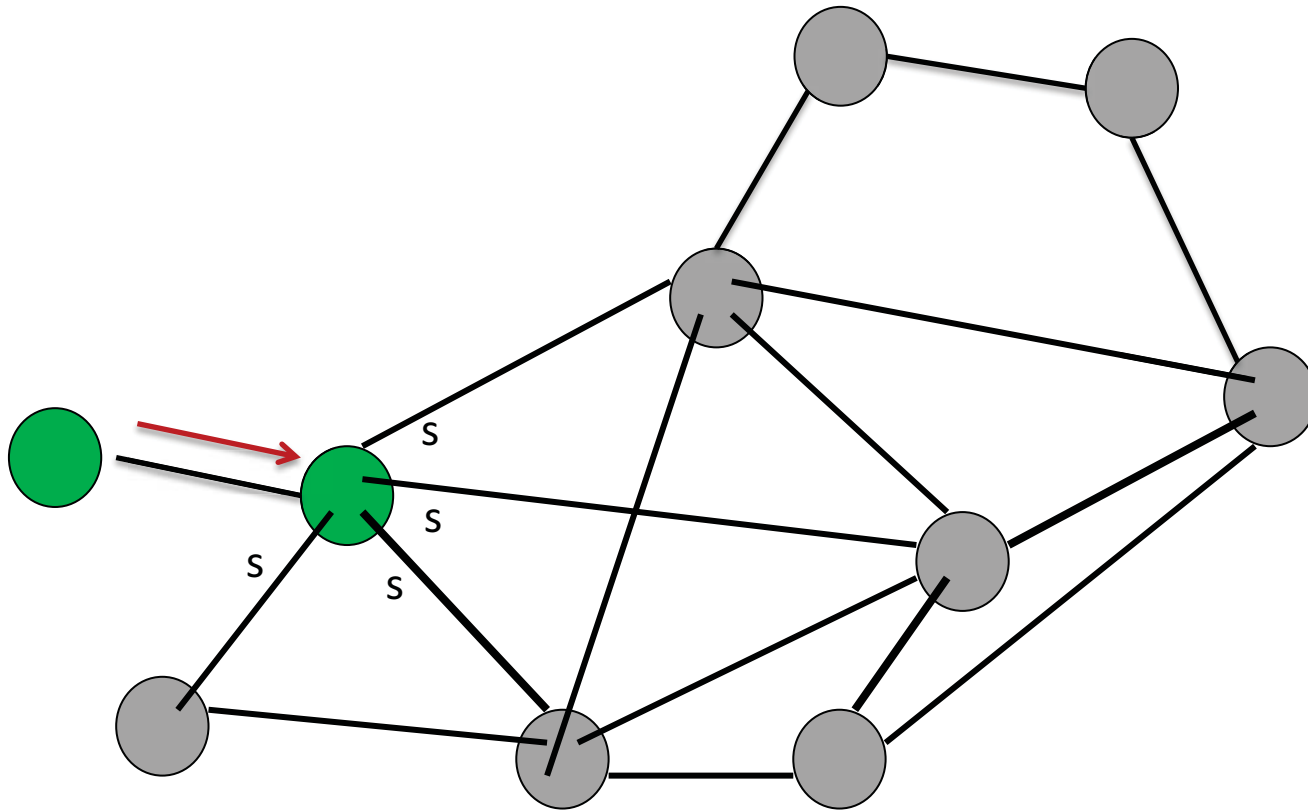
Round 1



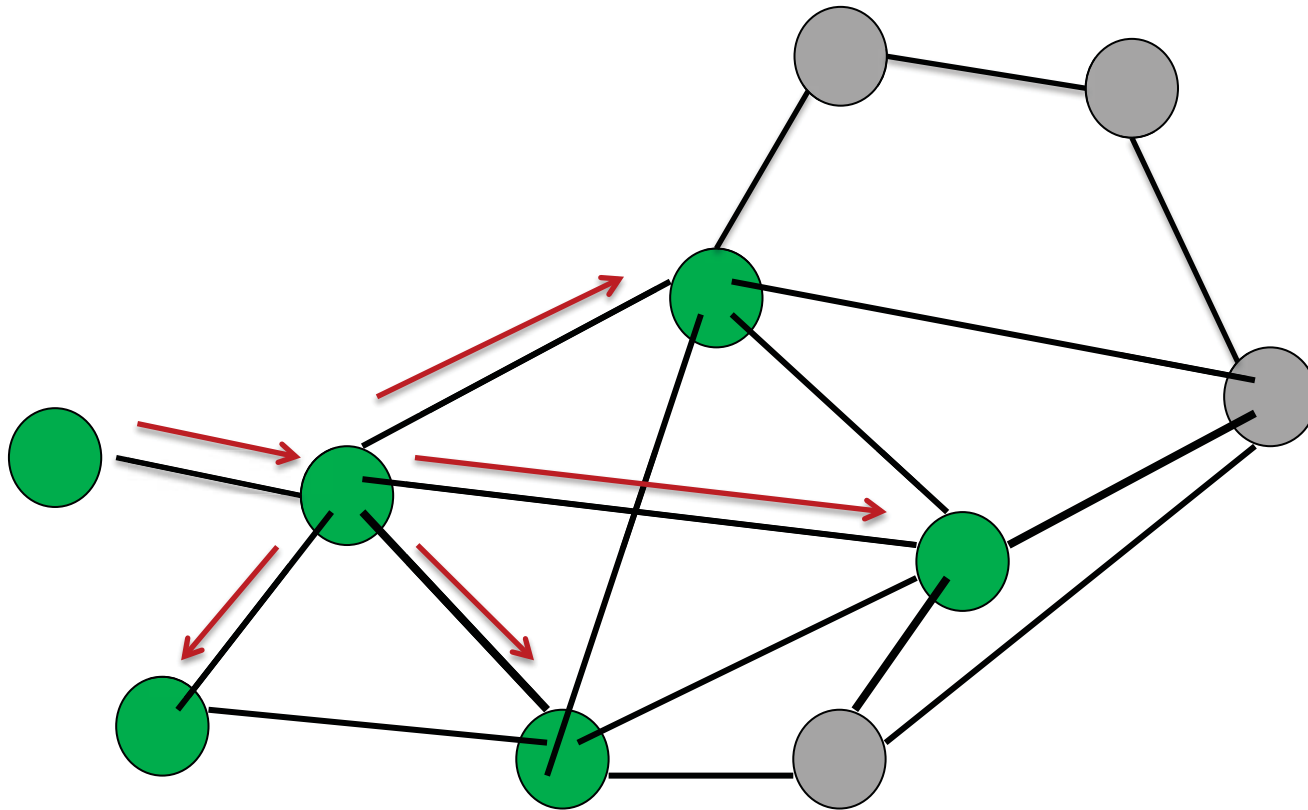
Round 1



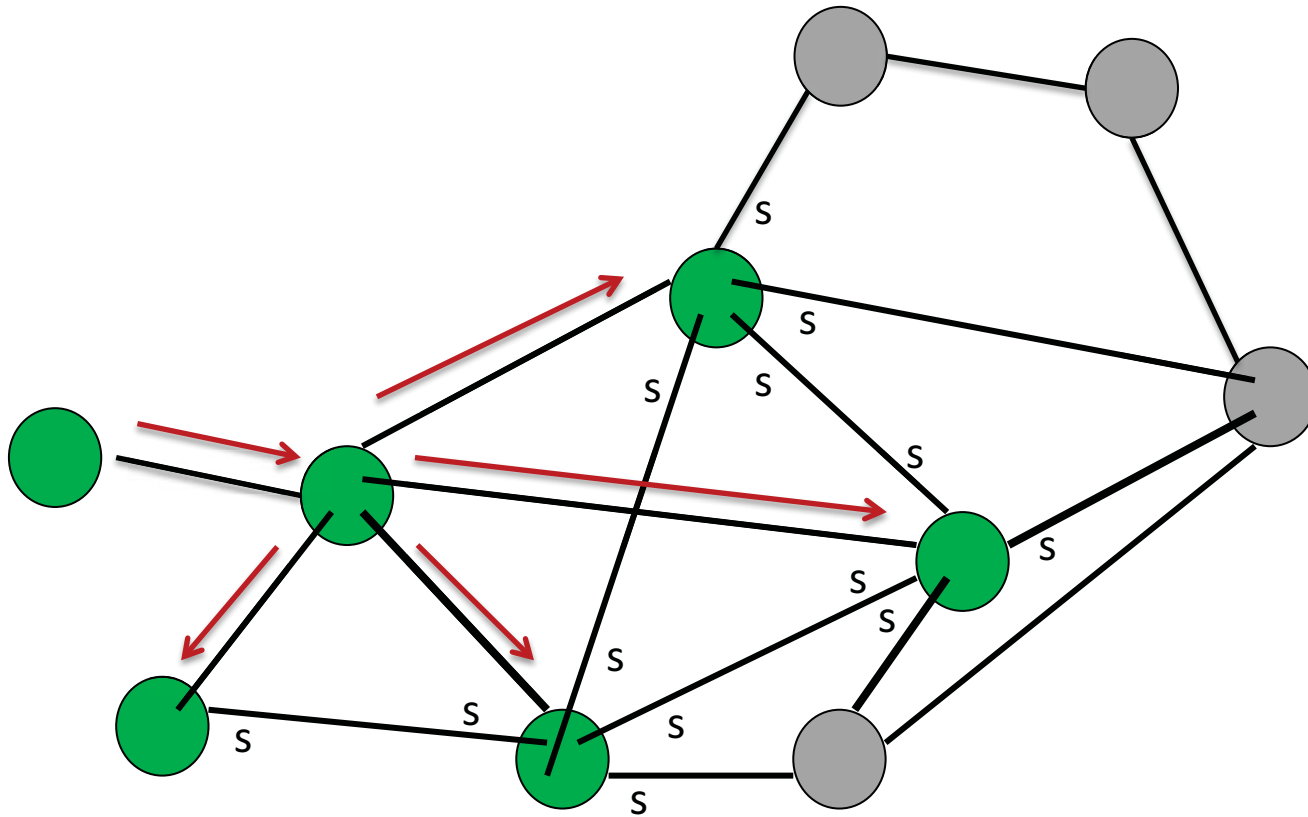
Round 2



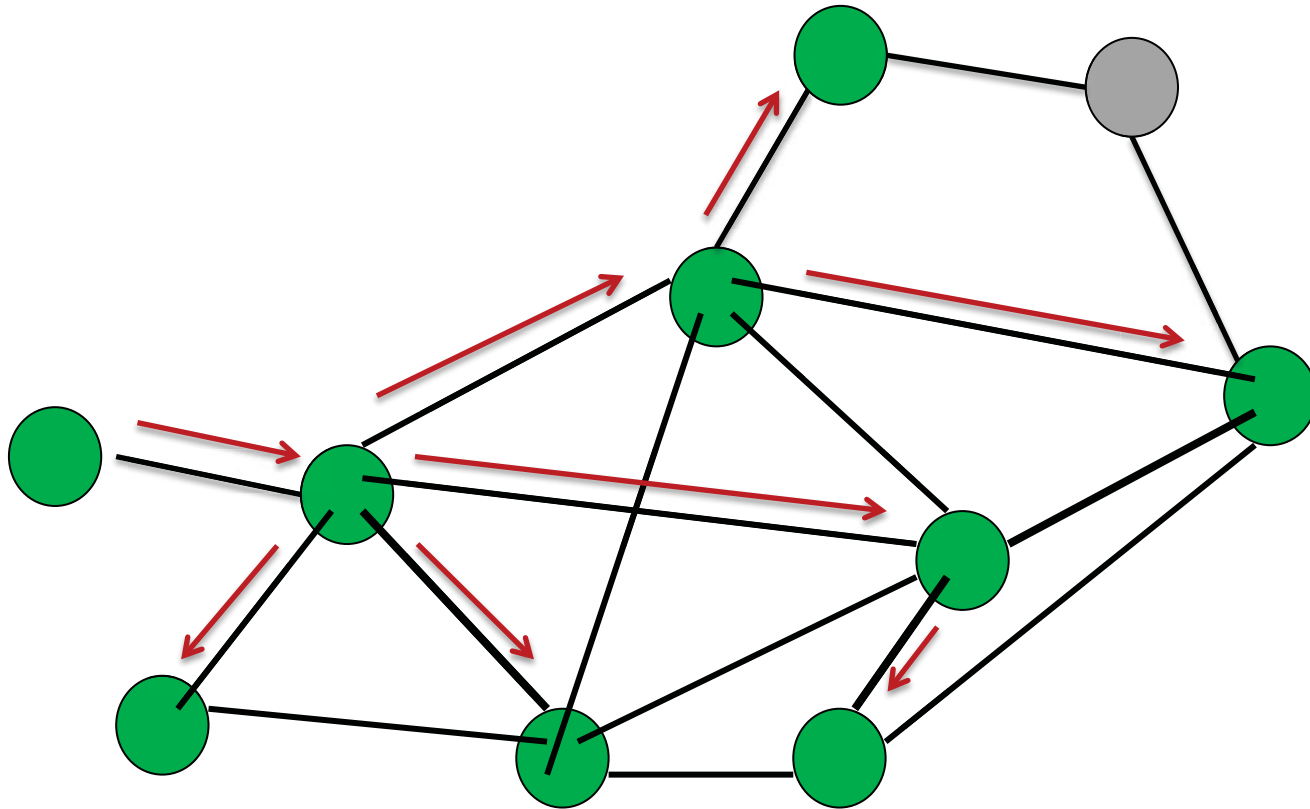
Round 2



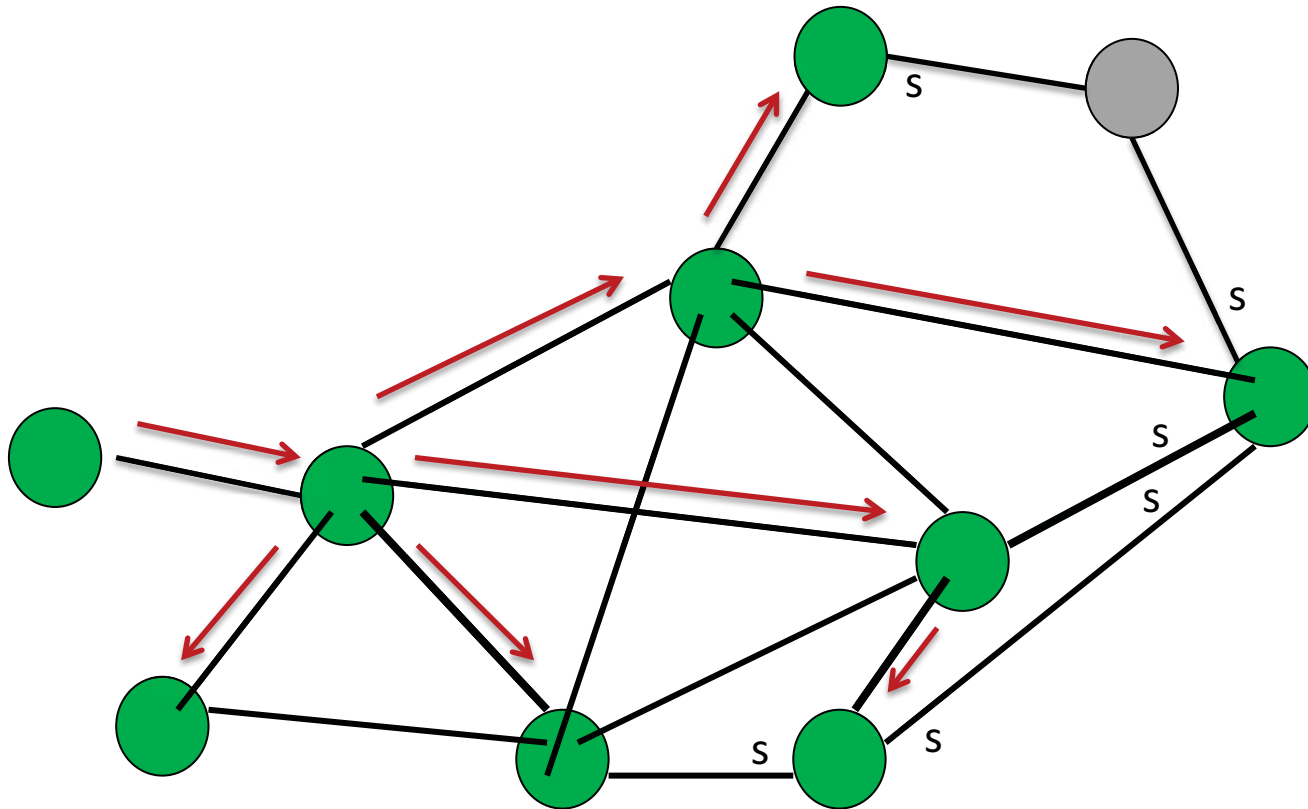
Round 3



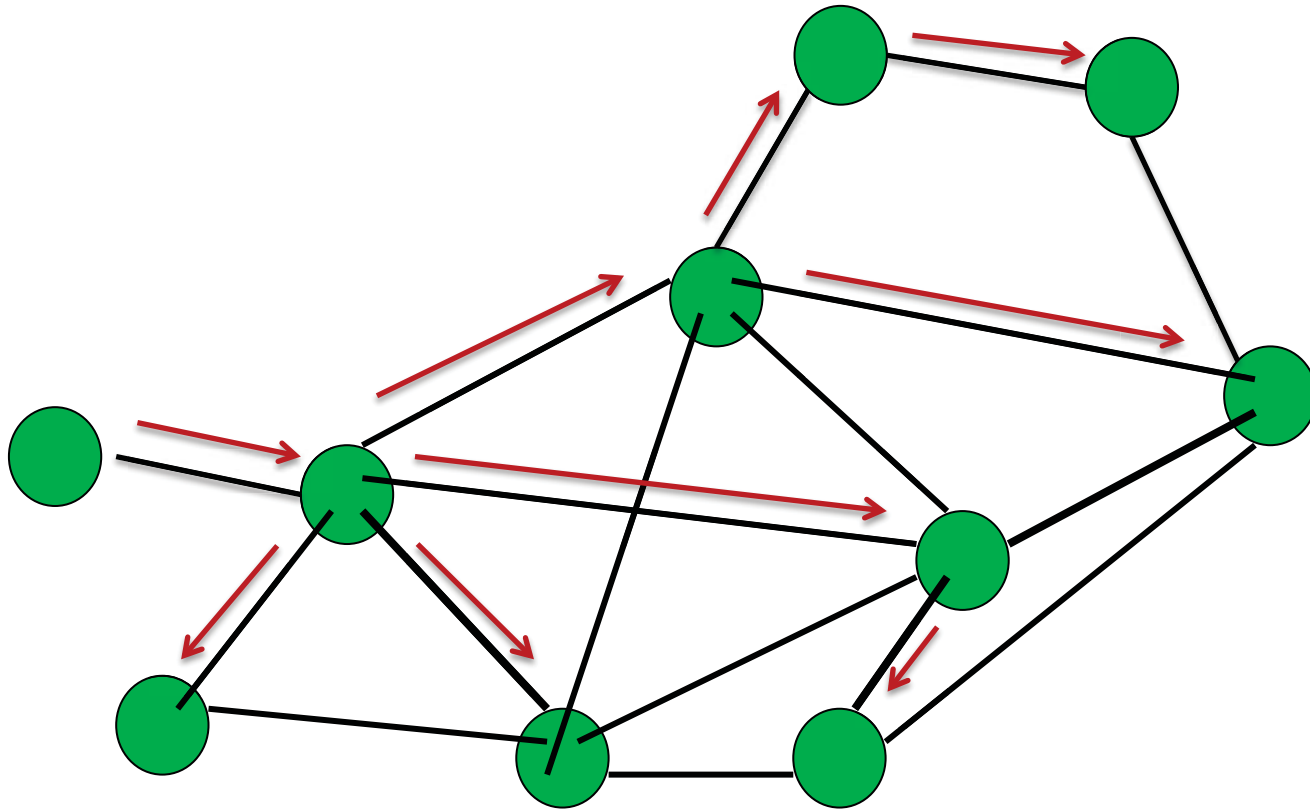
Round 3



Round 4



Round 4



Correctness

- **Q:** Key invariants describing the state after r rounds?
- **State variables, per process:**
 - *marked*, a Boolean, initially true for i_0 , false for others
 - *parent*, a UID or undefined
 - *send*, a Boolean, initially true for i_0 , false for others
 - *uid*
- **Invariants:**
 - At the end of r rounds, exactly the processes at distance $\leq r$ from v_0 are marked.
 - A process $\neq i_0$ has its *parent* defined iff it is marked.
 - For any process at distance d from v_0 , if its *parent* is defined, then it is the UID of a process at distance $d - 1$ from v_0 .
- Could use these in a formal correctness proof.

Complexity

- Time complexity:
 - Number of rounds until all nodes outputs their parent information.
 - Maximum distance of any node from v_0 , which is $\leq diam$
- Message complexity:
 - Number of messages sent by all processes during the entire execution.
 - $O(|E|)$

Bells and Whistles

- **Child pointers:**
 - So far, each process learns its parent, but not its children.
 - To add child pointers, everyone who receives a *search* message sends back a *parent* or *nonparent* response, at the next round.
- **Distances:**
 - Augment the algorithm so everyone also learns and outputs its distance from v_0 .
 - Each process records its distance from v_0 in a new *dist* variable, initially 0 for process i_0 , and ∞ for everyone else.
 - Include the *dist* value in every *search* message; when an unmarked process receives a *search*(d) message, it sets its own *dist* to $d + 1$.

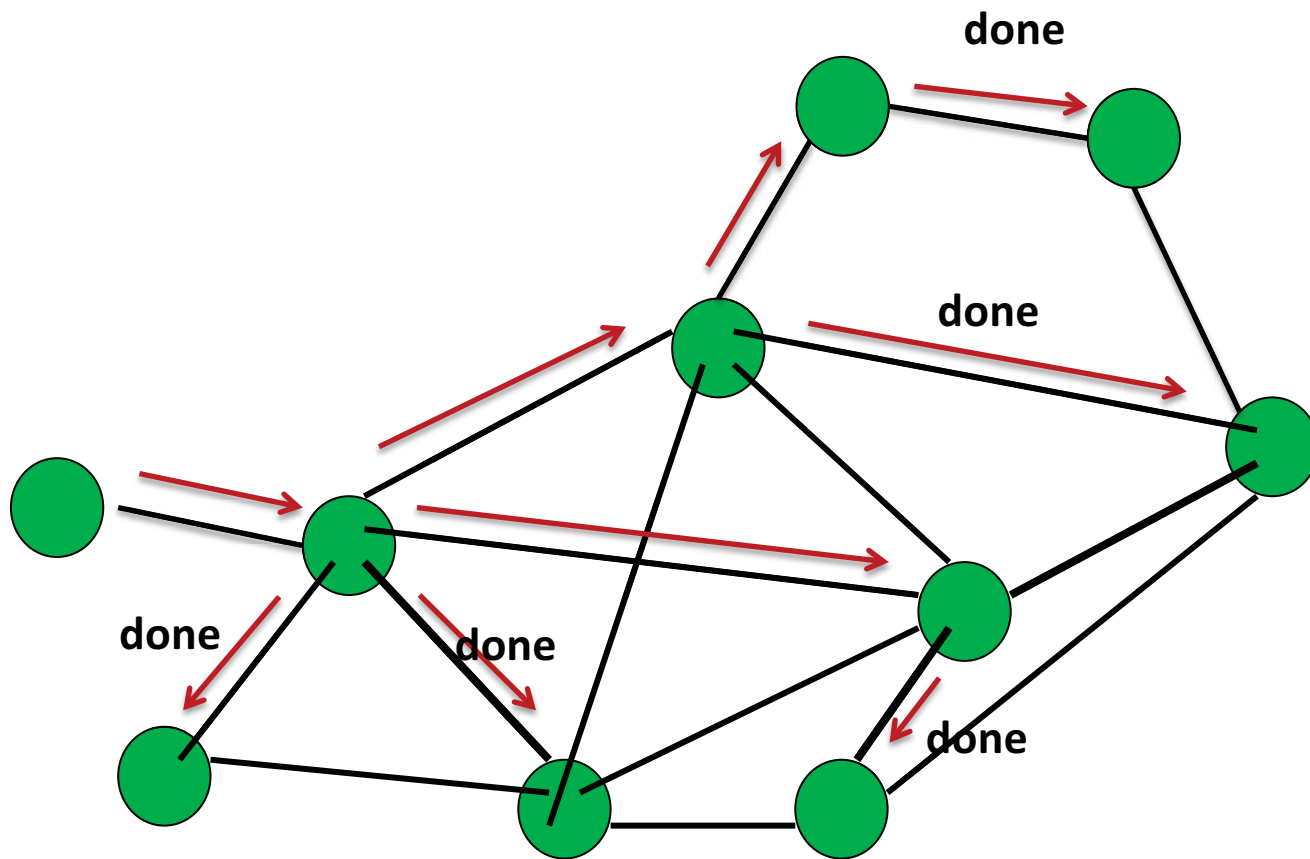
Termination

- **Q:** How can processes learn when the BFS tree is completed?
- If they knew an upper bound on *diam*, then they could simply wait until that number of rounds have passed.
- **Q:** What if they don't know anything about the graph?
- **Slightly simpler problem:** Process i_0 should learn when the tree is done.
- **Termination algorithm:**
 - Assume each *search* message receives a response, *parent* or *nonparent*.
 - After a node has received responses to all its outgoing *search* messages, it knows who its children are, and knows they are all marked.
 - The leaves of the tree learn who they are (they receive only *nonparent* responses).
 - Now use a **convergecast** strategy:

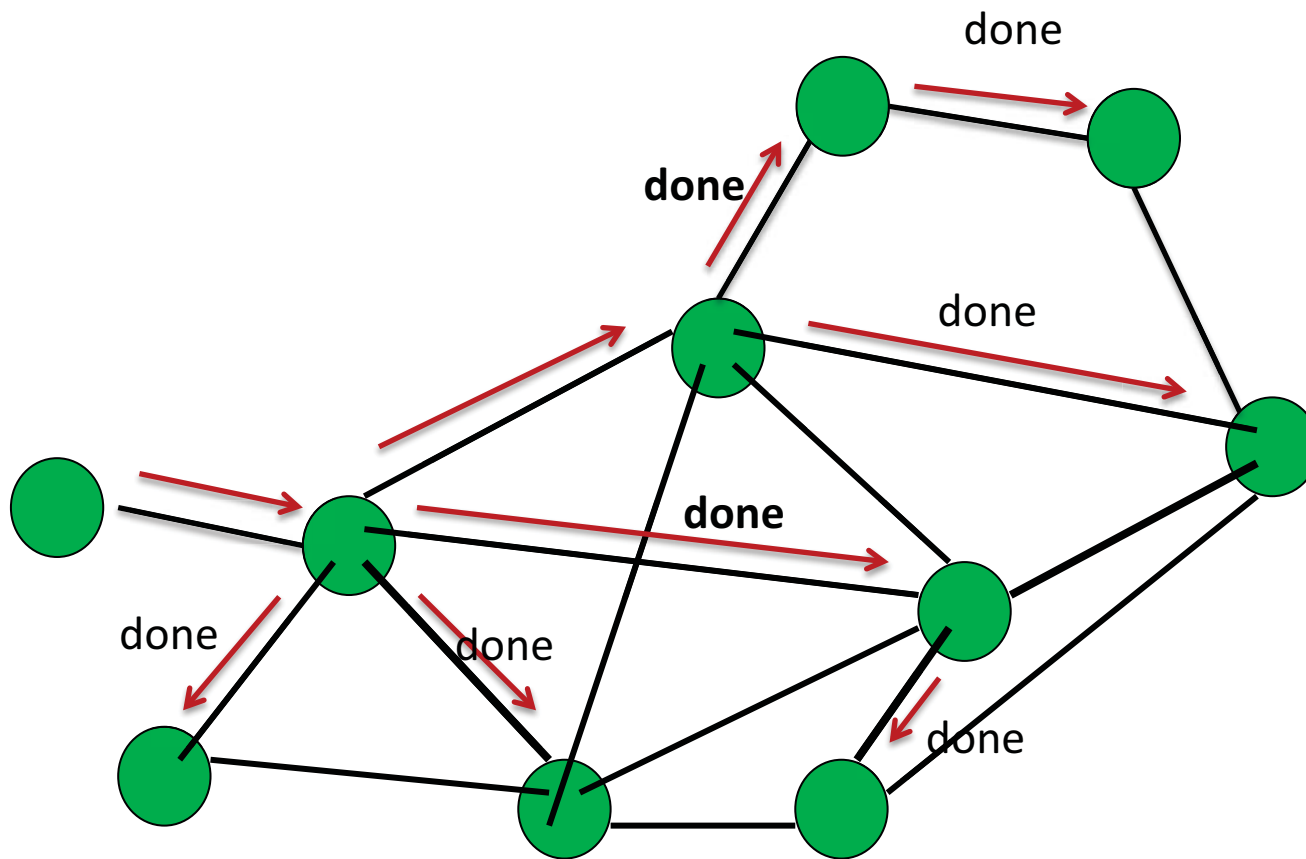
Termination

- Termination algorithm:
 - After a node has received responses to all its *search* messages, it knows who its children are, and knows they are marked.
 - The leaves of the tree learn who they are.
 - Use a *convergecast* strategy:
 - Starting from the leaves, the processes fan in *done* messages up the BFS tree, toward i_0 .
 - A process $\neq i_0$ can send a *done* message to its parent after:
 - It has received responses to all its *search* messages (so it knows who its children are), and
 - It has received *done* messages from all its children.
 - i_0 knows that the BFS tree is completed, when:
 - It has received responses to all its *search* messages (so it knows who its children are), and
 - It has received *done* messages from all its children.

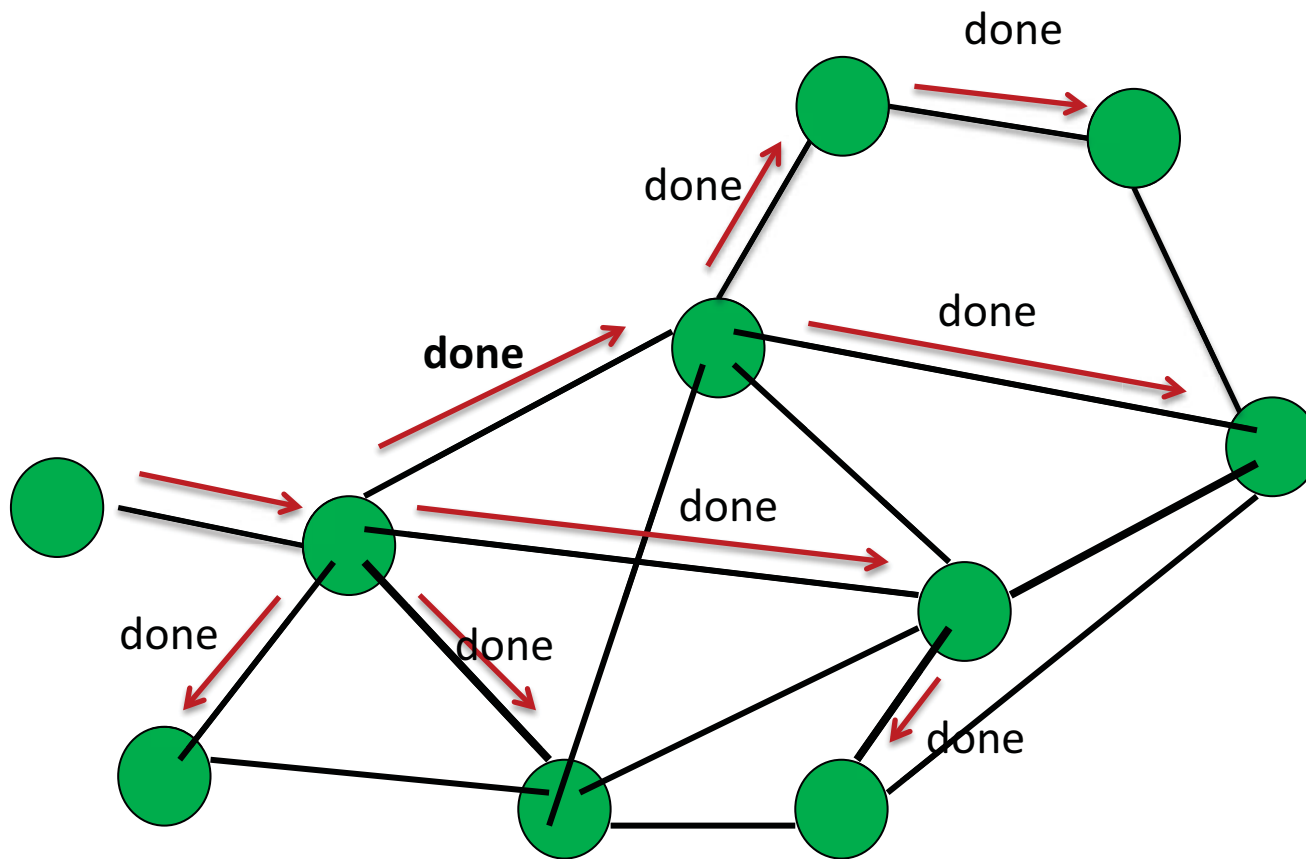
Termination Using Convergecast



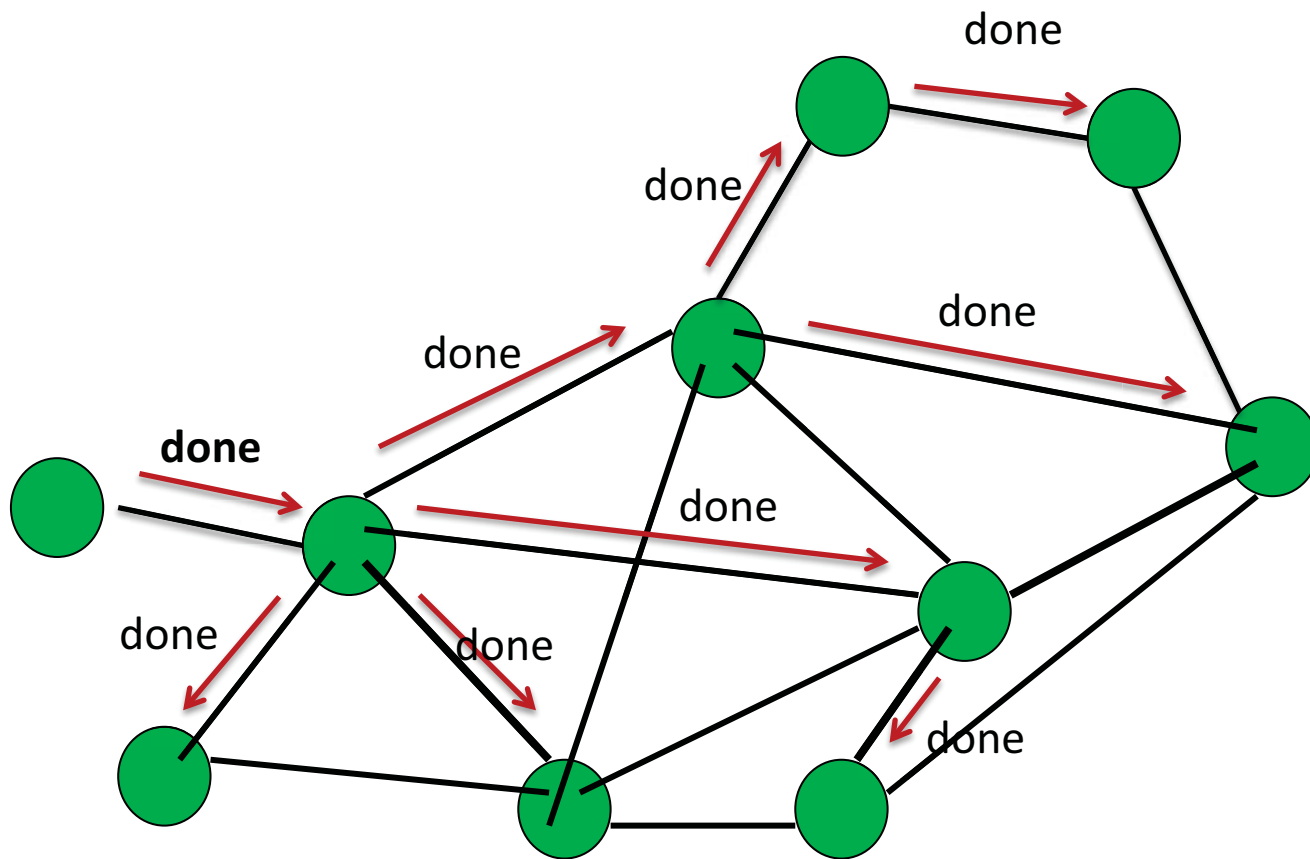
Termination Using Convergecast



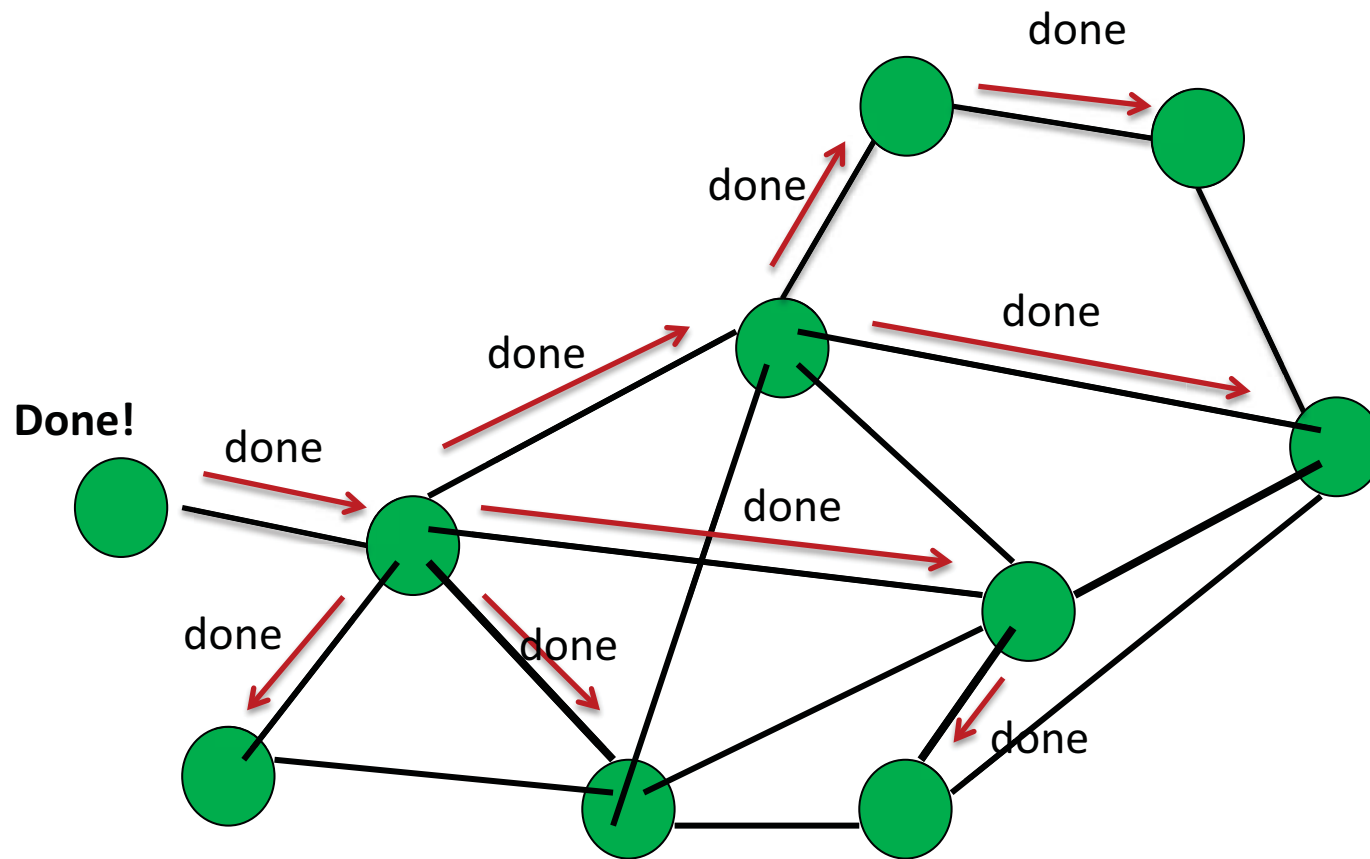
Termination Using Convergecast



Termination Using Convergecast



Termination Using Convergecast



Complexity for Termination

- After the tree is done, it takes $\leq diam$ rounds and n messages for the *done* information to reach i_0 .
- *Q*: How can all the processes learn that the BFS tree is completed?
- Process i_0 can broadcast a message to everyone along the edges of the tree.
- Takes an additional $diam$ rounds and n messages.

Applications of Breadth-First Spanning Trees

- **Message broadcast:**

- Process i_0 can use a BFS tree with child pointers to broadcast a sequence of messages to all processes in the network.
- Each takes $diam$ rounds and n messages.
- Could pipeline them, so k messages take only $diam + k$ rounds, not $diam \times k$.

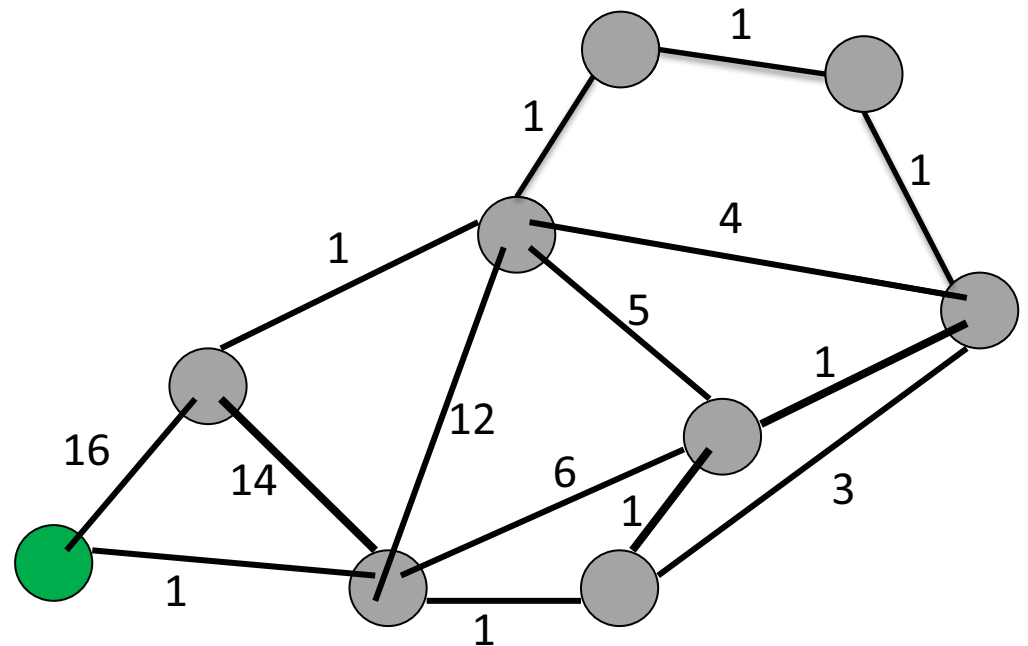
- **Global computation:**

- Suppose every process starts with some initial value, and process i_0 should determine the value of some function of the set of all processes' values.
 - min, max, sum, average,...
- Can use convergecast on a BFS tree.
- Cost is $diam$ rounds and n messages.
- In general, messages may be large, but for many functions, some data aggregation is possible along the way.

Application of the BFS construction

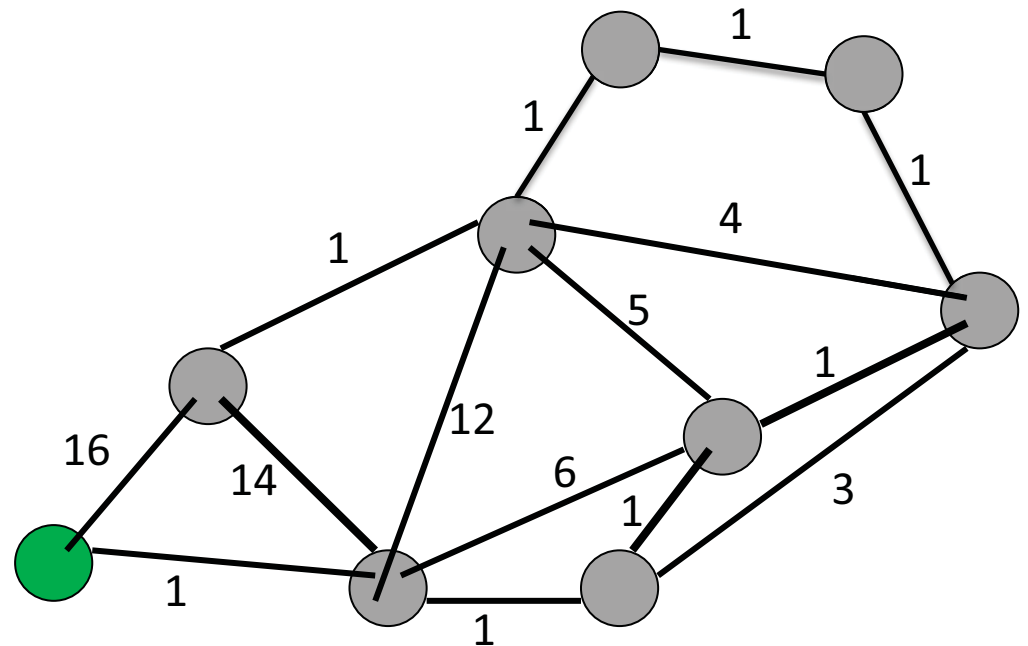
- Leader election in a general graph:
 - No distinguished process i_0 .
 - Processes don't know anything about the graph.
 - Everyone can start its own BFS, acting as the root.
 - Use this to determine the maximum UID; process with the max UID is the leader.
 - Cost is $O(diam)$ rounds, $O(n |E|)$ messages.

Shortest Paths Trees



Shortest Paths

- Generalize the BFS problem to allow weights on the graph edges, *weight*_{u,v} for edge {u, v}
- Connected graph $G = (V, E)$, root vertex v_0 , process i_0 .
- Processes have UIDs.
- Processes know their neighbors and the weights of their incident edges, but otherwise have no knowledge about the graph.



Shortest Paths

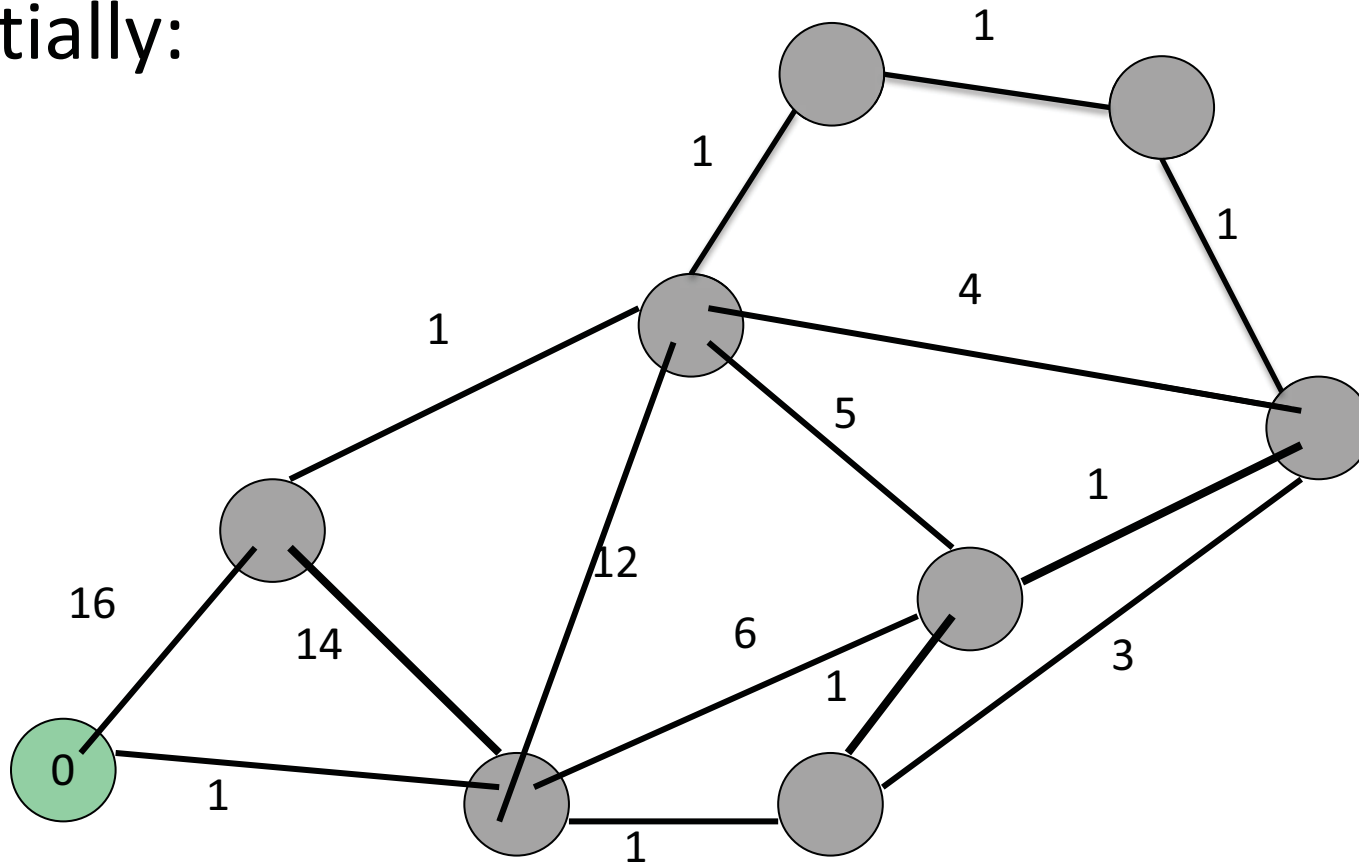
- Processes must produce a Shortest-Paths Spanning Tree rooted at vertex v_0 .
- Branches are directed paths from v_0 .
 - **Spanning:** Branches reach all vertices.
 - **Shortest paths:** The total weight of the tree branch to each node is the minimum total weight for any path from v_0 in G .
- **Output:** Each process $i \neq i_0$ should output *parent(j), distance(d)*, meaning that:
 - j 's vertex is the parent of i 's vertex on a shortest path from v_0 ,
 - d is the total weight of a shortest path from v_0 to j .

Bellman-Ford Shortest Paths Algorithm

- **State variables:**
 - *dist*, a nonnegative real or ∞ , representing the shortest known distance from v_0 . Initially 0 for process i_0 , ∞ for the others.
 - *parent*, a UID or undefined, initially undefined.
 - *uid*
- **Algorithm for process i :**
 - At each round:
 - Send a *distance*(*dist*) message to all neighbors.
 - Receive messages from neighbors; let d_j be the distance received from neighbor j .
 - Perform a **relaxation step**:
$$dist := \min(dist, \min_j (d_j + weight_{\{i,j\}})).$$
 - If *dist* decreases then set *parent* $:= j$, where j is any neighbor that produced the new *dist*.

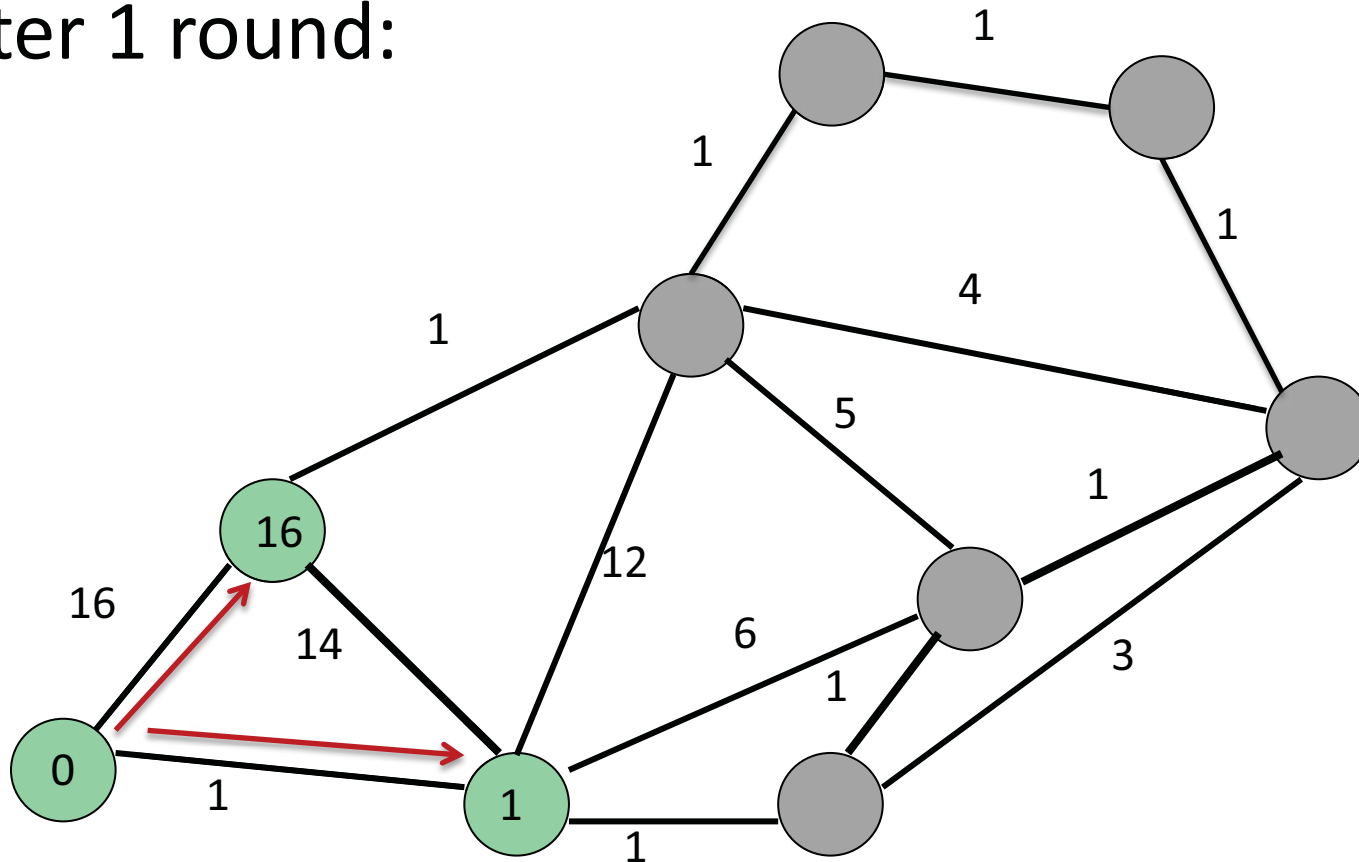
Example: Bellman-Ford

- Initially:



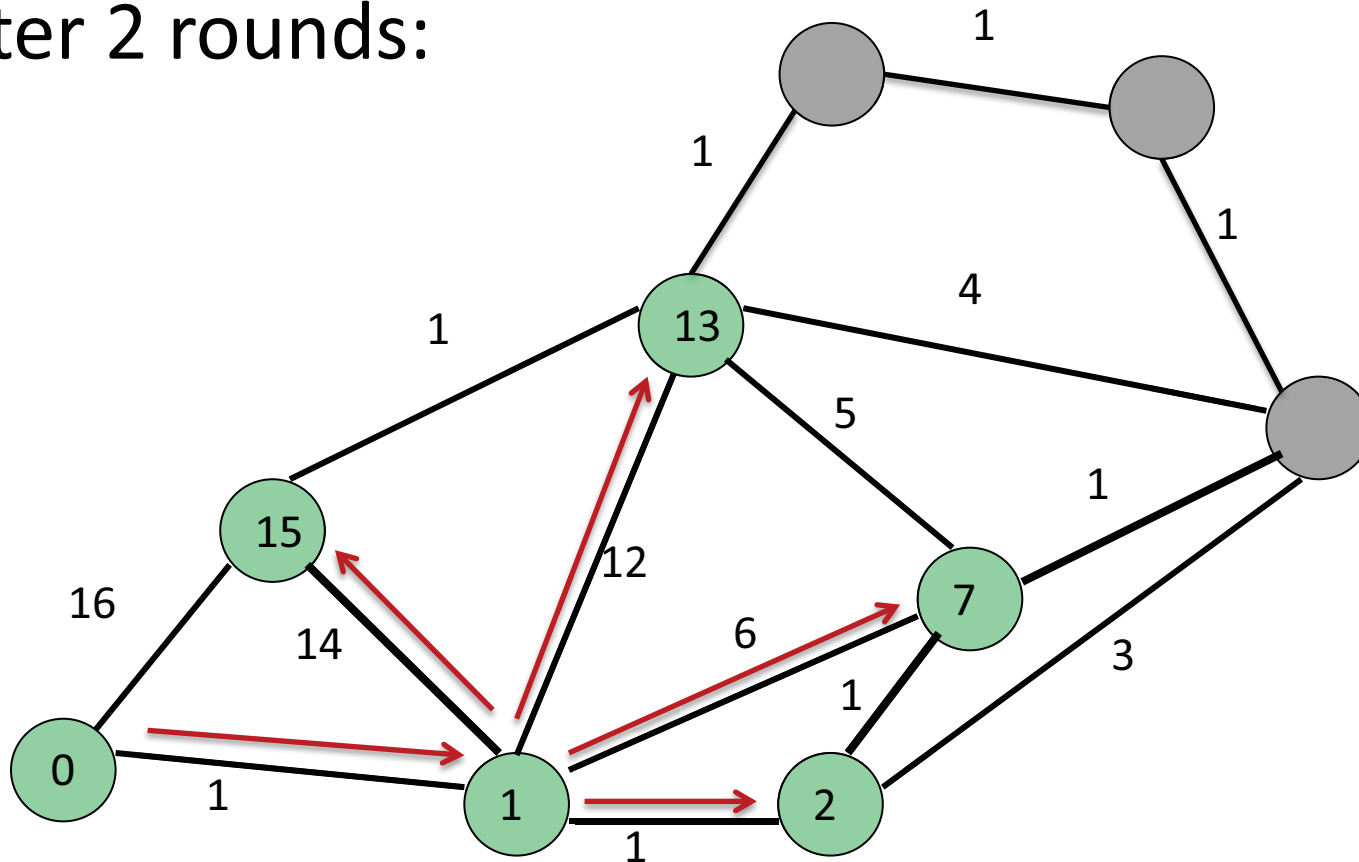
Bellman-Ford

- After 1 round:



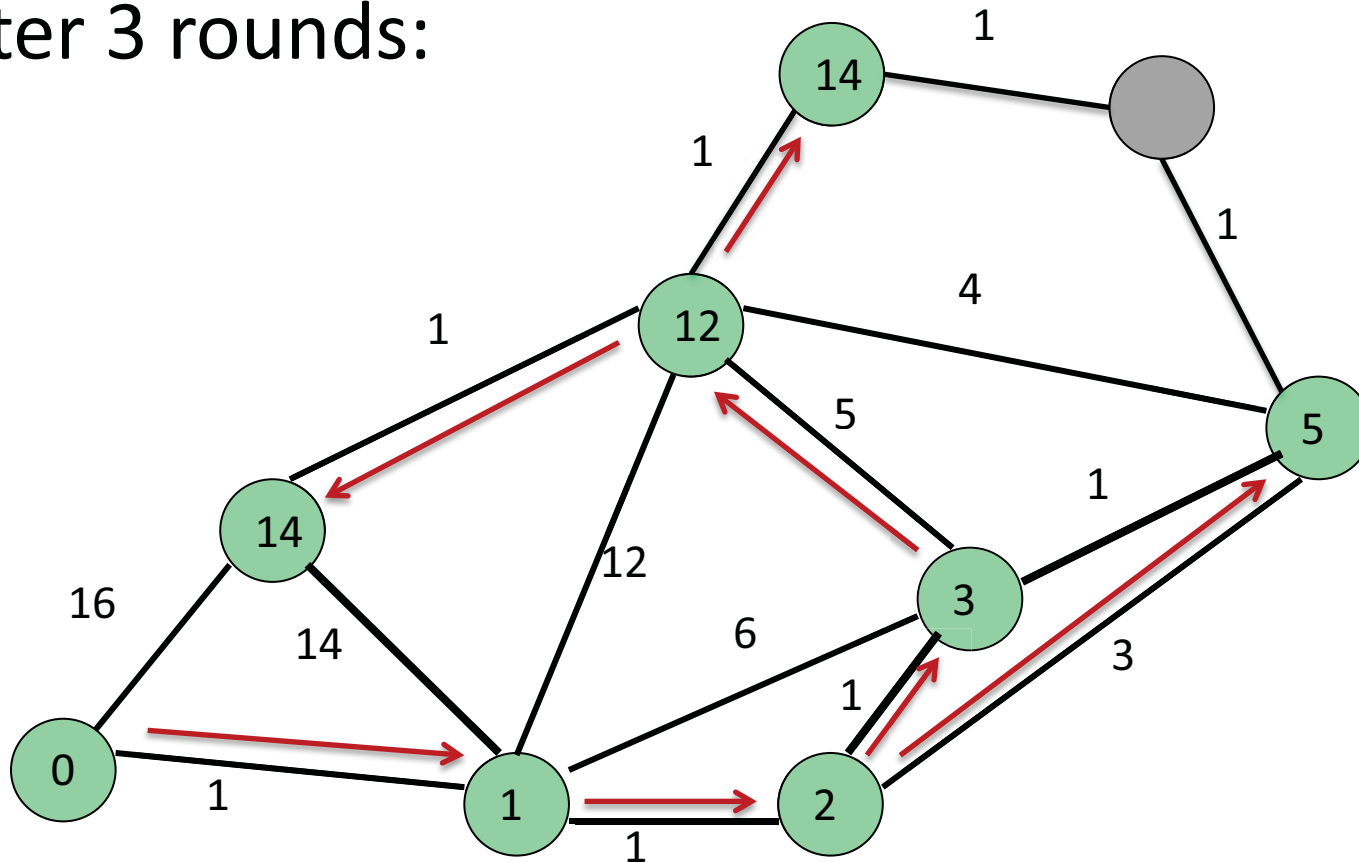
Bellman-Ford

- After 2 rounds:



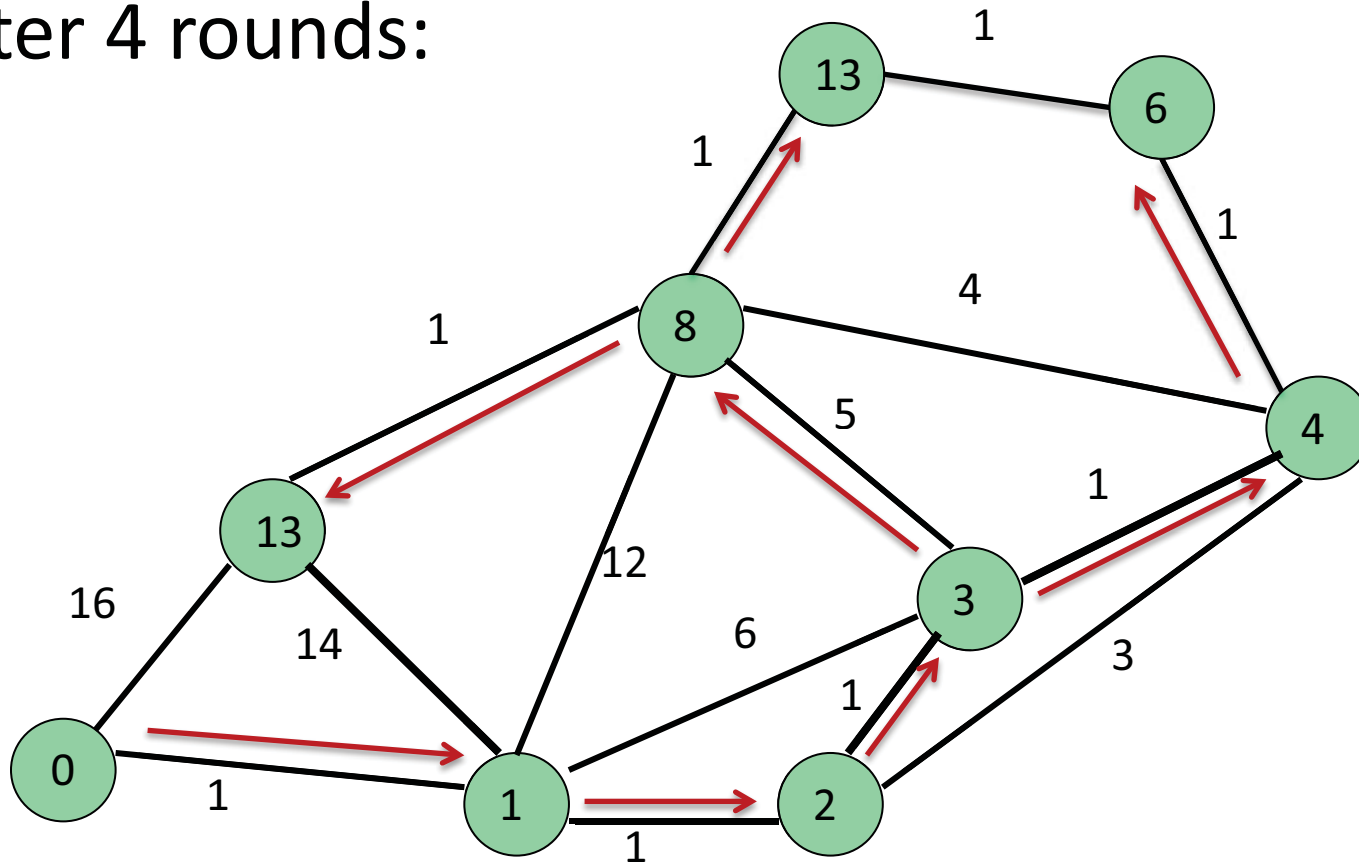
Bellman-Ford

- After 3 rounds:



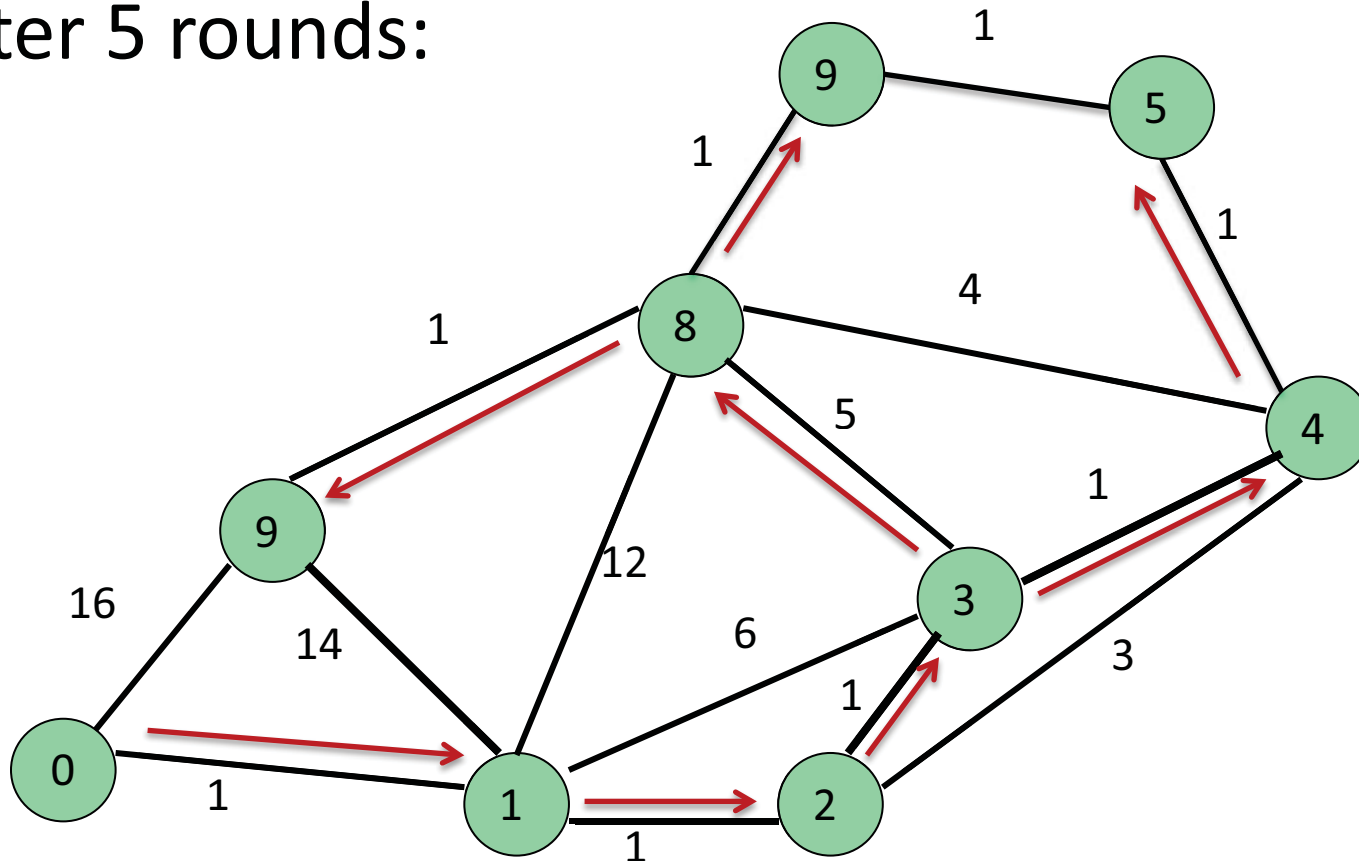
Bellman-Ford

- After 4 rounds:



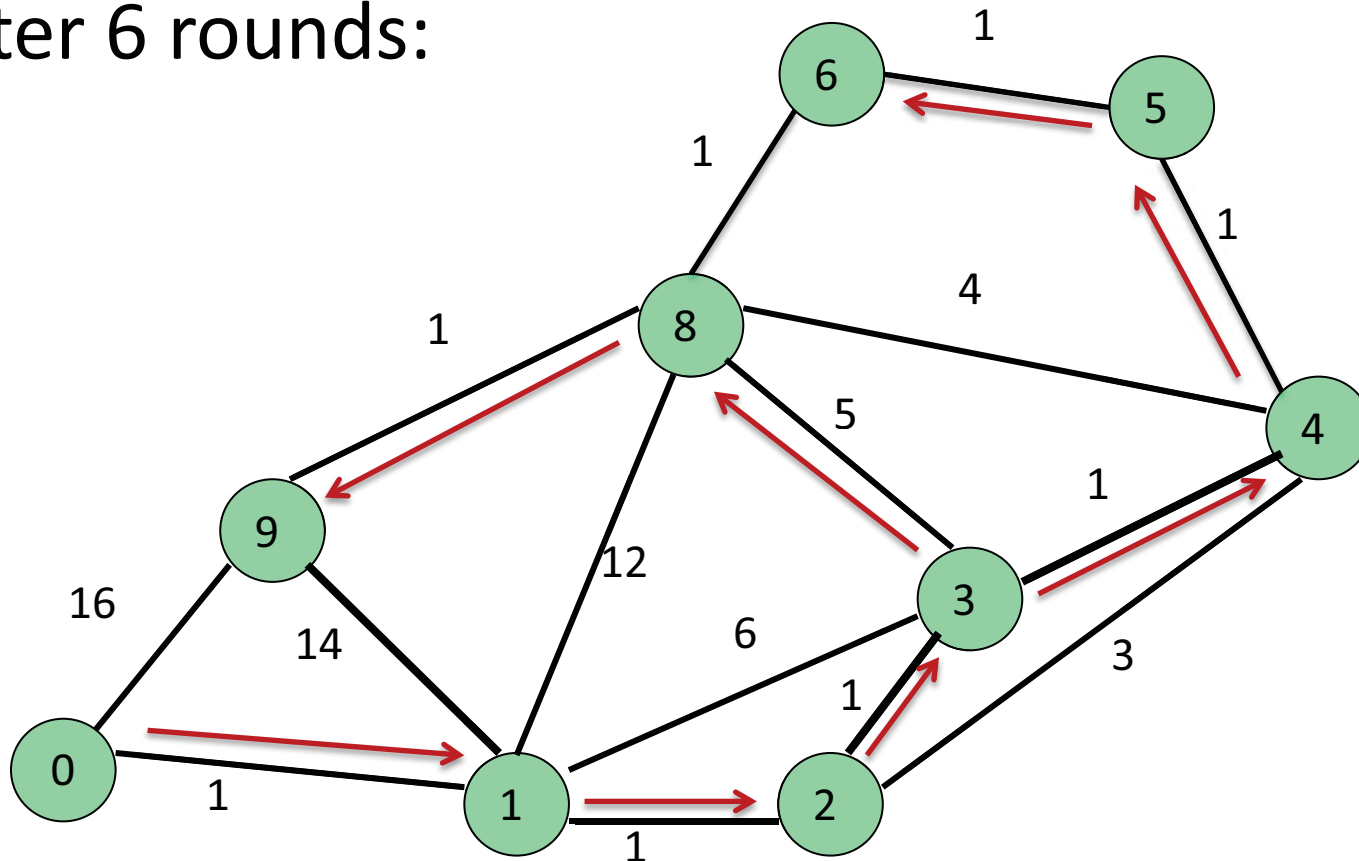
Bellman-Ford

- After 5 rounds:



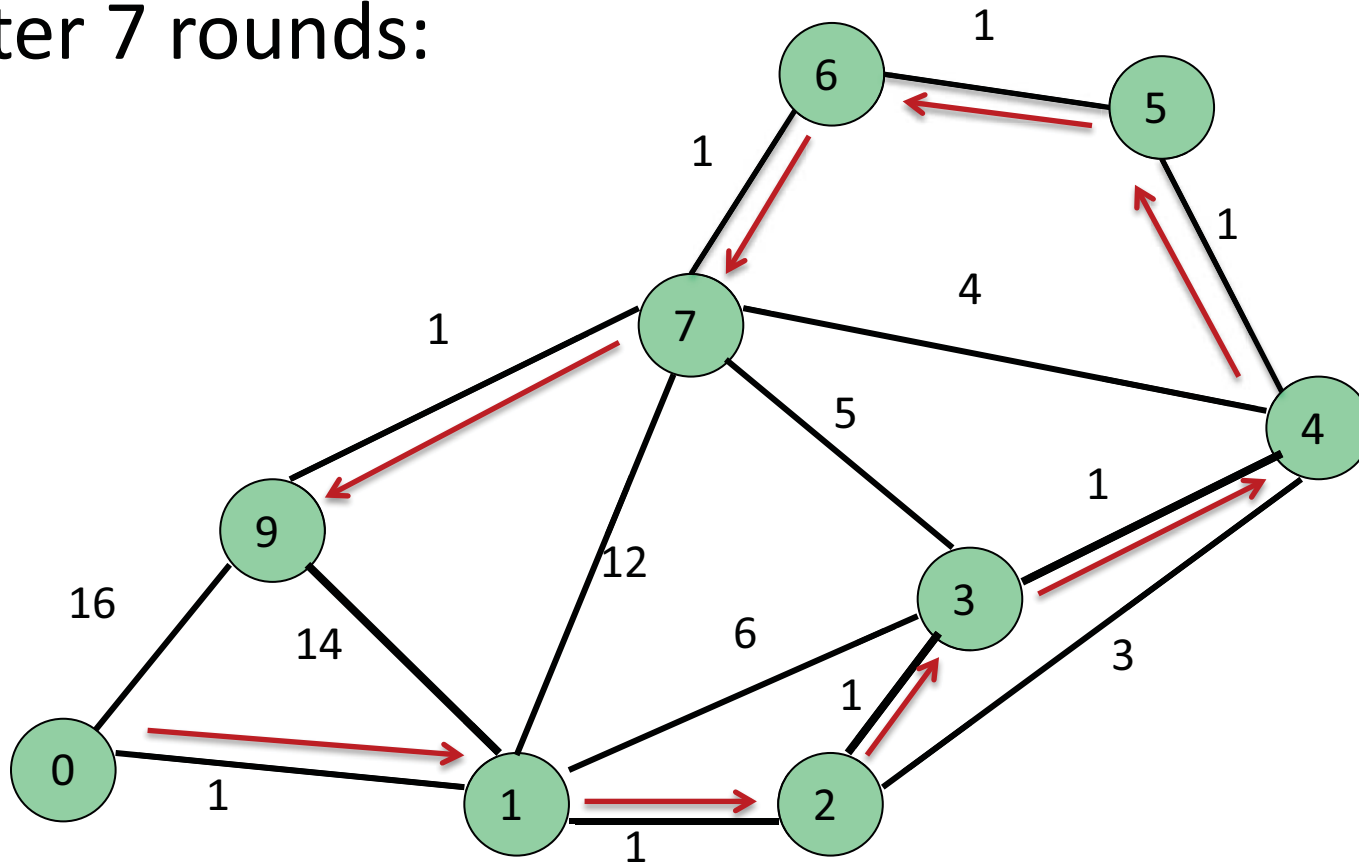
Bellman-Ford

- After 6 rounds:



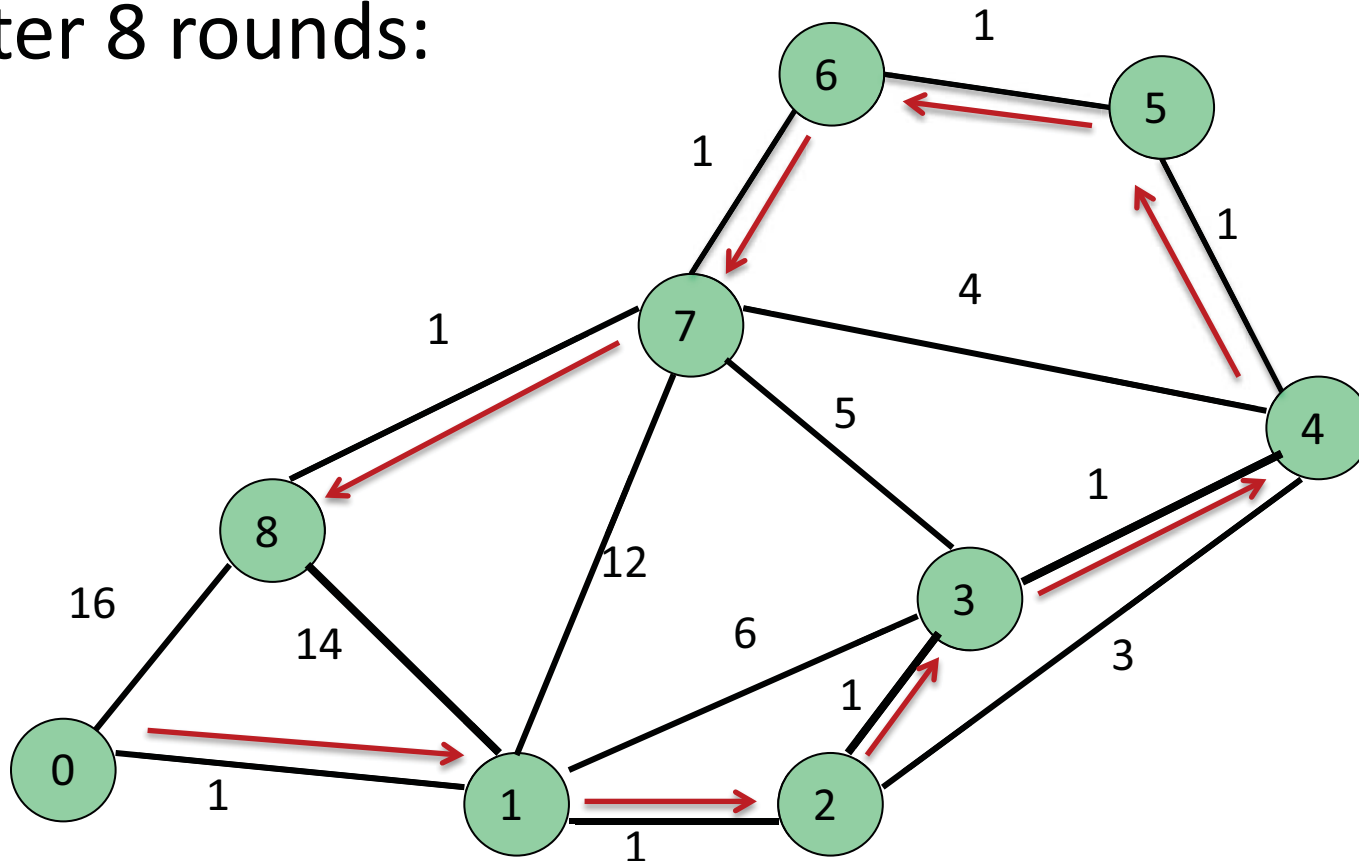
Bellman-Ford

- After 7 rounds:



Bellman-Ford

- After 8 rounds:



Correctness

- **Claim:** Eventually, every process i has:
 - $dist$ = minimum weight of a path from i_0 to i , and
 - if $i \neq i_0$, $parent$ = the previous node on some shortest path from i_0 to i .
- **Key invariant:**
 - For every r , at the end of r rounds, every process $i \neq i_0$ has its $dist$ and $parent$ corresponding to a shortest path from i_0 to i among those paths that consist of at most r edges; if there is no such path, then $dist = \infty$ and $parent$ is undefined.

Complexity

- Time complexity:
 - Number of rounds until all the variables stabilize to their final values.
 - $n - 1$ rounds
- Message complexity:
 - Number of messages sent by all processes during the entire execution.
 - $O(n \cdot |E|)$
- More expensive than BFS:
 - *diam* rounds,
 - $O(|E|)$ messages
- Q: Does the time bound really depend on n ?

Child Pointers

- Ignore repeated messages.
- When process i receives a message that it does not use to improve *dist*, it responds with a *nonparent* message.
- When process i receives a message that it uses to improve *dist*, it responds with a *parent* message, and also responds to any previous parent with a *nonparent* message.
- Process i records nodes from which it receives *parent* messages in a set *children*.
- When process i receives a *nonparent* message from a current child, it removes the sender from its *children*.
- When process i improves *dist*, it empties *children*.

Termination

- Q: How can the processes learn when the shortest-paths tree is completed?
- Q: How can a process know when it can output its own *parent* and *distance*?
- If processes knew an upper bound on n , then they could simply wait until that number of rounds have passed.
- But what if they don't know anything about the graph?
- Recall termination for BFS: Used **convergecast**.
- Q: Does that work here?

Termination

- **Q:** How can the processes learn when the shortest-paths tree is completed?
- **Q:** Does convergecast work here?
- Yes, but it's trickier, since the tree structure changes.
- **Key ideas:**
 - A process $\neq i_0$ can send a *done* message to its current parent after:
 - It has received responses to all its *distance* messages, so it believes it knows who its children are, and
 - It has received *done* messages from all of those children.
 - The same process may be involved several times in the convergecast, based on improved estimates.

MIT OpenCourseWare
<http://ocw.mit.edu>

6.046J / 18.410J Design and Analysis of Algorithms
Spring 2015

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Distributed Algorithms

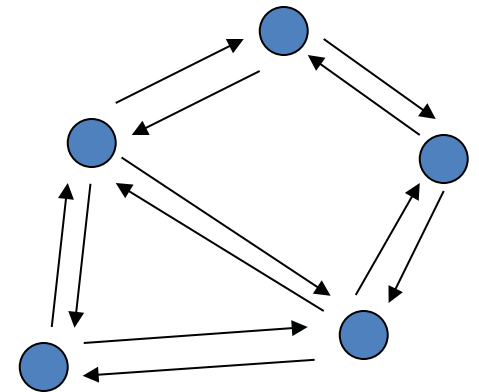
6.046J, Spring, 2015

Part 2

Nancy Lynch

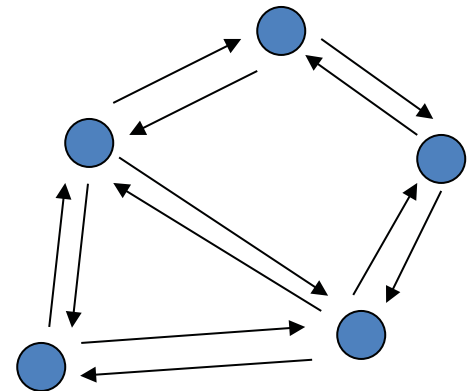
This Week

- Synchronous distributed algorithms:
 - Leader Election
 - Maximal Independent Set
 - Breadth-First Spanning Trees
 - Shortest Paths Trees (started)
 - Shortest Paths Trees (finish)
- Asynchronous distributed algorithms:
 - Breadth-First Spanning Trees
 - Shortest Paths Trees

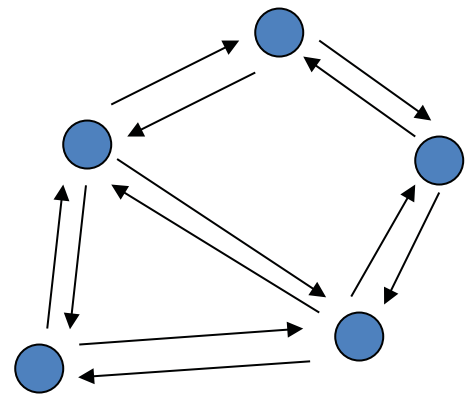


Distributed Networks

- Based on undirected graph $G = (V, E)$.
 - $n = |V|$
 - $\Gamma(u)$, set of neighbors of vertex u .
 - $\deg(u) = |\Gamma(u)|$, number of neighbors of vertex u .
- Associate a **process** with each graph vertex.
- Associate two directed **communication channels** with each edge.



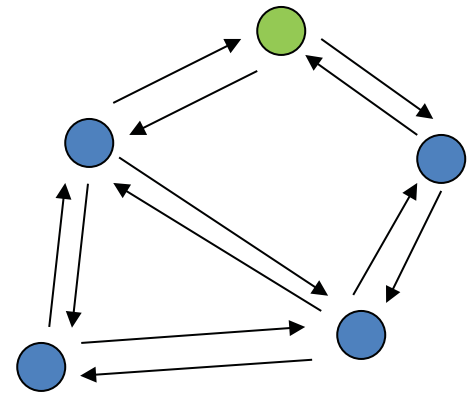
Synchronous Distributed Algorithms



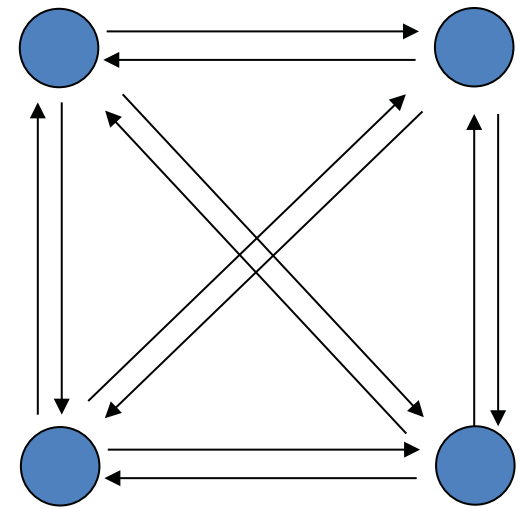
Synchronous Network Model

- Processes at graph vertices, communicate using messages.
- Each process has **output ports, input ports** that connect to communication channels.
- Algorithm executes in **synchronous rounds**.
- In each round:
 - Each process sends messages on its ports.
 - Each message gets put into the channel, delivered to the process at the other end.
 - Each process computes a new state based on the arriving messages.

Leader Election

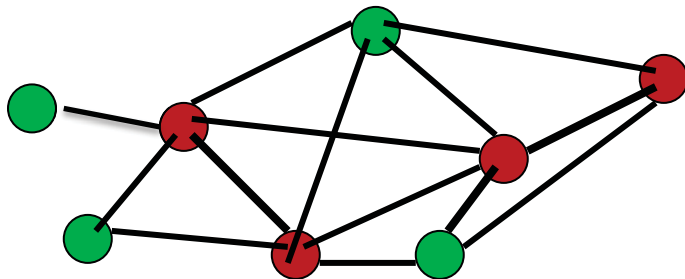


n -vertex Clique



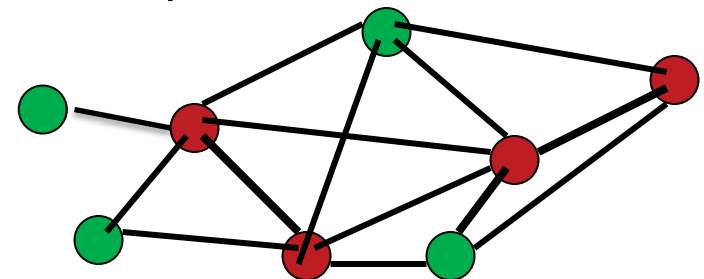
- Theorem: There is no algorithm consisting of **deterministic, indistinguishable processes** that is guaranteed to elect a leader in G .
- Theorem: There is an algorithm consisting of **deterministic processes with UIDs** that is guaranteed to elect a leader.
 - 1 round, n^2 messages.
- Theorem: There is an algorithm consisting of **randomized, indistinguishable processes** that eventually elects a leader, with probability 1.
 - Expected time $\leq \frac{1}{1-\epsilon}$.
 - With probability $\geq 1 - \epsilon$, finishes in one round.

Maximal Independent Set (MIS)



MIS

- **Independent:** No two neighbors are both in the set.
- **Maximal:** We can't add any more nodes without violating independence.
- Every node is either in S or has a neighbor in S .
- **Assume:**
 - No UIDs
 - Processes know a good upper bound on n .
- **Require:**
 - Compute an MIS S of the network graph.
 - Each process in S should output **in**, others output **out**.



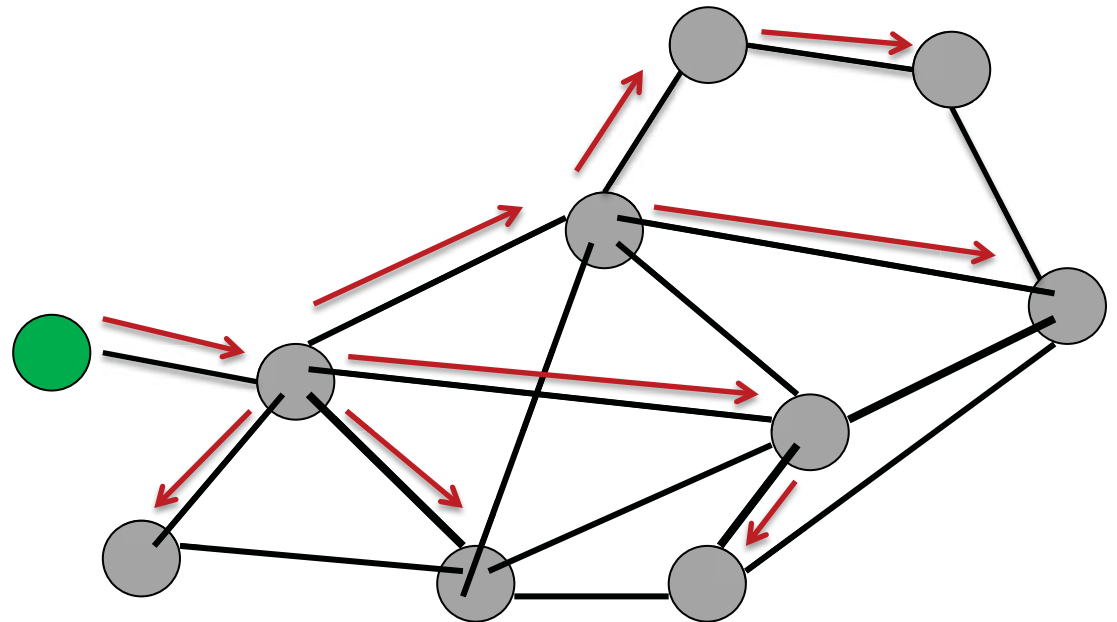
Luby's Algorithm

- Initially all nodes are **active**.
- At each phase, some active nodes decide to be **in**, others decide to be **out**, the rest continue to the next phase.
- **Behavior of active node at a phase:**
- **Round 1:**
 - Choose a random value r in $\{1, 2, \dots, n^5\}$, send it to all neighbors.
 - Receive values from all active neighbors.
 - If r is strictly greater than all received values, then join the MIS, output **in**.
- **Round 2:**
 - If you joined the MIS, announce it in messages to all (active) neighbors.
 - If you receive such an announcement, decide not to join the MIS, output **out**.
 - If you decided one way or the other at this phase, become **inactive**.

Luby's Algorithm

- **Theorem:** If Luby's algorithm ever terminates, then the final set S is an MIS.
- **Theorem:** With probability at least $1 - \frac{1}{n}$, all nodes decide within $4 \log n$ phases.

Breadth-First Spanning Trees



Breadth-First Spanning Trees

- Distinguished vertex v_0 .
- Processes must produce a Breadth-First Spanning Tree rooted at vertex v_0 .
- **Assume:**
 - UIDs.
 - Processes have no knowledge about the graph.
- **Output:** Each process $i \neq i_0$ should output *parent(j)*.

Simple BFS Algorithm

- Processes **mark** themselves as they get incorporated into the tree.
- Initially, only i_0 is marked.
- **Algorithm for process i :**
 - **Round 1:**
 - If $i = i_0$ then process i sends a **search** message to its neighbors.
 - If process i receives a message, then it:
 - Marks itself.
 - Selects i_0 as its parent, outputs **parent(i_0)**.
 - Plans to send at the next round.
 - **Round $r > 1$:**
 - If process i planned to send, then it sends a **search** message to its neighbors.
 - If process i is not marked and receives a message, then it:
 - Marks itself.
 - Selects one sending neighbor, j , as its parent, outputs **parent(j)**.
 - Plans to send at the next round.

Correctness

- **State variables, per process:**
 - *marked*, a Boolean, initially true for i_0 , false for others
 - *parent*, a UID or undefined
 - *send*, a Boolean, initially true for i_0 , false for others
 - *uid*
- **Invariants:**
 - At the end of r rounds, exactly the processes at distance $\leq r$ from v_0 are marked.
 - A process $\neq i_0$ has its *parent* defined iff it is marked.
 - For any process at distance d from v_0 , if its *parent* is defined, then it is the UID of a process at distance $d - 1$ from v_0 .

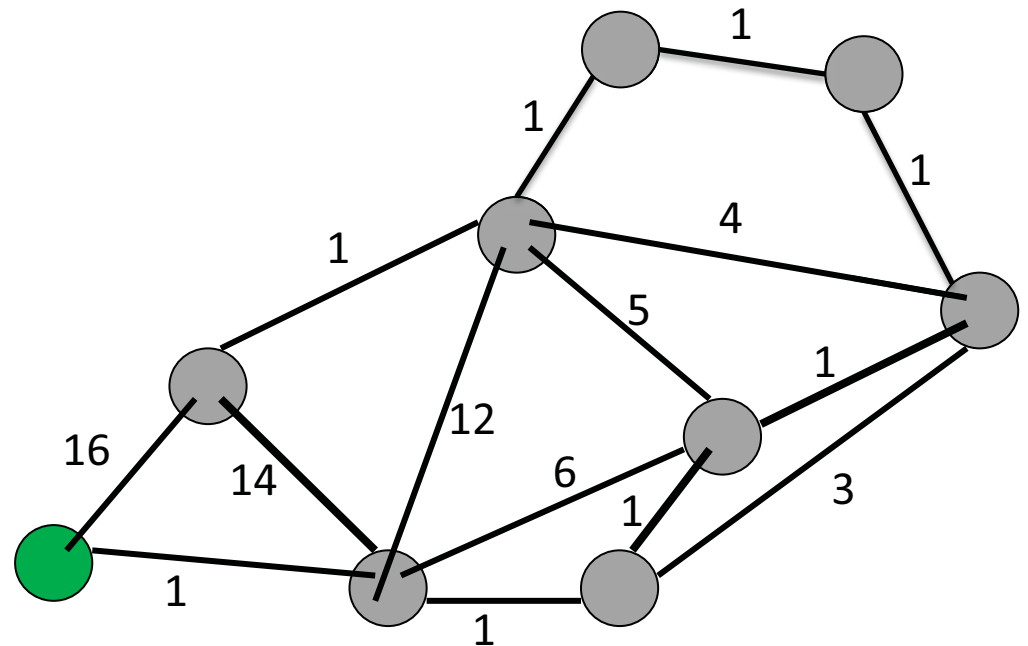
Complexity

- Time complexity:
 - Number of rounds until all nodes outputs their parent information.
 - Maximum distance of any node from v_0 , which is $\leq diam$
- Message complexity:
 - Number of messages sent by all processes during the entire execution.
 - $O(|E|)$

Bells and Whistles

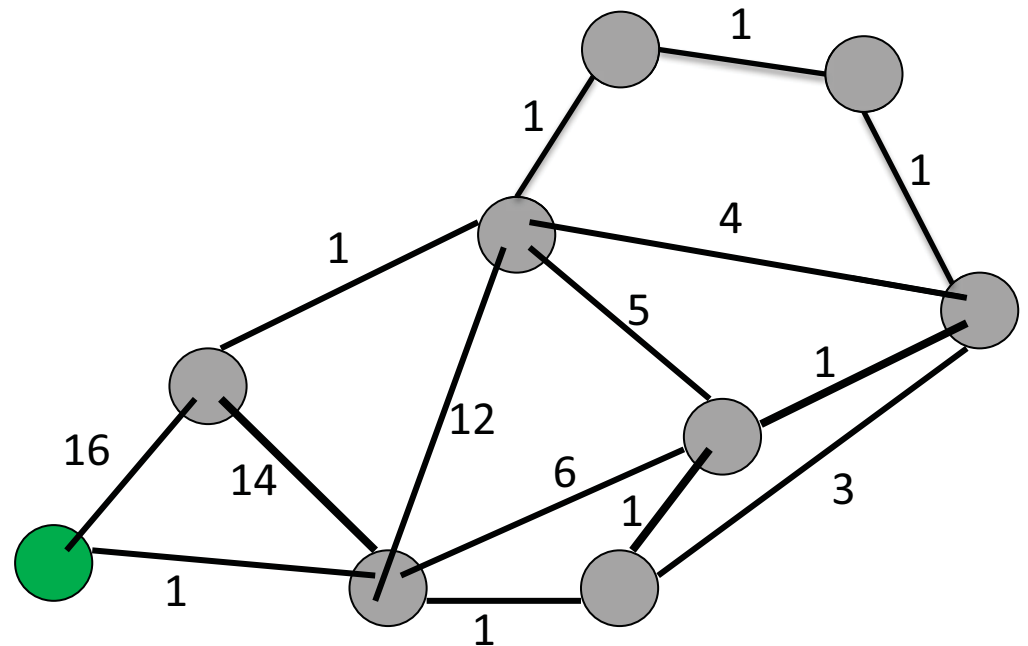
- Child pointers:
 - Send *parent/nonparent* responses to search messages.
- Distances:
 - Piggyback distances on *search* messages.
- Termination:
 - Convergecast starting from the leaves.
- Applications:
 - Message broadcast from the root
 - Global computation

Shortest Paths Trees



Shortest Paths

- Generalize the BFS problem to allow weights on the graph edges, *weight*_{u,v} for edge {u, v}
- Connected graph $G = (V, E)$, root vertex v_0 , process i_0 .
- Processes have UIDs.
- Processes know their neighbors and the weights of their incident edges, but otherwise have no knowledge about the graph.



Shortest Paths

- Processes must produce a Shortest-Paths Spanning Tree rooted at vertex v_0 .
- Branches are directed paths from v_0 .
 - **Spanning:** Branches reach all vertices.
 - **Shortest paths:** The total weight of the tree branch to each node is the minimum total weight for any path from v_0 in G .
- **Output:** Each process $i \neq i_0$ should output *parent(j), distance(d)*, meaning that:
 - j 's vertex is the parent of i 's vertex on a shortest path from v_0 ,
 - d is the total weight of a shortest path from v_0 to j .

Bellman-Ford Shortest Paths Algorithm

- **State variables:**
 - *dist*, a nonnegative real or ∞ , representing the shortest known distance from v_0 . Initially 0 for process i_0 , ∞ for the others.
 - *parent*, a UID or undefined, initially undefined.
 - *uid*
- **Algorithm for process i :**
 - At each round:
 - Send a *distance*(*dist*) message to all neighbors.
 - Receive messages from neighbors; let d_j be the distance received from neighbor j .
 - Perform a **relaxation step**:
$$dist := \min(dist, \min_j (d_j + weight_{\{i,j\}})).$$
 - If *dist* decreases then set *parent* $:= j$, where j is any neighbor that produced the new *dist*.

Correctness

- **Claim:** Eventually, every process i has:
 - $dist$ = minimum weight of a path from i_0 to i , and
 - if $i \neq i_0$, $parent$ = the previous node on some shortest path from i_0 to i .
- **Key invariant:**
 - For every r , at the end of r rounds, every process $i \neq i_0$ has its $dist$ and $parent$ corresponding to a shortest path from i_0 to i among those paths that consist of at most r edges; if there is no such path, then $dist = \infty$ and $parent$ is undefined.

Complexity

- Time complexity:
 - Number of rounds until all the variables stabilize to their final values.
 - $n - 1$ rounds
- Message complexity:
 - Number of messages sent by all processes during the entire execution.
 - $O(n \cdot |E|)$
- More expensive than BFS:
 - *diam* rounds,
 - $O(|E|)$ messages
- Q: Does the time bound really depend on n ?

Child Pointers

- Ignore repeated messages.
- When process i receives a message that it does not use to improve *dist*, it responds with a *nonparent* message.
- When process i receives a message that it uses to improve *dist*, it responds with a *parent* message, and also responds to any previous parent with a *nonparent* message.
- Process i records nodes from which it receives *parent* messages in a set *children*.
- When process i receives a *nonparent* message from a current child, it removes the sender from its *children*.
- When process i improves *dist*, it empties *children*.

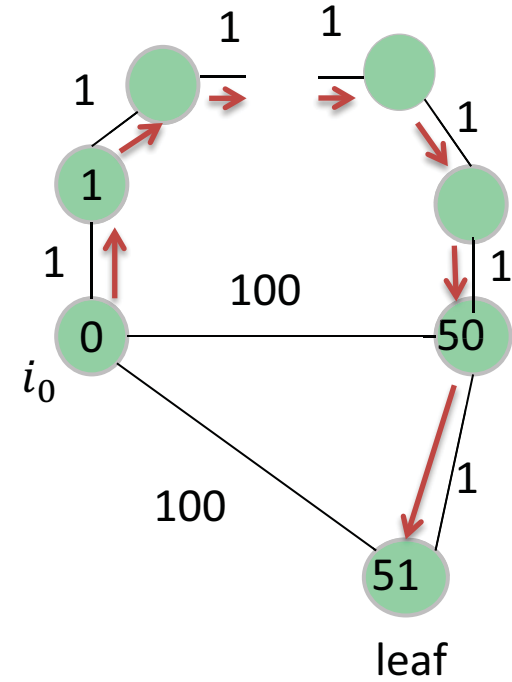
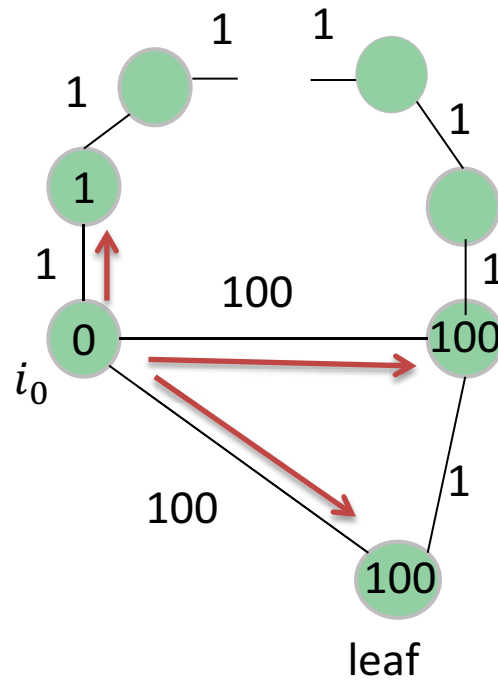
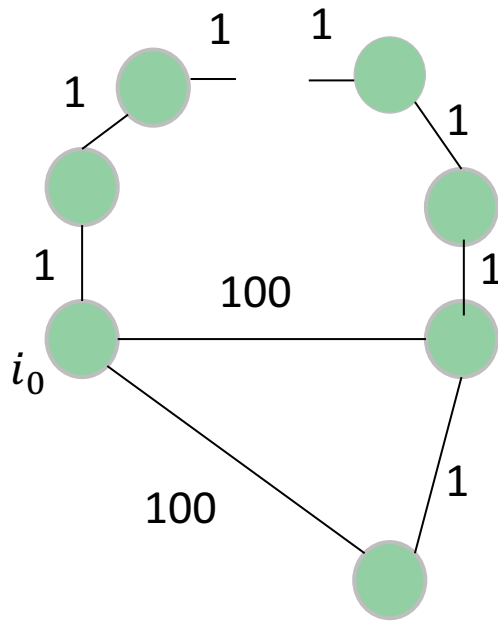
Termination

- Q: How can the processes learn when the shortest-paths tree is completed?
- Q: How can a process even know when it can output its own *parent* and *distance*?
- If processes knew an upper bound on n , then they could simply wait until that number of rounds have passed.
- But what if they don't know anything about the graph?
- Recall termination for BFS: Used **convergecast**.
- Q: Does that work here?

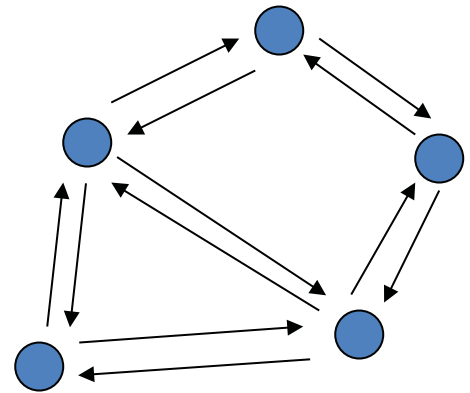
Termination

- Q: How can the processes learn when the shortest-paths tree is completed?
- Q: Does convergecast work here?
- Yes, but it's trickier, since the tree structure changes.
- Key ideas:
 - A process $\neq i_0$ can send a *done* message to its current parent after:
 - It has received responses to all its *distance* messages, so it believes it knows who its children are, and
 - It has received *done* messages from all of those children.
 - The same process may be involved several times in the convergecast, based on improved estimates.

Termination

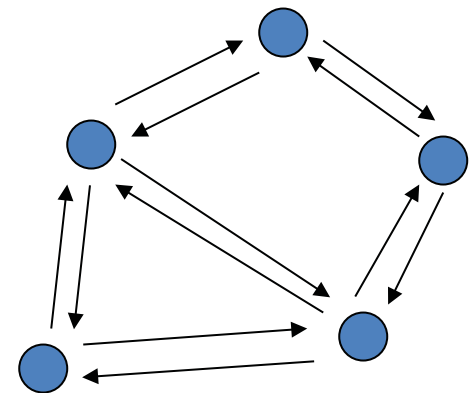


Asynchronous Distributed Algorithms



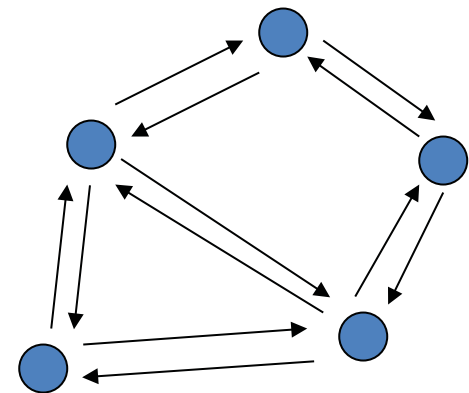
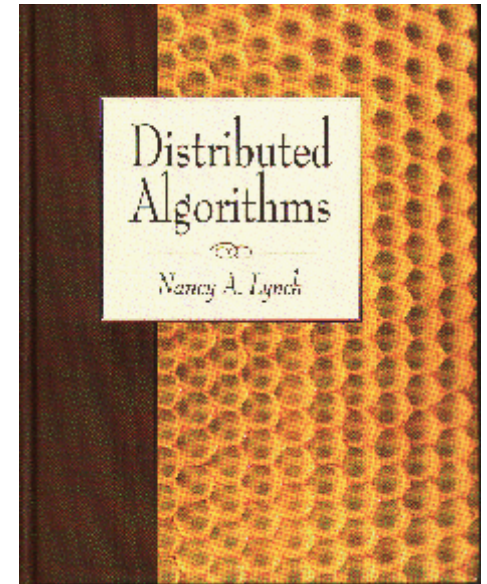
Asynchronous Network Model

- Complications so far:
 - Processes act concurrently.
 - A little nondeterminism.
- Now things get much worse:
 - No rounds---process steps and message deliveries happen at arbitrary times, in arbitrary orders.
 - Processes get out of synch.
 - Much more nondeterminism.
- Understanding asynchronous distributed algorithms is hard because we can't understand **exactly how they execute.**
- Instead, we must understand **abstract properties of executions.**



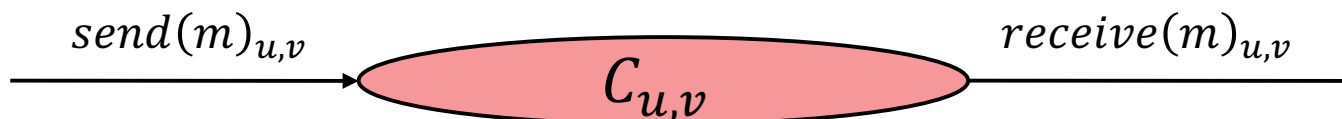
Asynchronous Network Model

- Lynch, Distributed Algorithms, Chapter 8.
- **Processes** at nodes of an undirected graph $G = (V, E)$, communicate using messages.
- **Communication channels** associated with edges (one in each direction on each edge).
 - $C_{u,v}$, channel from vertex u to vertex v .
- Each process has **output ports and input ports** that connect it to its communication channels.
- Processes need not be distinguishable.



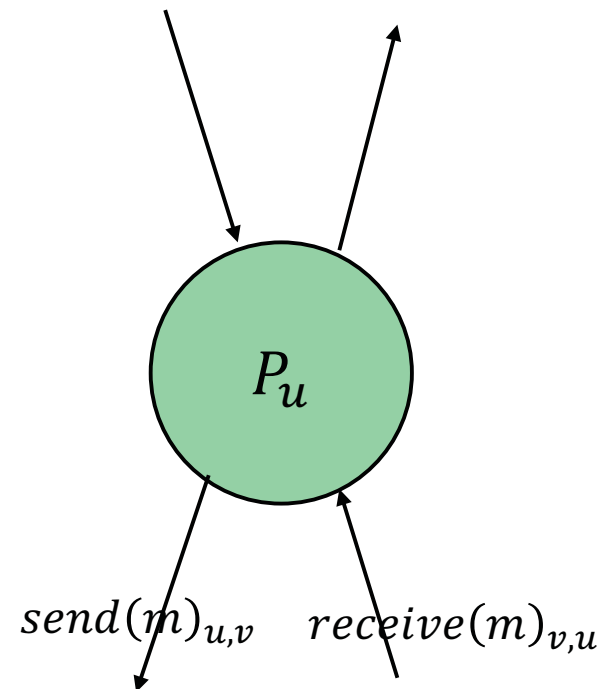
Channel Automaton $C_{u,v}$

- Formally, an **input/output automaton**.
- Input actions: $send(m)_{u,v}$
- Output actions: $receive(m)_{u,v}$
- State variable:
 - $mqueue$, a FIFO queue, initially empty.
- Transitions:
 - $send(m)_{u,v}$
 - Effect: add m to $mqueue$.
 - $receive(m)_{u,v}$
 - Precondition: $m = head(mqueue)$
 - Effect: remove head of $mqueue$



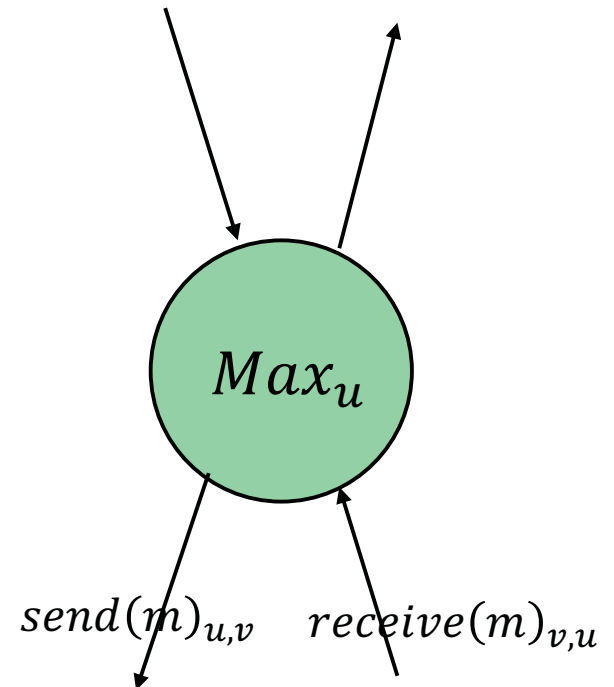
Process Automaton P_u

- Associate a process automaton with each vertex of G .
- To simplify notation, let P_u denote the process automaton at vertex u .
 - But the process does not “know” u .
- P_u has $\text{send}(m)_{u,v}$ outputs and $\text{receive}(m)_{v,u}$ inputs.
- May also have external inputs and outputs.
- Has state variables.
- Keeps taking steps (eventually).



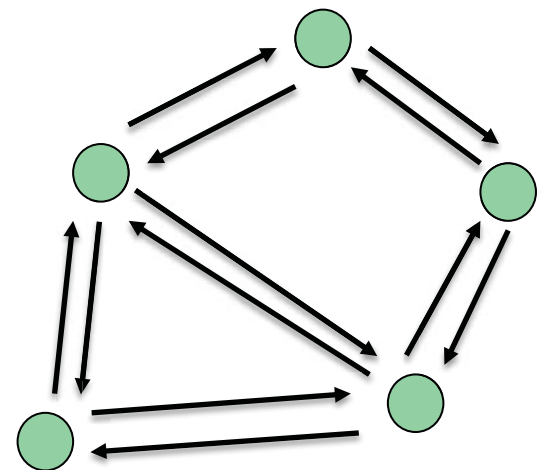
Example: Max_u Process Automaton

- Input actions: $receive(m)_{v,u}$
- Output actions: $send(m)_{u,v}$
- State variables:
 - max , a natural number, initially x_u
 - For each neighbor v :
 - $send(v)$, a Boolean, initially $true$
- Transitions:
 - $receive(m)_{v,u}$
 - Effect: if $m > max$ then
 - $max := m$
 - for every w , $send(w) := true$
 - $send(m)_{u,v}$
 - Precondition: $send(v) = true$ and $m = max$
 - Effect: $send(v) := false$



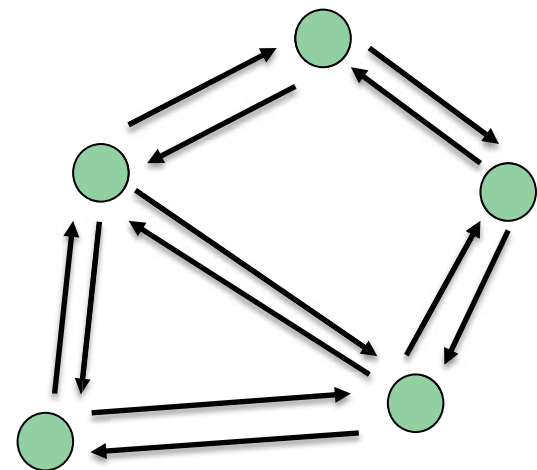
Combining Processes and Channels

- Undirected graph $G = (V, E)$.
- Process P_u at each vertex u .
- Channels $C_{u,v}$ and $C_{v,u}$, associated with each edge $\{u, v\}$.
- *send*(m) _{u,v} output of process P_u gets identified with *send*(m) _{u,v} input of channel $C_{u,v}$.
- *receive*(m) _{v,u} output of channel $C_{v,u}$ gets identified with *receive*(m) _{v,u} input of process P_u .
- Steps involving such a shared action involve simultaneous state transitions for a process and a channel.



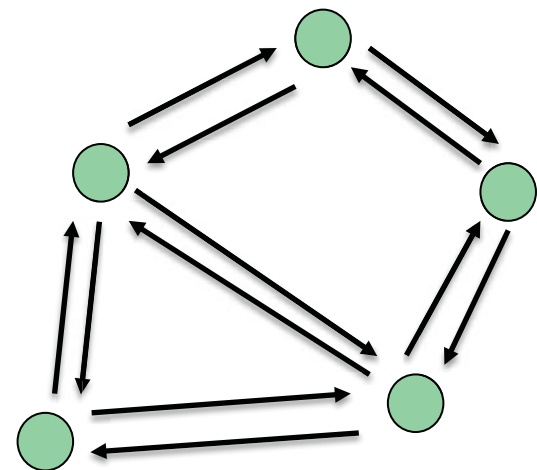
Execution

- No synchronous rounds anymore.
- The system executes by performing enabled steps, one at a time, **in any order**.
- Formally, an execution is modeled as a **sequence of individual steps**.
- Different from the synchronous model, in which all processes take steps concurrently at each round.
- Assume enabled steps eventually occur:
 - Each channel always eventually delivers the first message in its queue.
 - Each process always eventually performs some enabled step.

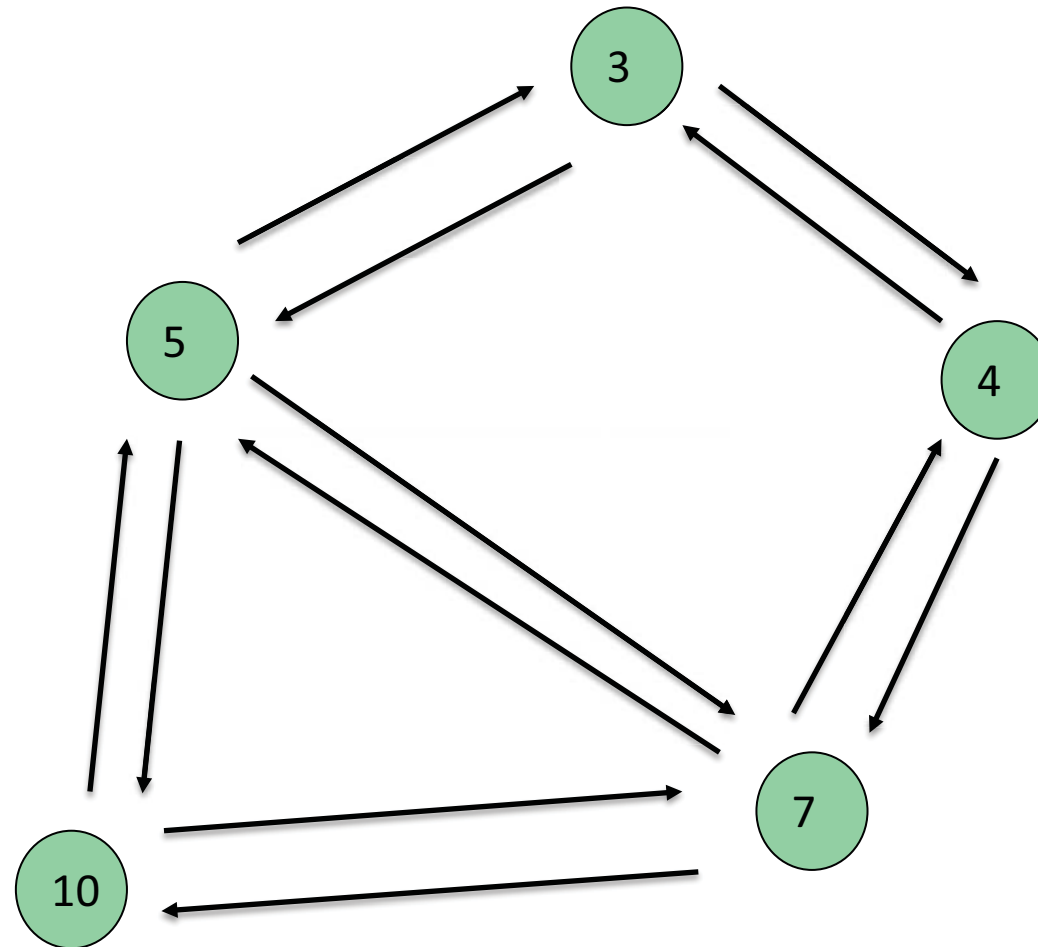


Combining *Max* Processes and Channels

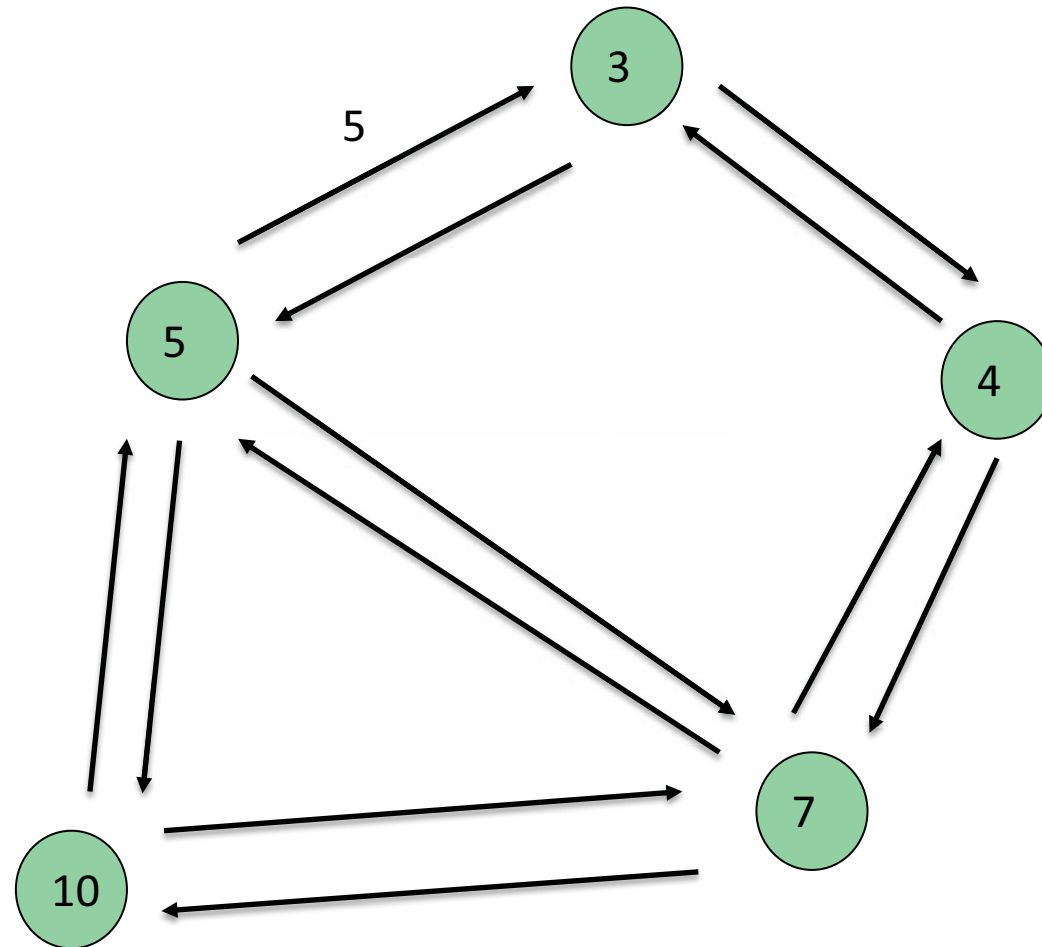
- Each process Max_u starts with an initial value x_u .
- They all send out their initial values, and propagate their *max* values, until everyone has the globally-maximum value.
- Sending and receiving steps can happen in many different orders, but in all cases the global max will eventually arrive everywhere.



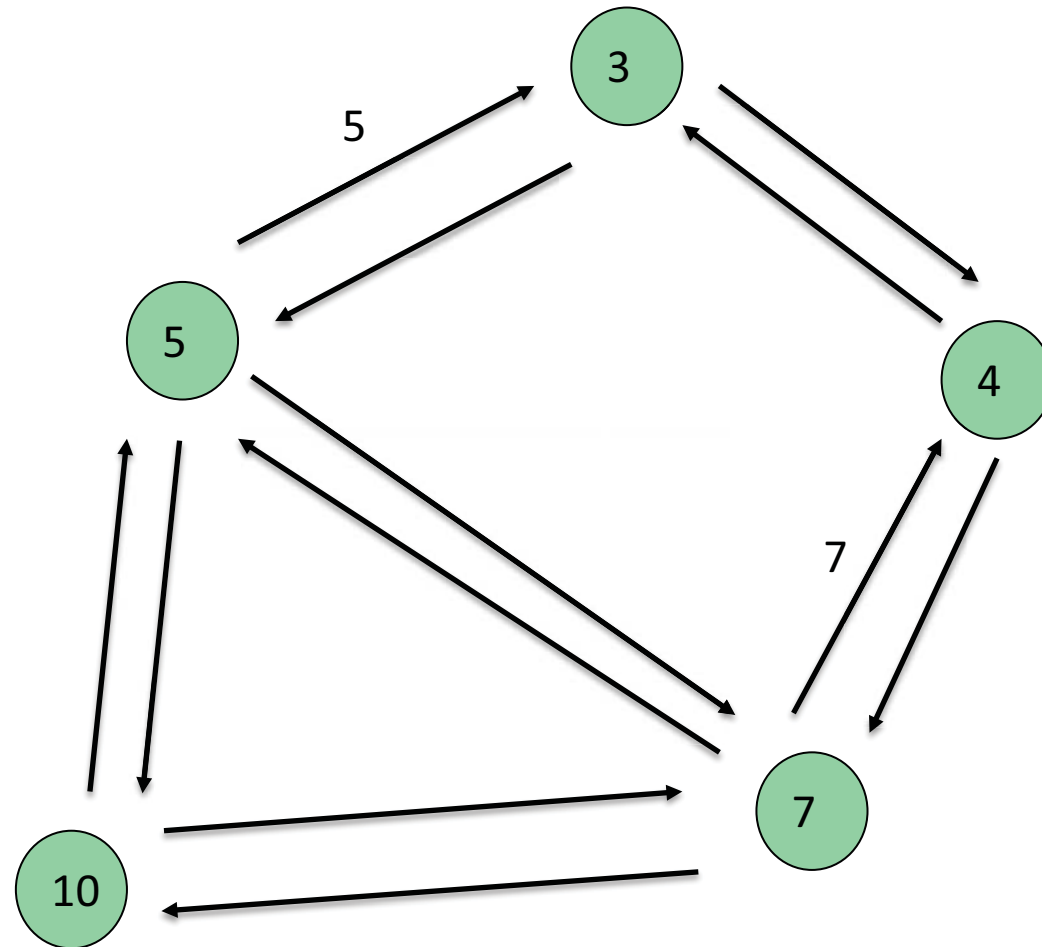
Max System



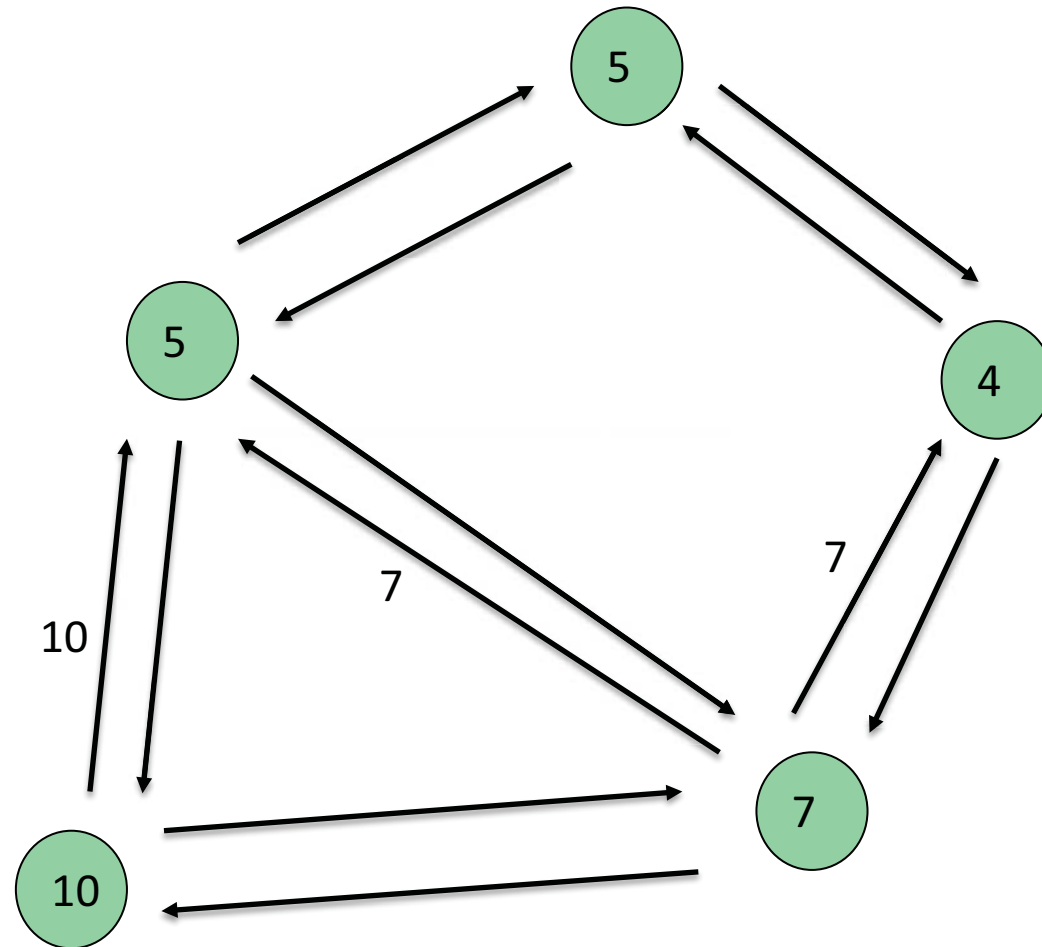
Max System



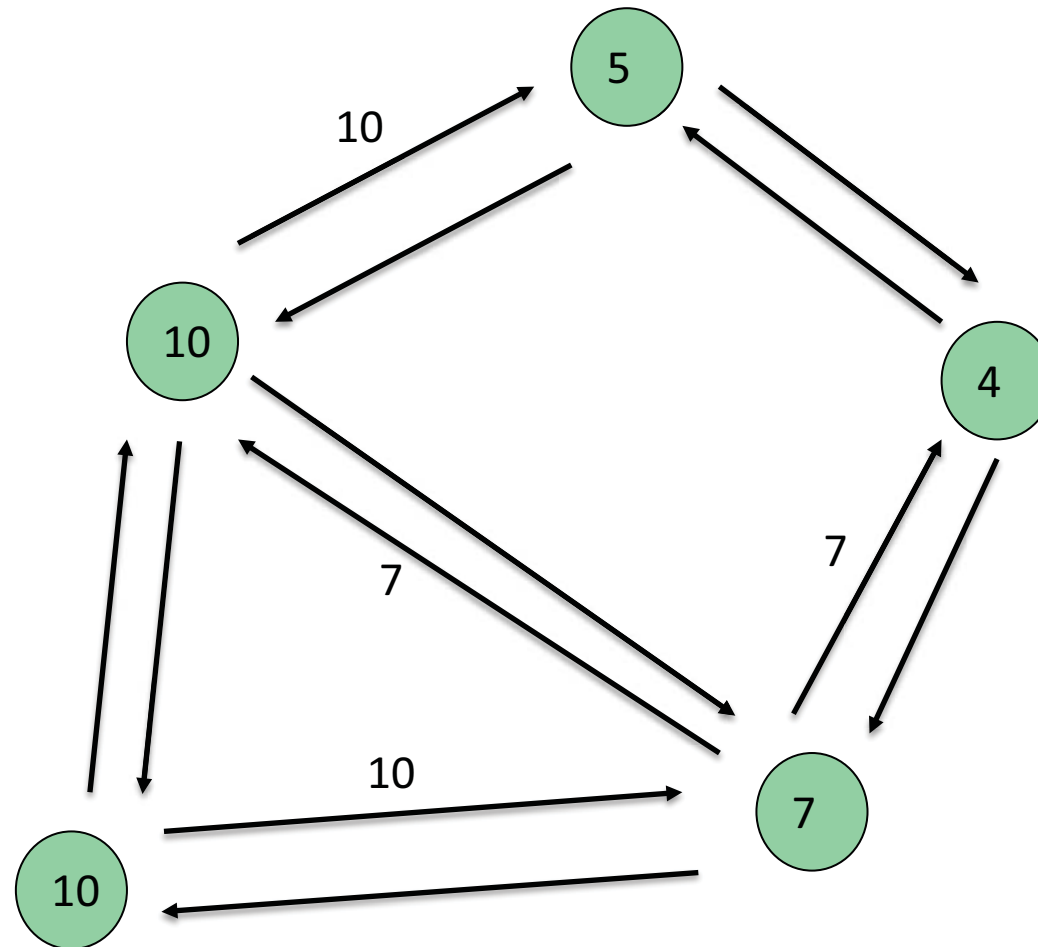
Max System



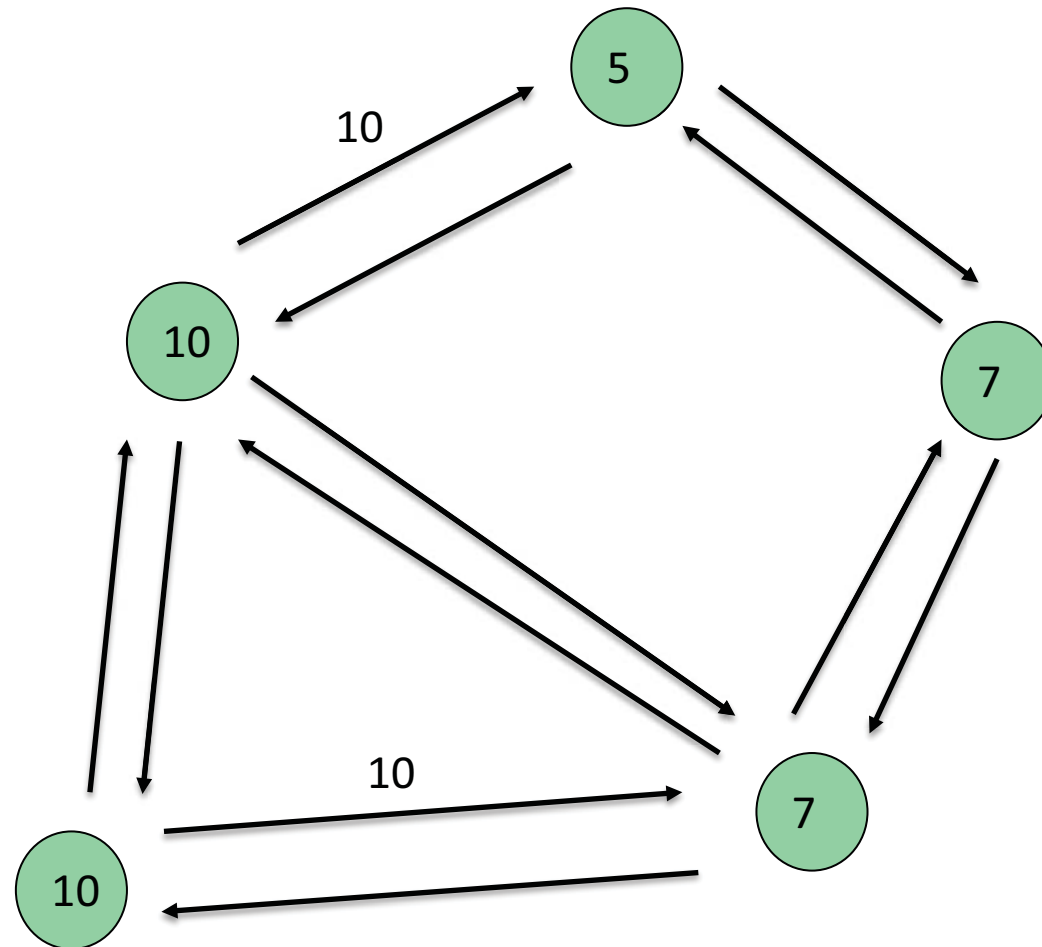
Max System



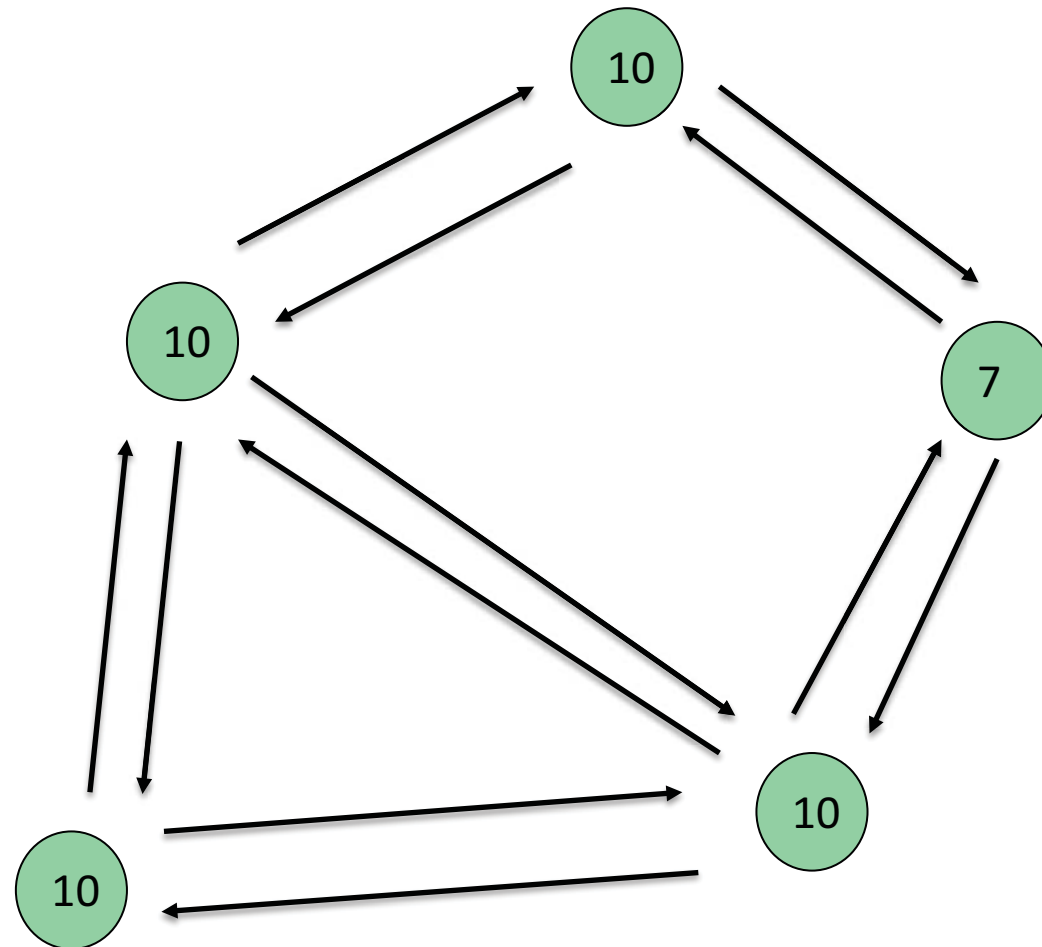
Max System



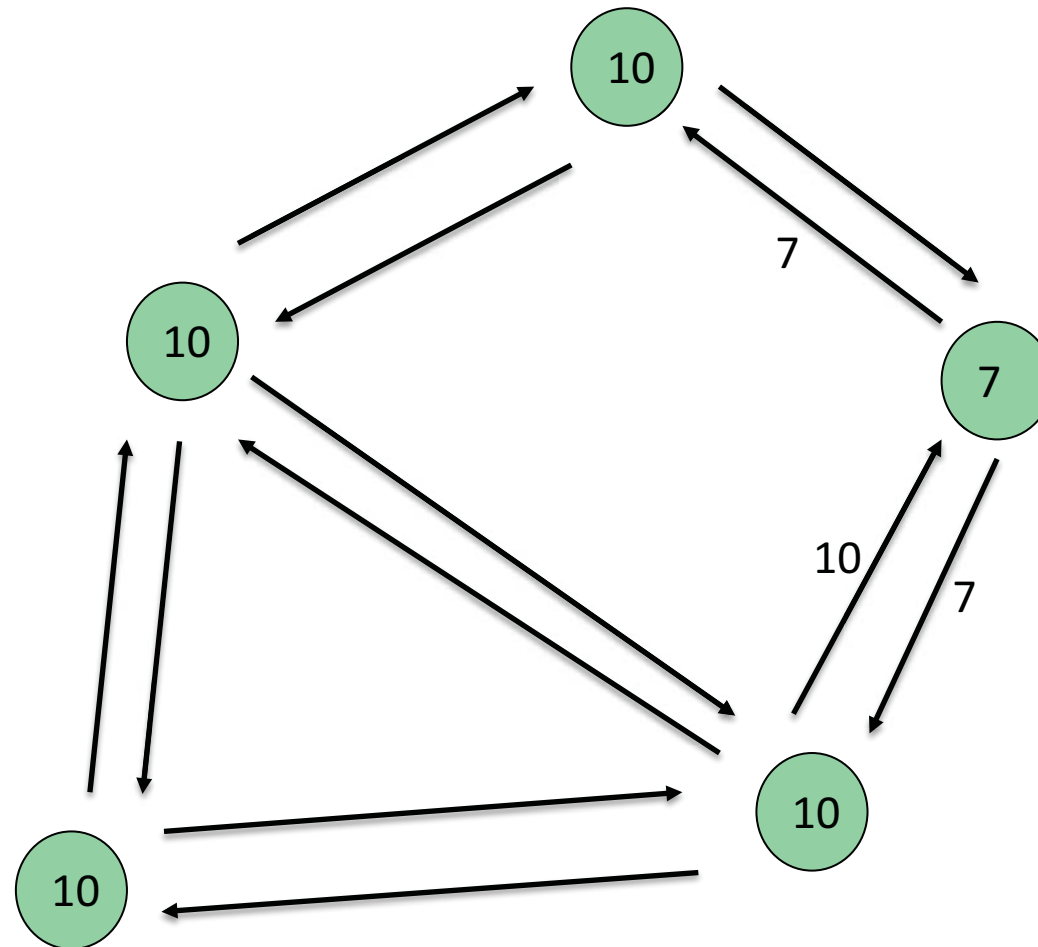
Max System



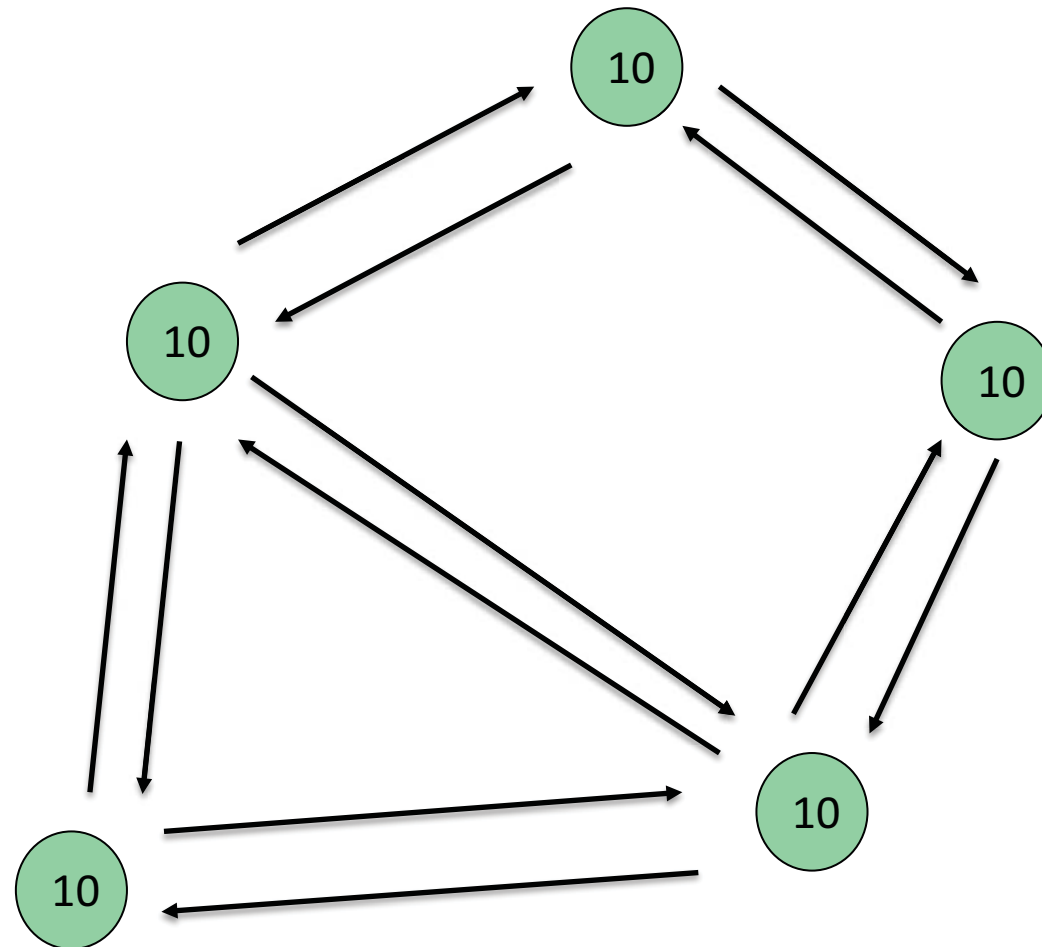
Max System



Max System



Max System



Complexity

- **Message complexity:**

- Number of messages sent by all processes during the entire execution.
- $O(n \cdot |E|)$

- **Time complexity:**

- **Q:** What should we measure?
- Not obvious, because the various components are taking steps in arbitrary orders---no “rounds”.
- A common approach:
 - Assume **real-time upper bounds** on the time to perform basic steps:
 - d for a channel to deliver the next message, and
 - l for a process to perform its next step.
 - Infer a real-time upper bound for solving the overall problem.

Complexity

- Time complexity:
 - Assume real-time upper bounds on the time to perform basic steps:
 - d for a channel to deliver the next message, and
 - l for a process to perform its next step.
 - Infer a real-time upper bound for solving the problem.
- For the *Max* system:
 - Ignore local processing time ($l = 0$), consider only channel sending time.
 - Straightforward upper bound: $O(\text{diam} \cdot n \cdot d)$
 - Consider the time for the max to reach any particular vertex u , along a shortest path in the graph.
 - At worst, it waits in each channel on the path for every other value, which is at most time $n \cdot d$ for that channel.

Breadth-First Spanning Trees

- **Problem:** Compute a Breadth-First Spanning Tree in an asynchronous network.
- Connected graph $G = (V, E)$.
- Distinguished root vertex v_0 .
- Processes have no knowledge about the graph.
- Processes have UIDs
 - i_0 is the UID of the root v_0 .
 - Processes know UIDs of their neighbors, and know which ports are connected to each neighbor.
- Processes must produce a BFS tree rooted at v_0 .
- Each process $i \neq i_0$ should output *parent(j)*, meaning that j 's vertex is the parent of i 's vertex in the BFS tree.

First Attempt

- Just run the simple synchronous BFS algorithm asynchronously.
- Process i_0 sends *search* messages, which everyone propagates the first time they receive it.
- Everyone picks the first node from which it receives a *search* message as its parent.
- **Nondeterminism:**
 - No longer any nondeterminism in process decisions.
 - But plenty of new nondeterminism: orders of message deliveries and process steps.

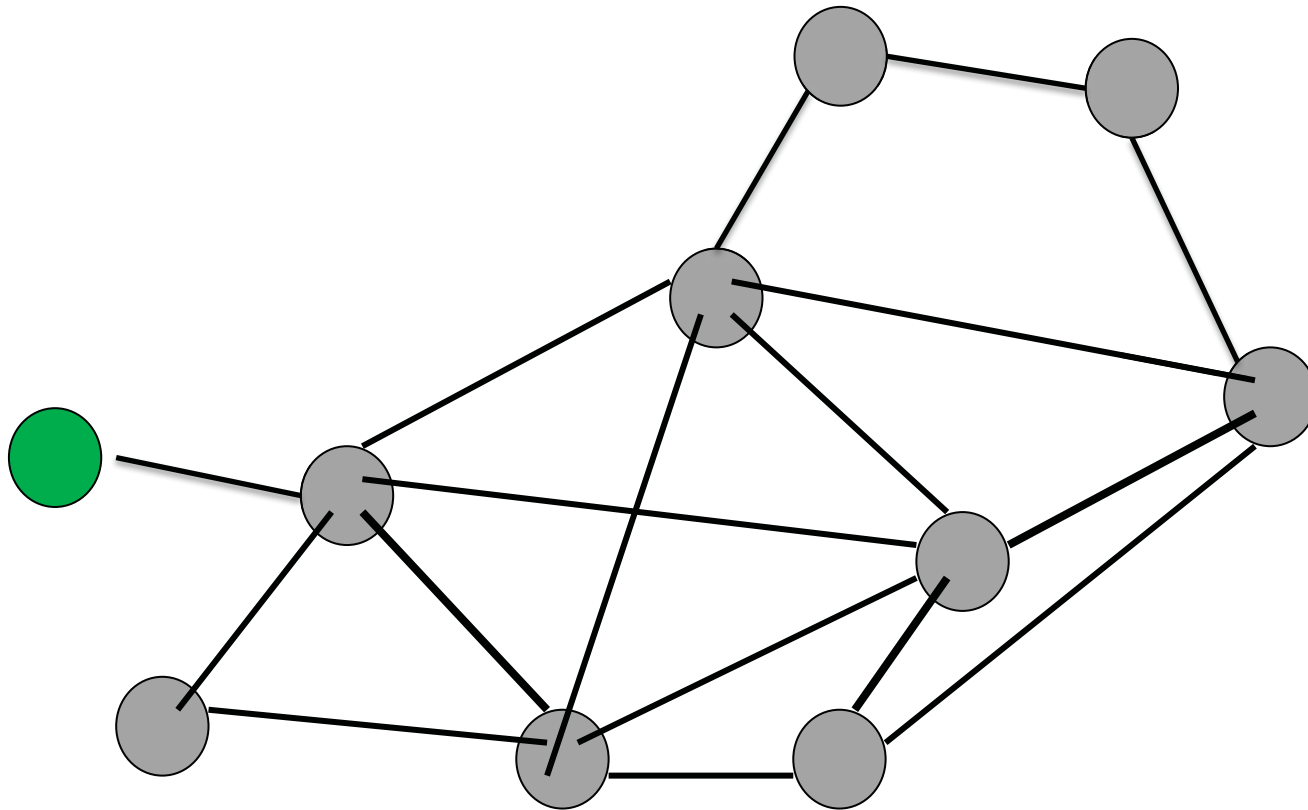
Process Automaton P_u

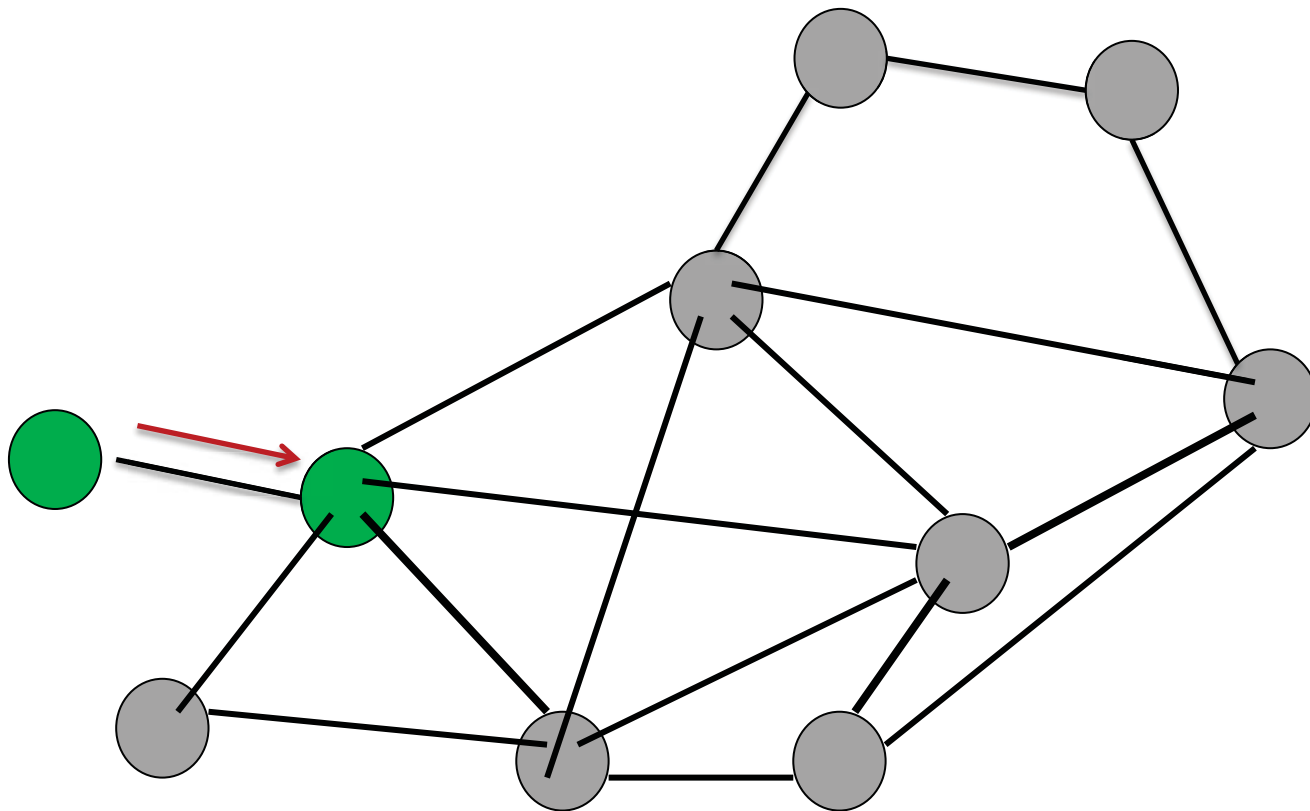
- Input actions: $receive(search)_{v,u}$
- Output actions: $send(search)_{u,v}; parent(v)_u$
- State variables:
 - $parent$: $\Gamma(u) \cup \{\perp\}$, initially $\perp$
 - $reported$: Boolean, initially false
 - For every $v \in \Gamma(u)$:
 - $send(v) \in \{search, \perp\}$, initially $search$ if $u = v_0$, else $\perp$
- Transitions:
 - $receive(search)_{v,u}$
 - Effect: if $u \neq v_0$ and $parent = \perp$ then
 - $parent := v$
 - for every w , $send(w) := search$

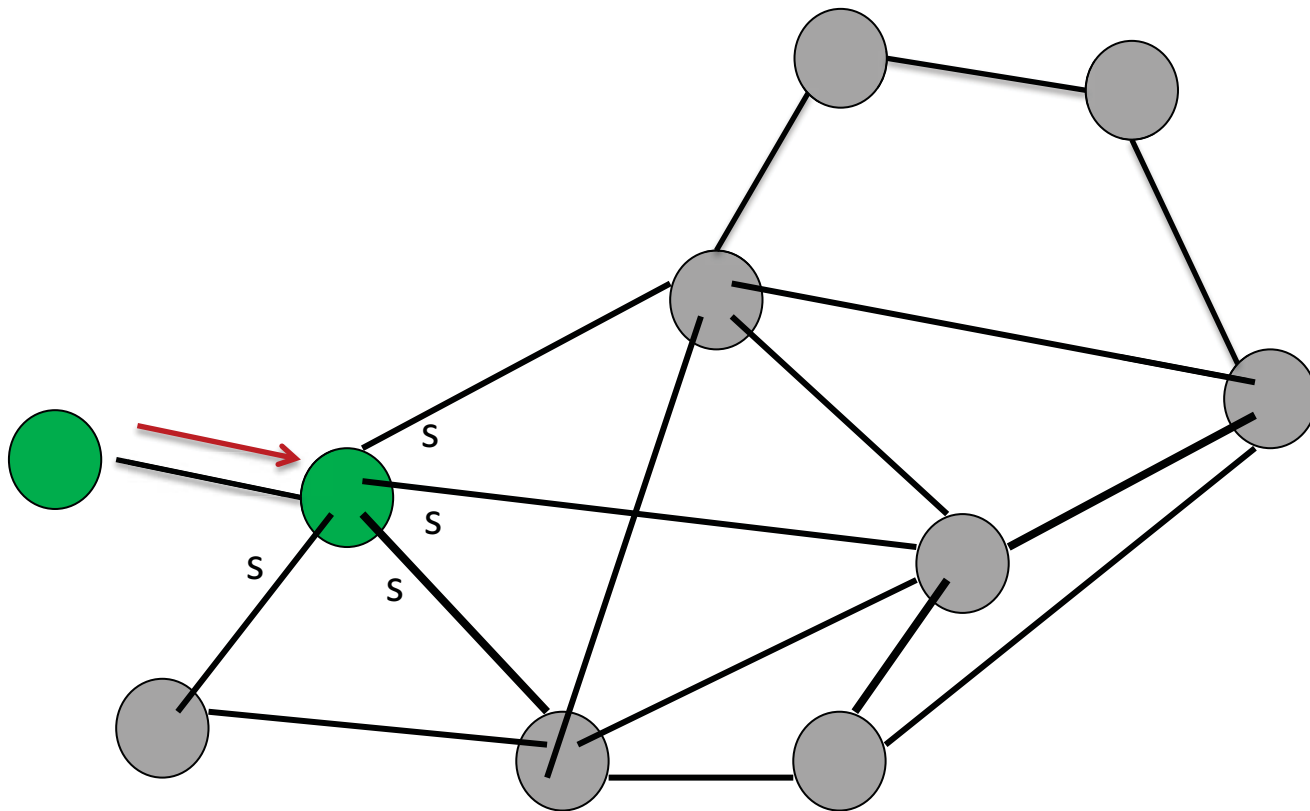
Process Automaton P_u

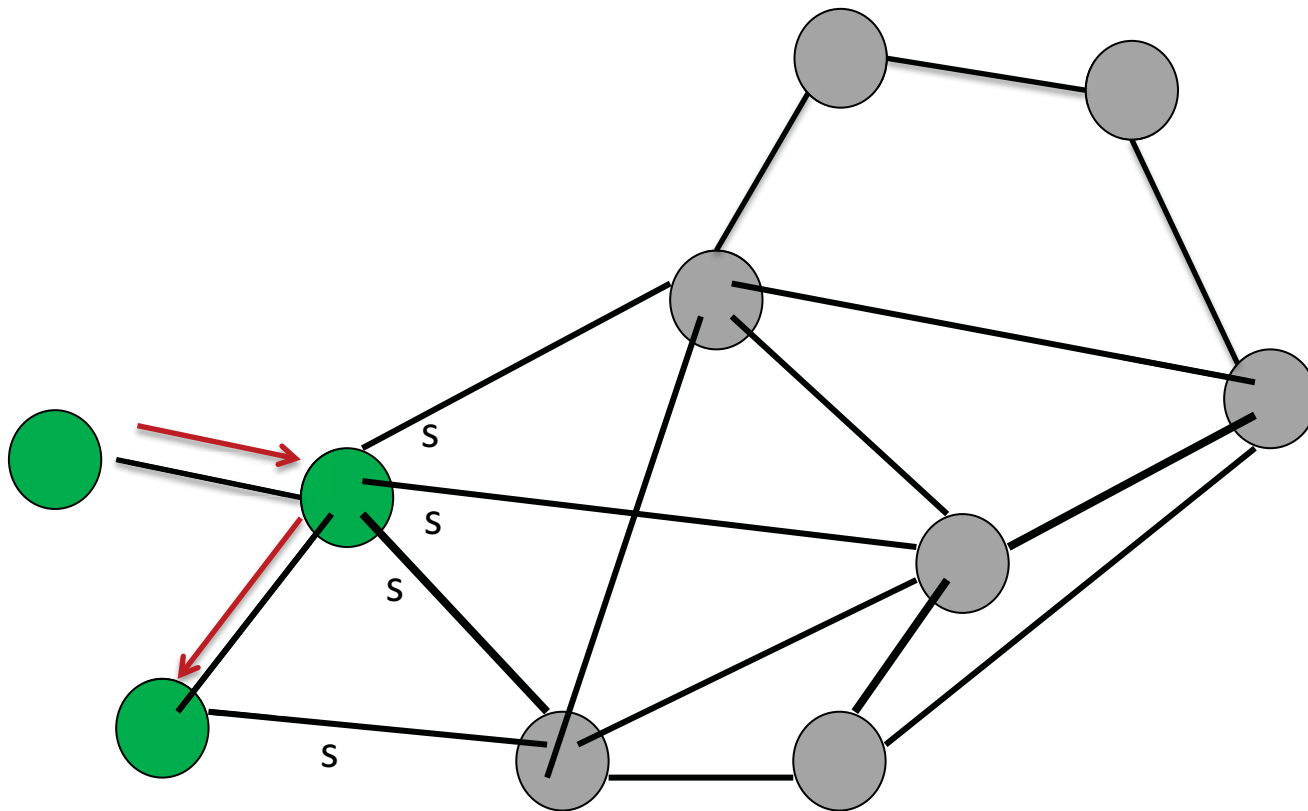
- Transitions:
 - $receive(search)_{v,u}$
 - Effect: if $u \neq v_0$ and $parent = \perp$ then
 - $parent := v$
 - for every w , $send(w) := search$
 - $send(search)_{u,v}$
 - Precondition: $send(v) = search$
 - Effect: $send(v) := \perp$
 - $parent(v)_u$
 - Precondition: $parent = v$ and $reported = false$
 - Effect: $reported := true$

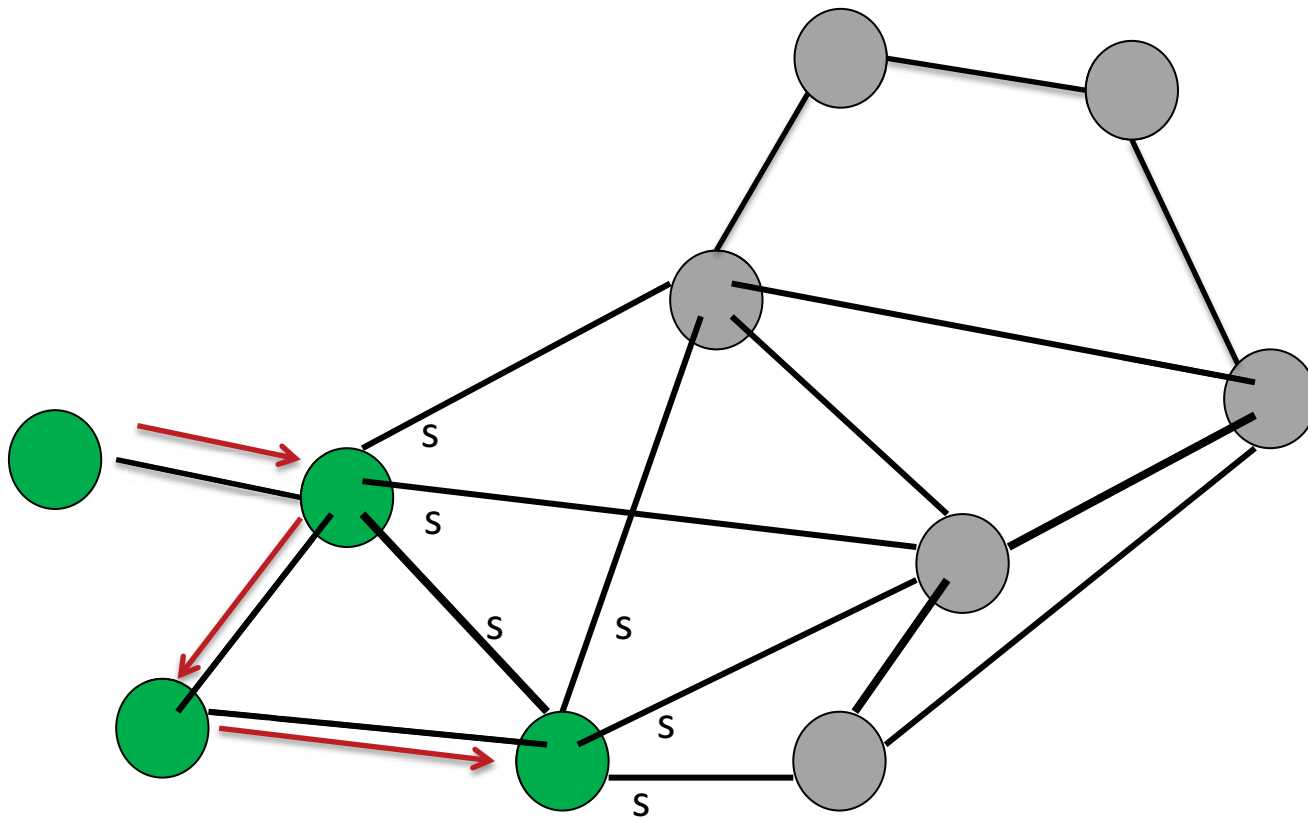
Running Simple BFS Asynchronously

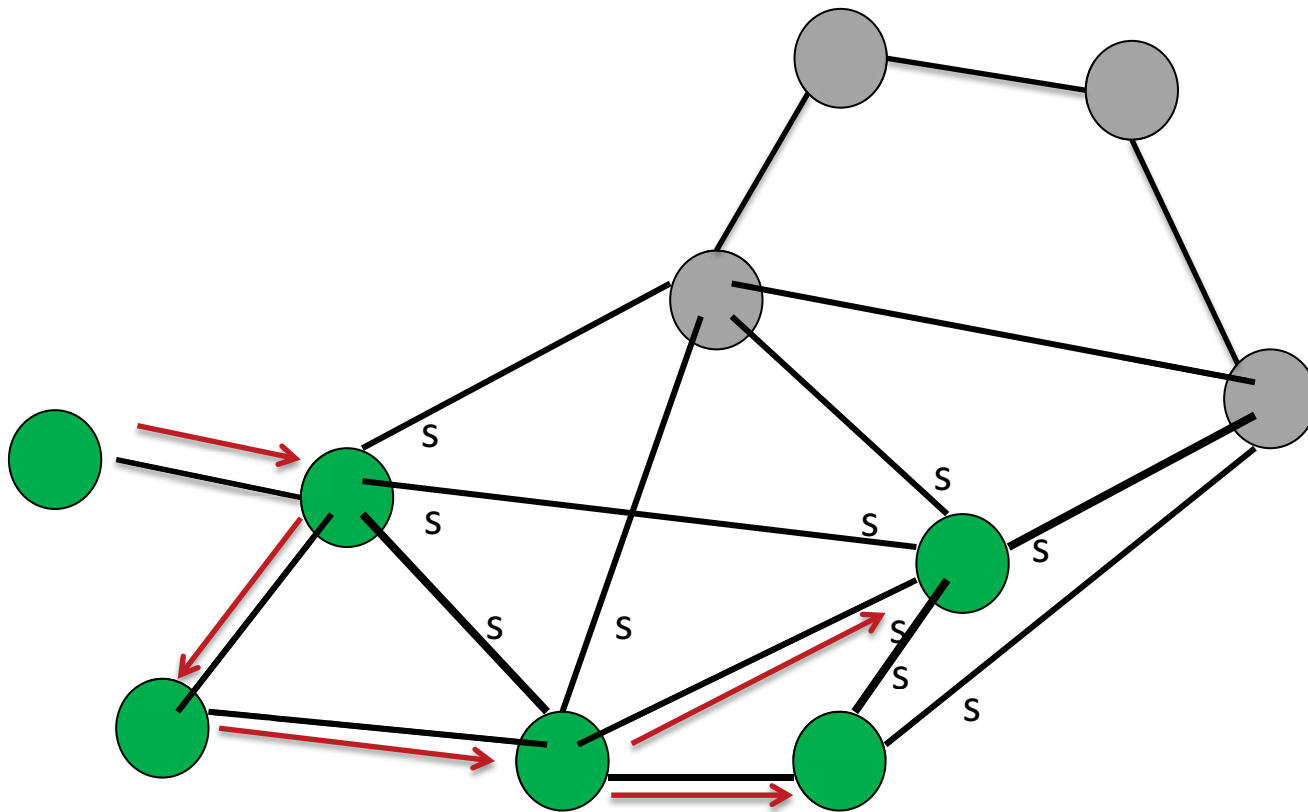


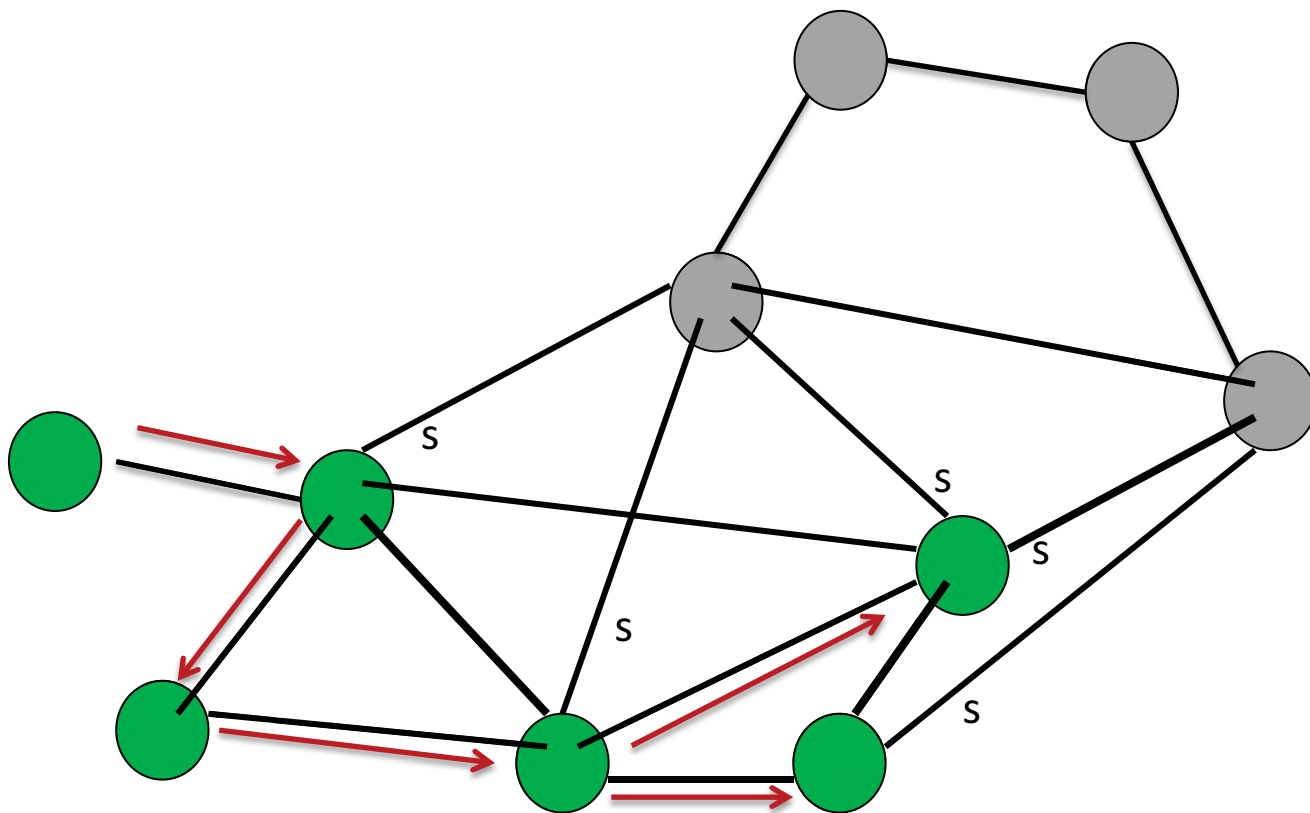


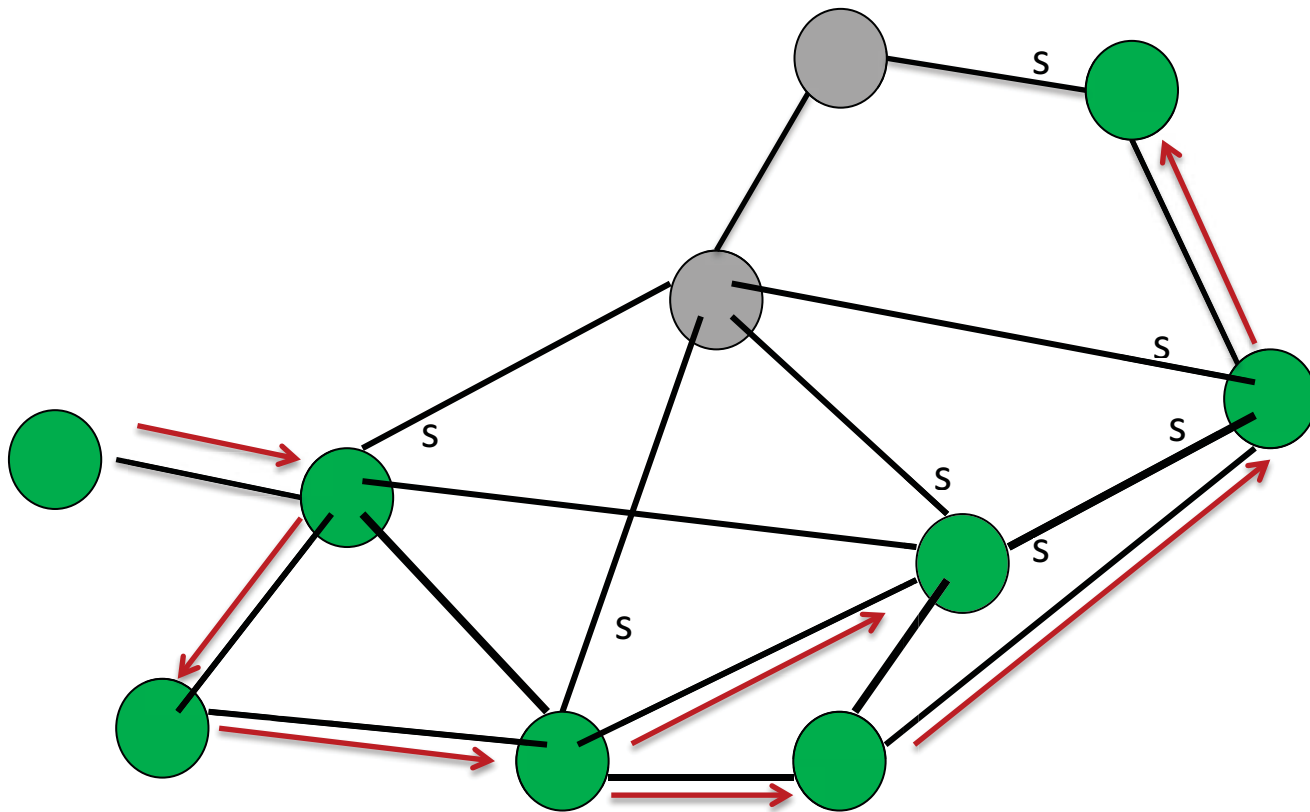




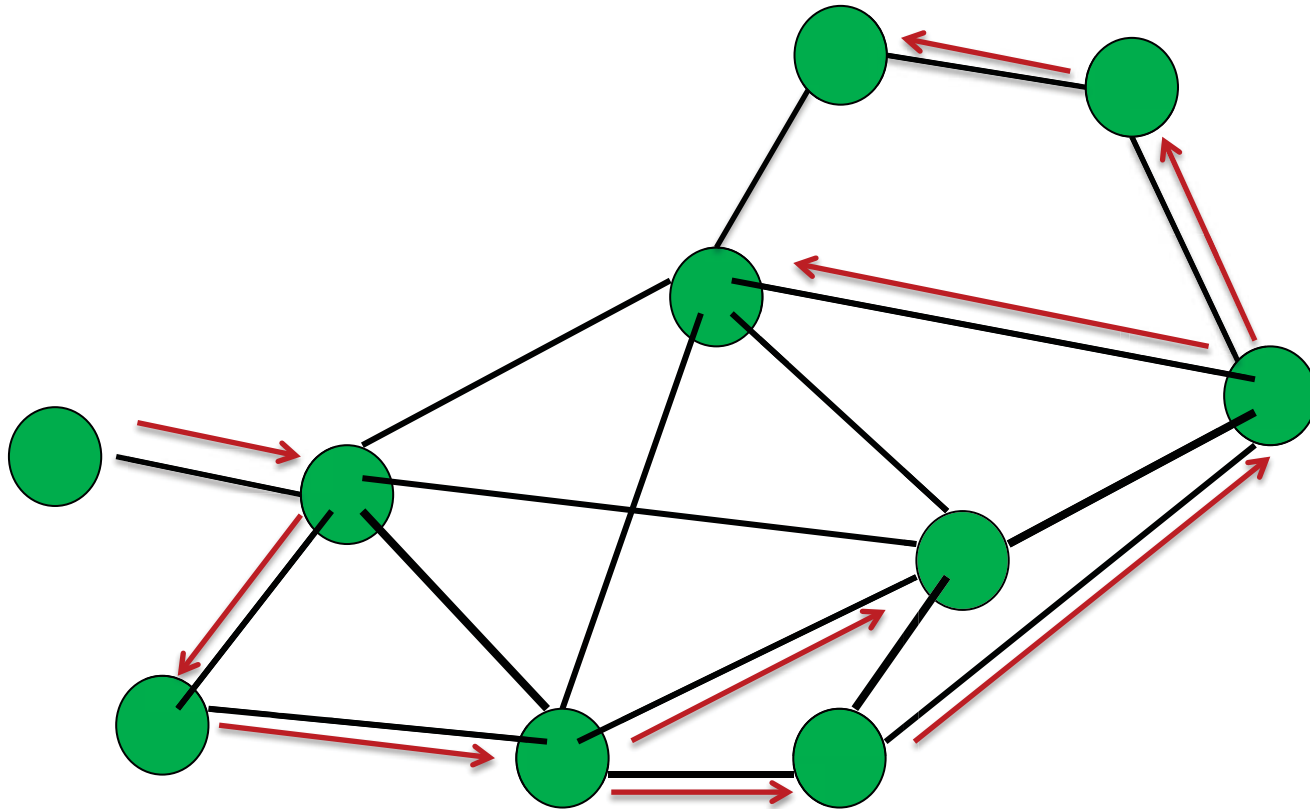




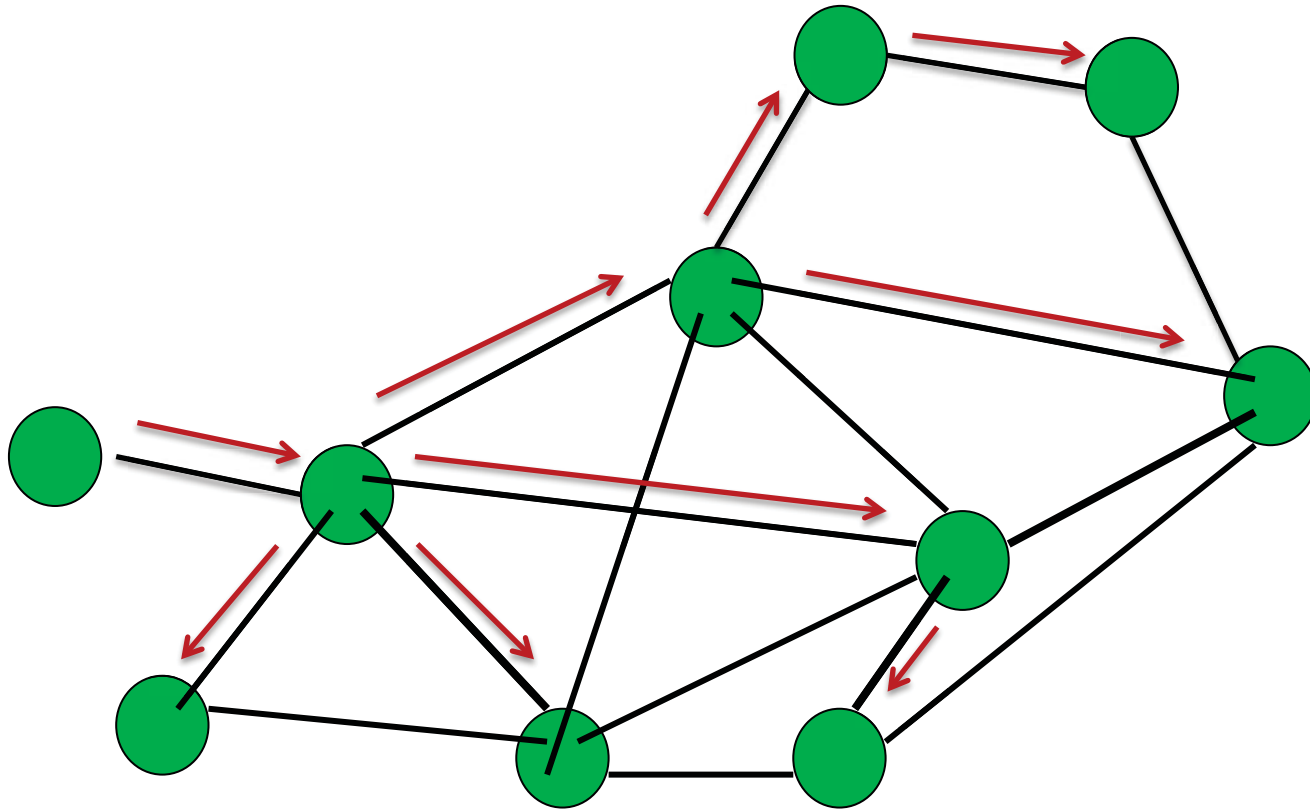




Final Spanning Tree

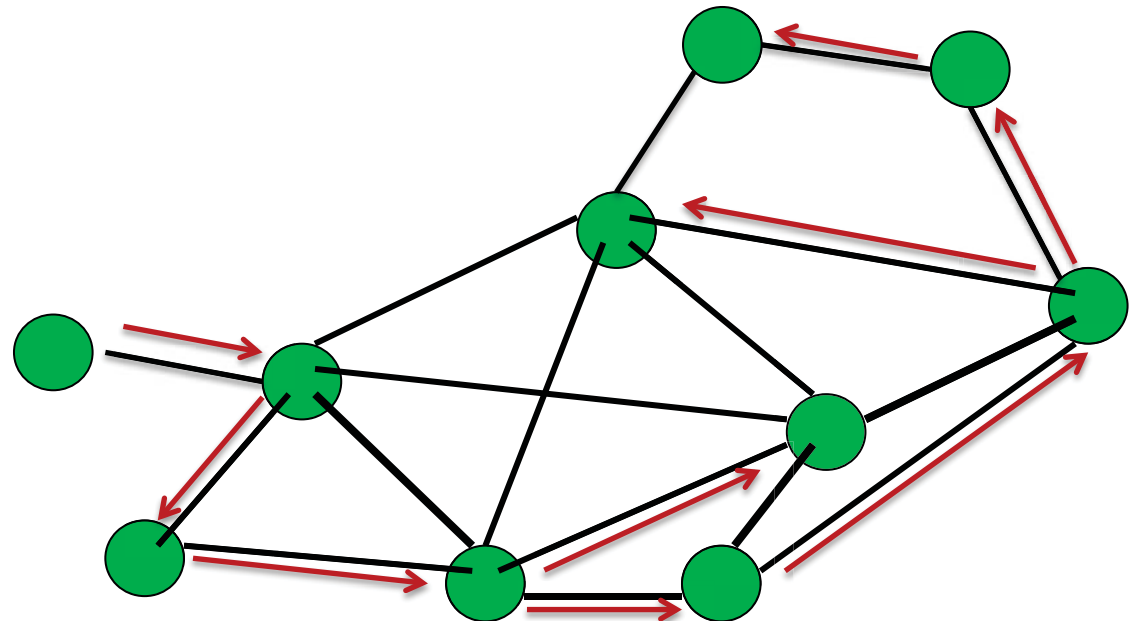


Actual BFS



Anomaly

- Paths produced by the algorithm may be longer than the shortest paths.
- Because in asynchronous networks, messages may propagate faster along longer paths.



Complexity

- **Message complexity:**

- Number of messages sent by all processes during the entire execution.
- $O(|E|)$

- **Time complexity:**

- Time until all processes have chosen their parents.
- Neglect local processing time.
- $O(diam \cdot d)$
- **Q:** Why *diam*, when some of the paths are longer?
- The time until a node receives a *search* message is at most the time it would take on a shortest path.

Extensions

- Child pointers:
 - As for synchronous BFS.
 - Everyone who receives a *search* message sends back a *parent* or *nonparent* response.
- Termination:
 - After a node has received responses to all its *search* its messages, it knows who its children are, and knows they are marked.
 - The leaves of the tree learn who they are.
 - Use a *convergecast* strategy, as before.
 - **Time complexity:** After the tree is done, it takes time $O(n \cdot d)$ for the *done* information to reach i_0 .
 - **Message complexity:** $O(n)$

Applications

- **Message broadcast:**
 - Process i_0 can use the tree (with child pointers) to broadcast a message.
 - Takes $O(n \cdot d)$ time and n messages.
- **Global computation:**
 - Suppose every process starts with some initial value, and process i_0 should determine the value of some function of the set of all processes' values.
 - Use convergecast on the tree.
 - Takes $O(n \cdot d)$ time and n messages.

Second Attempt

- A **relaxation algorithm**, like synchronous Bellman-Ford.
- Before, we corrected for paths with many hops but low weights.
- Now, instead, correct for errors caused by asynchrony.
- **Strategy:**
 - Each process keeps track of the hop distance, changes its parent when it learns of a shorter path, and propagates the improved distances.
 - Eventually stabilizes to a breadth-first spanning tree.

Process Automaton P_u

- Input actions: $receive(m)_{v,u}$, m a nonnegative integer
- Output actions: $send(m)_{u,v}$, m a nonnegative integer
- State variables:
 - $parent$: $\Gamma(u) \cup \{\perp\}$, initially $\perp$
 - $dist \in N \cup \{\infty\}$, initially 0 if $u = v_0$, ∞ otherwise
 - For every $v \in \Gamma(u)$:
 - $send(v)$, a FIFO queue of N , initially (0) if $u = v_0$, else empty
- Transitions:
 - $receive(m)_{v,u}$
 - Effect: if $m + 1 < dist$ then
 - $dist := m + 1$
 - $parent := v$
 - for every w , add $dist$ to $send(w)$

Process Automaton P_u

- Transitions:
 - $\text{receive}(m)_{v,u}$
 - Effect: if $m + 1 < \text{dist}$ then
 - $\text{dist} := m + 1$
 - $\text{parent} := v$
 - for every w , add $m + 1$ to $\text{send}(w)$
 - $\text{send}(m)_{u,v}$
 - Precondition: $m = \text{head}(\text{send}(v))$
 - Effect: remove head of $\text{send}(v)$
- No terminating actions...

Correctness

- For synchronous BFS, we characterized precisely the situation after r rounds.
- We can't do that now.
- Instead, state abstract properties, e.g., invariants and timing properties, e.g.:
- **Invariant:** At any point, for any node $u \neq v_0$, if its *dist* $\neq \infty$, then it is the actual distance on some path from v_0 to u , and its *parent* is u 's predecessor on such a path.
- **Timing property:** For any node u , and any r , $0 \leq r \leq \text{diam}$, if there is an at-most- r -hop path from v_0 to u , then by time $r \cdot n \cdot d$, node u 's *dist* is $\leq r$.

Complexity

- **Message complexity:**
 - Number of messages sent by all processes during the entire execution.
 - $O(n |E|)$
- **Time complexity:**
 - Time until all processes' *dist* and *parent* values have stabilized.
 - Neglect local processing time.
 - $O(diam \cdot n \cdot d)$
 - Time until each node receives a message along a shortest path, counting time $O(n \cdot d)$ to traverse each link.

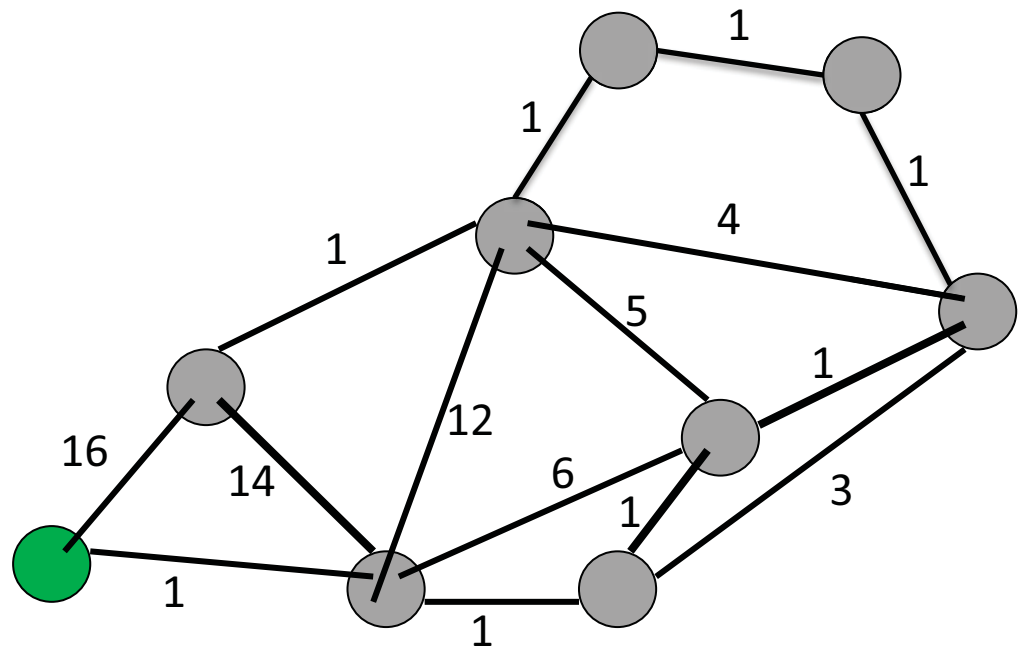
Termination

- Q: How can processes learn when the tree is completed?
- Q: How can a process know when it can output its own *dist* and *parent*?
- Knowing a bound on n doesn't help here: can't use it to count rounds.
- Can use **convergecast**, as for synchronous Bellman-Ford:
 - Compute and recompute child pointers.
 - Process $\neq v_0$ sends *done* to its current parent after:
 - It has received responses to all its messages, so it believes it knows all its children, and
 - It has received *done* messages from all of those children.
 - The same process may be involved several times, based on improved estimates.

Uses of Breadth-First Spanning Trees

- Same as in synchronous networks, e.g.:
 - Broadcast a sequence of messages
 - Global function computation
- Similar costs, but now count time d instead of one round.

Shortest Paths Trees



Shortest Paths

- **Problem:** Compute a **Shortest Paths Spanning Tree** in an **asynchronous network**.
- Connected weighted graph, root vertex v_0 .
- $weight_{\{u,v\}}$ for edge $\{u, v\}$.
- Processes have no knowledge about the graph, except for weights of incident edges.
- **UIDs**
- Processes must produce a Shortest Paths spanning tree rooted at v_0 .
- Each process $u \neq v_0$ should output its distance and parent in the tree.

Shortest Paths

- Use a **relaxation algorithm**, once again.
- **Asynchronous Bellman-Ford**.
- Now, it handles two kinds of corrections:
 - Because of long, small-weight paths (as in synchronous Bellman-Ford).
 - Because of asynchrony (as in asynchronous Breadth-First search).
- The combination leads to surprisingly high message and time complexity, much worse than either type of correction alone (exponential).

Asynch Bellman-Ford, Process P_u

- Input actions: $receive(m)_{v,u}$, m a nonnegative integer
- Output actions: $send(m)_{u,v}$, m a nonnegative integer
- State variables:
 - $parent$: $\Gamma(u) \cup \{\perp\}$, initially $\perp$
 - $dist \in N \cup \{\infty\}$, initially 0 if $u = v_0$, ∞ otherwise
 - For every $v \in \Gamma(u)$:
 - $send(v)$, a FIFO queue of N , initially (0) if $u = v_0$, else empty
- Transitions:
 - $receive(m)_{v,u}$
 - Effect: if $m + weight_{\{v,u\}} < dist$ then
 - $dist := m + weight_{\{v,u\}}$
 - $parent := v$
 - for every w , add $dist$ to $send(w)$

Asynch Bellman-Ford, Process P_u

- Transitions:
 - $receive(m)_{v,u}$
 - Effect: if $m + weight_{\{v,u\}} < dist$ then
 - $dist := m + weight_{\{v,u\}}$
 - $parent := v$
 - for every w , add $dist$ to $send(w)$
 - $send(m)_{u,v}$
 - Precondition: $m = head(send(v))$
 - Effect: remove head of $send(v)$
- No terminating actions...

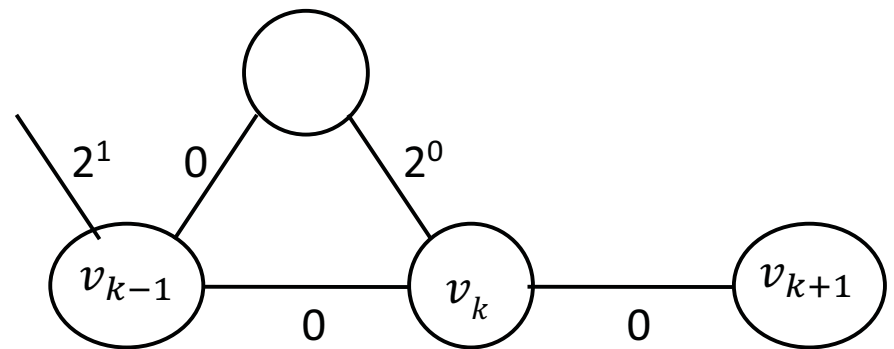
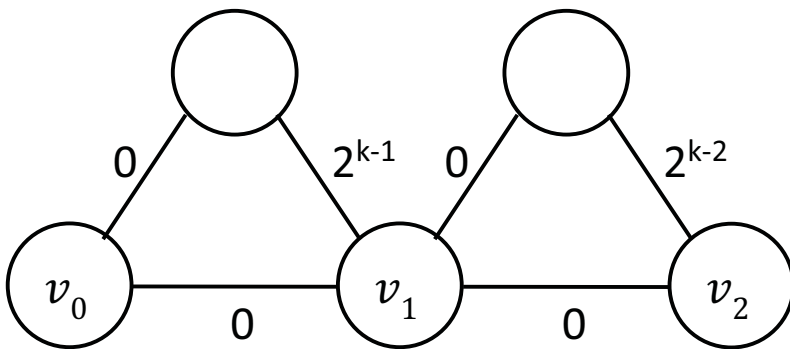
Correctness:

Invariants and Timing Properties

- **Invariant:** At any point, for any node $u \neq v_0$, if its *dist* $\neq \infty$, then it is the actual distance on some path from v_0 to u , and its *parent* is u 's predecessor on such a path.
- **Timing property:** For any node u , and any r , $0 \leq r \leq \text{diam}$, if p is any at-most- r -hop path from v_0 to u , then by time ???, node u 's *dist* is $\leq$ total weight of p .
- **Q:** What is ??? ?
- It depends on how many messages might pile up in a channel.
- This can be a lot!

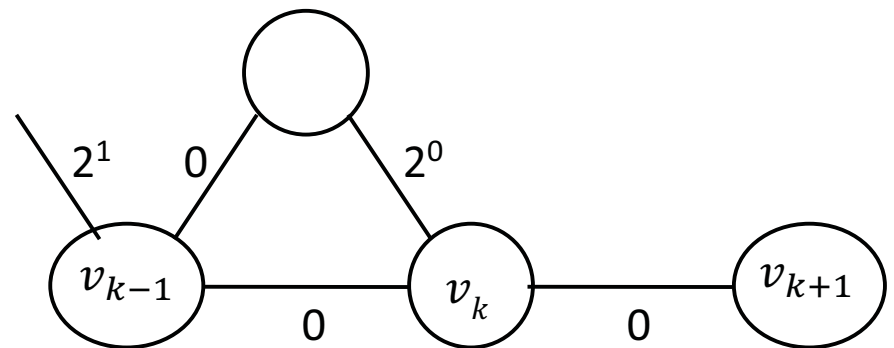
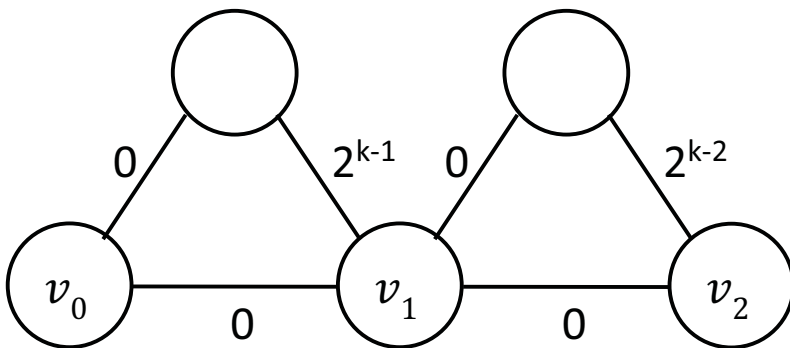
Complexity

- $O(n!)$ simple paths from v_0 to any other node u , which is $O(n^n)$.
- So the number of messages sent on any channel is $O(n^n)$.
- **Message complexity:** $O(n^n |E|)$.
- **Time complexity:** $O(n^n \cdot n \cdot d)$.
- **Q:** Are such exponential bounds really achievable?



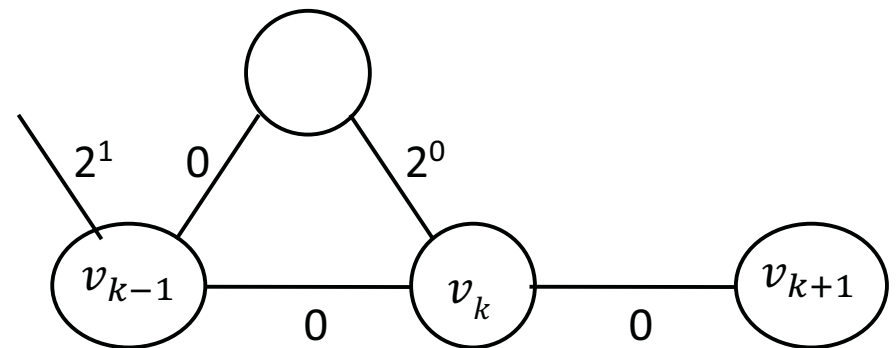
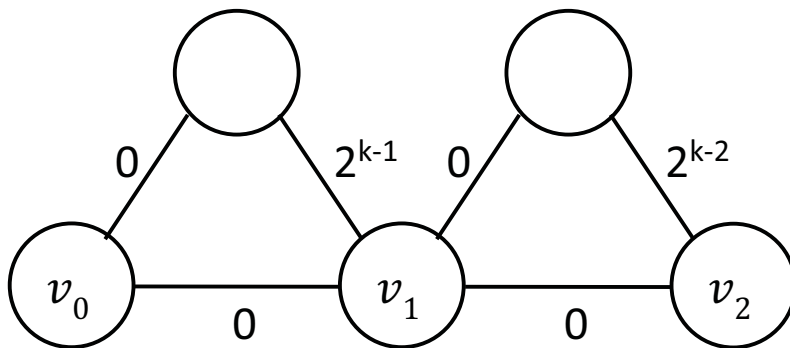
Complexity

- **Q:** Are such exponential bounds really achievable?
- **Example:**
 - There is an execution of the network below in which node v_k sends $2^k \approx 2^{n/2}$ messages to node v_{k+1} .
 - Message complexity is $\Omega(2^{n/2})$.
 - Time complexity is $\Omega(2^{n/2} d)$.



Complexity

- Execution in which node v_k sends 2^k messages to node v_{k+1} .
- Possible distance estimates for v_k are $2^k - 1, 2^k - 2, \dots, 0$.
- Moreover, v_k can take on all these estimates in sequence:
 - First, messages traverse upper links, $2^k - 1$.
 - Then last lower message arrives at v_k , $2^k - 2$.
 - Then lower message $v_{k-2} \rightarrow v_{k-1}$ arrives, reduces v_{k-1} 's estimate by 2, message $v_{k-1} \rightarrow v_k$ arrives on upper links, $2^k - 3$.
 - Etc. Count down in binary.
 - If this happens quickly, get pileup of 2^k search messages in $C_{k,k+1}$.



Termination

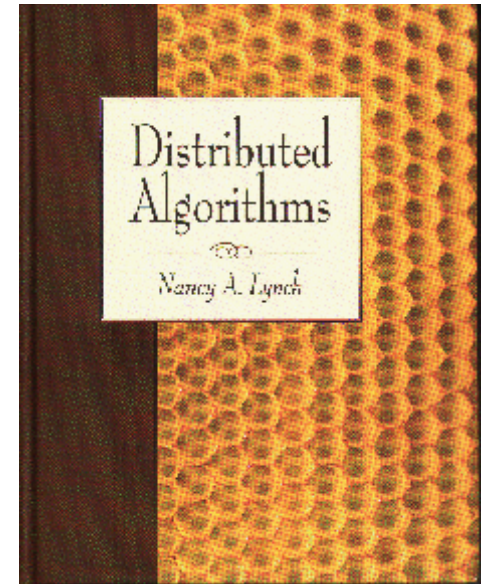
- **Q:** How can processes learn when the tree is completed?
- **Q:** How can a process know when it can output its own *dist* and *parent*?
- **Convergecast**, once again
 - Compute and recompute child pointers.
 - Process $\neq v_0$ sends *done* to its current parent after:
 - It has received responses to all its messages, so it believes it knows all its children, and
 - It has received *done* messages from all of those children.
 - The same process may be involved several (many) times, based on improved estimates.

Shortest Paths

- Moral: Unrestrained asynchrony can cause problems.
- What to do?
- Find out in 6.852/18.437, Distributed Algorithms!

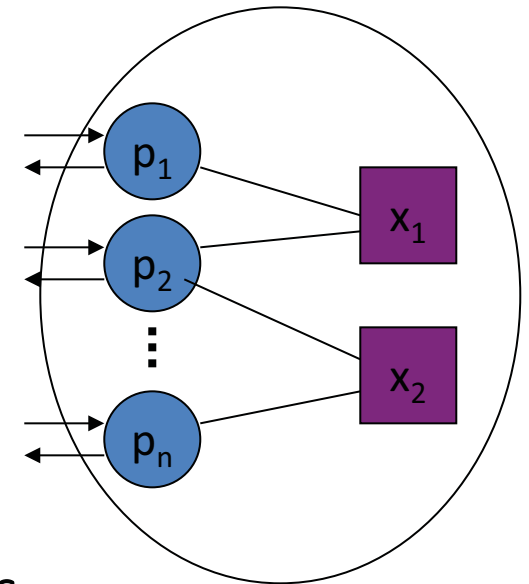
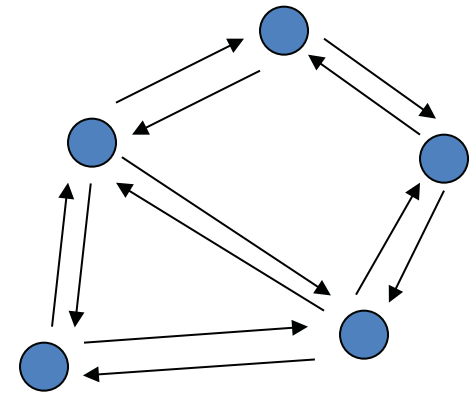
What's Next?

- 6.852/18.437 Distributed Algorithms
- Basic grad course
- Covers synchronous, asynchronous, and timing-based algorithms
- Synchronous algorithms:
 - Leader election
 - Building various kinds of spanning trees
 - Maximal Independent Sets and other network structures
 - Fault tolerance
 - Fault-tolerant consensus, commit, and related problems



Asynchronous Algorithms

- Asynchronous network model
- Leader election, network structures.
- Algorithm design techniques:
 - Synchronizers
 - Logical time
 - Global snapshots, stable property detection.
- Asynchronous shared-memory model
- Mutual exclusion, resource allocation
- Fault tolerance
- Fault-tolerant consensus and related problems
- Atomic data objects, atomic snapshots
- Transformations between models.
- Self-stabilizing algorithms



And More

- Timing-based algorithms
 - Models
 - Revisit some problems
 - New problems, like clock synchronization.
- Newer work (maybe):
 - Dynamic network algorithms
 - Wireless networks
 - Insect colony algorithms and other biological distributed algorithms

MIT OpenCourseWare
<http://ocw.mit.edu>

6.046J / 18.410J Design and Analysis of Algorithms
Spring 2015

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Lecture 21: Cryptography: Hashing

In this lecture, we will be studying some basics of cryptography. Specifically, we will be covering

- Hash functions
- Random oracle model
- Desirable Properties
- Applications to security

1 Hash Functions

A hash function h maps arbitrary strings of data to fixed length output. The function is deterministic and public, but the mapping should look “random”. In other words,

$$h : \{0, 1\}^* \rightarrow \{0, 1\}^d$$

for a fixed d . Hash functions do not have a secret key. Since there are no secrets and the function itself is public, anyone can evaluate the function. To list some examples,

Hash function	MD4	MD5	SHA-1	SHA-256	SHA-512
d	128	128	160	256	512

In practice, hash functions are used for “digesting” large data. For example, if you want to check the validity of a large file (potentially much larger than a few megabytes), you can check the hash value of that file with the expected hash. Therefore, it is desirable (especially for cryptographic hash functions covered here) that the function is collision resistant. That is, it should be “hard” to find two inputs m_1 and m_2 for hash function h such that $h(m_1) = h(m_2)$. Most modern hash functions hope to achieve security level of 2^{64} or better, which means that the attacker needs to test more than 2^{64} different inputs to find a collision. Unfortunately, MD4 and MD5 aimed to provide 2^{64} security, but has been shown to be broken using 2^6 and 2^{37} inputs respectively. SHA-1 aimed to provide 2^{80} security, but has been shown (at least theoretically) to be no more than 2^{61} security.

1.1 Random Oracle

The Random Oracle model is an ideal model of the hash function that is not achievable in practice. In this model, we assume there exists an oracle h such that on input $x \in \{0,1\}^*$, if h has not seen x before, then it outputs a random value as $h(x)$. Otherwise, it returns $h(x)$ it previously output. The random oracle gives a random value for all new inputs, and gives deterministic answers to all inputs it has seen before. Unfortunately, a random oracle does not exist since it requires infinite storage, so in practice we use pseudo-random functions.

1.2 Desirable Properties

There are many desirable properties of a hash function.

1. One-way (pre-image resistance): Given $y \in \{0,1\}^d$, it is hard to find an x such that $h(x) = y$.
2. Strong collision-resistance: It is hard to find any pair of inputs x, x' such that $h(x) = h(x')$.
3. Weak collision-resistance (target collision resistance, 2nd pre-image resistance): Given x , it is hard to find x' such that $h(x) = h(x')$.
4. Pseudo-random: The function behaves indistinguishable from a random oracle.
5. Non-malleability: Given $h(x)$, it is hard to generate $h(f(x))$ for any function f .

Some of the properties imply others, and some others do not. For example,

- $2 \Rightarrow 3$
- $1 \not\Rightarrow 2, 3$.

Furthermore, collision can be found in $O(2^{d/2})$ (using birthday paradox), and inversion can be found in $O(2^d)$.

To give more insight as to why some properties do not imply others, we provide examples here. Consider h that satisfies 1 and 2. We can construct a new h' such that h' takes in one extra bit of input, and XORs the first two bits together to generate an input for h . That is, $h'(a, b, x_2, \dots, x_n) = h((a \oplus b), x_2, \dots, x_n)$. h' is still one-way, but is not weak collision resistant. Now consider a different h'' that evaluates to 0 if $|x| \leq n$, and $1 || h(x)$ otherwise. h'' is weak collision resistant, but is not one-way.

1.3 Applications

There are many applications of hash functions.

1. Password Storage: We can store hash $h(p)$ for password p instead of p directly, and check $h(p)$ to authenticate a user. If it satisfies the property 1, adversary comprising $h(p)$ will not learn p .
2. File Authenticity: For each file F , we can store $h(F)$ in a secure location. To check authenticity of a file, we can recompute $h(F)$. This requires property 3.
3. Digital Signature: We can use hash functions to generate a signature that guarantees that the message came from a said source. For further explanation, refer to Recitation 11.
4. Commitments: In a secure bidding, Alice wants to bid value x , but does not want to reveal the bid until the auction is over. Alice then computes $h(x)$, and publicize it, which serves as her commitment. When bidding is over, then she can reveal x , and x can be verified using $h(x)$.

It should be “binding” so that Alice cannot generate a new x that has the same commitment, and it should “hide” x so no one learns anything before x is revealed. Furthermore, it should be non-malleable. To guarantee secrecy, we need more than the properties from the previous section, as $h'(x) = h(x) || MSB(x)$ actually satisfies 1, 2, and 5. In practice, this problem is bypassed by adding randomness in the commitment, $C(x) = h(r || x)$ for $r \in_R \{0, 1\}^{256}$, and reveal both randomness and x at the end.

MIT OpenCourseWare
<http://ocw.mit.edu>

6.046J / 18.410J Design and Analysis of Algorithms
Spring 2015

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Lecture 22: Cryptography: Encryption

- Symmetric key encryption
- Key exchange
- Asymmetric key encryption
- RSA
- NP-complete problems and cryptography
 - graph coloring
 - knapsack

Symmetric key encryption

$$c = e_k(m)$$

$$m = d_k(c)$$

Here c is the *ciphertext*, m is the *plaintext*, e is the encryption function, d is the decryption function and k is the secret key. e , d permute and reverse-permute the space of all messages.

Reversible operations: $\oplus$, $+/-$, shift left/right.

Symmetric algorithms: AES, RC5, DES

Key Management Question

How does secret key k get exchanged/shared?

Alice wants to send a message to Bob. There are pirates between Alice and Bob, that will take any keys or messages in unlocked box(es), but won't touch locked boxes. How can Alice send a message or a key to Bob (without pirates knowing what was sent)?

Solution:

- Alice puts m in box, locks it with k_A
- Box sent to Bob

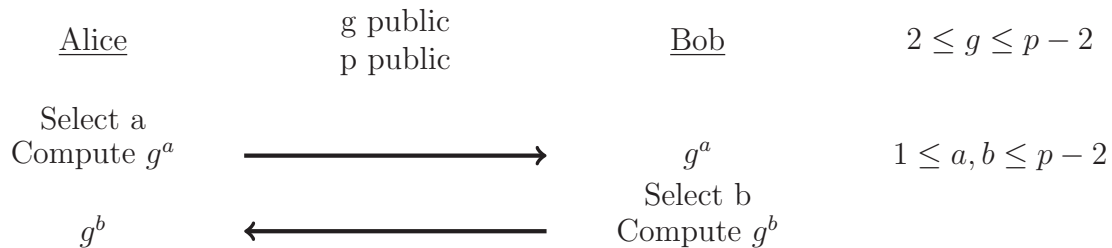
- Bob locks box with k_B
- Box sent to Alice
- Alice unlocks k_A
- Box sent to Bob
- Bob unlocks k_B , reads m

Notice that this method relied on the commutativity of the locks. This means that the order of the lock and unlock operations doesn't matter.

Diffie-Hellman Key Exchange

$$G = F_p^*$$

Here F_p^* is a finite field (mod p , a prime). * means invertible elements only ($\{1, 2, \dots, p-1\}$)



Alice can compute $(g^b)^a \mod p = k$.

Bob can compute $(g^a)^b \mod p = k$.

Assumes the Discrete Log Problem is hard (given g^a , compute a) and Diffie Hellman Problem is hard (given g^a, g^b compute g^{ab}).

Can we attack this? **Man-in-the-middle:**

- Alice doesn't know she is communicating with Bob.
- Alice agrees to a key exchange with Eve (thinking she is Bob).
- Bob agrees to a key exchange with Eve (thinking she is Alice).
- Eve can see all communications.

Public Key Encryption

message + public key = ciphertext
 ciphertext + private key = message

The two keys need to be linked in a mathematical way. Knowing the public key should tell you nothing about the private key.

RSA

- Alice picks two large secret primes p and q .
- Alice computes $N = p \cdot q$.
- Chooses an encryption exponent e which satisfies $\gcd(e, (p-1)(q-1)) = 1$,
 $e = 3, 17, 65537$.
- Alice's public key = (N, e) .
- Decryption exponent obtained using Extended Euclidean Algorithm by Alice such that $e \cdot d \equiv 1 \pmod{(p-1)(q-1)}$.
- Alice private key = (d, p, q) (storing p and q is not absolutely necessary, but we do it for efficiency).

Encryption and Decryption with RSA

$$\begin{array}{ll} c = m^e \pmod{N} & \text{encryption} \\ m = c^d \pmod{N} & \text{decryption} \end{array}$$

Why it works

$$\phi = (p-1)(q-1)$$

Since $ed \equiv 1 \pmod{\phi}$ there exists an integer k such that $ed = 1 + k\phi$.

Two cases:

Case 1 $\gcd(m, p) = 1$. By Fermat's theorem,

$$\begin{aligned} m^{p-1} &\equiv 1 \pmod{p} \\ (m^{p-1})^{k(q-1)} \cdot m &\equiv m \pmod{p} \\ m^{1+k(p-1)(q-1)} &= m^{ed} \equiv m \pmod{p} \end{aligned}$$

Case 2 $\gcd(m, p) = p$. This means that $m \bmod p = 0$ and so $m^{ed} \equiv m$

Thus, in both cases, $m^{ed} \equiv m \bmod p$. Similarly, $m^{ed} \equiv m \bmod q$. Since p, q are distinct primes, $m^{ed} \equiv m \bmod N$. So $c^d = (m^e)^d \equiv m \bmod N$

Hardness of RSA

- Factoring: given N , hard to factor into p, q .
- RSA Problem: given e such that $\gcd(e, (p-1)(q-1)) = 1$ and c , find m such that $m^e \equiv c \bmod N$.

NP-completeness

Is N composite with a factor within a range? [unknown if NP-complete](#)

Is a graph k -colorable? In other words: can you assign one of k colors to each vertex such that no two vertices connected by an edge share the same color? [NP-complete](#)

Given a pile of n items, each with different weights w_i , is it possible to put items in a knapsack such that we get a specific total weight S ? [NP-complete](#)

NP-completeness and Cryptography

- NP-completeness: about worst-case complexity
- Cryptography: want a problem instance, with suitably chosen parameters that is hard on average

Most knapsack cryptosystems have failed.

Determining if a graph is 3-colorable is NP-complete, but very easy on average. This is because an average graph, beyond a certain size, is not 3-colorable!

Consider a standard backtracking search to determine 3-colorability.

- Order vertices $v_1, \dots, v_t$. Colors = $\{1, 2, 3\}$
- Traverse graph in order of vertices.
- On visiting a vertex, choose smallest possible color that “works”.
- If you get stuck, backtrack to previous choice, and try next choice.

- Run out of colors for 1st vertex → output 'NO'
- Successfully color last vertex → output 'YES'

On a random graph of t vertices, average number of vertices traveled < 197 , regardless of t !

Knapsack Cryptography

General knapsack problem: NP-complete

Super-increasing knapsack: linear time solvable. In this problem, the weights are constrained as follows:

$$w_j \geq \sum_{i=1}^{j-1} w_i$$

Merkle Hellman Cryptosystem

Private key → super-increasing knapsack problem $\xrightarrow{\text{Private transform}}$ “hard” general knapsack problem → public key.

Transform: two private integers N, M s.t. $\gcd(N, M) = 1$.

Multiply all values in the sequence by N and then take mod M .

Example: $N = 31$, $M = 105$, private key = {2, 3, 6, 14, 27, 52},
public key = {62, 93, 81, 88, 102, 37}

Merkle Hellman Example

Message =	011000	110101	101110
Ciphertext:	011000	$93 + 81 = 174$	
	110101	$62 + 93 + 88 + 37 = 280$	
	101110	$62 + 81 + 88 + 102 = 333$	
		$= 174, 280, 333$	

Recipient knows $N = 31$, $M = 105$, {2, 3, 6, 14, 27, 52}. Multiplies each ciphertext block by $N^{-1} \pmod{M}$. In this case, $N^{-1} = 61 \pmod{105}$.

$174 \cdot 61 = 9 = 3 + 6 = 011000$
$280 \cdot 61 = 70 = 2 + 3 + 13 + 52 = 110101$
$333 \cdot 61 = 48 = 2 + 6 + 13 + 27 = 101110$

Beautiful but broken

Lattice based techniques break this scheme.

Density of knapsack $d = \frac{n}{\max\{\log_2 w_i : 1 \leq i \leq n\}}$

Lattice basis reduction can solve knapsacks of low density. Unfortunately, M-H scheme always produces knapsacks of low density.

MIT OpenCourseWare
<http://ocw.mit.edu>

6.046J / 18.410J Design and Analysis of Algorithms
Spring 2015

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

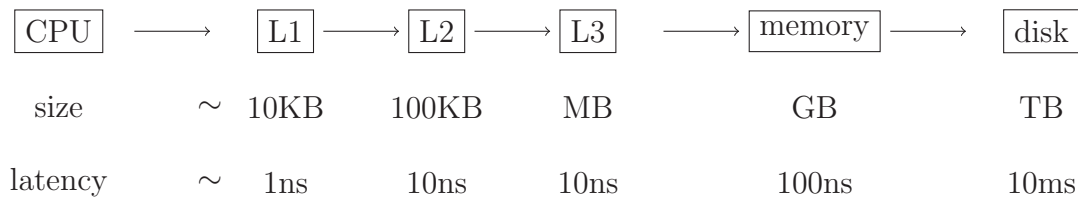
Lecture 23: Cache-oblivious Algorithms I

This lecture introduces cache-oblivious algorithms. Topics include

- memory hierarchy
- external memory vs. cache-oblivious model
- cache-oblivious scanning
- cache-oblivious divide & conquer algorithms: median finding & matrix multiplication

1 Modern Memory Hierarchy

So far in this class, we have viewed all operations and memory accesses as equal cost. However, modern computers have memory hierarchy.

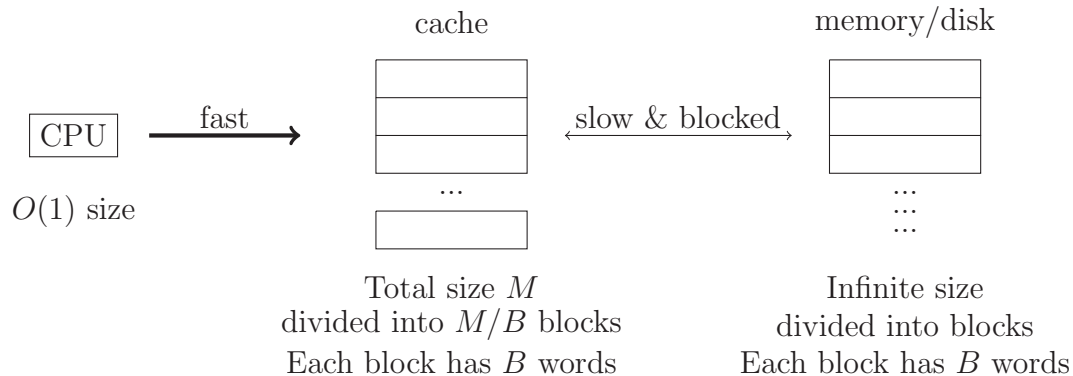


Each hierarchy on the right is bigger, but has longer latency due to the longer distance data has to travel. Yet, bandwidth between different hierarchies is usually matched.

A common technique to mitigate latency is *blocking*: when fetching a word of data, get the entire block containing it. Using algorithmic terminology, the idea is to amortize latency over a whole block. For this idea to work, we additionally require the algorithm to use all words in a block (spatial locality) and reuse blocks in cache (temporal locality).

2 Memory Model of Algorithms

2.1 External Memory Model



In this model, cache accesses are free. The algorithm explicitly reads and writes memory in blocks. We will count the number of memory transfers between the cache and the memory, as an important metric of the algorithm's performance.

2.2 Cache-oblivious Model

The only change from the external memory model is that the algorithm no longer knows B and M . Accessing a word in the memory automatically fetches an entire block into the cache, and evicts the least recently used (LRU) block from the cache if the cache is full.

Every algorithm is a cache-oblivious algorithm, but we would like to find the algorithm that minimizes our new metric — the number of memory transfers.

Why do we like cache-oblivious algorithms (as opposed to letting the algorithm know B and M)? Because this way, an algorithm can auto-tune itself and run efficiently on different computers (possibly with different B and M). Besides, it is cool research!

3 Scanning

Example program:

```
for  $i$  in range( $N$ ): sum +=  $A[i]$ 
```

Assume A is stored contiguously in memory. External memory model can align A with a block boundary, so it needs $\lceil N/B \rceil$ memory transfers.

Cache-oblivious algorithms cannot control alignment (because it does not know B), so it needs $\lceil N/B \rceil + 1 = N/B + O(1)$ memory transfers. $O(1)$ parallel scans still need $O(N/B + 1)$ memory transfers.

4 Divide & Conquer

Divide & Conquer algorithms divide problems down to $O(1)$ size. The base case of the recursion is either when

- problem fits in cache i.e., $\leq M$, or
- problem fits in $O(1)$ blocks, i.e., $O(B)$.

Below we will see one example for each.

4.1 Median Finding / Order Statistics

Recall the steps of the algorithm

1. view array as partitioned into columns of 5 (each column is $O(1)$ size).
2. sort each column
3. recursively find the median of the column medians
4. partition array by x
5. recurse on one side

We will now analyze the number of memory transfers in each step. Let $MT(N)$ be the total number of memory transfers.

1. free
2. a scan, $O(N/B + 1)$
3. $MT(N/5)$, this involves a pre-processing step that coalesces the $N/5$ elements in a consecutive array
4. 3 parallel scans, $O(N/B + 1)$
5. $MT(7N/10)$

Therefore, we get the recursion

$$MT(N) = MT(N/5) + MT(7N/10) + O(N/B + 1).$$

Solving the recursion requires setting a base case. An obvious base case is $MT(O(1)) = O(1)$.

But we can get a stronger base case: $MT(O(B)) = O(1)$. Using this base case, the recursion solves to $MT(N) = O(N/B + 1)$. (Intuition: cost at level of the recursion decreases geometrically, so the cost at root dominates.)

4.2 Matrix Multiplication

Problem: compute $Z = X \cdot Y$ where X, Y, Z are all $N \times N$ matrices. Also suppose X is stored in row-major order, and Y is stored in column-major order to improve locality.

If we use the basic algorithm, computing one element in Z requires one or two parallel scans, because it either requires scanning either a new row from X , or a new column from Y . Computing each element takes $O(N/B + 1)$ memory transfers, so computing the entire Z costs $O(N^3/B + N^2)$ memory transfers.

Instead we will use the blocked matrix multiplication algorithm (not Strassen). Note that key block should be stored consecutively. We get recursion

$$MT(N) = 8MT(N/2) + O(N^2/B + 1)$$

The first term is recursive sub-matrix multiplication, and the second term is matrix addition which requires scanning the matrices.

Again, we can have different base cases

- weak: $MT(O(1)) = O(1)$
- better: $MT(O(B)) = O(1)$
- even better: $MT(\sqrt{M}/3) = O(M/B)$

The third case represents the case that the three involved matrices can fit in the cache together. Therefore, to multiply them, we only need one scan to load all of them into cache.

If we draw the recursion tree, the cost at each level is geometrically increasing this time, $N^2/B, 8(\frac{N}{2})^2/B, 8^2(\frac{N}{4})^2/B, \dots$. Therefore, the cost at the leaves dominate, and the total cost = cost per leaf $\cdot$ number of leaves,

$$MT(N) = O(M/B) \cdot 8^{O(\lg N/\sqrt{M})} = O(M/B) \cdot O((N/\sqrt{M})^3) = O(N^3/B\sqrt{M}).$$

MIT OpenCourseWare
<http://ocw.mit.edu>

6.046J / 18.410J Design and Analysis of Algorithms
Spring 2015

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Lecture 24: Cache-oblivious algorithms II

- Search
 - binary
 - B-ary
 - cache-oblivious
- Sorting
 - mergesorts
 - cache-oblivious

Why LRU block replacement strategy?

$LRU_M \leq 2 \cdot OPT_{M/2}$ [Sleator and Tarjan 1985]

Proof.

- partition block access sequence into maximal phases of M/B distinct blocks
- LRU spends $\leq M/B$ memory transfers/phase
- OPT must spend $\geq \frac{M}{2}/B$ memory transfers per phase: at best, starts phase with entire $M/2$ cache with needed items. But there are M/B blocks during phase. So $\leq$ half free

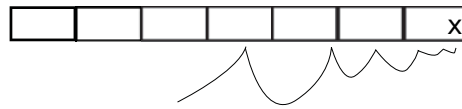
Search

Preprocess n elements in comparison model to support predecessor search for x .

B-trees

They support predecessor (and insert and delete) in $O(\log_{B+1} N)$ memory transfers.

- each node occupies $\Theta(1)$ blocks
- height = $\Theta(\log_B N)$
- need to know B

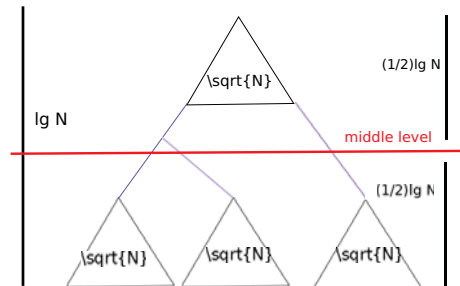


Binary search

Approximately, every iteration visits a different block until we are in x 's block. Thus, $MT(N) = \Theta(\log N - \log B) = \Theta(\log(N/B))$. **SLOW**

van Emde Boas layout

[Prokop 1999]



- store N elements in complete BST
- carve BST at middle level of edges
- recursively layout the pieces and concatenate
- like block matrix multiplication, order of pieces doesn't matter; just need each piece to be stored consecutively

Analysis of BST search in vEB layout:

- consider recursive level of refinement at which structure has $\leq B$ nodes
- the height of the vEB tree is between $\frac{1}{2} \lg B$ and $\lg B \implies$ size is between $\sqrt{B}$ and B
 $\implies$ any root-to-node path (search path) visits $\leq \frac{\lg N}{\frac{1}{2} \lg B} = 2 \log_B N$ trees that have size $\leq B$
- each tree of size $\leq B$ occupies ≤ 2 memory blocks

$\implies \leq 4 \log_B N = O(\log_B N)$ memory transfers

- this generalizes to heights that are not powers of 2, B-trees of constant branching factor and dynamic B-trees: $O(\log_B N)$ insert/delete. [Bender, Demaine, Farach-Colton 2000]

Sorting

B-trees

N inserts into (cache-oblivious) B-tree $\implies MT(N) = \Theta(N \log_B N)$ **NOT OPTIMAL**. By contrast, BST sort is optimal $O(N \lg N)$

Binary mergesort

- binary mergesort is cache-oblivious.
- the merge is 3 parallel scans
 $\implies MT(N) = 2MT(N/2) + O(N/B + 1)$
 $MT(M) = O(M/B)$
- the recursion tree has $\lg(N/M)$ levels, and each level contributes $O(N/B)$
 $\implies MT(N) = \frac{N}{B} \lg \frac{N}{M} \leftarrow \frac{B}{\lg B}$ faster than the B-tree version discussed earlier!

M/B -way mergesort

- split array into M/B equal subarrays
- recursively sort each
- merge via M/B parallel scans (keeping one “current” block per list)

$$\begin{aligned}\implies MT(N) &= \frac{M}{B} MT\left(\frac{N}{M/B}\right) + O(N/B + 1) \\ MT(M) &= O(M/B)\end{aligned}$$

$$\begin{aligned}\implies \text{height becomes } &\log_{M/B} \frac{N}{M} + 1 \\ &= \log_{M/B} \frac{N}{B} \cdot \frac{B}{M} + 1 \\ &= \log_{M/B} \frac{N}{B} - \log_{M/B} \frac{M}{B} + 1 \\ &= \log_{M/B} \frac{N}{B}\end{aligned}$$

$$\implies MT(N) = O\left(\frac{N}{B} \log_{M/B} \frac{N}{B}\right)$$

This is asymptotically optimal, in the comparison model.

Cache-oblivious Sorting

This requires the tall-cache assumption: $M = \Omega(B^{1+\epsilon})$ for some fixed $\epsilon > 0$, e.g., $M = \Omega(B^2)$ or $M/B = \Omega(B)$.

Then, $\approx N^\epsilon$ -way mergesort with recursive (“funnel”) merge works.

Priority Queues

- $O(\frac{1}{B} \log_{M/B} \frac{N}{B})$ per insert or delete-min
- generalizes sorting
- external memory and cache-oblivious
- see 6.851

MIT OpenCourseWare
<http://ocw.mit.edu>

6.046J / 18.410J Design and Analysis of Algorithms
Spring 2015

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.